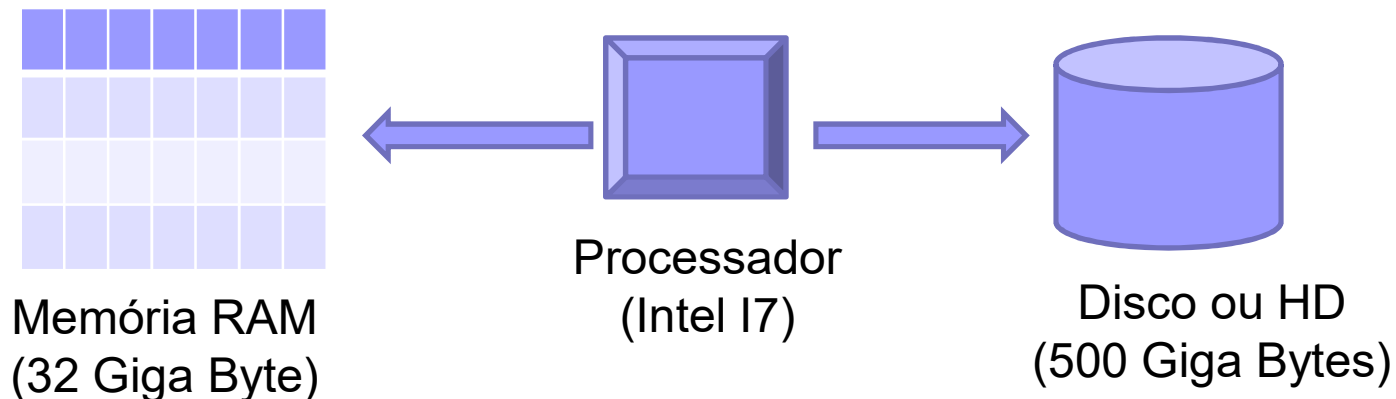




Computação Eletrônica

Arquivos

Arquivo



■ Por que temos 2 memórias?

- HD: permanente (pode desligar o computador), barato e lento
 - O HD é representado por um cilindro 
- RAM: volátil (ao desligar o computador, perde tudo), caro e rápido



Arquivo

- Até o momento, todos os programas em C usavam apenas a memória RAM
- As variáveis são armazenadas na memória RAM



Arquivo

- Suponha um programa em C, o qual lê o nome e a nota de 70 alunos armazenando em 2 vetores.
 - Se o computador for desligado no meio da digitação, todos os dados são perdidos
 - Pior: assim que o programa terminar, todos os dados são perdidos também
 - Como fazer para salvar estas informações no disco?
 - Ou seja, como manipular arquivos em C?



Arquivo

■ Texto

- ☐ Sequência de caracteres organizados em linhas
- ☐ Pode ser manipulado por editores de textos (como Bloco de Notas, Word, etc)
- ☐ O formato interno é projetado pelo programador

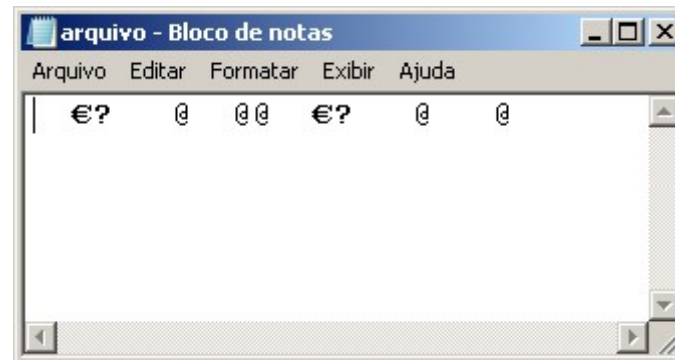
■ Binário

- ☐ Representados de forma similar como os dados são guardados na memória principal
- ☐ Mais rápidos de serem manipulados
- ☐ Difícil de serem lidos por seres humanos (zeros e uns)

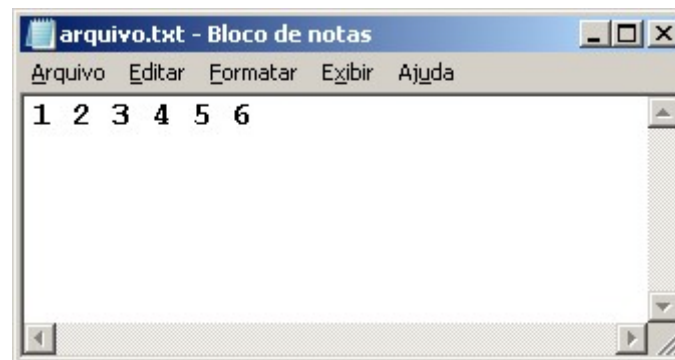
Arquivo

- Conteúdo (Ex: números inteiros)

- ☐ Binário



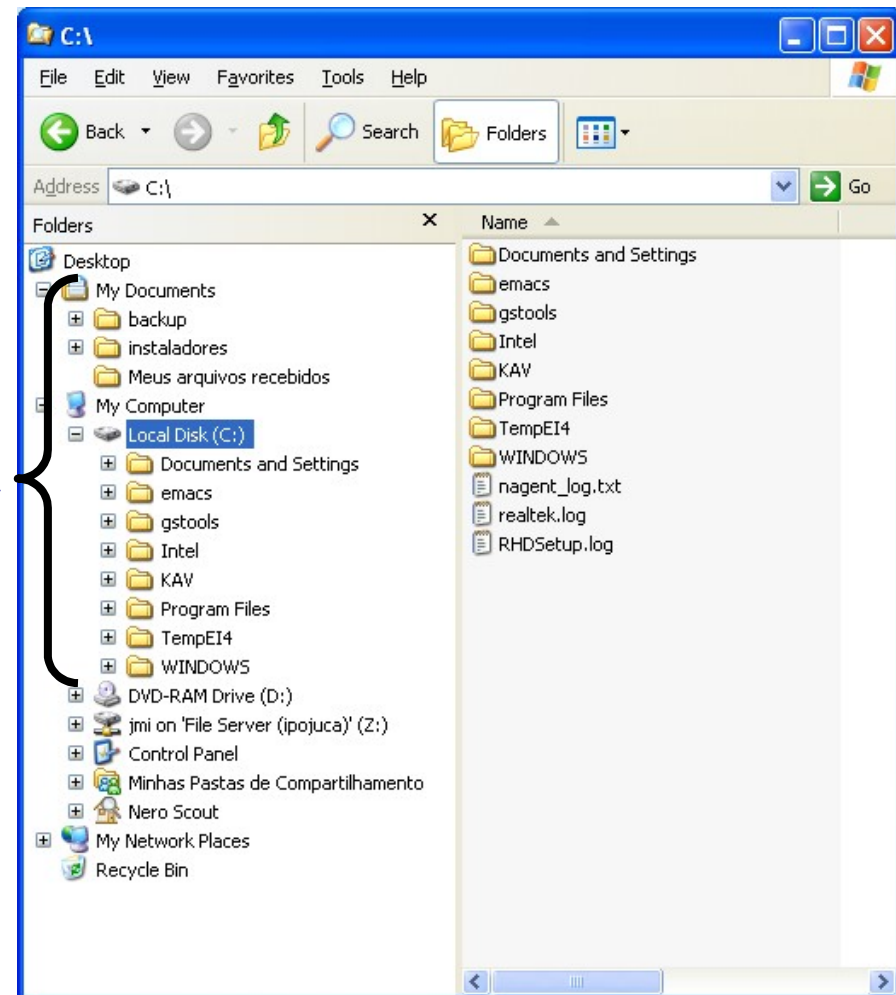
- ☐ Texto



Arquivo

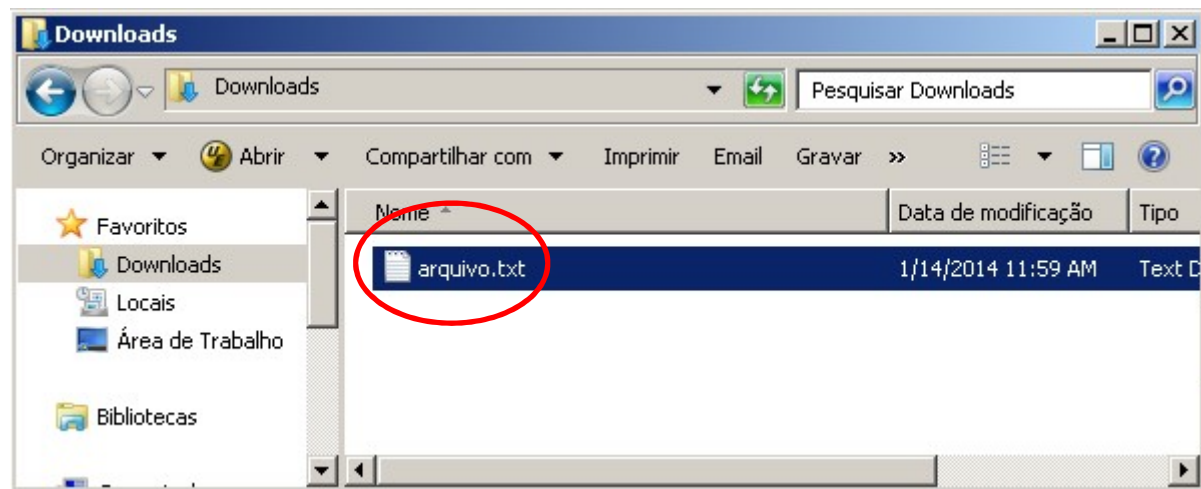
- No disco (ou HD) é onde ficam armazenados os arquivos

Todas estas pastas e estes arquivos estão armazenados no disco. Ou seja, você pode desligar o computador sem perder esses dados.



Arquivo

- Exemplo de um arquivo cujo nome é `arquivo.txt`
- Este arquivo pode ser salvo em qualquer pasta do seu Windows. No caso abaixo, a pasta é Downloads





Arquivo Texto

- Como criar um arquivo texto em branco?



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;

    p = fopen("arquivo.txt", "W");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

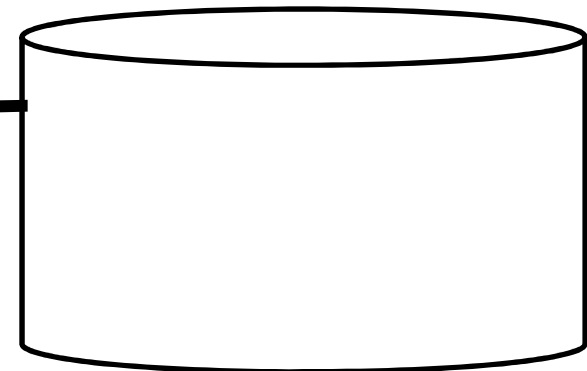
Arquivo Texto

Monitor / Teclado

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Memória

000	001	002
003	004	005

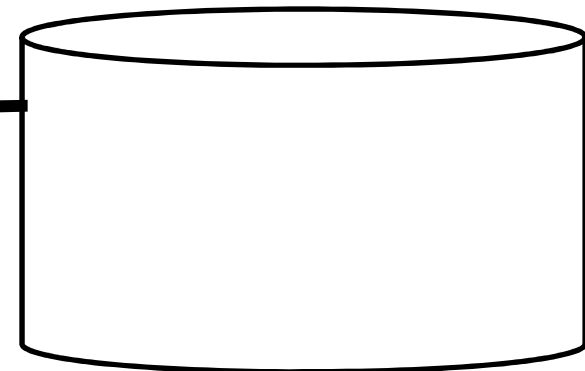
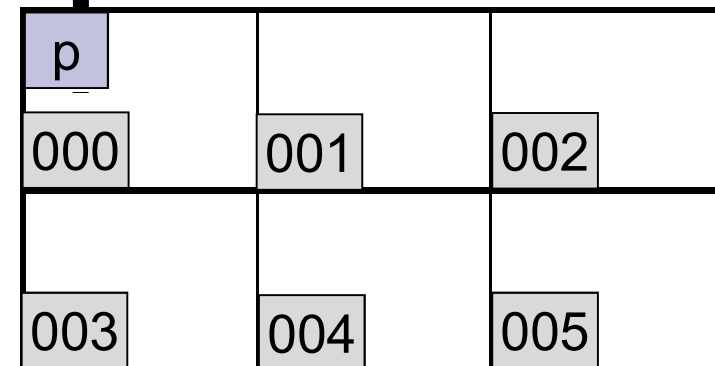


Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



Arquivo Texto

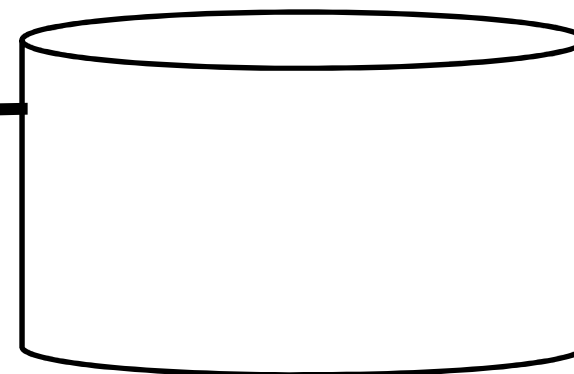
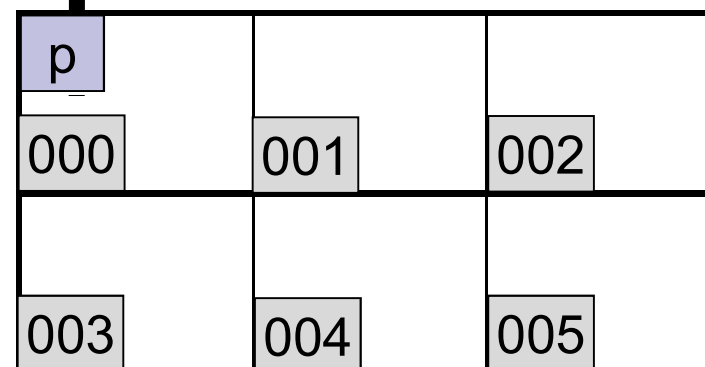
*Tenta criar arquivo.txt
no HD.*

Monitor / Teclado

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

*O parâmetro "w" (escrita)
significa que o arquivo será
criado e alterado.*

Memória



Arquivo Texto

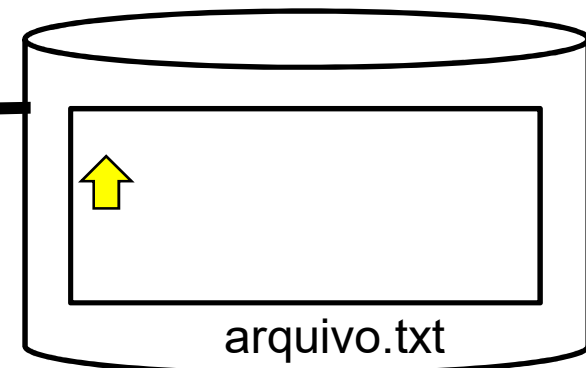
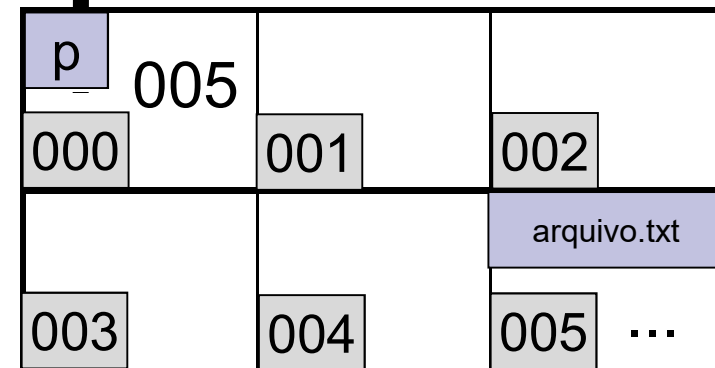
Assumimos que foi bem sucedido.

Monitor / Teclado

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

p aponta para um struct que armazena informações sobre o arquivo (posição atual, tamanho, localização no HD, etc). Não temos acesso a esta estrutura diretamente nem precisamos conhecê-la. A seta amarela indica a posição atual do arquivo.

Memória



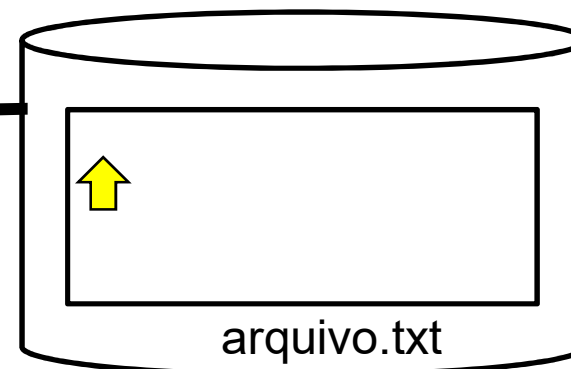
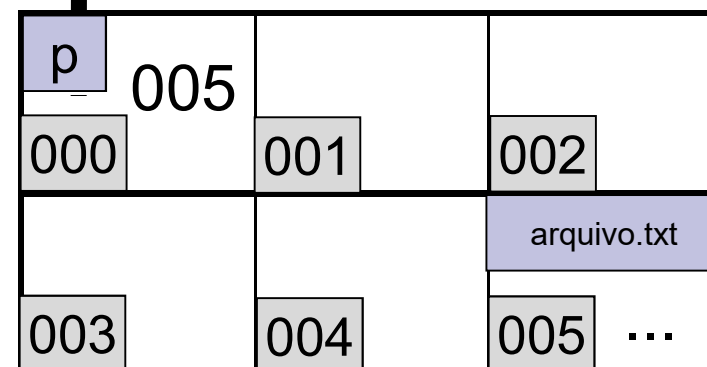
Arquivo Texto

Como a criação do arquivo foi bem sucedida, p não é NULL.

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



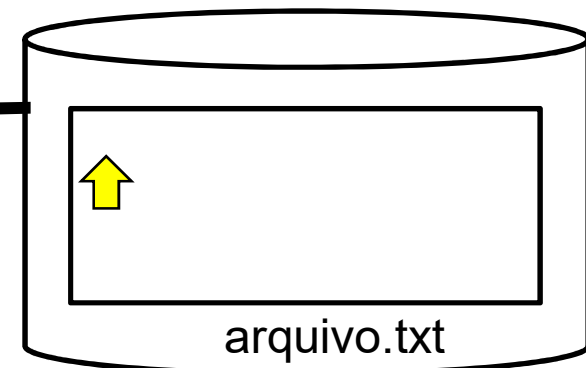
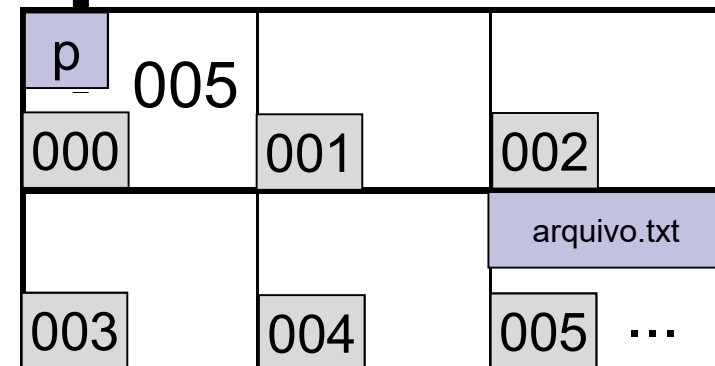
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Criacao com sucesso!

Memória



Arquivo Texto

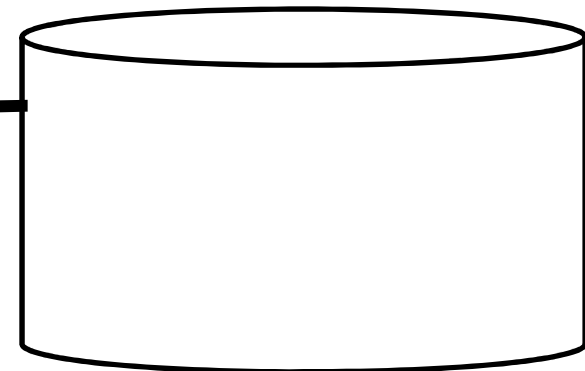
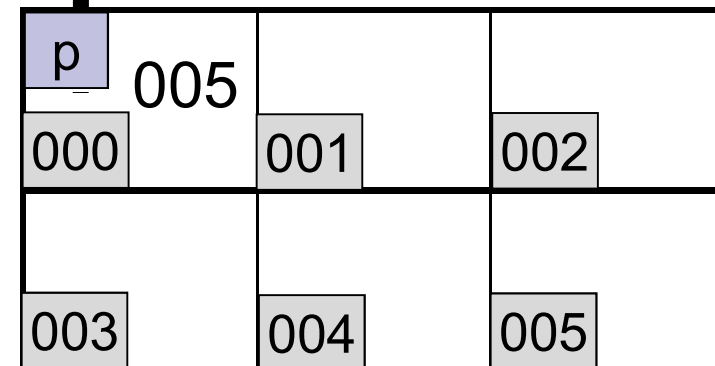
```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

fclose() fecha o arquivo

Monitor / Teclado

Criacao com sucesso!

Memória



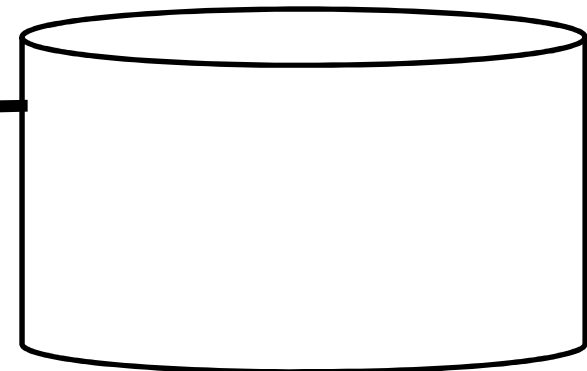
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        printf("Criacao com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005





Arquivo Texto

- Cuidado!

- ☐ `fopen("arquivo.txt", "w")`
- ☐ Se arquivo.txt já existir, ele vai apagar! E depois cria arquivo.txt em branco.
- ☐ Tudo que havia antes será perdido!



Arquivo Texto

- Como criar um arquivo texto e gravar um número nele?



Arquivo Texto

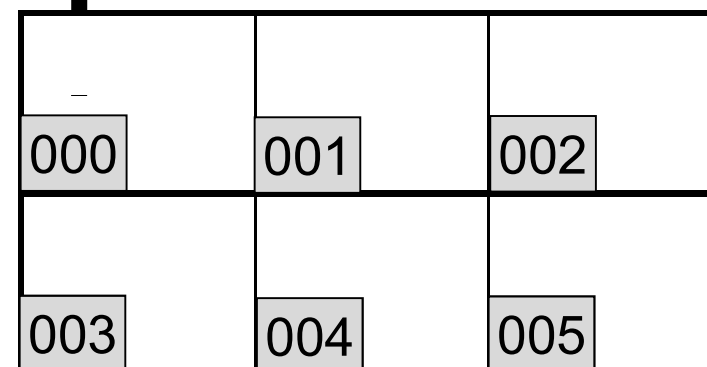
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p,"%d",i) ;
    }
    fclose(p);
    return 0;
}
```

Arquivo Texto

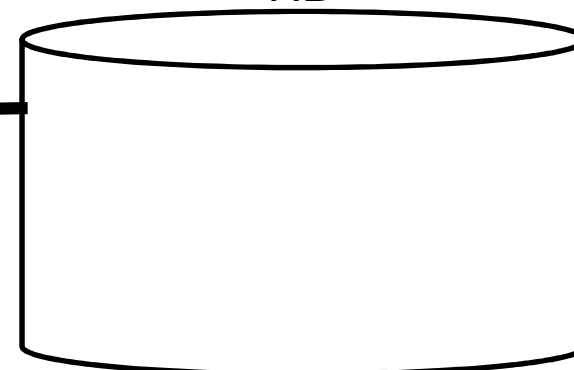
Monitor / Teclado

```
#include<stdio.h>
→ int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p,"%d",i);
    }
    fclose(p);
    return 0;
}
```

Memória



HD

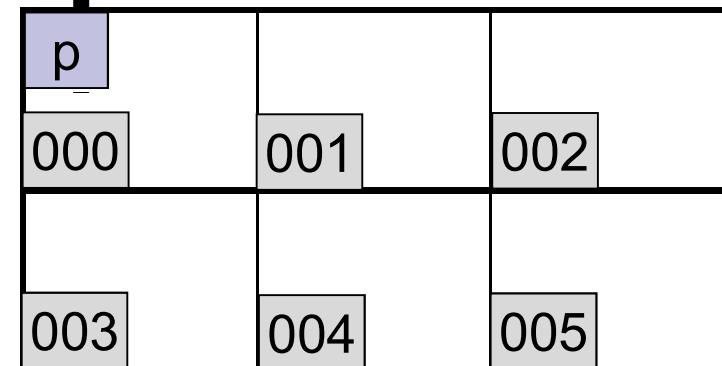


Arquivo Texto

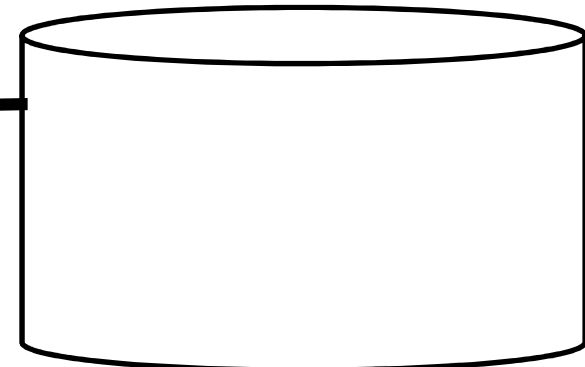
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

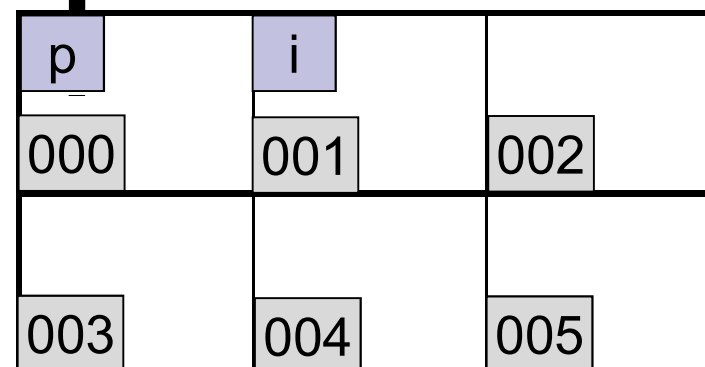


Arquivo Texto

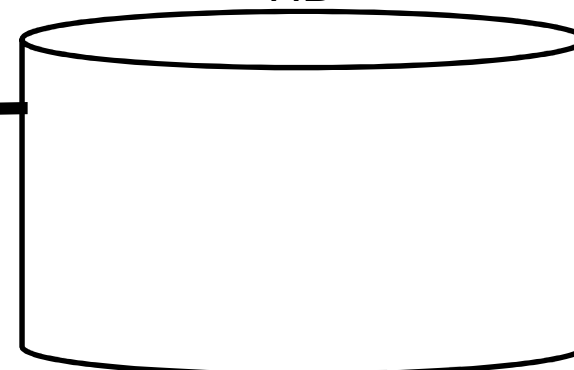
```
#include<stdio.h>
int main() {
    FILE *p;
    → int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



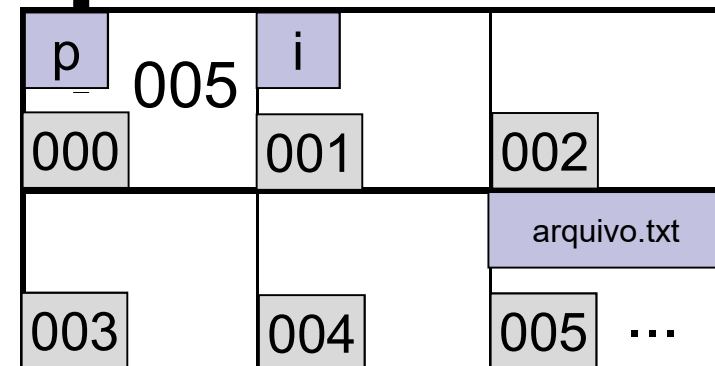
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

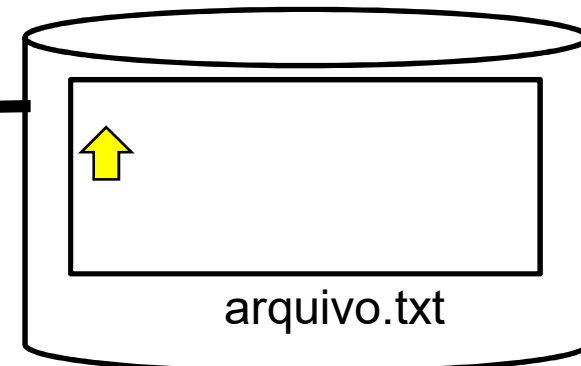
Assumimos que a criação do arquivo foi bem sucedida.

Monitor / Teclado

Memória



HD



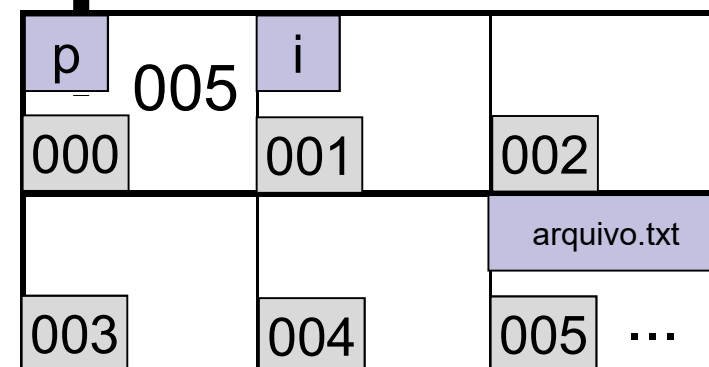
Arquivo Texto

$p == NULL$
 $= 0$

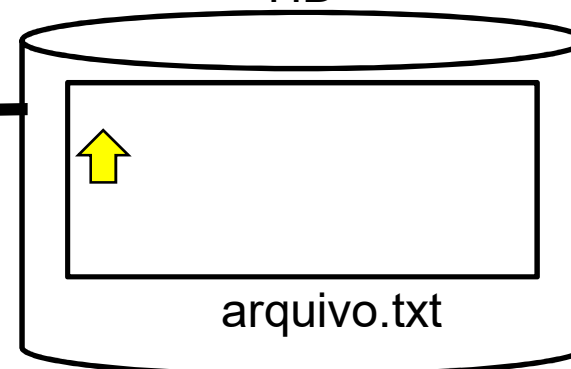
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

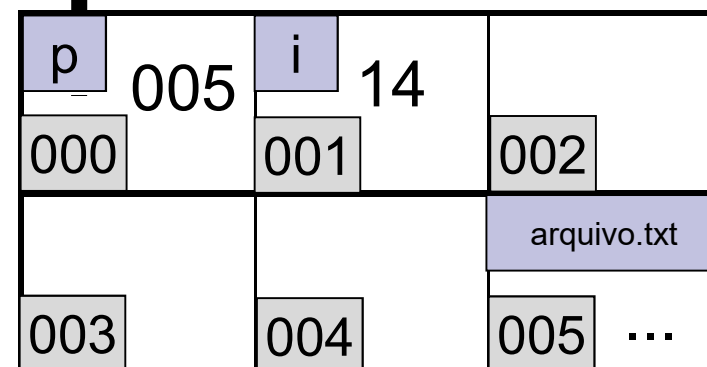


Arquivo Texto

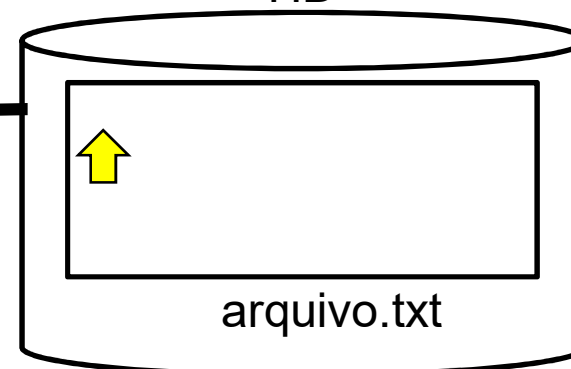
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

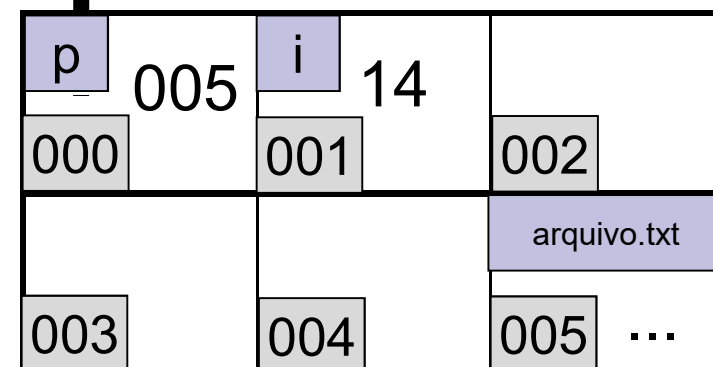


Arquivo Texto

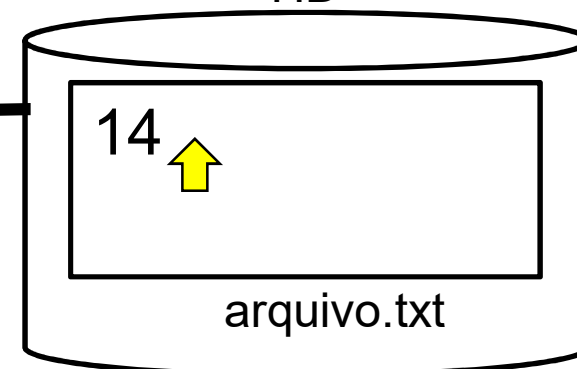
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        ➡ fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

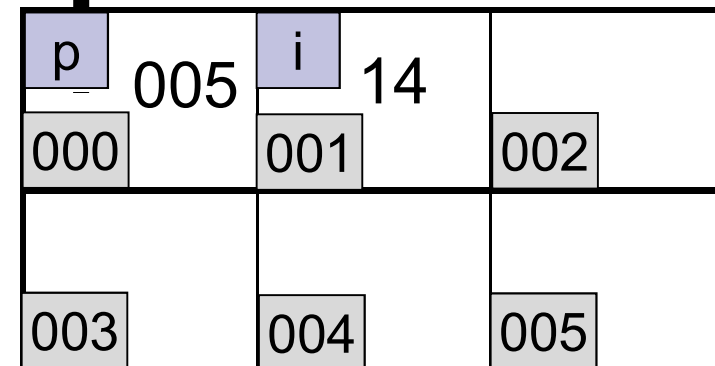


Arquivo Texto

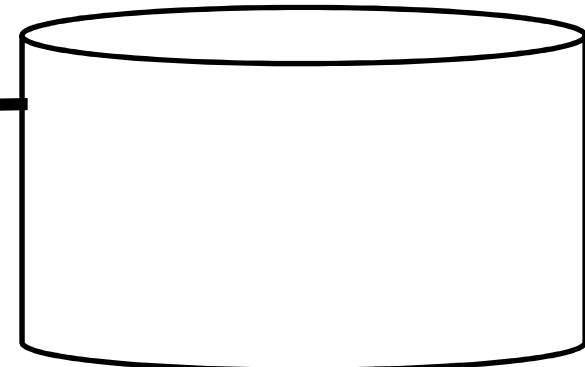
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

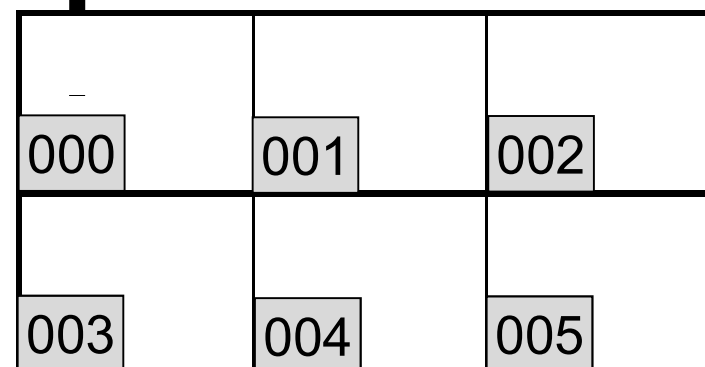


Arquivo Texto

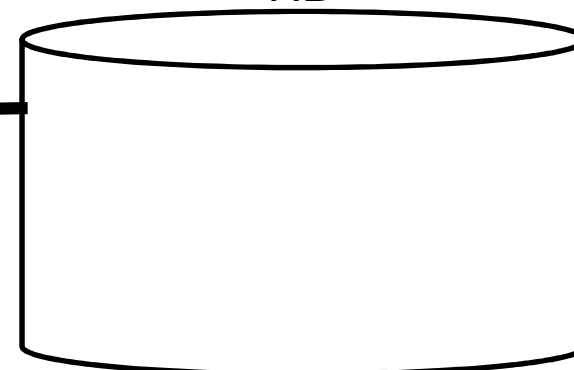
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        i = 14;
        fprintf(p, "%d", i);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD





Arquivo Texto

- Como abrir um arquivo texto e ler um número dele?



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p, "%d", &n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```


Arquivo Texto

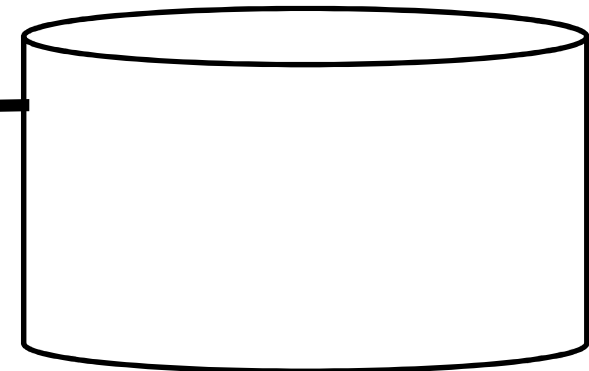
```
#include<stdio.h>
➡ int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005

HD

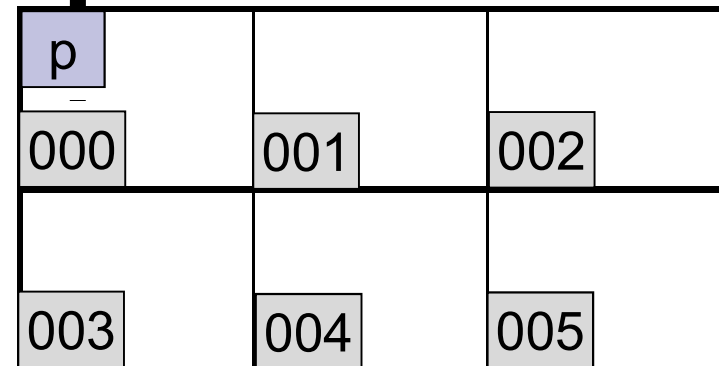


Arquivo Texto

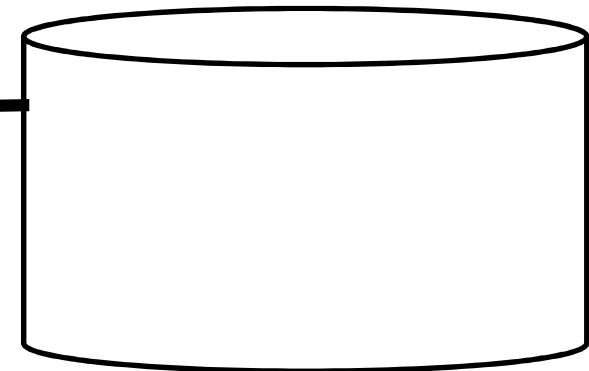
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

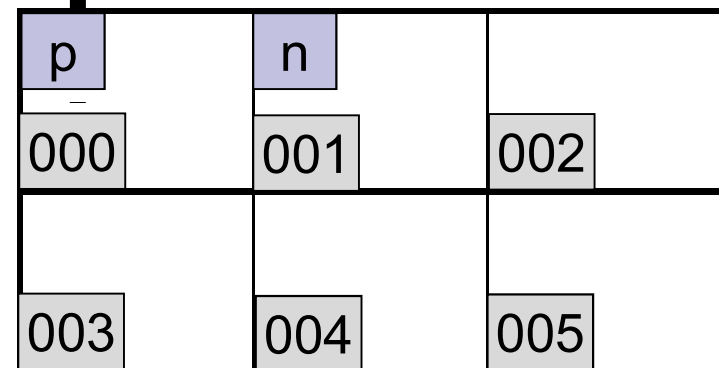


Arquivo Texto

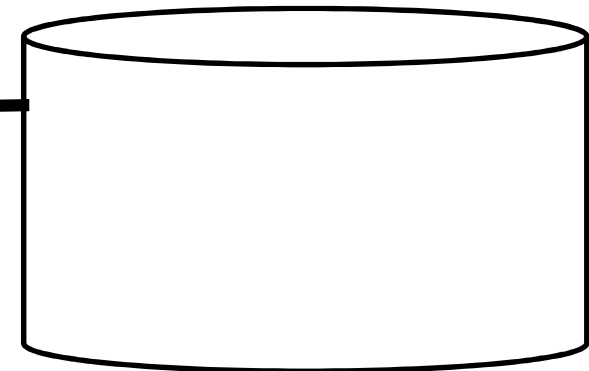
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



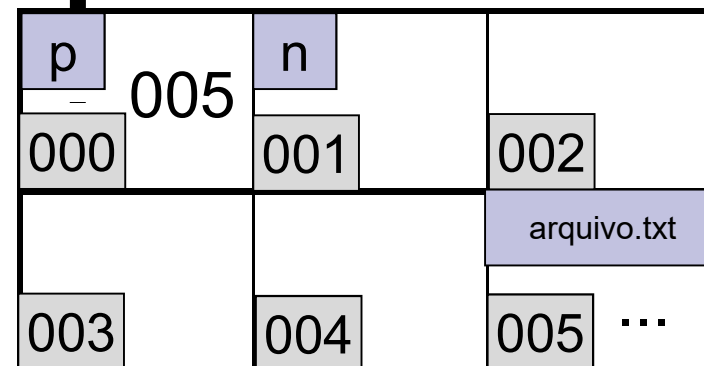
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p, "%d", &n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

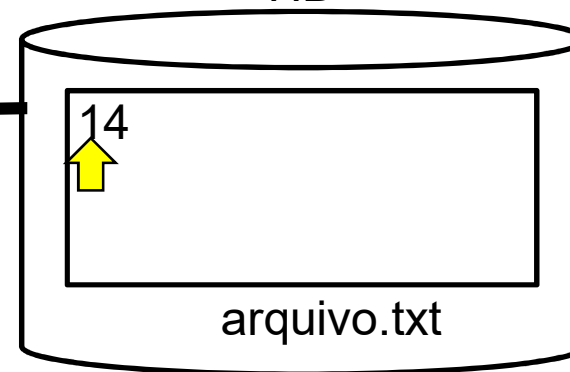
Assumimos que a abertura do arquivo foi bem sucedida. O parâmetro "r" (apenas leitura) significa que não iremos alterar o arquivo.

Monitor / Teclado

Memória



HD



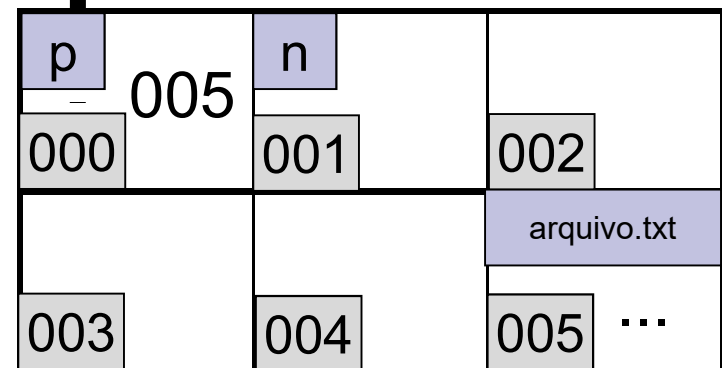
Arquivo Texto

$p == \text{NULL}$
 $= 0$

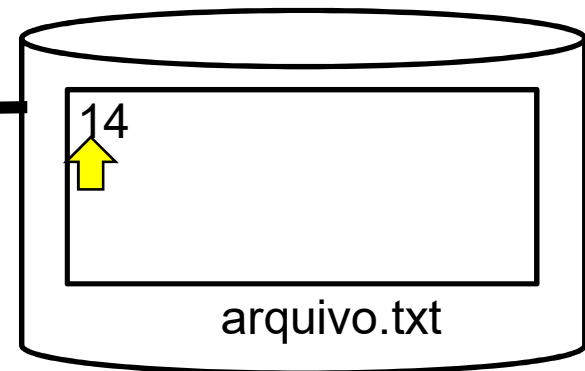
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p, "%d", &n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



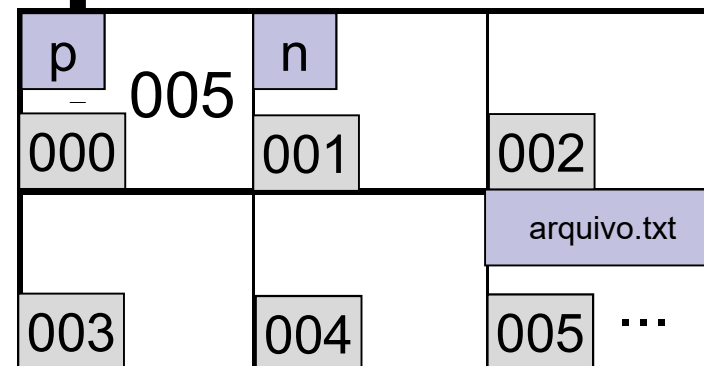
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

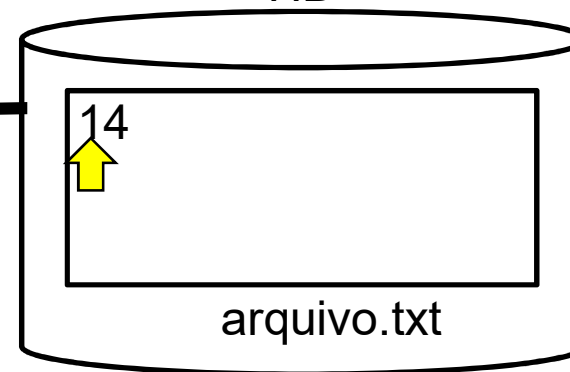
fscanf() vai tentar ler 1 inteiro do arquivo. Se conseguir, ele retorna 1. Se não conseguir, retorna 0. Se o arquivo já estava no fim, retorna -1.

Monitor / Teclado

Memória



HD



Arquivo Texto

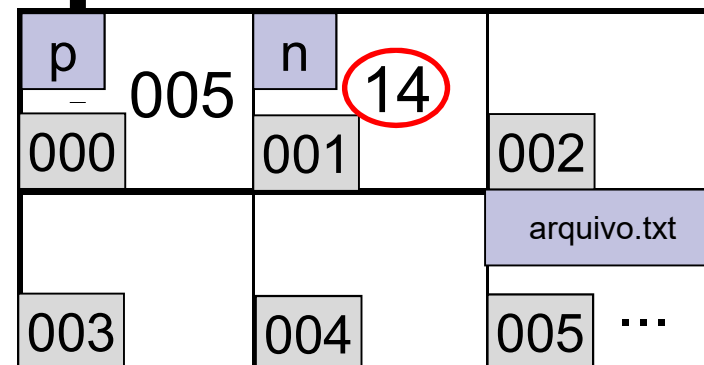
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

*fscanf(p,"%d",&n) == 1
= 1 == 1
= 1*

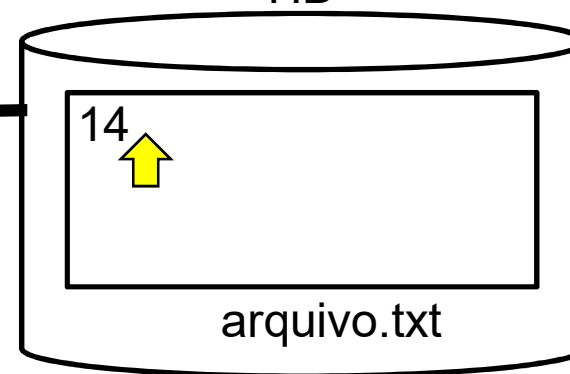
*Vamos assumir que fscanf()
retornou 1. Neste caso, ele grava
na variável n o valor 14 retorna
1 para o main()*

Monitor / Teclado

Memória



HD



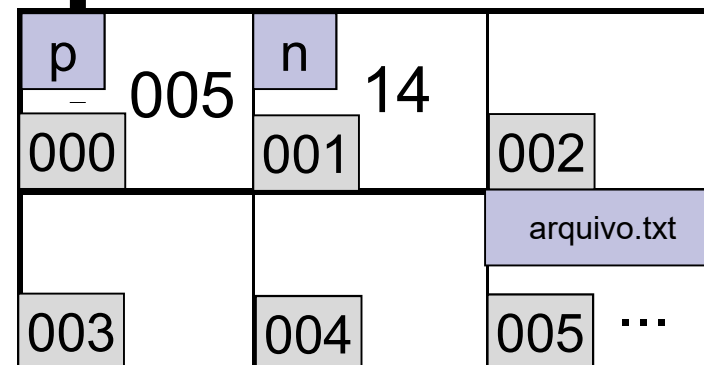
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

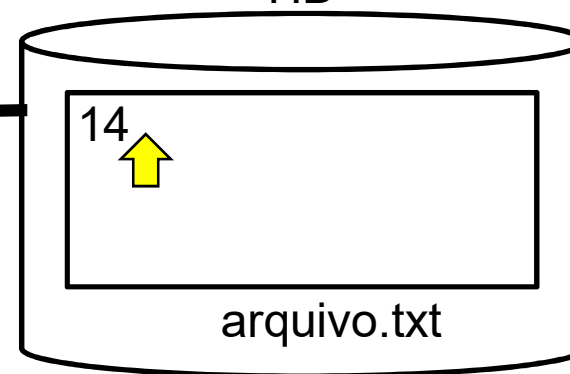
Monitor / Teclado

Numero lido: 14

Memória



HD



Arquivo Texto

Monitor / Teclado

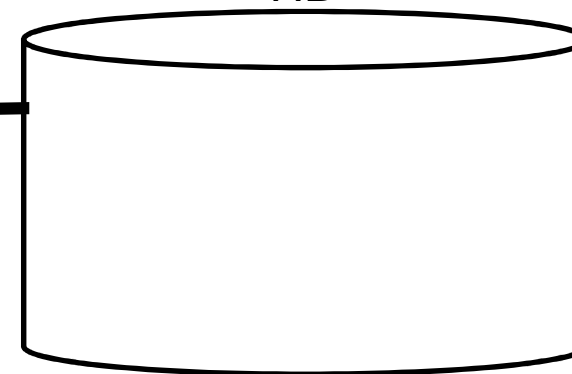
Numero lido: 14

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

Memória

p	n	
005	14	
000	001	002
003	004	005

HD



Arquivo Texto

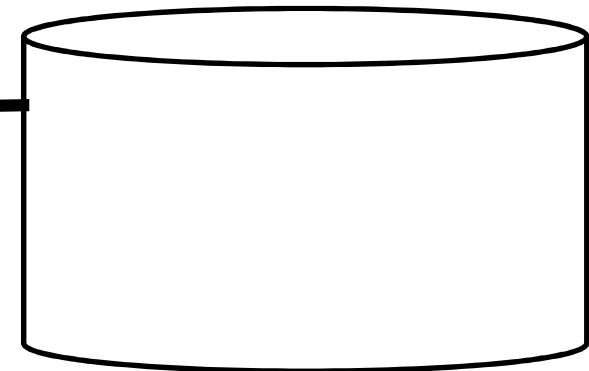
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005

HD





Arquivo Texto

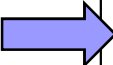
- Gravação de vários números em um arquivo



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p,"%d",i);
        }
    }
    fclose(p);
    return 0;
}
```

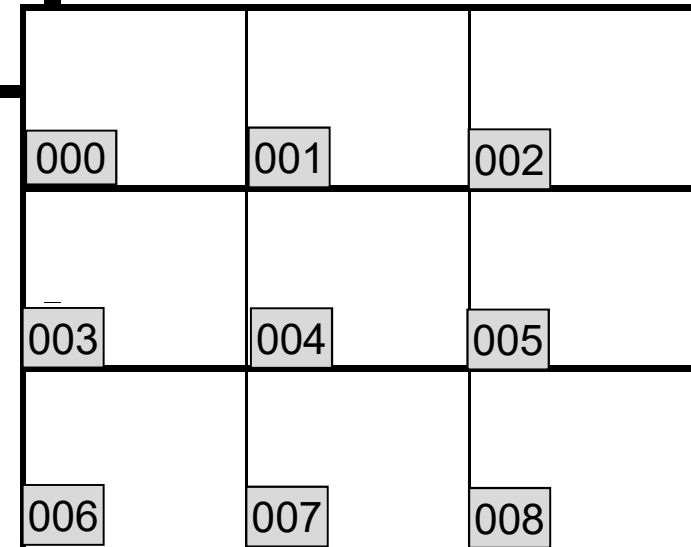
Arquivo Texto



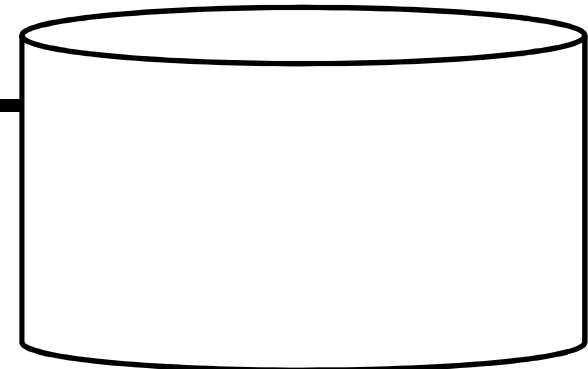
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

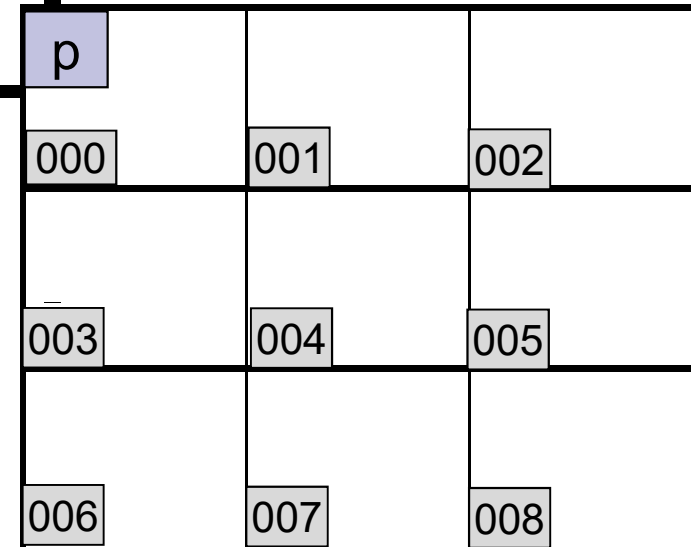


Arquivo Texto

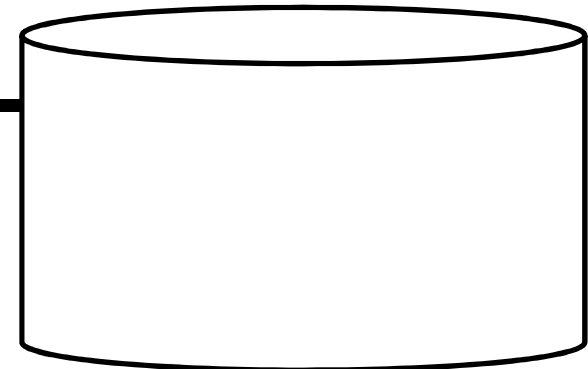
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

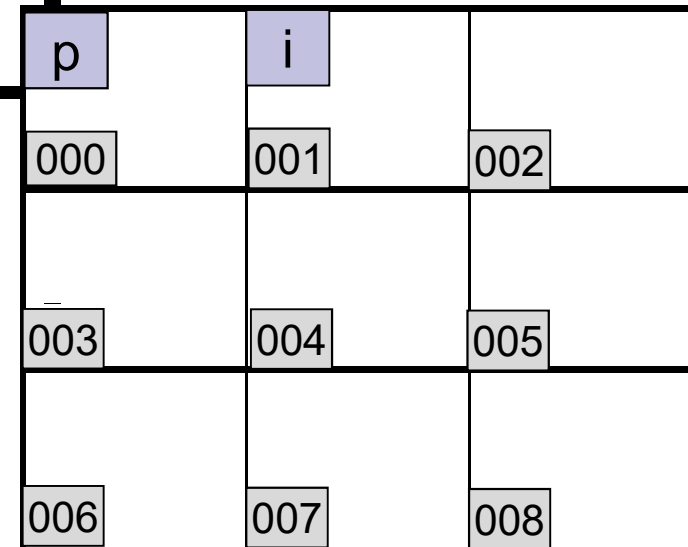


Arquivo Texto

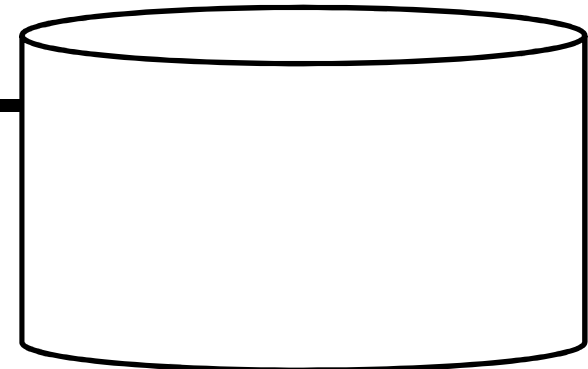
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



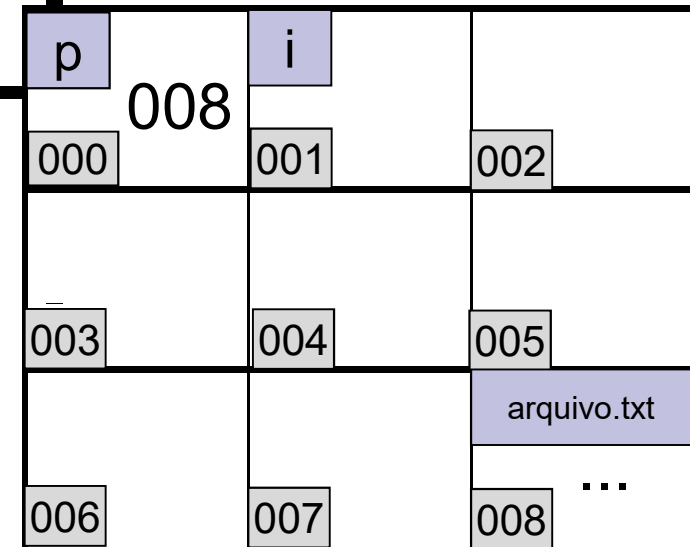
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

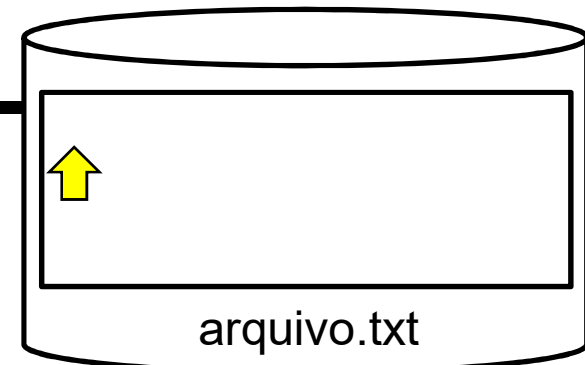
Assumimos que a criação do arquivo foi bem sucedida.

Monitor / Teclado

Memória



HD



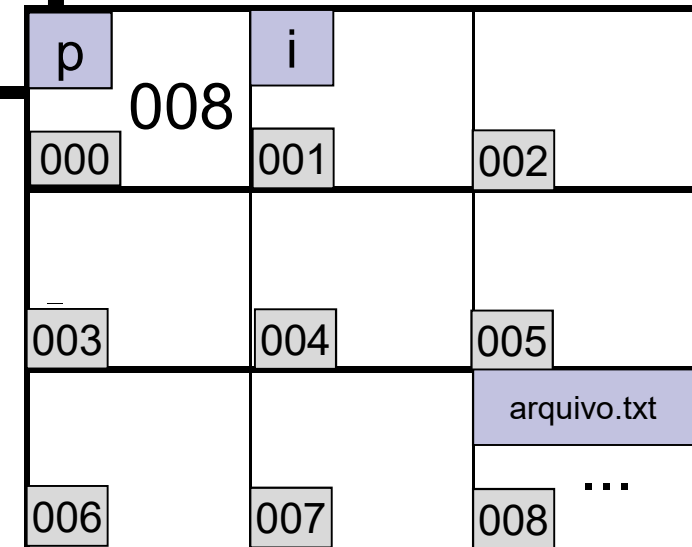
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

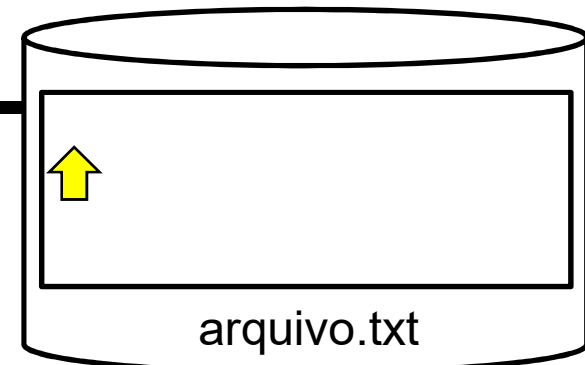
$p == \text{NULL}$
 $= 0$

Monitor / Teclado

Memória



HD

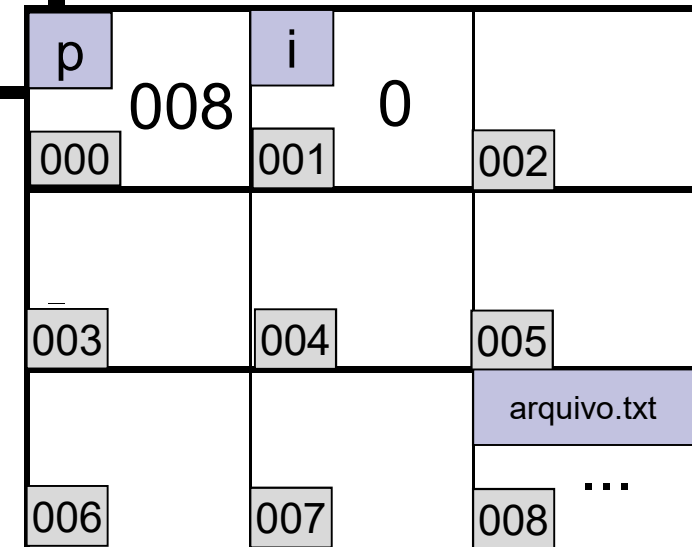


Arquivo Texto

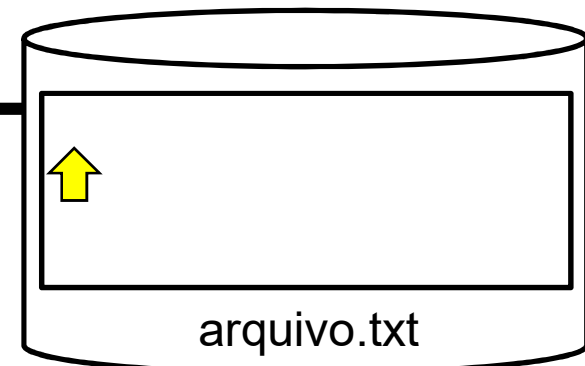
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for ((i = 0) i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



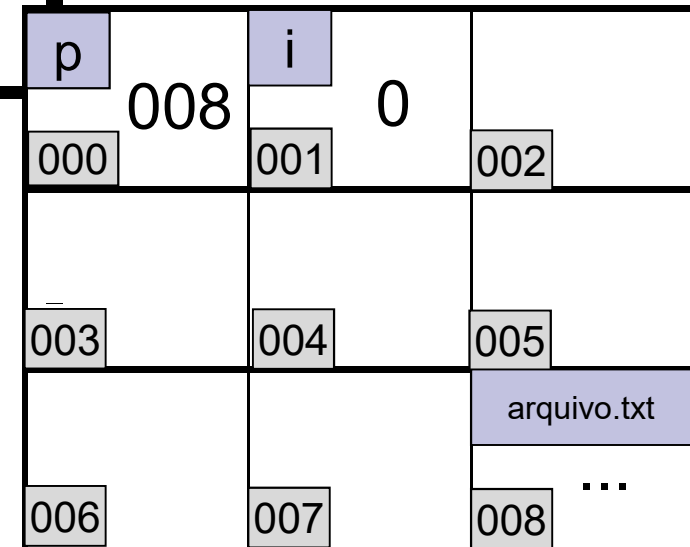
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

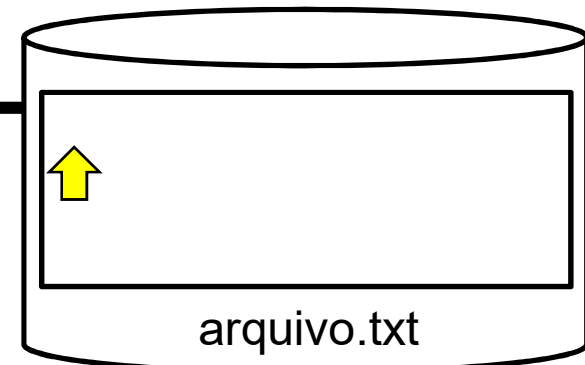
$0 < 5$
 $= 1$

Monitor / Teclado

Memória



HD

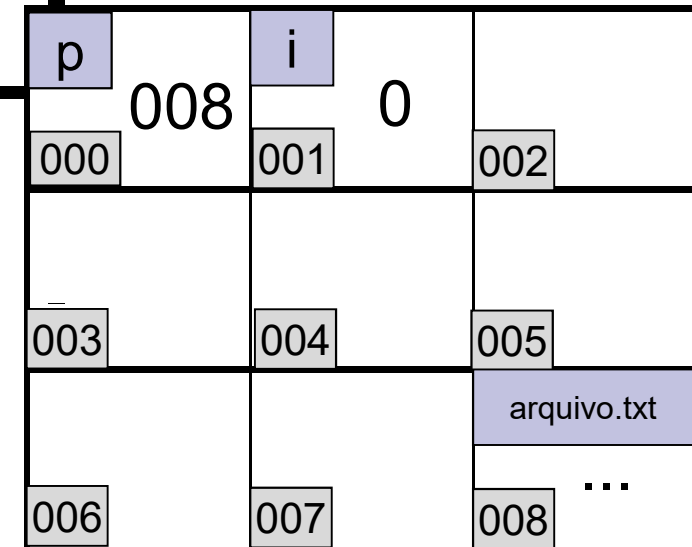


Arquivo Texto

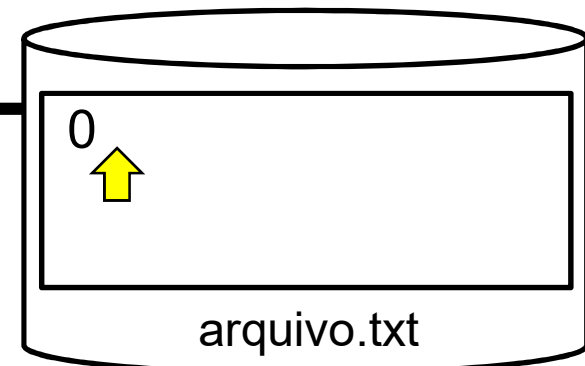
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            → fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

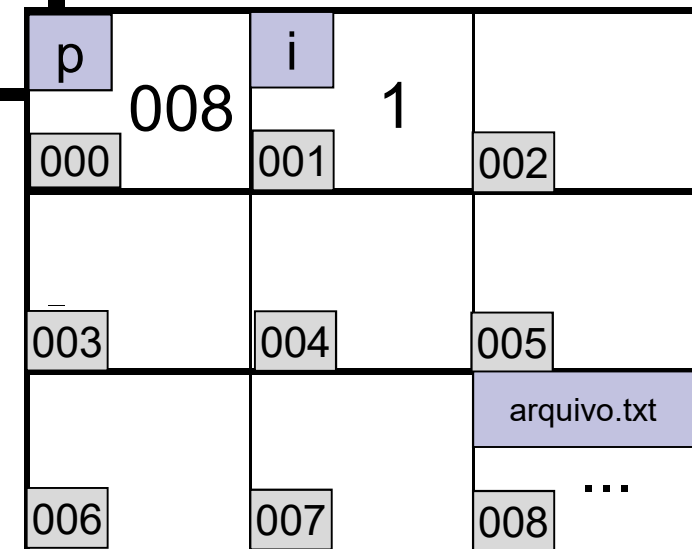


Arquivo Texto

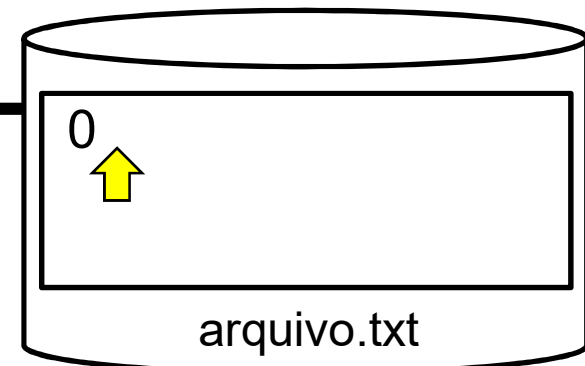
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



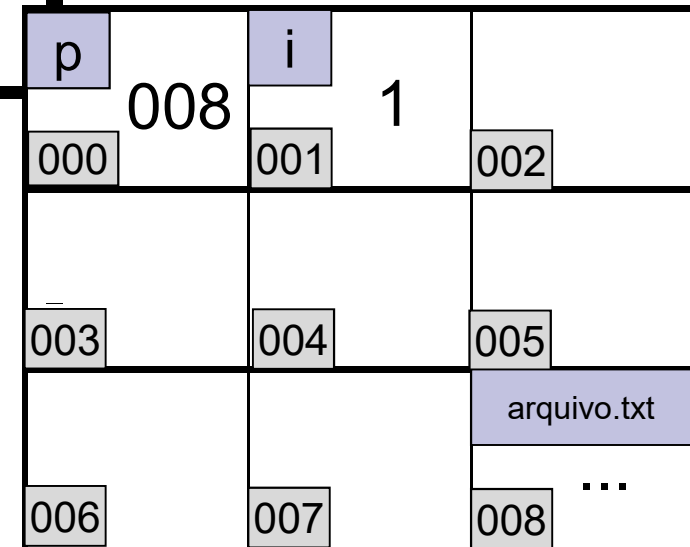
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

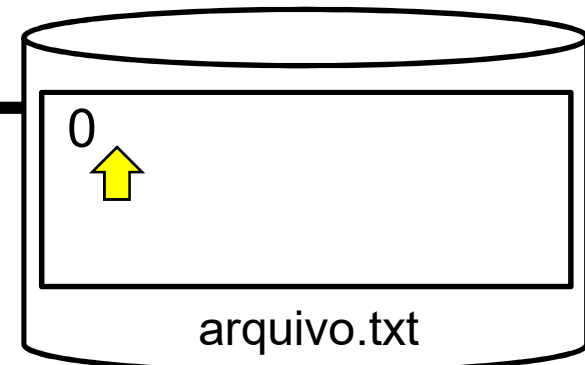
$1 < 5$
 $= 1$

Monitor / Teclado

Memória



HD

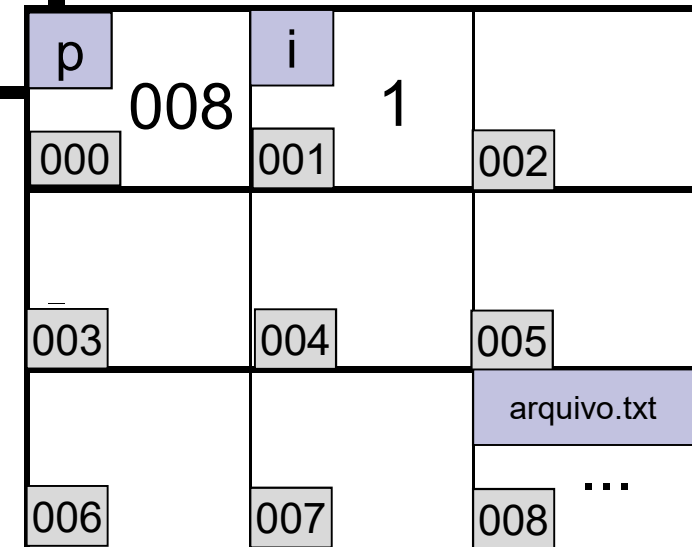


Arquivo Texto

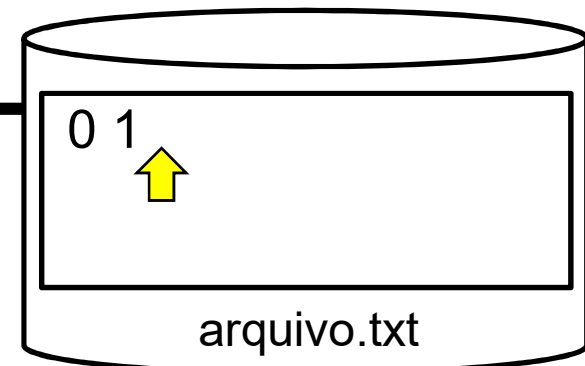
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            ➡ fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

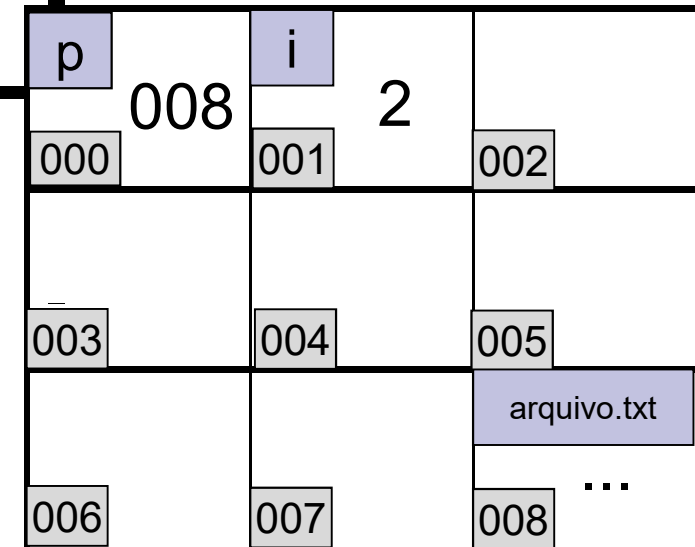


Arquivo Texto

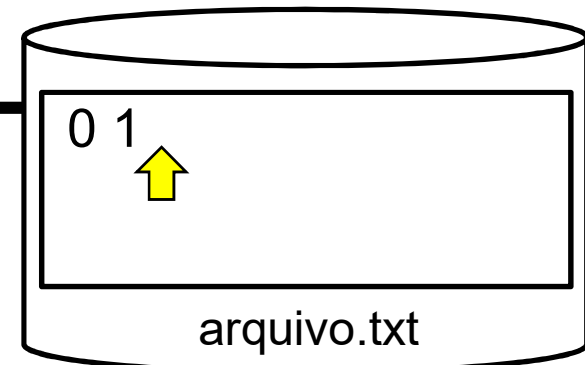
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5, i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



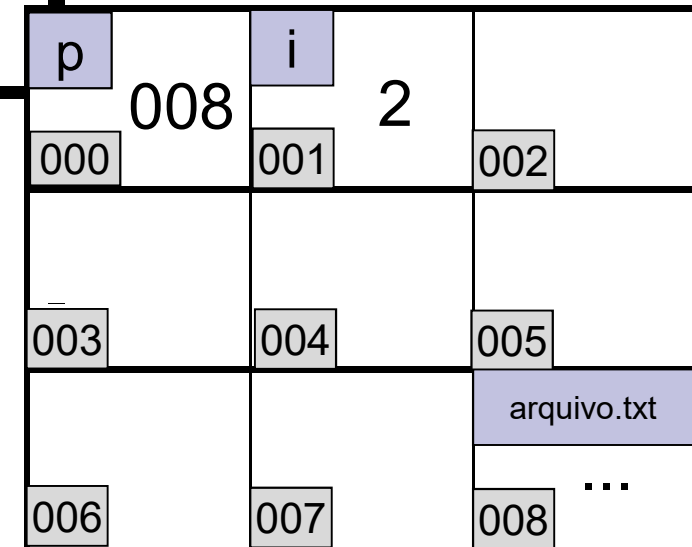
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

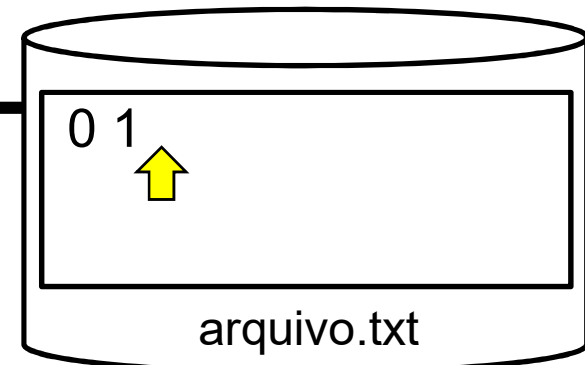
$2 < 5$
 $= 1$

Monitor / Teclado

Memória



HD

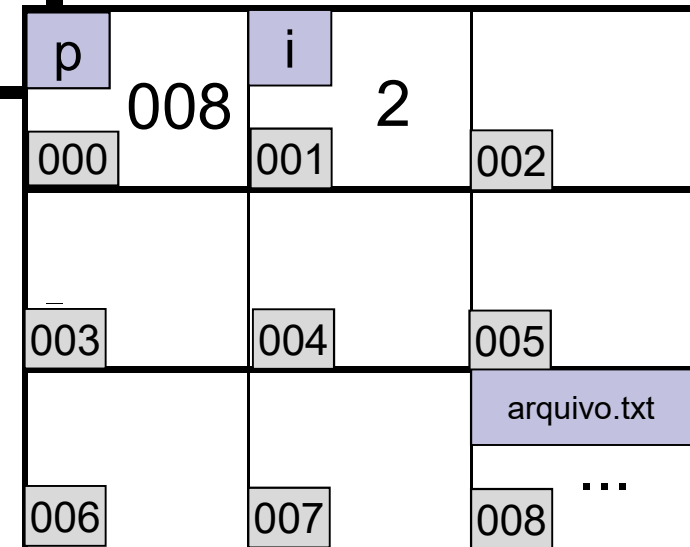


Arquivo Texto

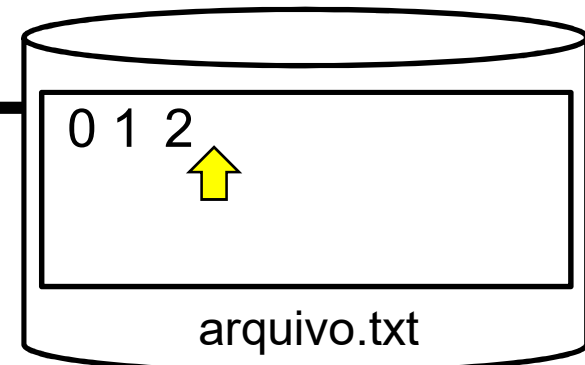
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            → fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

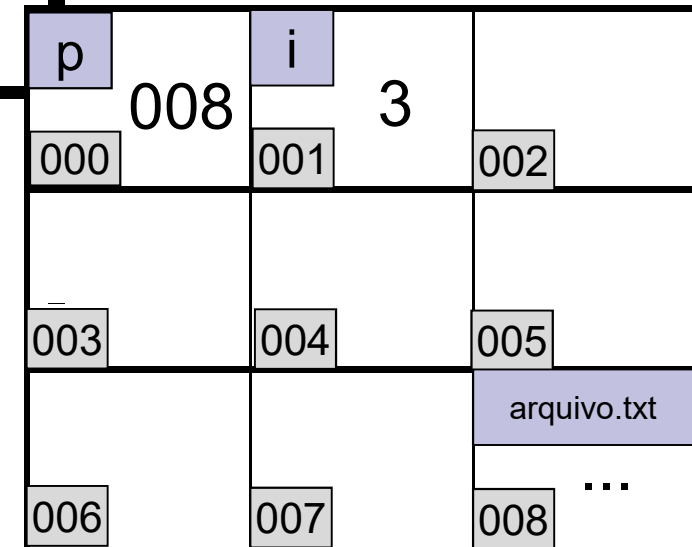


Arquivo Texto

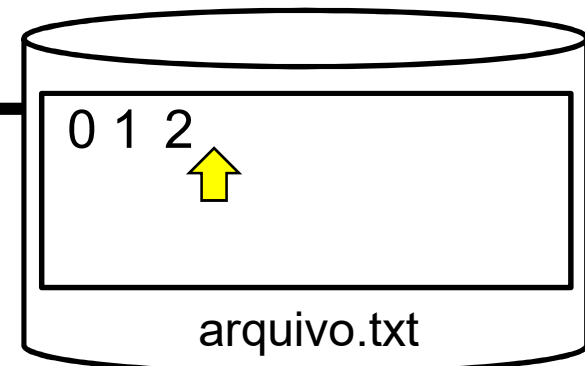
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



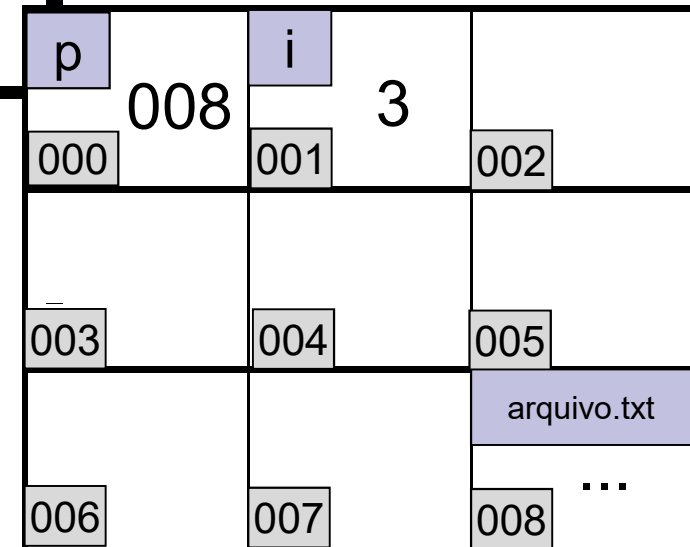
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

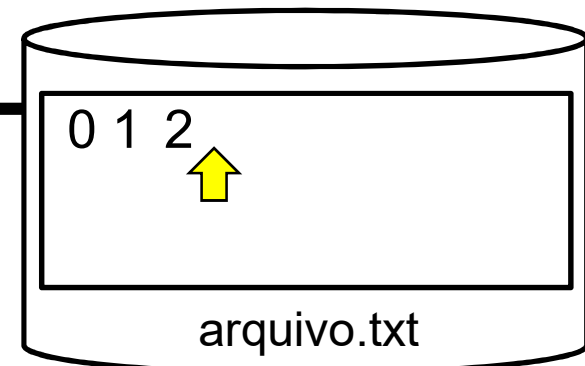
$3 < 5$
 $= 1$

Monitor / Teclado

Memória



HD

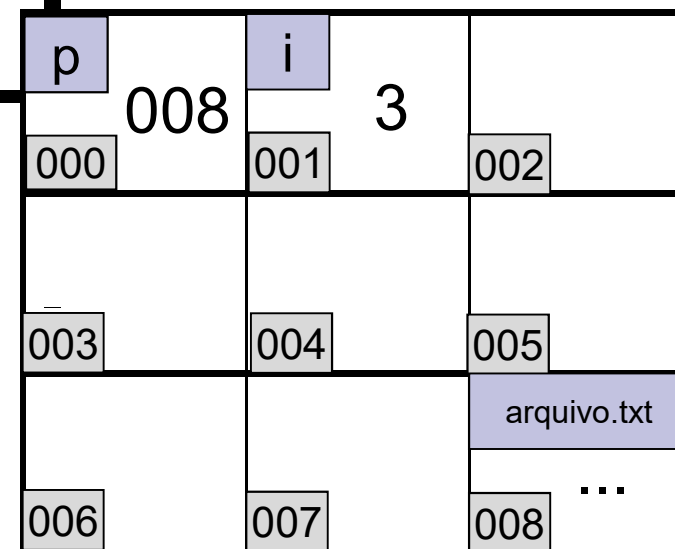


Arquivo Texto

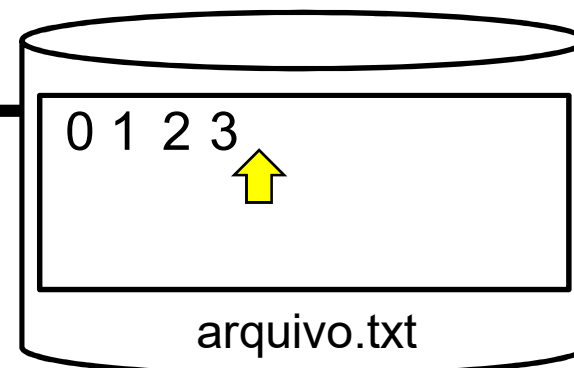
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            ➡ fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

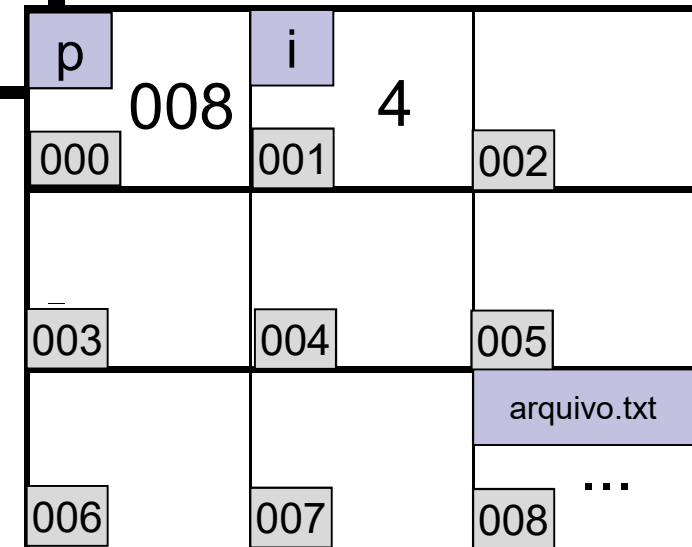


Arquivo Texto

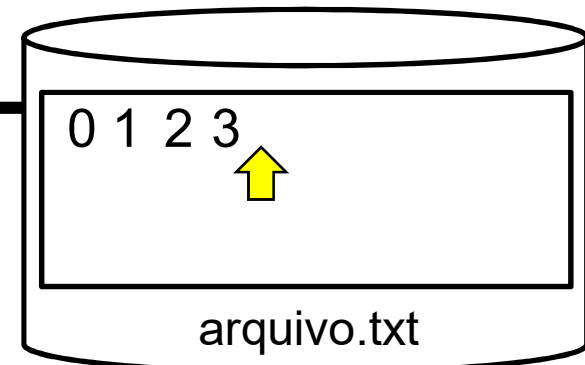
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



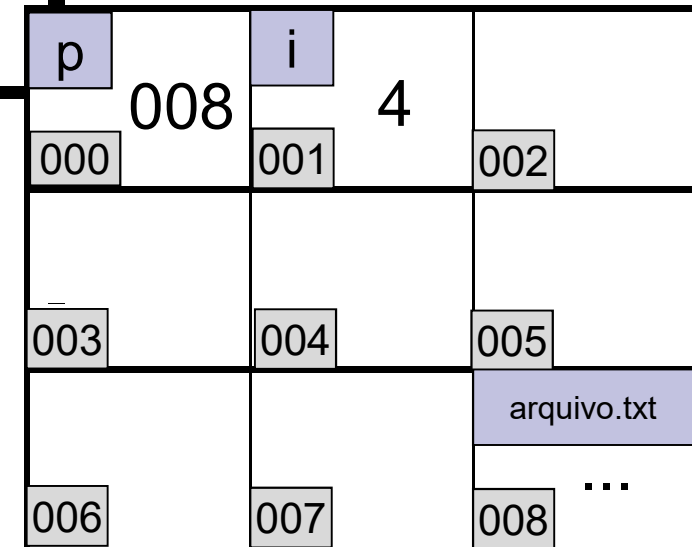
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        ➡ for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

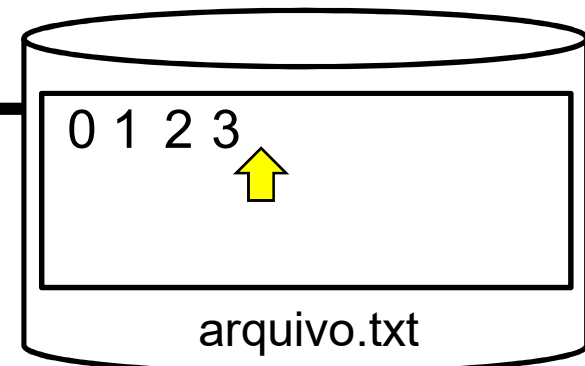
$4 < 5$
 $= 1$

Monitor / Teclado

Memória



HD

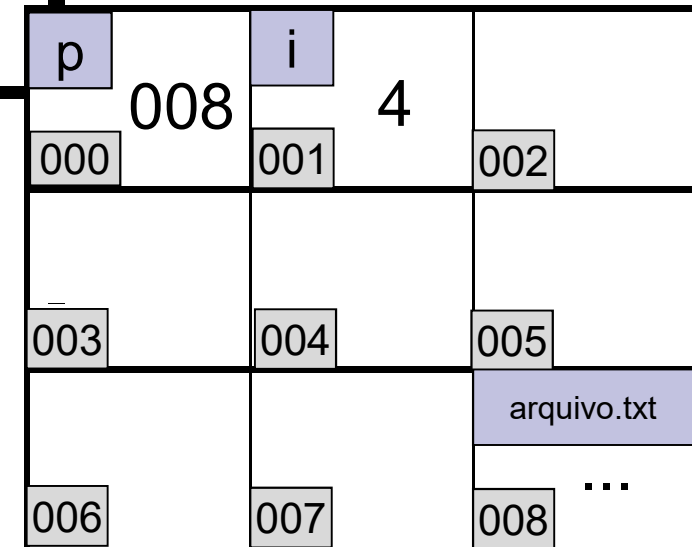


Arquivo Texto

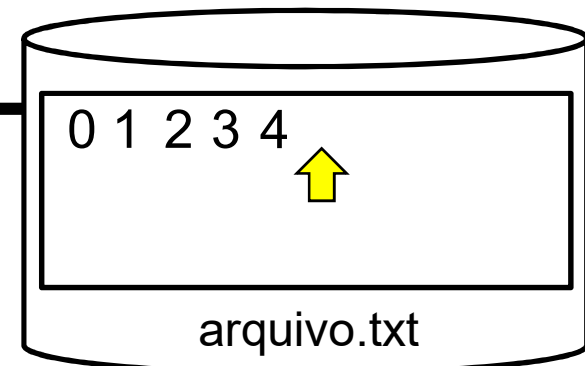
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            → fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

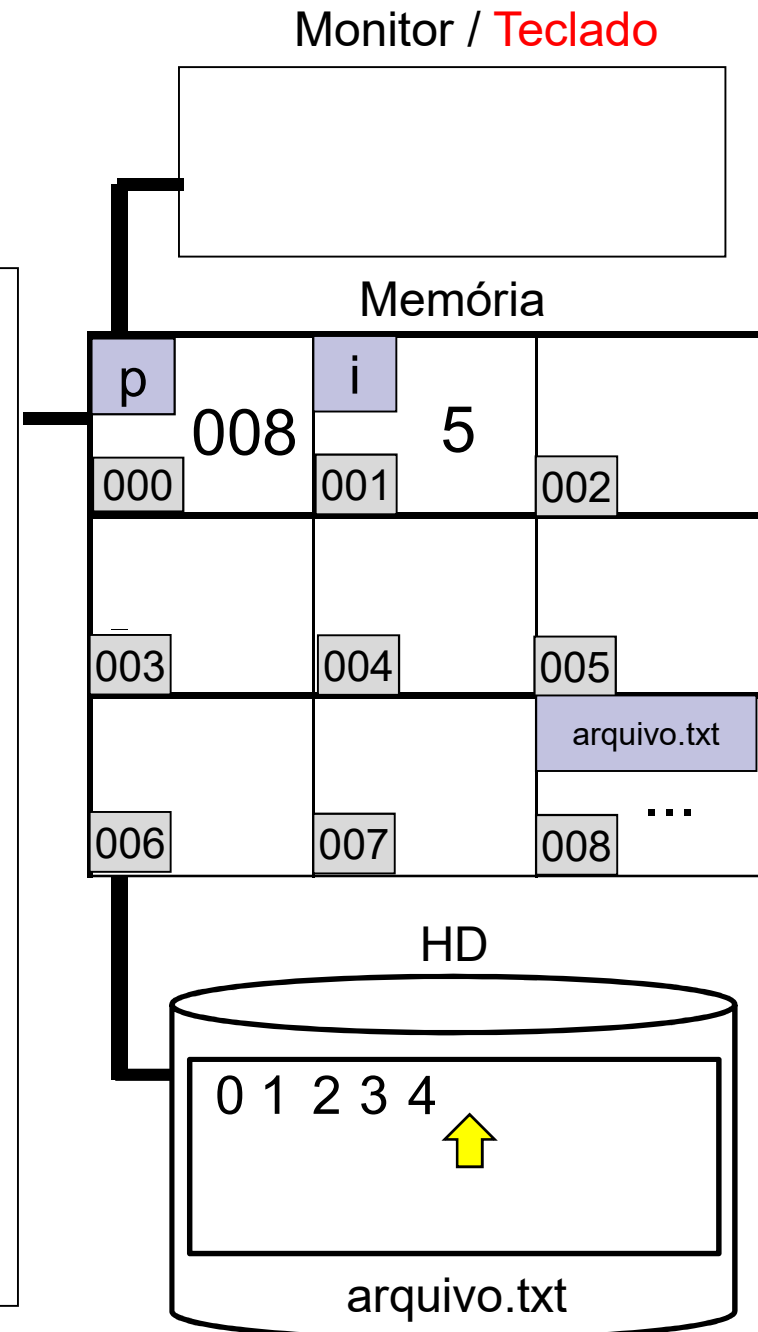
Monitor / Teclado

Memória



HD





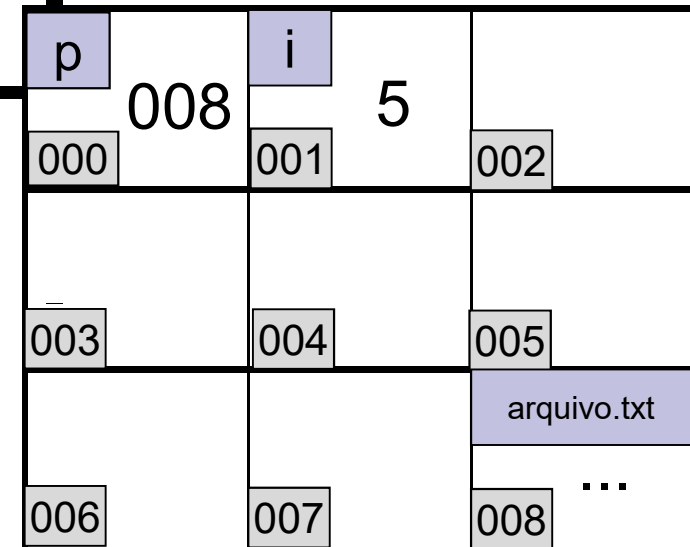
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

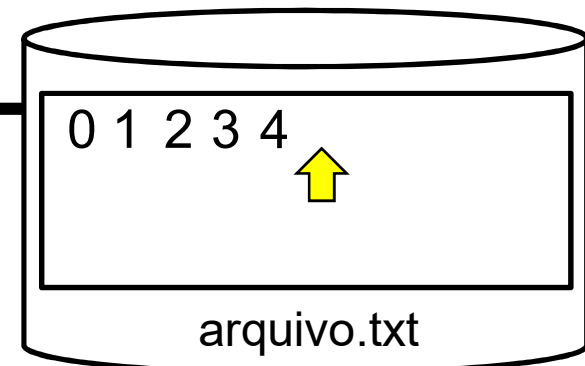
$5 < 5$
 $= 0$

Monitor / Teclado

Memória



HD



Arquivo Texto

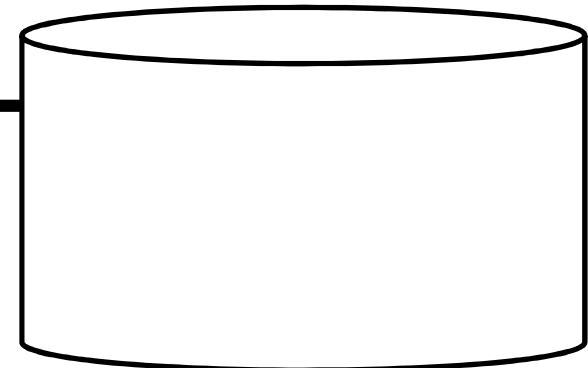
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

p	i	
008	5	
000	001	002
003	004	005
006	007	008

HD

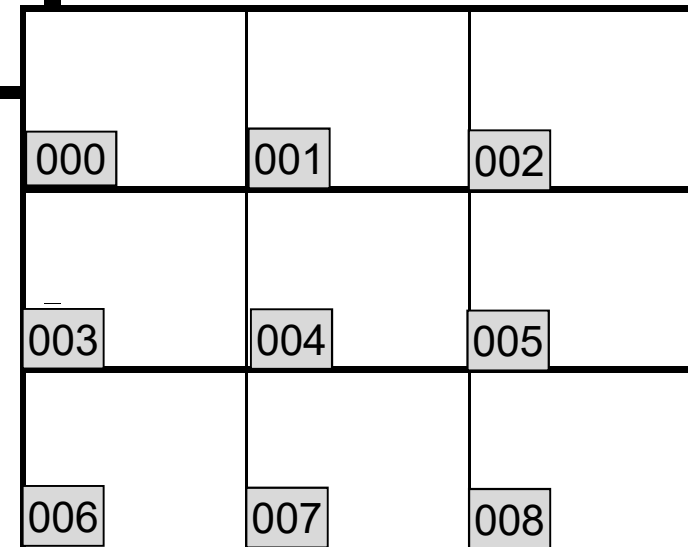


Arquivo Texto

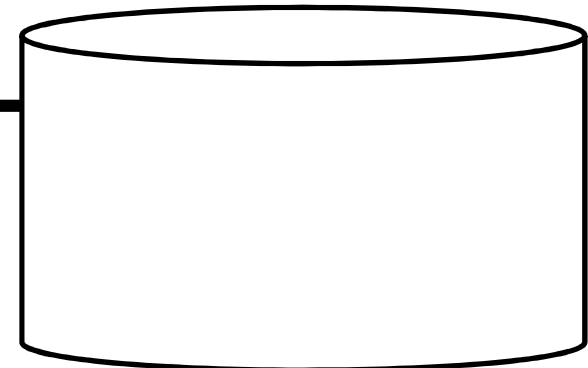
```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p, "%d", i);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD





Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        if (fscanf(p,"%d",&n) == 1) {
            printf("Numero lido: %d\n", n);
        }
        else {
            printf("Ou erro ou fim de arquivo.\n");
        }
    }
    fclose(p);
    return 0;
}
```

*Se arquivo.txt contém
01234
O que este programa
imprime na tela?*



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for (i = 0; i < 5; i++) {
            fprintf(p,"%d\n",i);
        }
    }
    fclose(p);
    return 0;
}
```

*Como ficará o
arquivo desta vez?*



Arquivo Texto

- Leitura de um arquivo texto com o seguinte conteúdo:

0

1

2

3

4

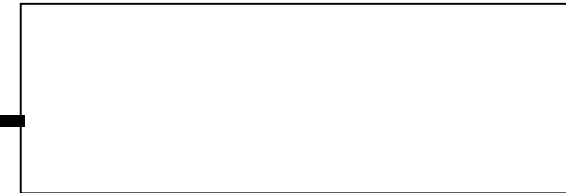


Arquivo Texto

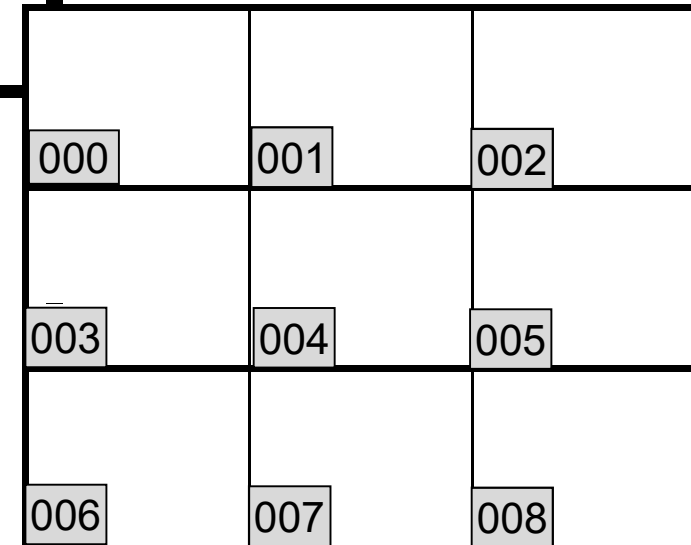
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```


Arquivo Texto

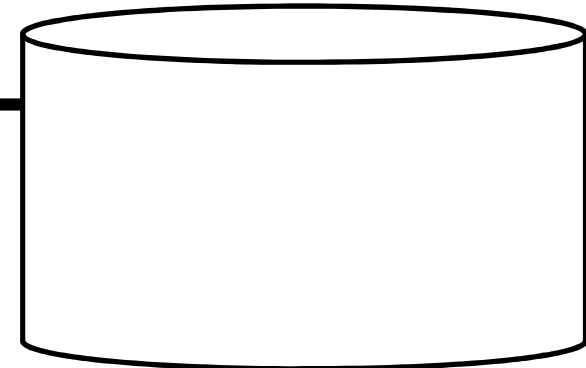
Monitor / Teclado



Memória



HD



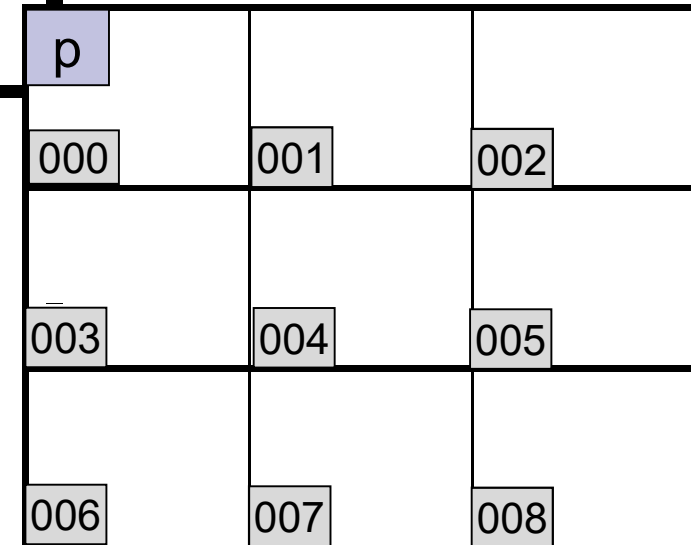
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p,"%d",&n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

Arquivo Texto

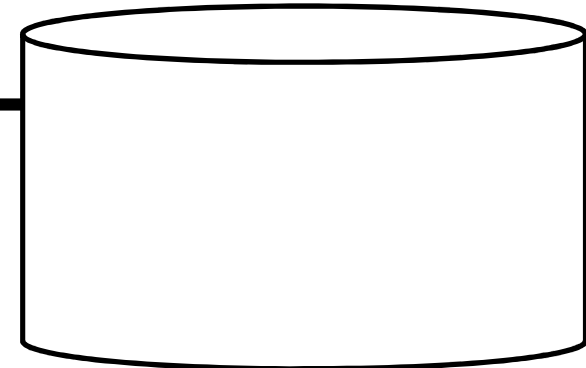
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

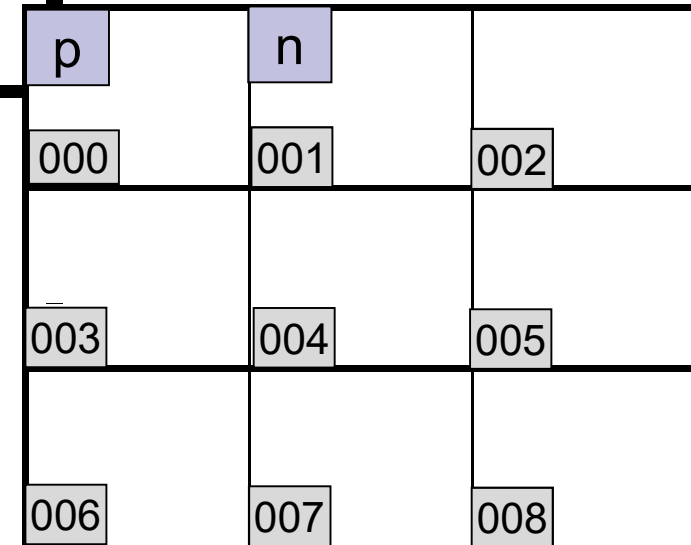


Arquivo Texto

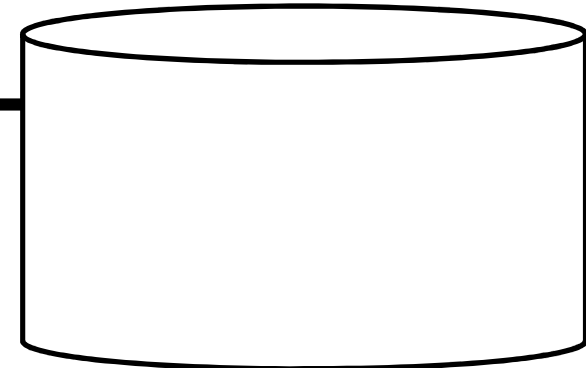
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



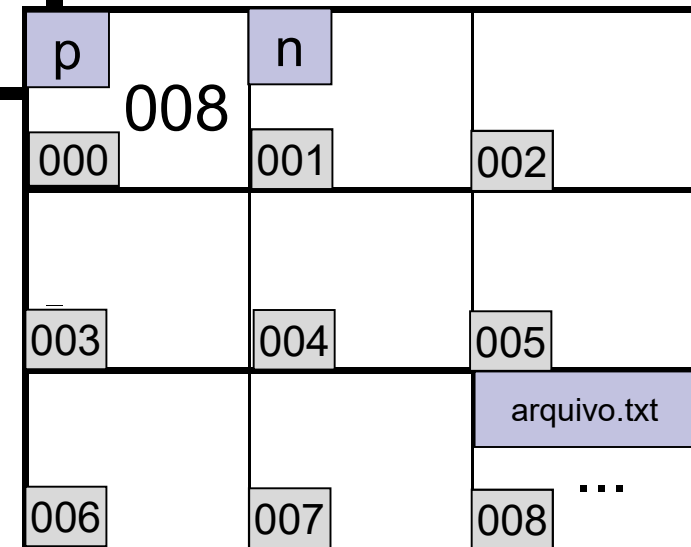
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

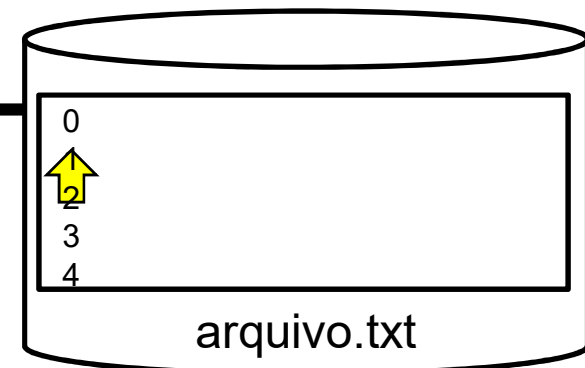
Assumimos que a abertura do arquivo foi bem sucedida.

Monitor / Teclado

Memória



HD





Monitor / Teclado

Memória

p	n	
000	001	002
003	004	005
		arquivo.txt
006	007	008 ...

HD

Diagram illustrating a file named `arquivo.txt` (represented as a cylinder). A vertical list of indices (0, 1, 2, 3, 4) is shown on the left. A yellow arrow points to index 1, which is highlighted with a yellow box.

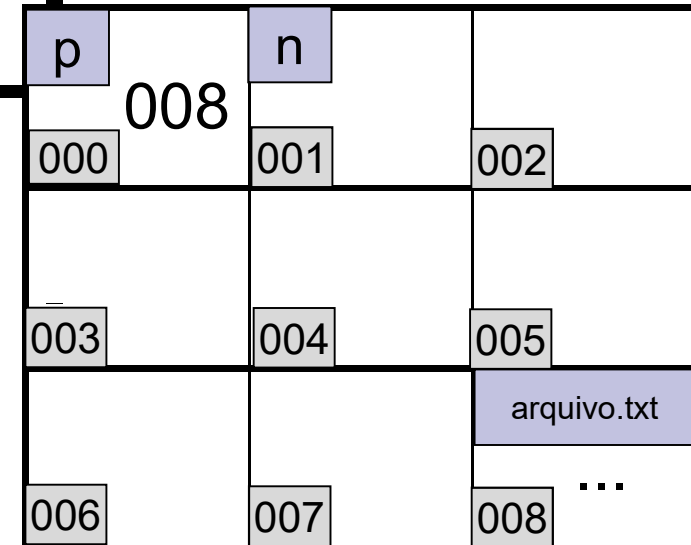
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

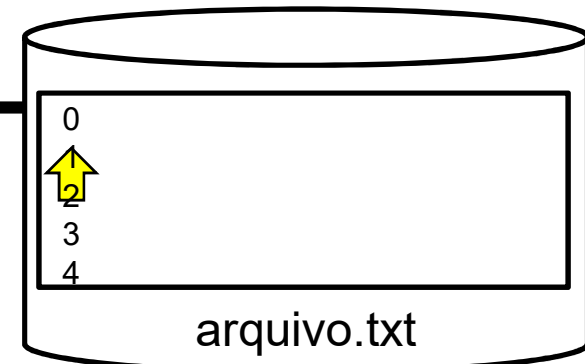
fscanf() tenta ler 1 inteiro. Se conseguir, retorna 1. Se não conseguir ler 1 inteiro, retorna 0. Se atingir o fim do arquivo, retorna -1.

Monitor / Teclado

Memória



HD



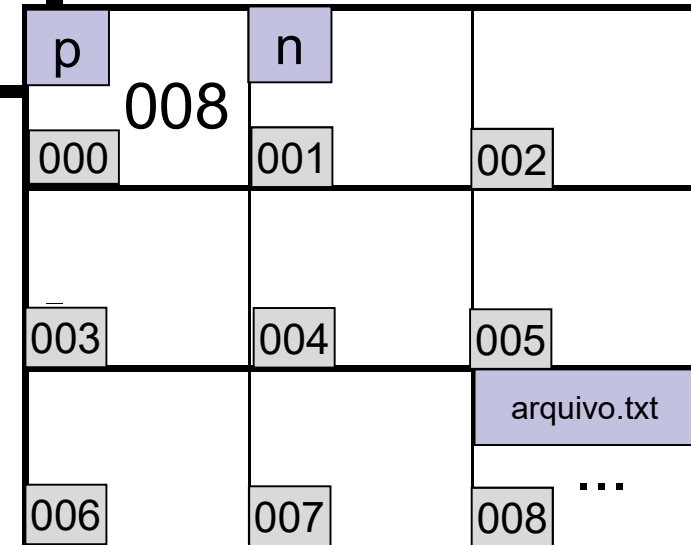
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

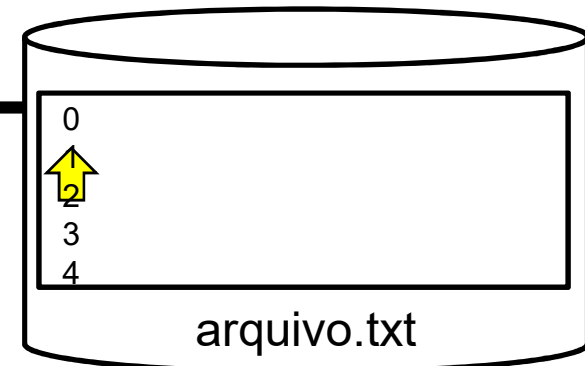
*A leitura vai ser bem sucedida.
Ou seja,
fscanf(p, "%d", &n) == 1
= 1 == 1
= 1*

Monitor / Teclado

Memória



HD



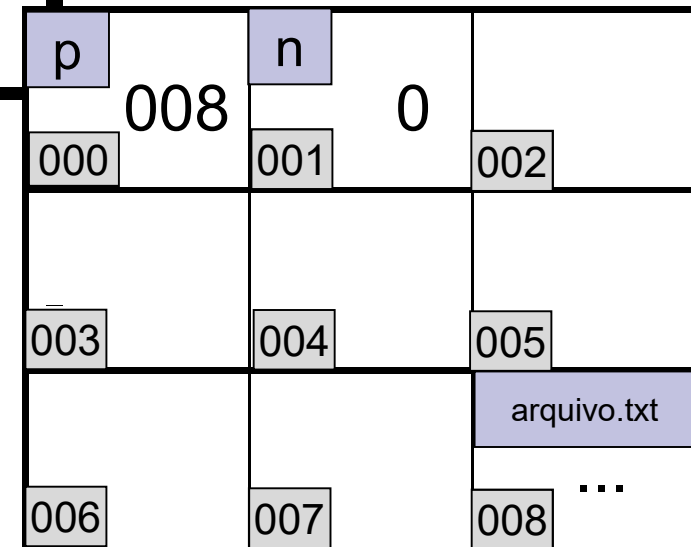
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

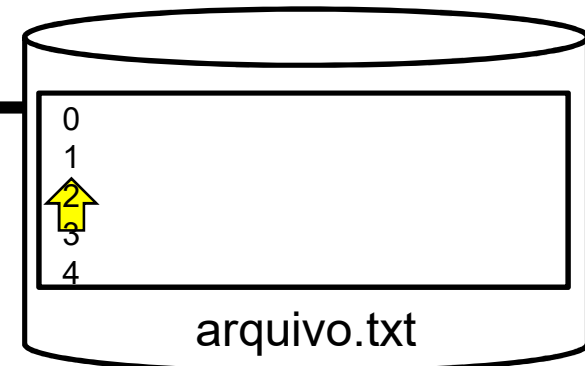
fscanf(p, "%d", &n) salva 0 em n e move a seta amarela à frente.

Monitor / Teclado

Memória



HD



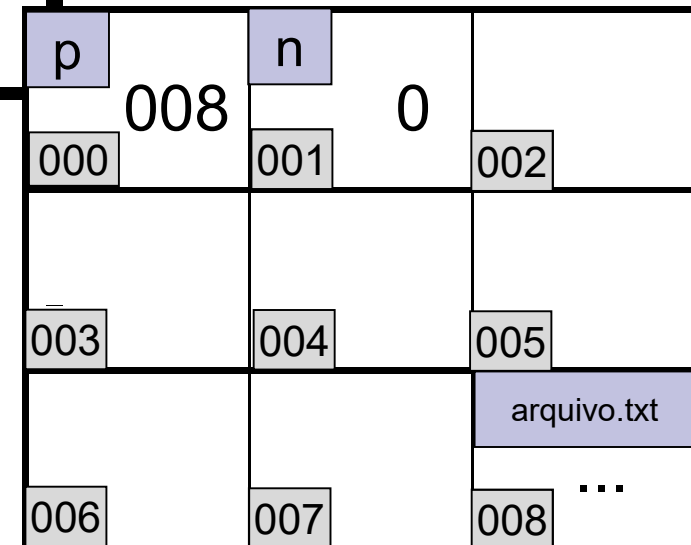
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

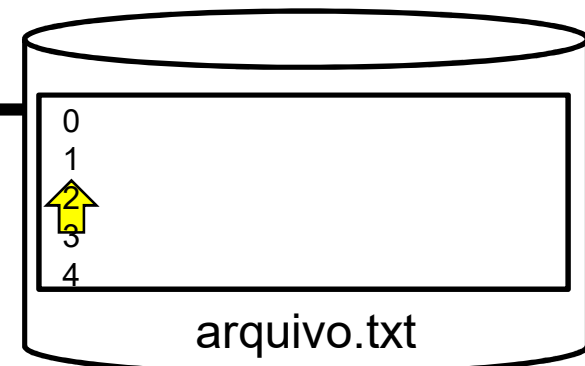
Monitor / Teclado

Numero: 0

Memória



HD



Arquivo Texto

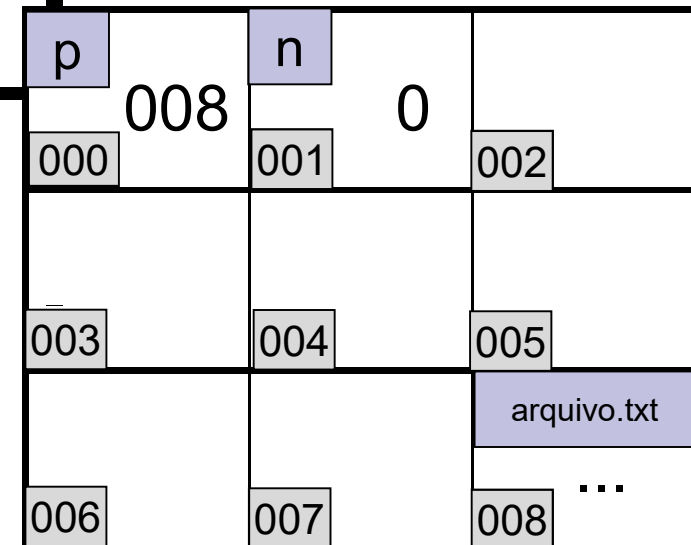
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
Ou seja,
fscanf(p, "%d", &n) == 1
= 1 == 1
= 1*

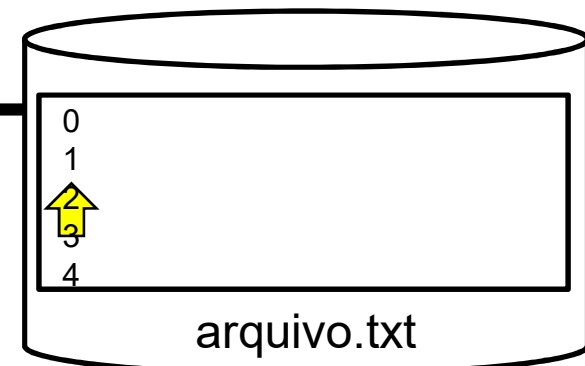
Monitor / Teclado

Numero: 0

Memória



HD



Arquivo Texto

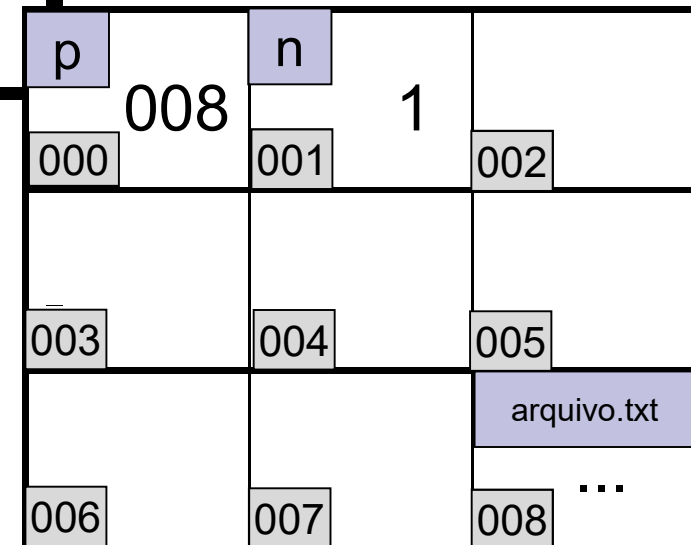
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

fscanf(p, "%d", &n) salva 1 em n e move a seta amarela à frente.

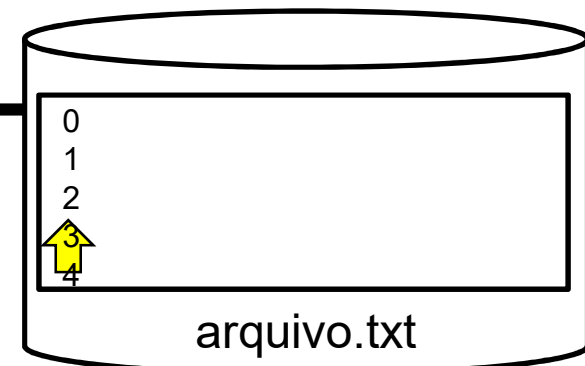
Monitor / Teclado

Numero: 0

Memória



HD



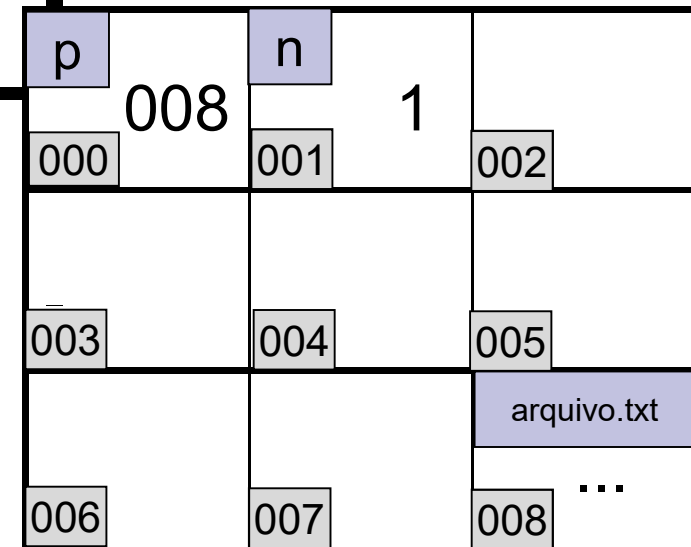
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

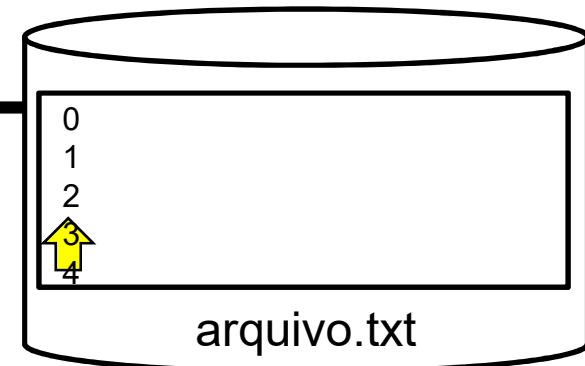
Monitor / Teclado

Numero: 0
Numero: 1

Memória



HD



Arquivo Texto

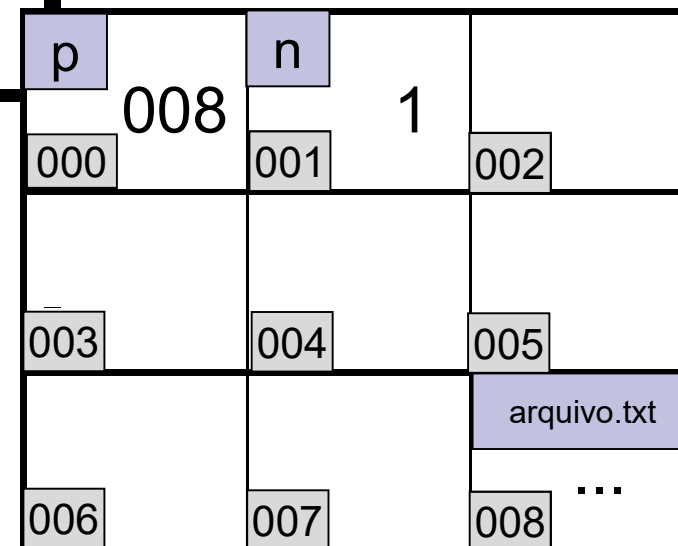
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
Ou seja,
fscanf(p, "%d", &n) == 1
= 1 == 1
= 1*

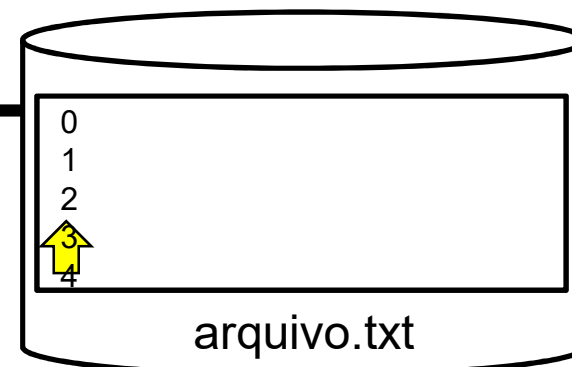
Monitor / Teclado

Numero: 0
Numero: 1

Memória



HD



Arquivo Texto

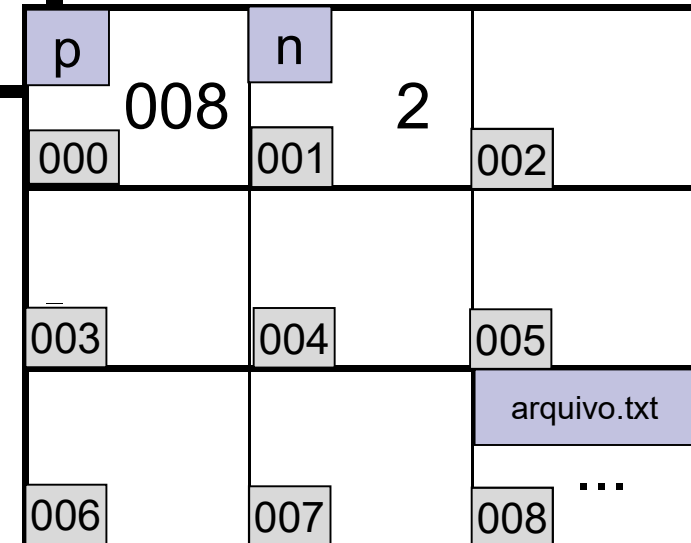
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

fscanf(p, "%d", &n) salva 2 em n e move a seta amarela à frente.

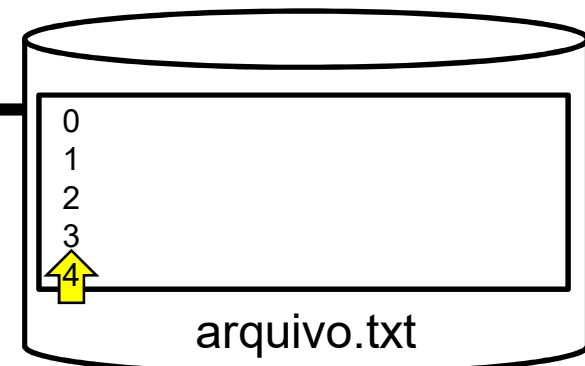
Monitor / Teclado

Numero: 0
Numero: 1

Memória



HD



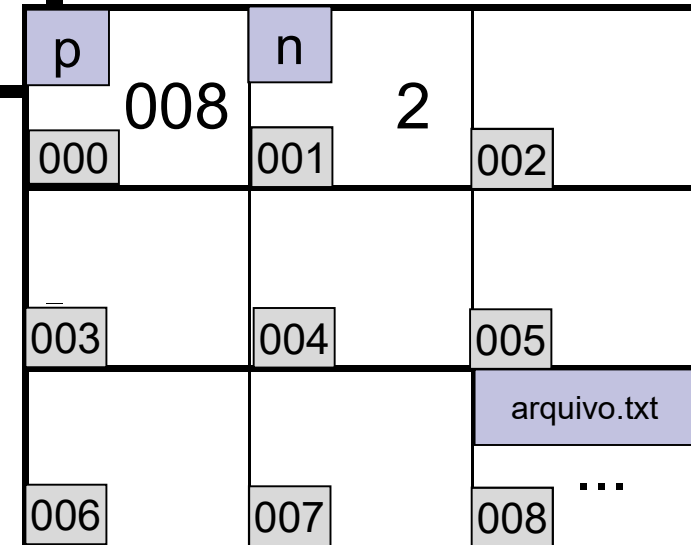
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

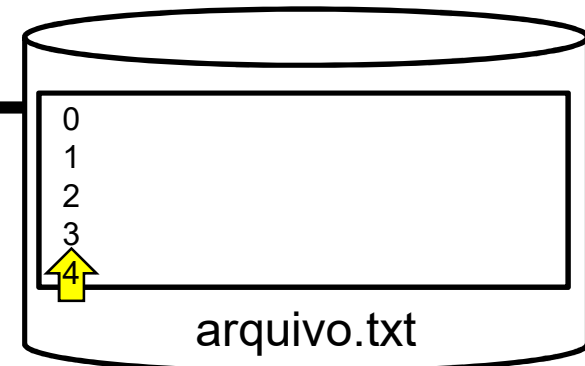
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2

Memória



HD



Arquivo Texto

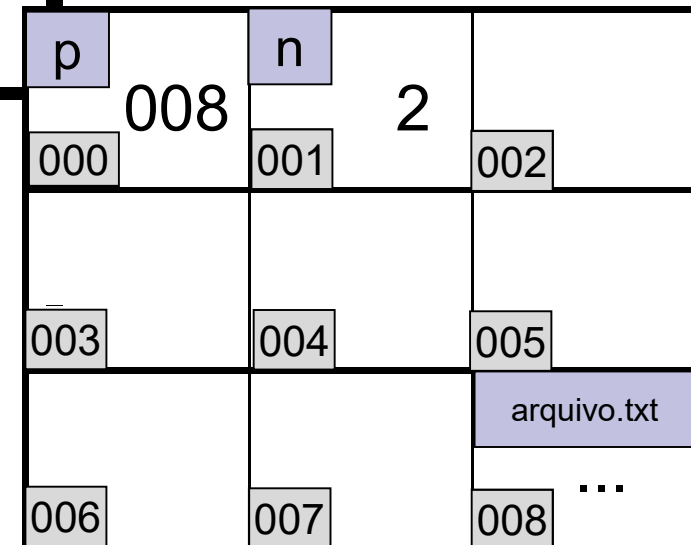
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
Ou seja,
fscanf(p, "%d", &n) == 1
= 1 == 1
= 1*

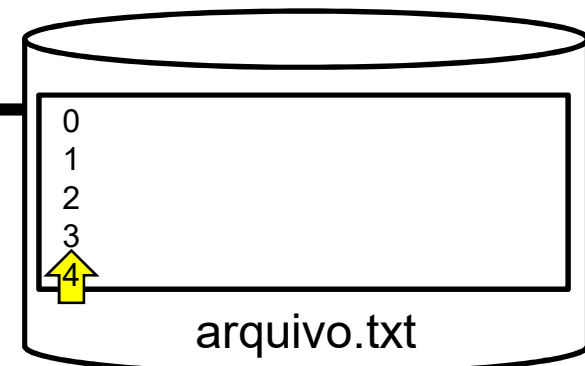
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2

Memória



HD



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

fscanf(p, "%d", &n) salva 3 em n e move a seta amarela à frente.

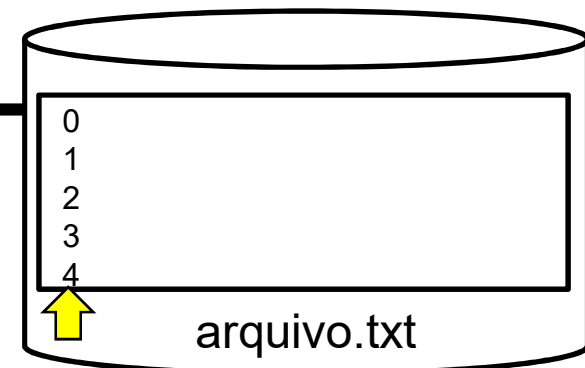
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2

Memória

p	n	
000	008	001 3 002
003	004	005
006	007	008 arquivo.txt ...

HD



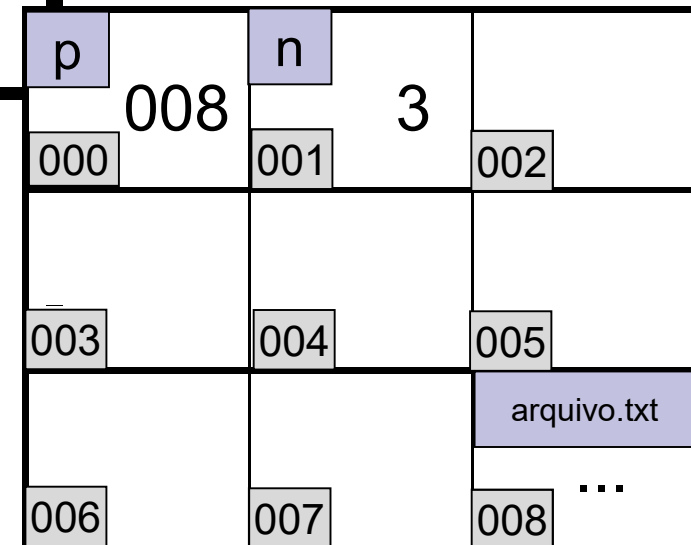
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

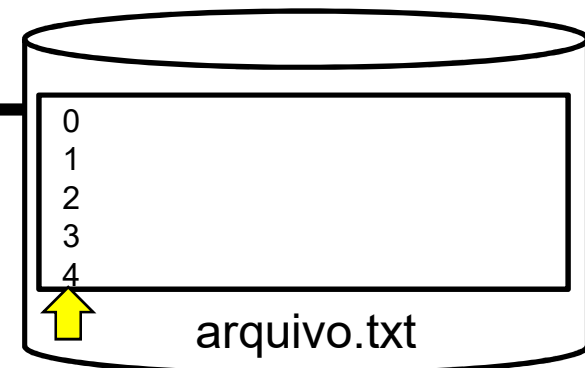
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2
Numero: 3

Memória



HD



Arquivo Texto

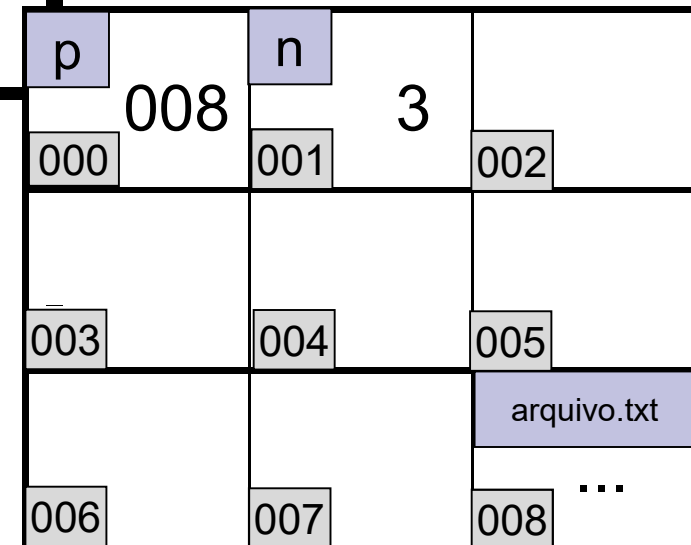
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
Ou seja,
fscanf(p, "%d", &n) == 1
= 1 == 1
= 1*

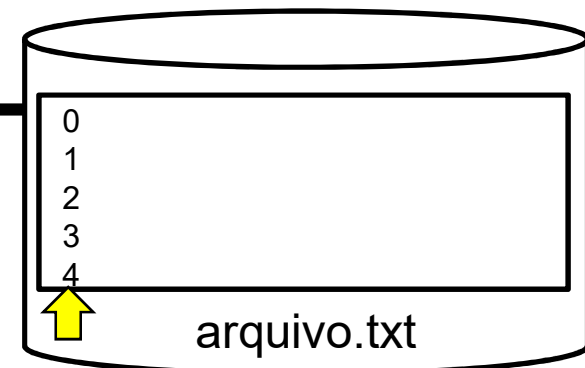
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2
Numero: 3

Memória



HD



Arquivo Texto

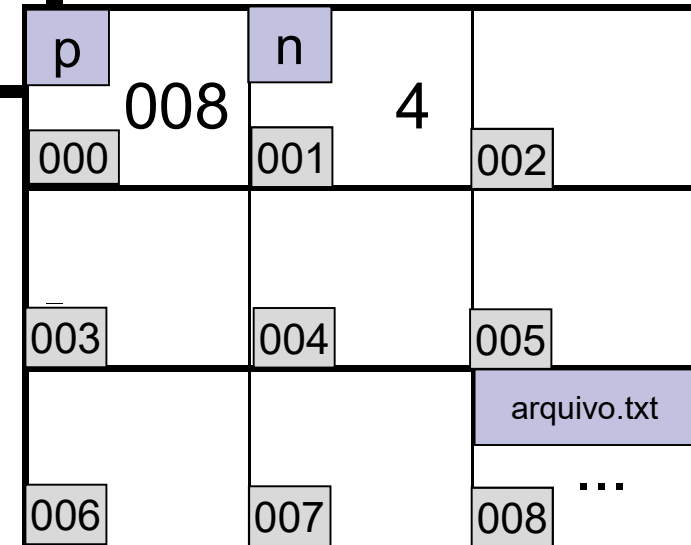
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

fscanf(p, "%d", &n) salva 4 em n e move a seta amarela à frente.

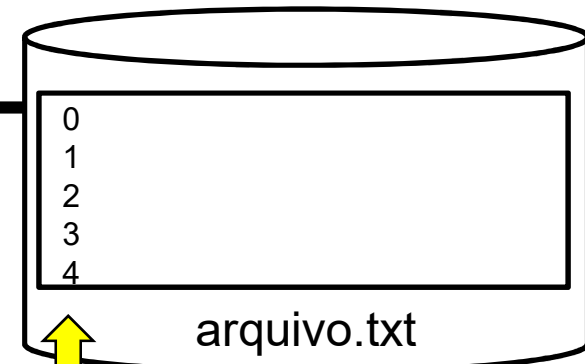
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2
Numero: 3

Memória



HD



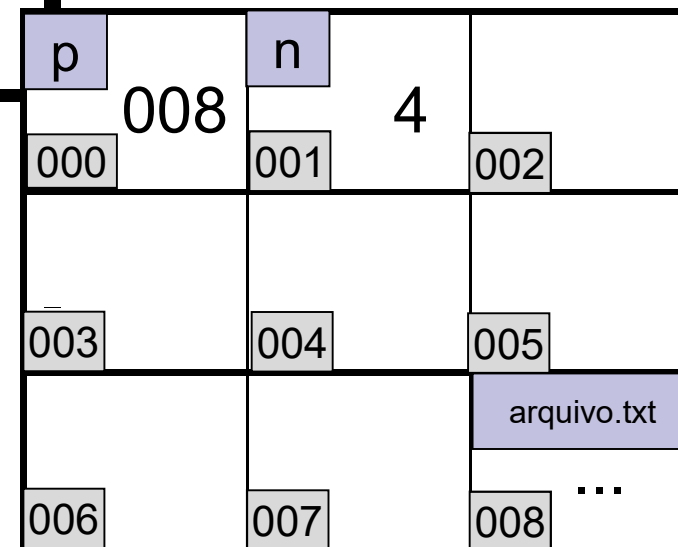
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

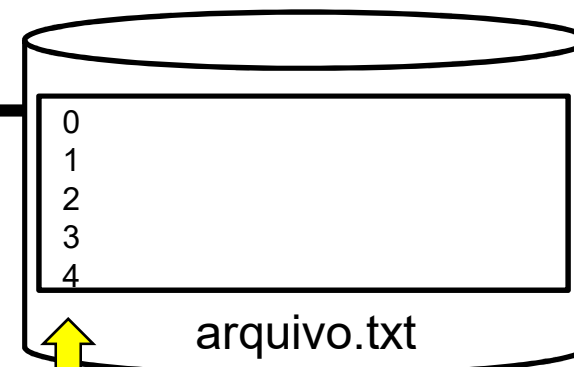
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2
Numero: 3
Numero: 4

Memória



HD



Arquivo Texto

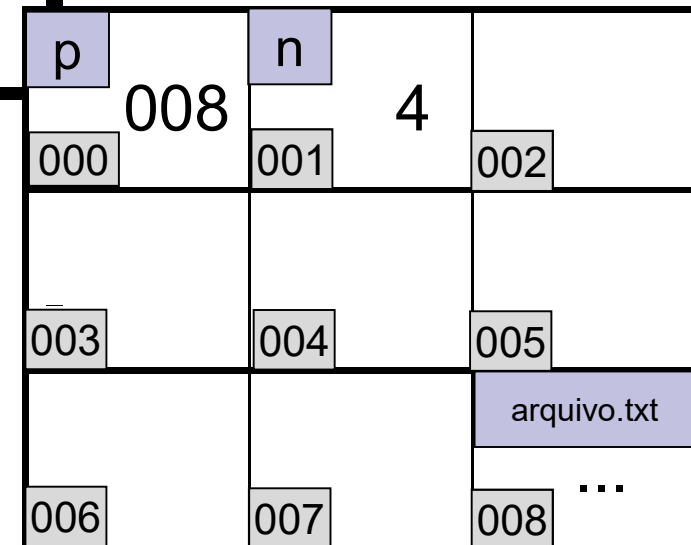
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

A leitura vai dar erro, pois o fim do arquivo foi atingido.
fscanf(p, "%d", &n) == 1
= -1 == 1
= 0

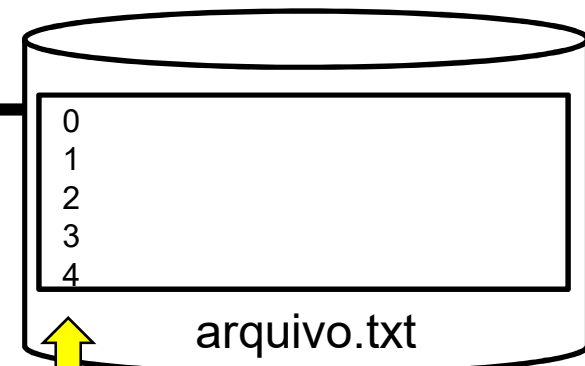
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2
Numero: 3
Numero: 4

Memória



HD



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p, "%d", &n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

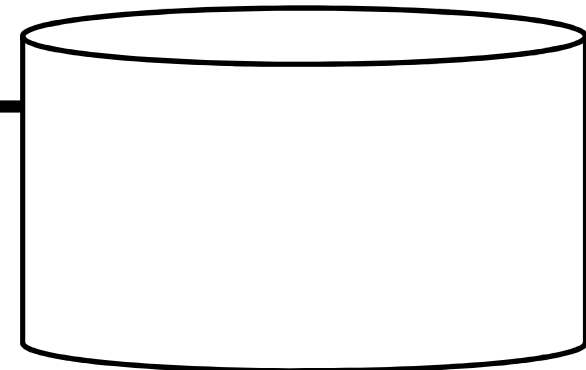
Monitor / Teclado

Numero: 0
Numero: 1
Numero: 2
Numero: 3
Numero: 4

Memória

p	n	
008	4	
000	001	002
003	004	005
006	007	008

HD

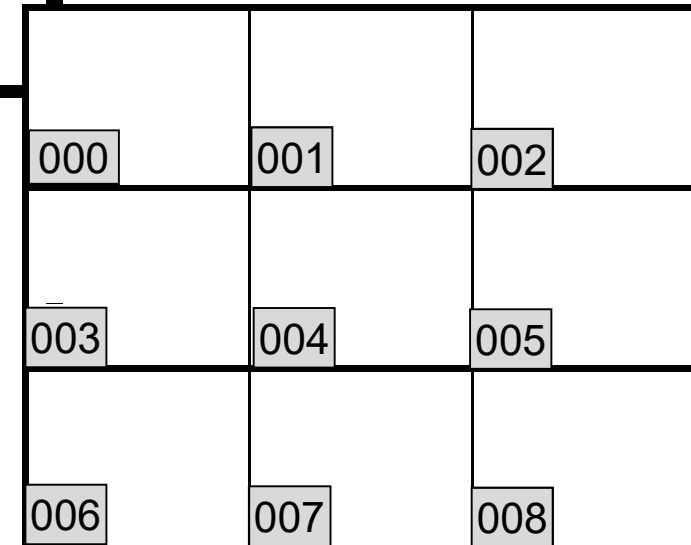


Arquivo Texto

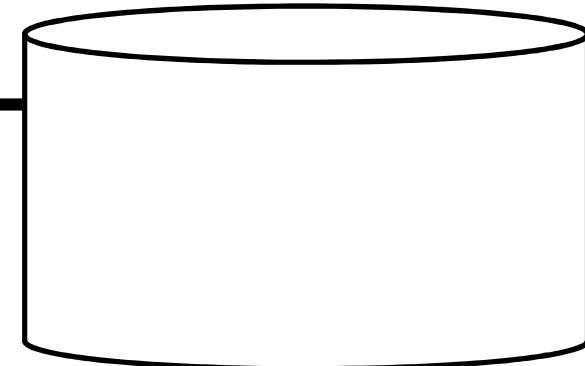
```
#include<stdio.h>
int main() {
    FILE *p;
    int n;
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        while (fscanf(p,"%d",&n) == 1) {
            printf("Numero: %d\n", n);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD





Arquivo Texto

- Gravação de vetores de números e strings



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt","w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p,"%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

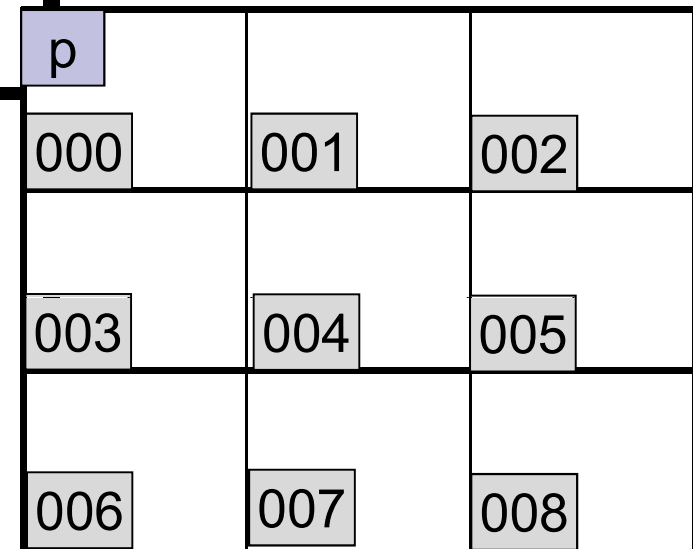
HD

Arquivo Texto

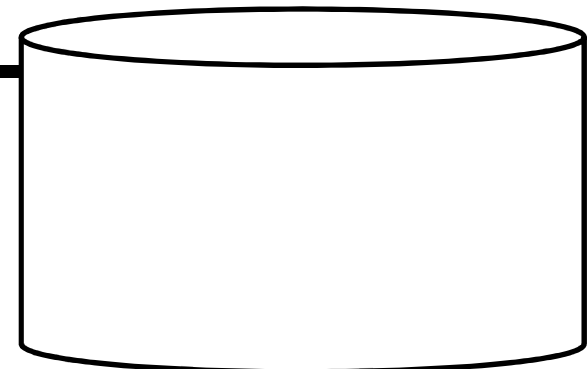
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

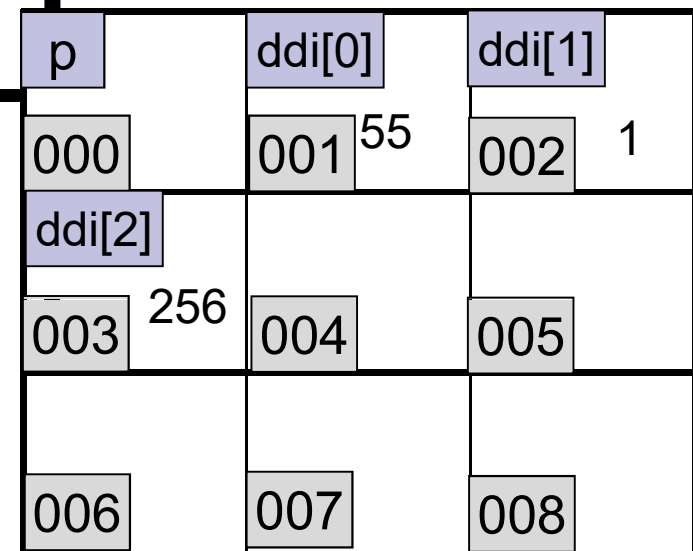


Arquivo Texto

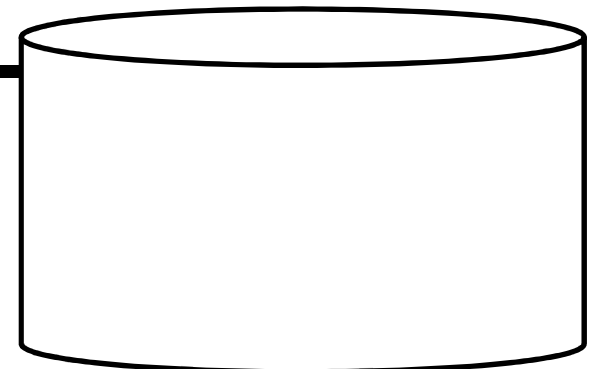
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

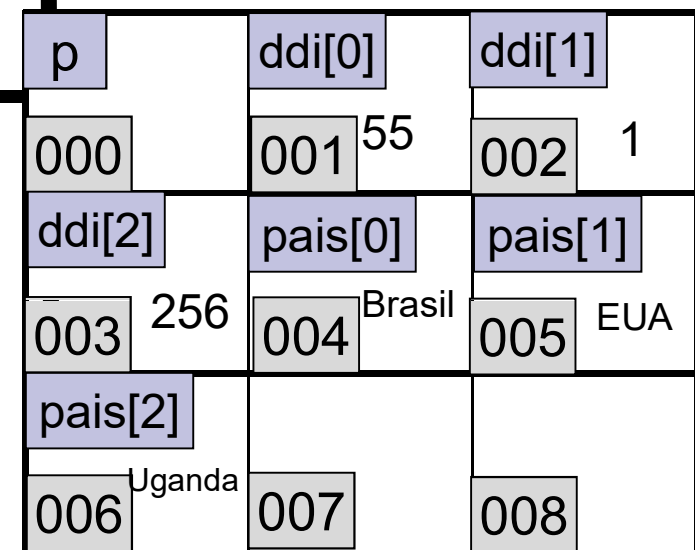


Arquivo Texto

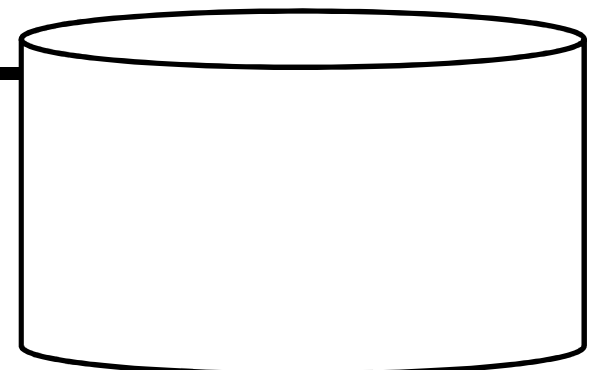
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

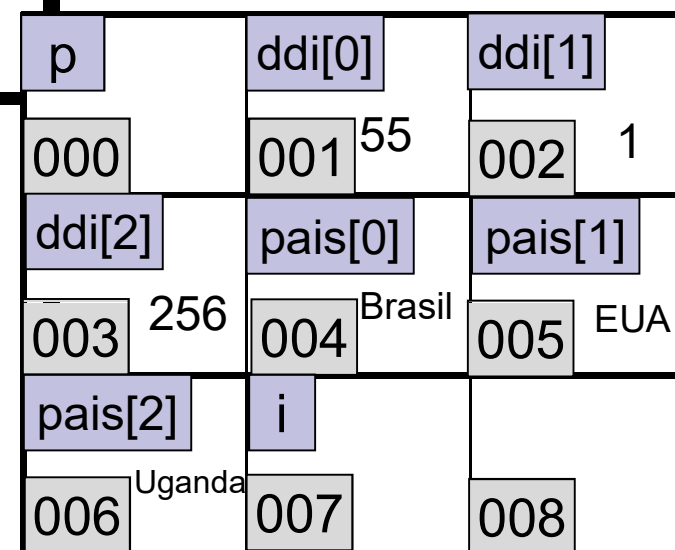


Arquivo Texto

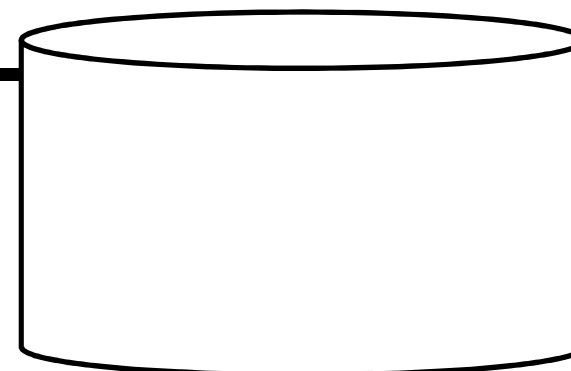
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

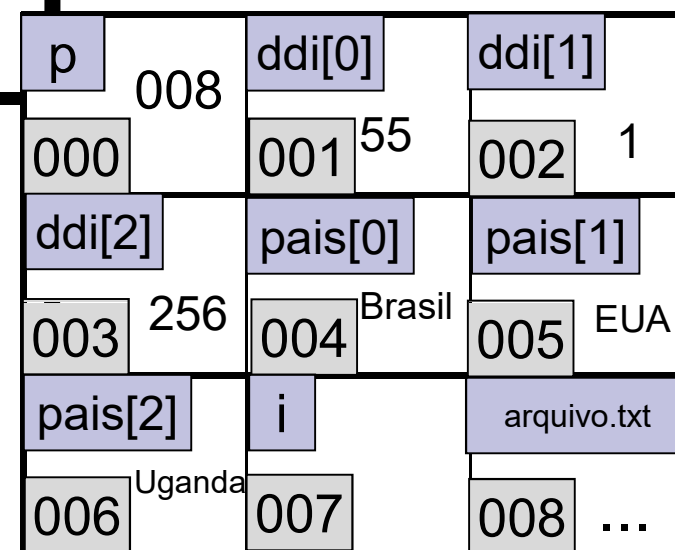


Arquivo Texto

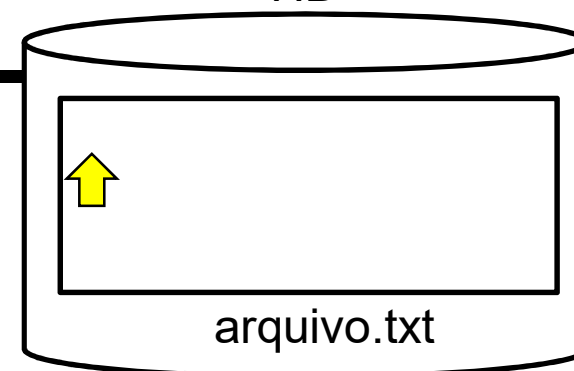
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



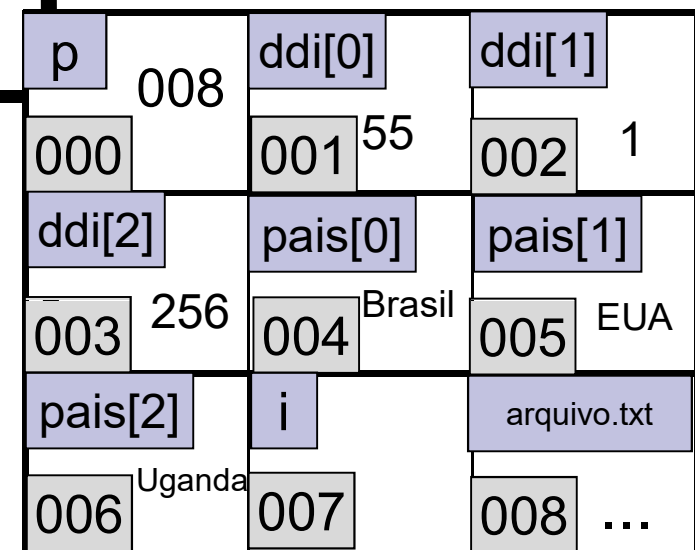
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

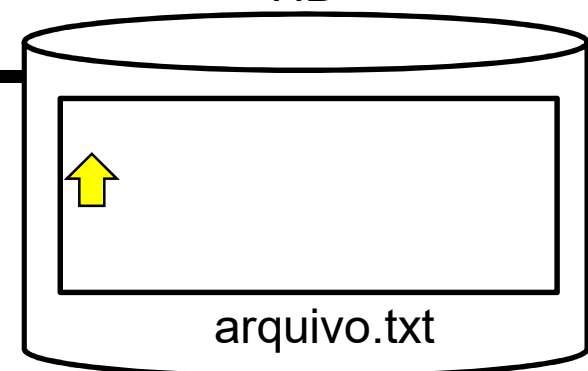
$p == NULL$
 $= 0$

Monitor / Teclado

Memória



HD

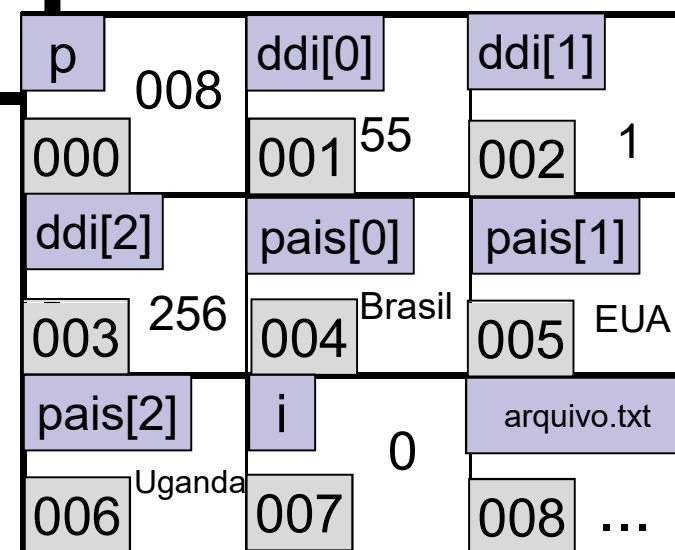


Arquivo Texto

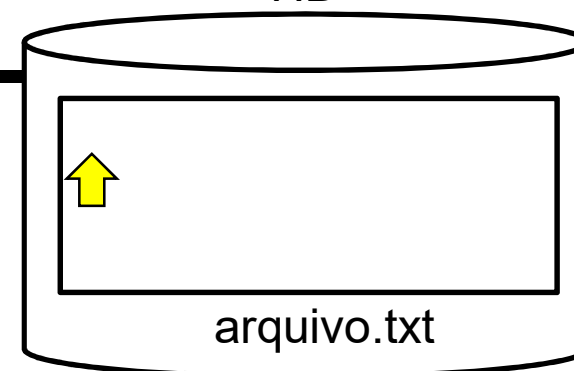
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



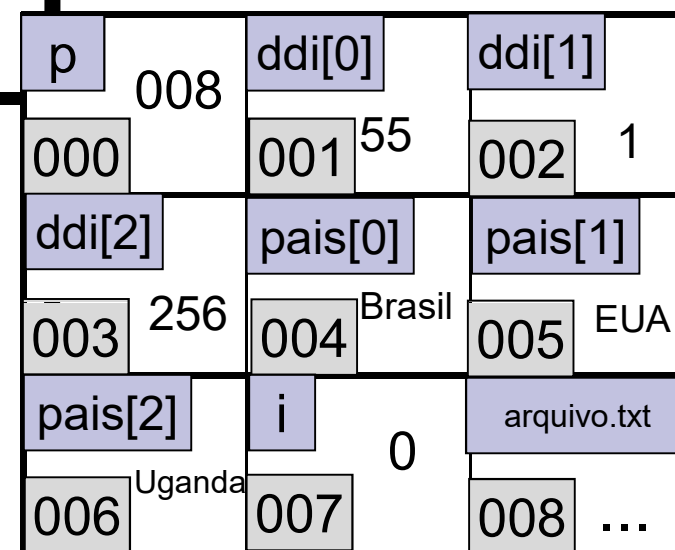
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

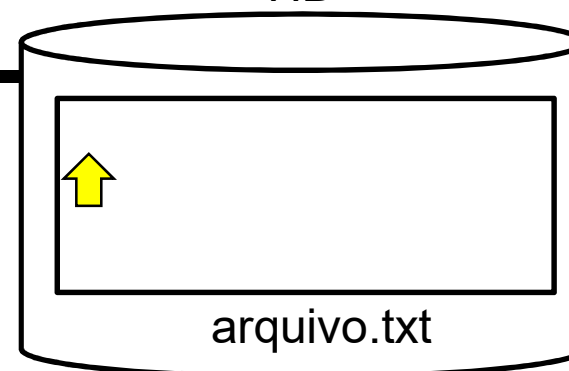
$0 < 3$
 $= 1$

Monitor / Teclado

Memória



HD



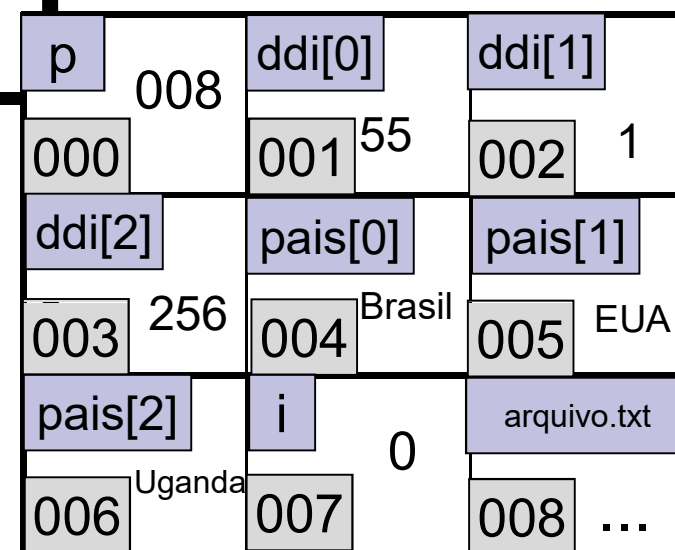
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

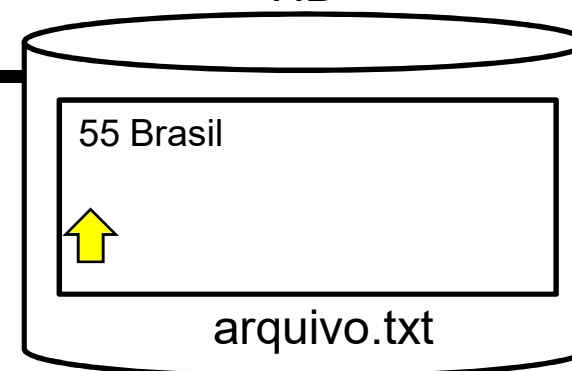
*O \n faz a seta amarela
descer uma linha.*

Monitor / Teclado

Memória



HD

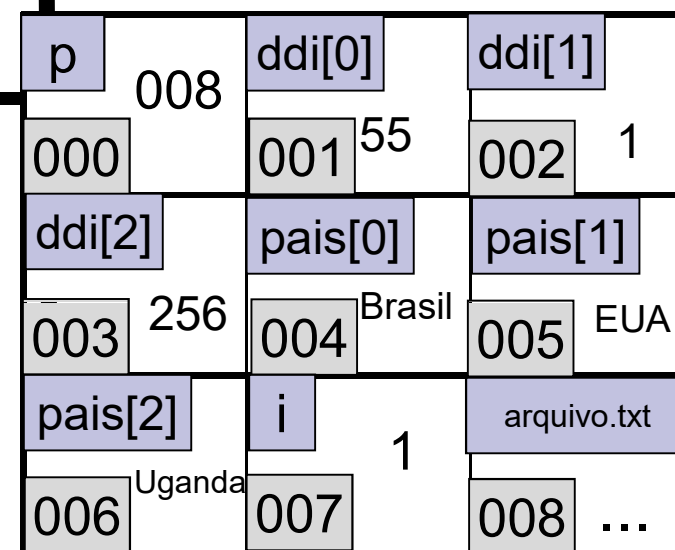


Arquivo Texto

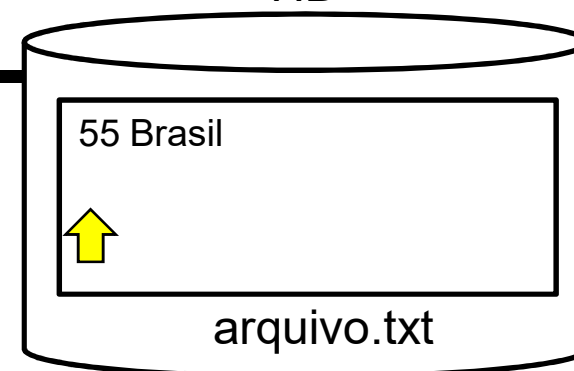
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



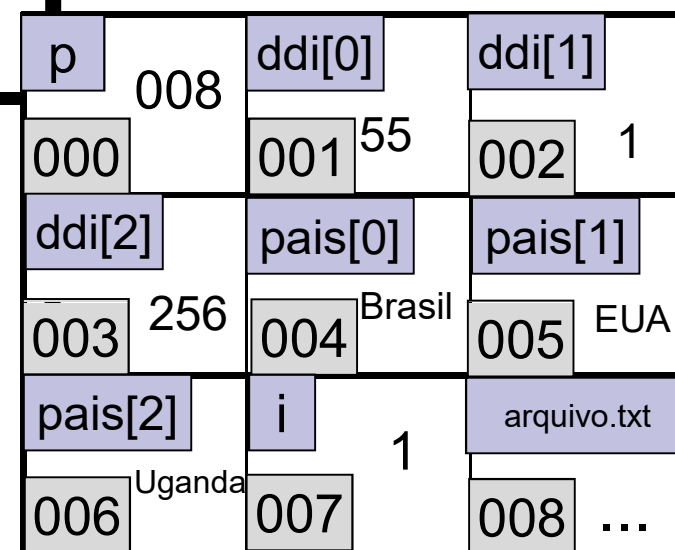
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

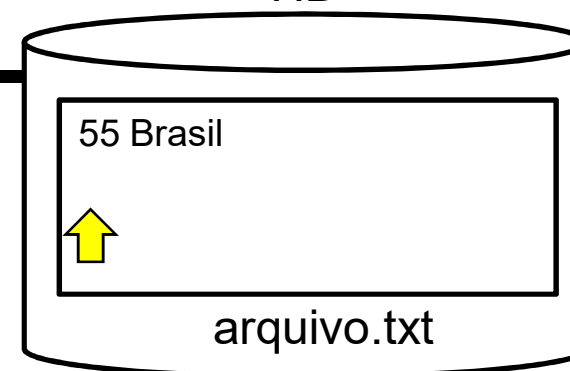
$1 < 3$
 $= 1$

Monitor / Teclado

Memória



HD

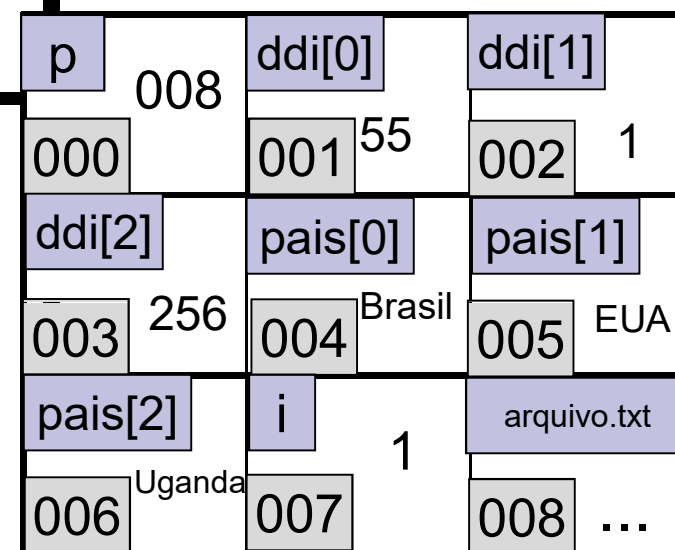


Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            ➡ fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

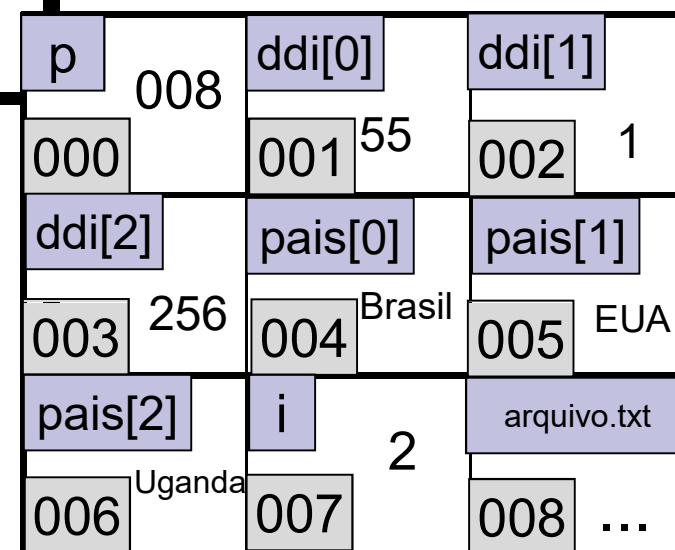


Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for(i = 0; i < 3: i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



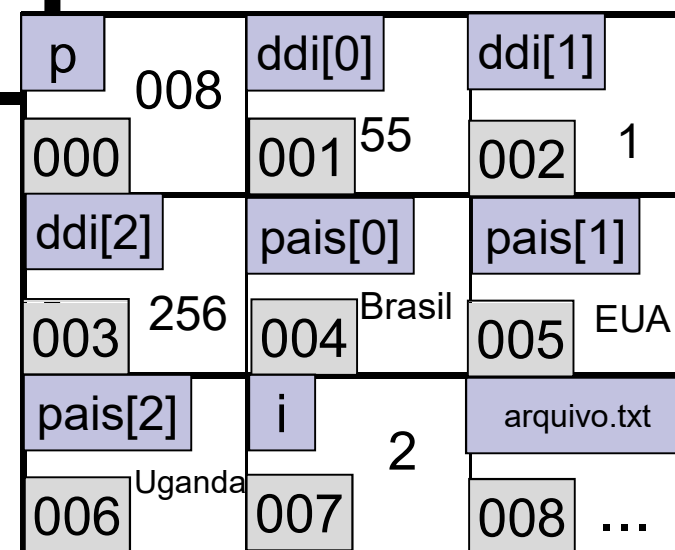
Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for(i = 0: i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

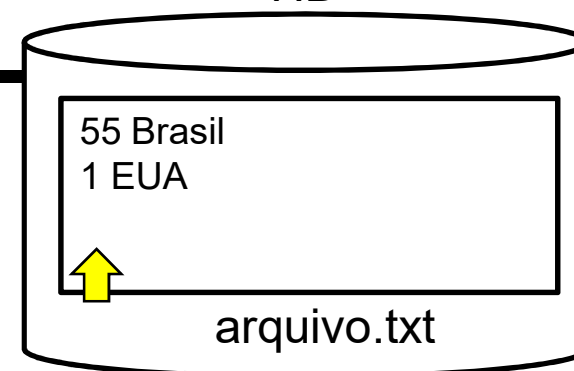
$2 < 3$
 $= 1$

Monitor / Teclado

Memória



HD

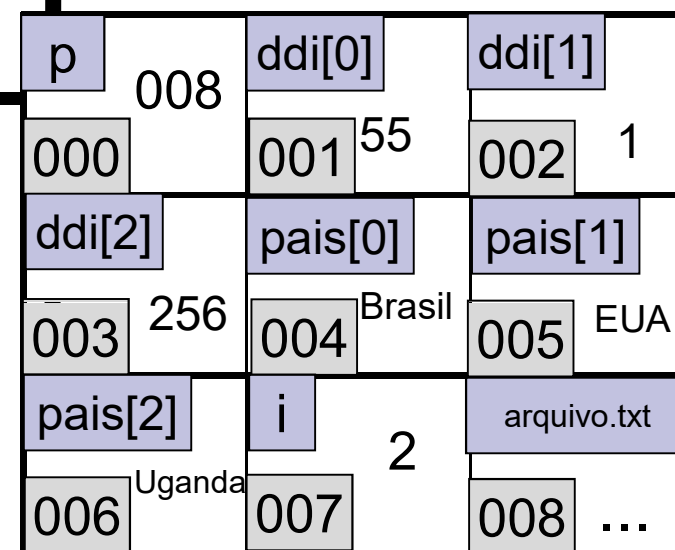


Arquivo Texto

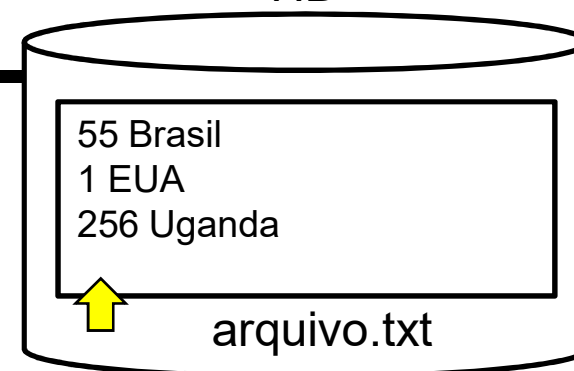
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

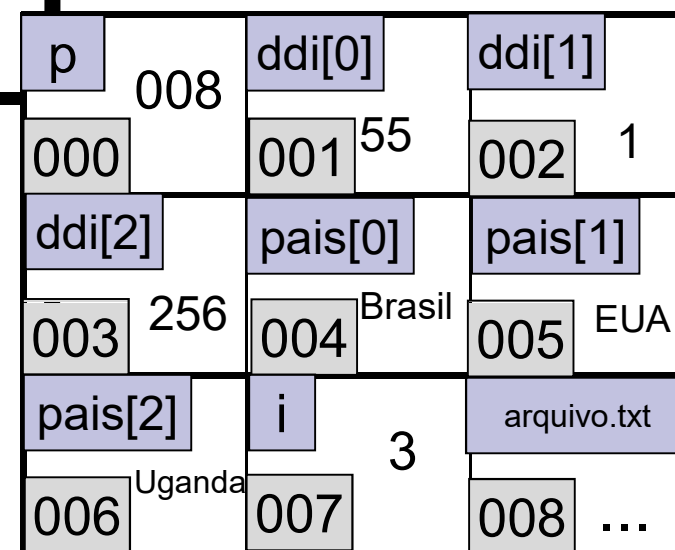


Arquivo Texto

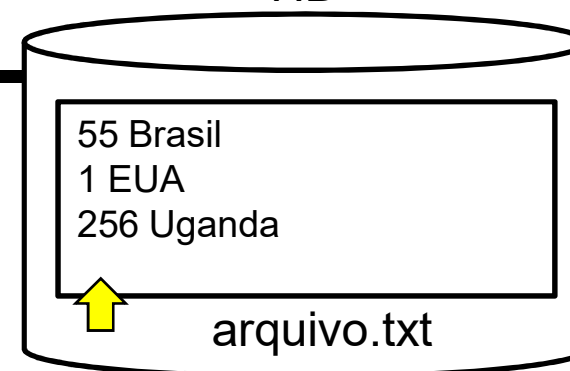
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        → for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Arquivo Texto

```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

$3 < 3$
 $= 0$

Monitor / Teclado

Memória

p	008	ddi[0]	ddi[1]
000		001 55	002 1
ddi[2]		pais[0]	pais[1]
003 256		004 Brasil	005 EUA
pais[2]		i 3	arquivo.txt
006 Uganda		007	008 ...

HD

55 Brasil
1 EUA
256 Uganda

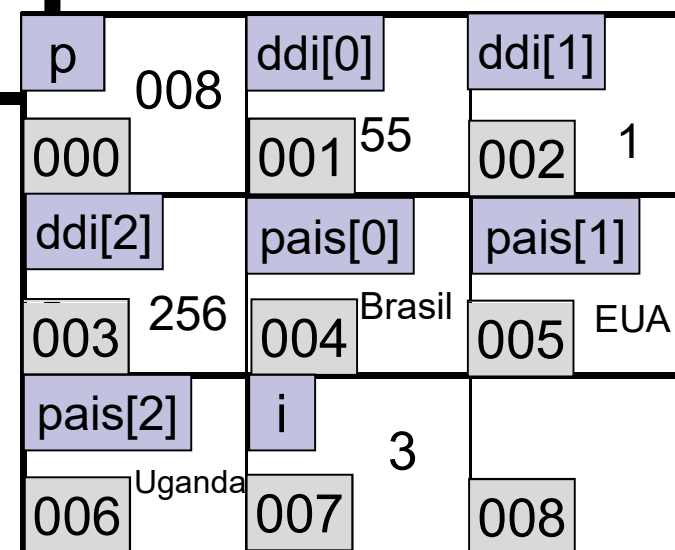
arquivo.txt

Arquivo Texto

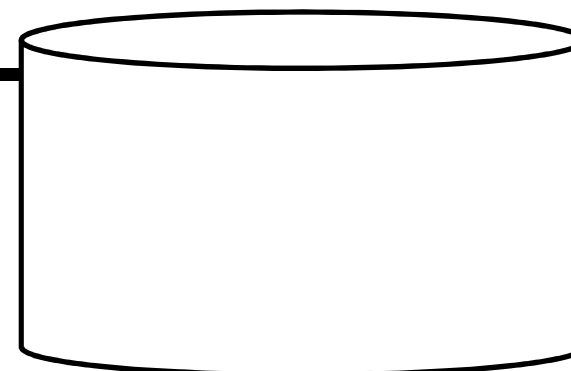
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

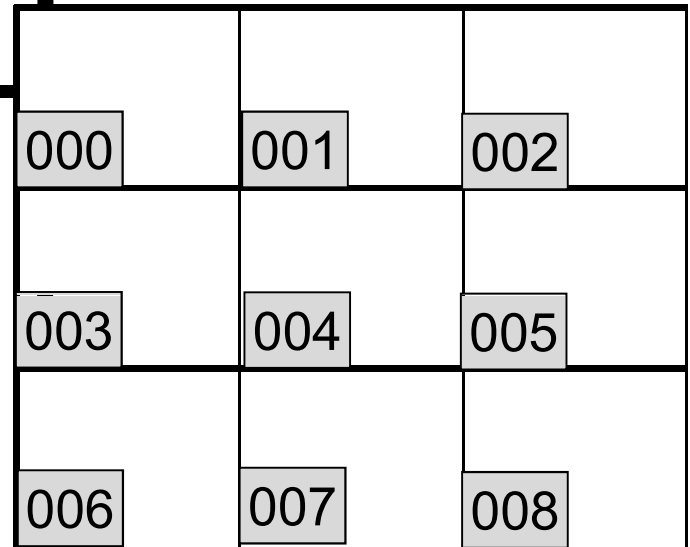


Arquivo Texto

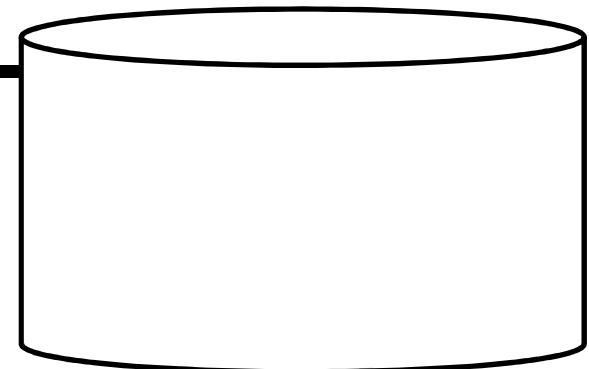
```
#include<stdio.h>
int main() {
    FILE *p;
    int ddi[3] = {55, 1, 256};
    char pais[3][7] = {"Brasil", "EUA", "Uganda"};
    int i;
    p = fopen("arquivo.txt", "w");
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n", ddi[i], pais[i]);
        }
    }
    fclose(p);
    return 0;
}
```

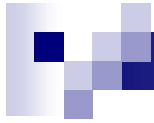
Monitor / Teclado

Memória



HD





Arquivo Texto

Exercício

- Refaça o programa anterior usando um vetor de struct codigo.

```
struct codigo {  
    int ddi;  
    char pais[7];  
}
```



Arquivo Texto

■ Resposta

```
#include<stdio.h>
#include<string.h>
struct codigo {
    int ddi;
    char pais[7];
};
int main() {
    FILE *p;
    struct codigo vetor[3];
    vetor[0].ddi = 55;
    strcpy(vetor[0].pais, "Brasil");
    vetor[1].ddi = 1;
    strcpy(vetor[1].pais, "EUA");
    vetor[2].ddi = 256;
    strcpy(vetor[2].pais, "Uganda");
    int i;
    p = fopen("arquivo.txt", "w");
```

```
    if (p == NULL) {
        printf("Erro na criacao!\n");
    }
    else {
        for(i = 0; i < 3; i++) {
            fprintf(p, "%d %s\n",
                vetor[i].ddi, vetor[i].pais);
        }
    }
    fclose(p);
    return 0;
}
```




Arquivo Texto

- Leitura de vetores de números e strings

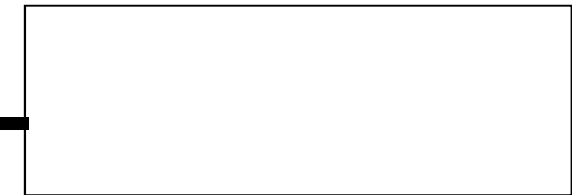


Arquivo Texto

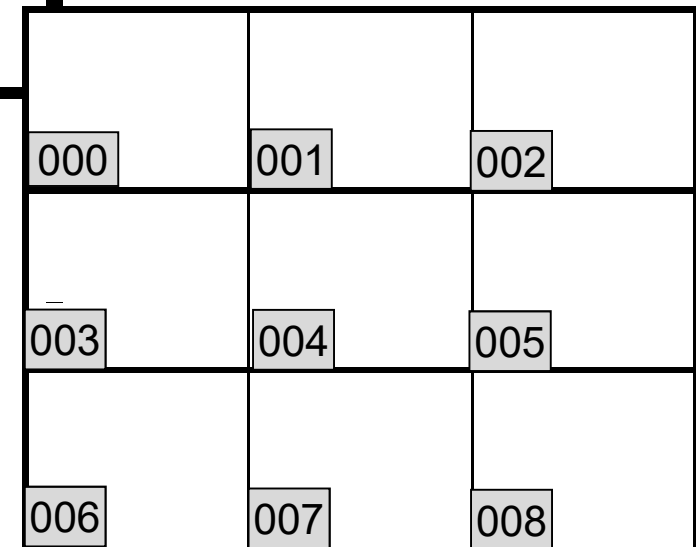
```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Arquivo Texto

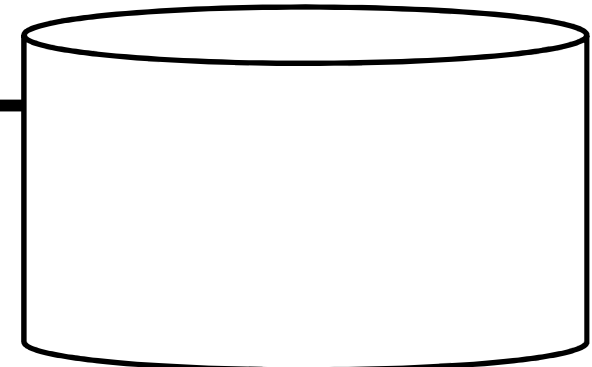
Monitor / Teclado



Memória



HD



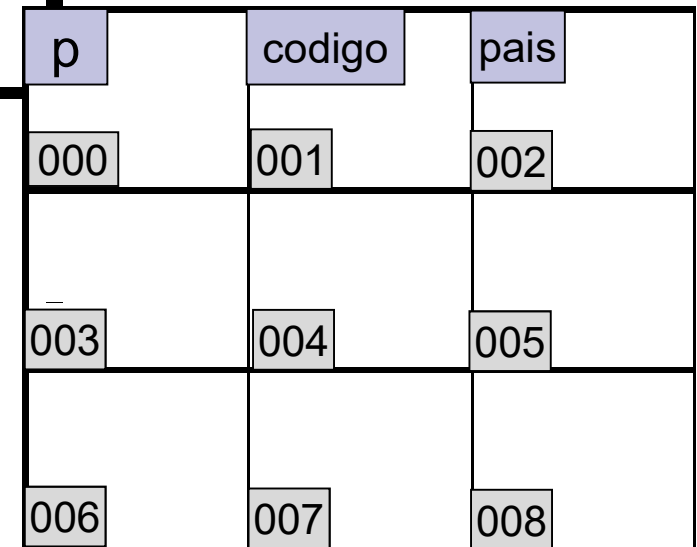
```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Arquivo Texto

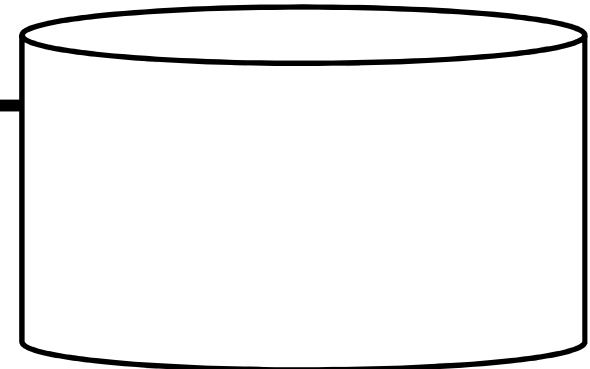
```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

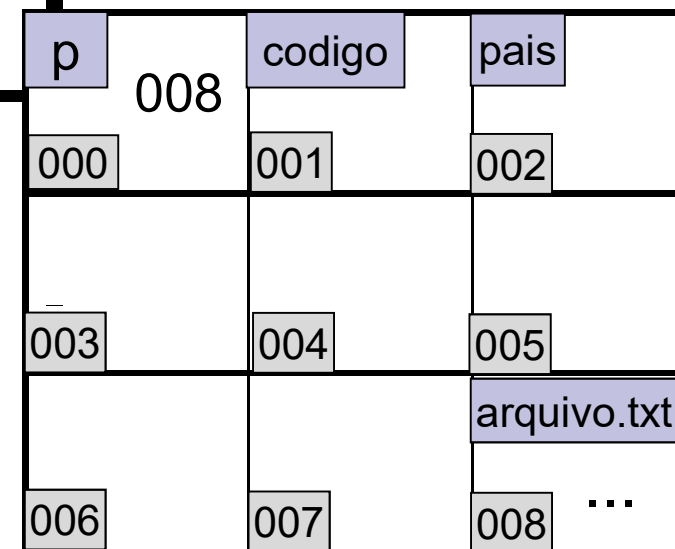


Arquivo Texto

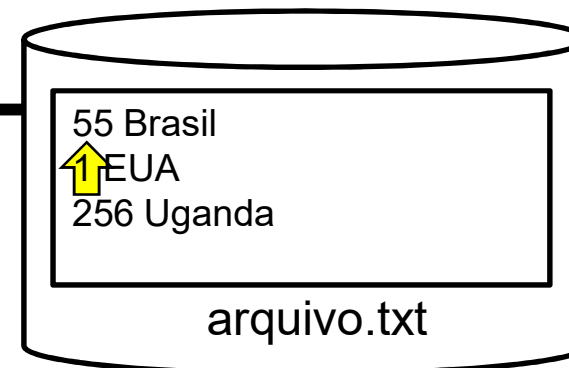
```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p, "%d %s", &codigo, pais) == 2) {
            printf("%d %s\n", codigo, pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



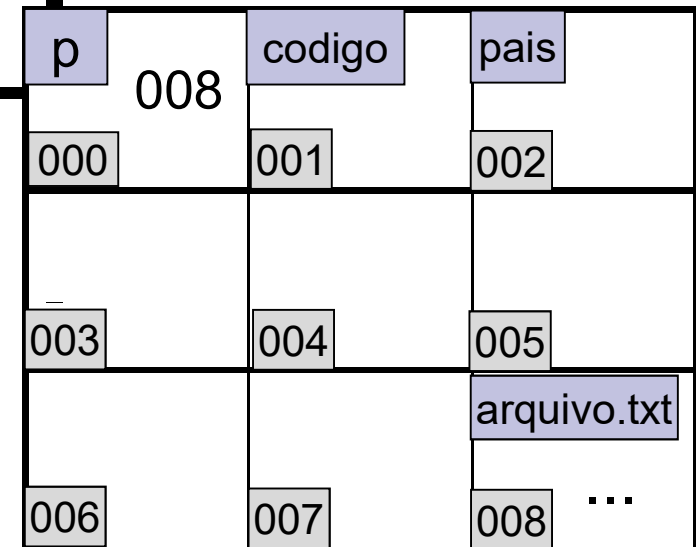
Arquivo Texto

$p == \text{NULL}$
 $= 0$

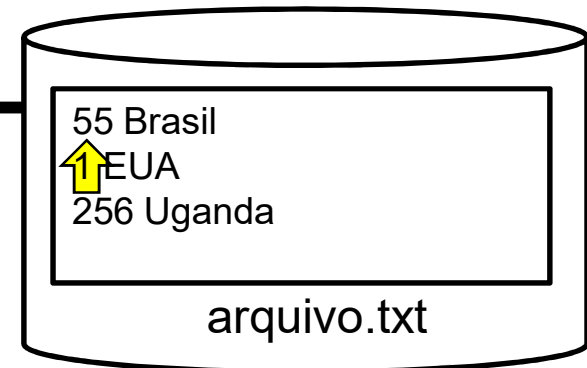
```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p, "%d %s", &codigo, pais) == 2) {
            printf("%d %s\n", codigo, pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



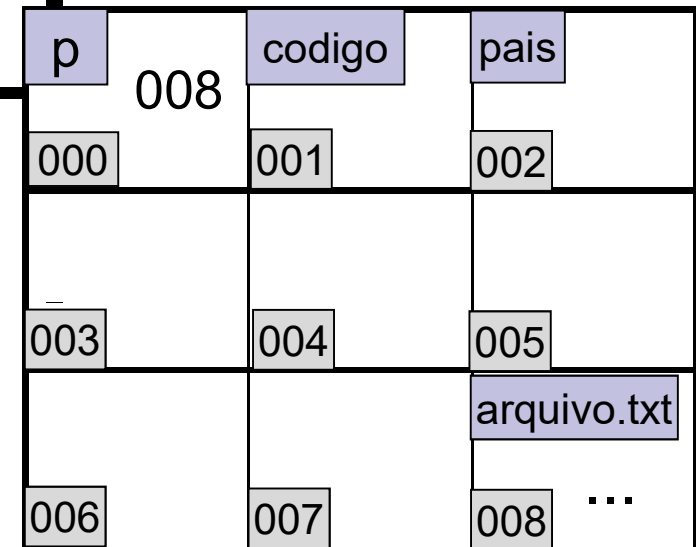
Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        → while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

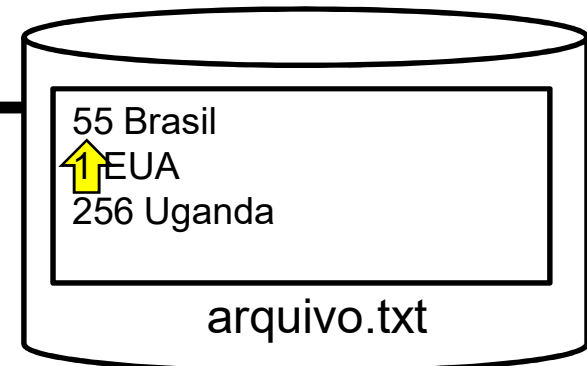
fscanf() tenta ler 2 valores. Se conseguir, retorna 2. Se não conseguir, retorna 0. Se atingir o fim do arquivo, retorna -1.

Monitor / Teclado

Memória



HD



Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        → while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

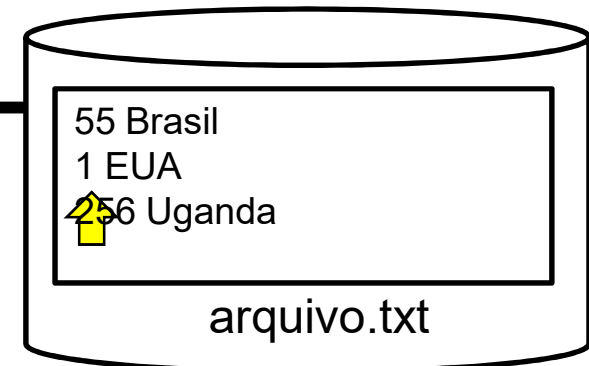
*A leitura vai ser bem sucedida.
fscanf() salva 55 em codigo,
"Brasil" em pais e move a seta
amarela à frente.*

Monitor / Teclado

Memória

p	008	codigo	55	pais	Brasil
000		001		002	
003		004		005	
					arquivo.txt
006		007		008	...

HD



Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

55 Brasil

Memória

p	008	codigo	pais
000		001 55	002 Brasil
003		004	005
			arquivo.txt
006		007	008 ...

HD

55 Brasil
1 EUA
256 Uganda

arquivo.txt

Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        → while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fscanf() salva 1 em codigo,
"EUA" em pais e move a seta
amarela à frente.*

Monitor / Teclado

55 Brasil

Memória

p	008	codigo	pais
000		001 1	002 EUA
003		004	005
			arquivo.txt
006		007	008 ...

HD

55 Brasil
1 EUA
256 Uganda

arquivo.txt

Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

55 Brasil
1 EUA

Memória

p	008	codigo	pais
000		001 1	002 EUA
003		004	005
			arquivo.txt
006		007	008 ...

HD

55 Brasil
1 EUA
256 Uganda

arquivo.txt

Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        → while (fscanf(p, "%d %s", &codigo, pais) == 2) {
            printf("%d %s\n", codigo, pais);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fscanf() salva 256 em codigo,
"Uganda" em pais e move a
seta amarela à frente.*

Monitor / Teclado

55 Brasil
1 EUA

Memória

p	008	codigo	256	pais	Uganda
000		001		002	
003		004		005	
					arquivo.txt
006		007		008	...

HD

55 Brasil
1 EUA
256 Uganda

arquivo.txt

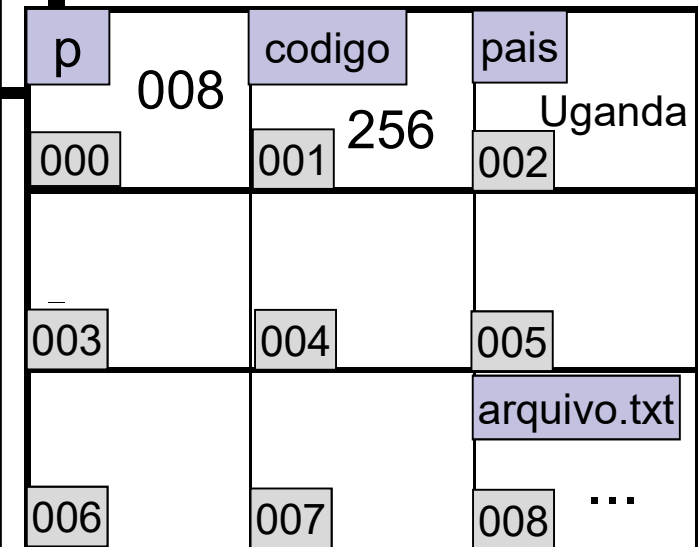
Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

55 Brasil
1 EUA
256 Uganda

Memória



HD

55 Brasil
1 EUA
256 Uganda

arquivo.txt

Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt", "r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        → while (fscanf(p, "%d %s", &codigo, pais) == 2) {
            printf("%d %s\n", codigo, pais);
        }
    }
    fclose(p);
    return 0;
}
```

A leitura vai dar erro, pois o fim do arquivo foi atingido e fscanf() vai retornar -1, que é diferente de 2.

Monitor / Teclado

55 Brasil
1 EUA
256 Uganda

Memória

p	008	codigo	pais
000		001 256	002 Uganda
003		004	005
			arquivo.txt
006		007	008 ...

HD

55 Brasil
1 EUA
256 Uganda

arquivo.txt

Arquivo Texto

```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

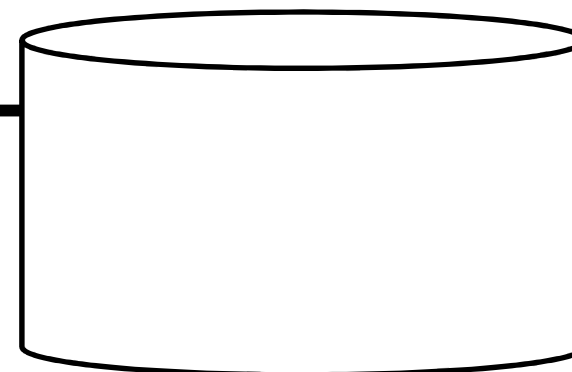
Monitor / Teclado

55 Brasil
1 EUA
256 Uganda

Memória

p	codigo	pais
008	256	Uganda
000	001	002
003	004	005
006	007	008

HD

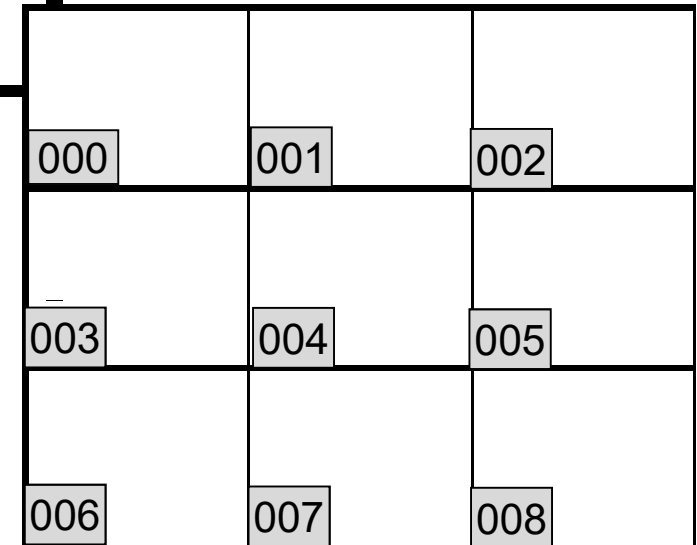


Arquivo Texto

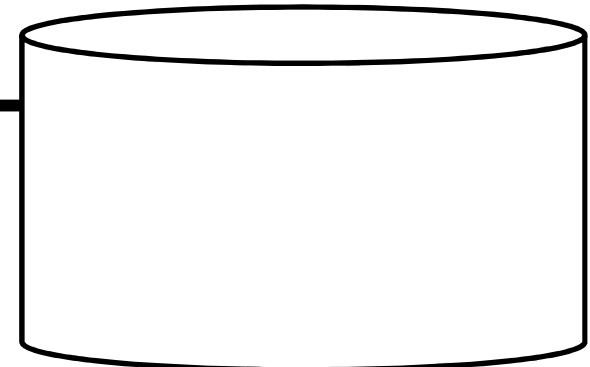
```
#include <stdio.h>
int main() {
    FILE* p; int codigo; char pais[7];
    p = fopen("arquivo.txt","r");
    if (p == NULL) {
        printf("Erro ao abrir arquivo\n");
    }
    else {
        while (fscanf(p,"%d %s",&codigo,pais) == 2) {
            printf("%d %s\n",codigo,pais);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



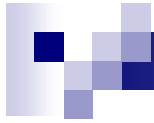
HD





Arquivo Binário

- O Bloco de Notas não consegue interpretar os bits
- Temos que interpretar usando comandos da linguagem C



Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Arquivo Binário

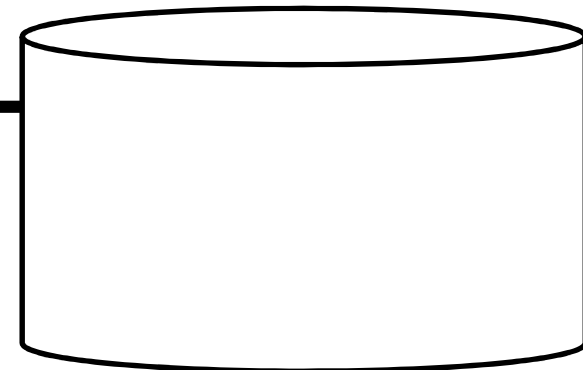
Monitor / Teclado

```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Memória

000	001	002
003	004	005
006	007	008

HD

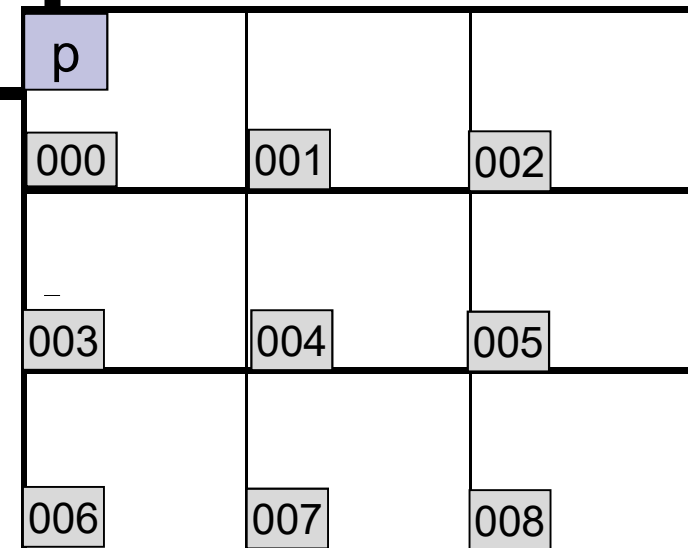


Arquivo Binário

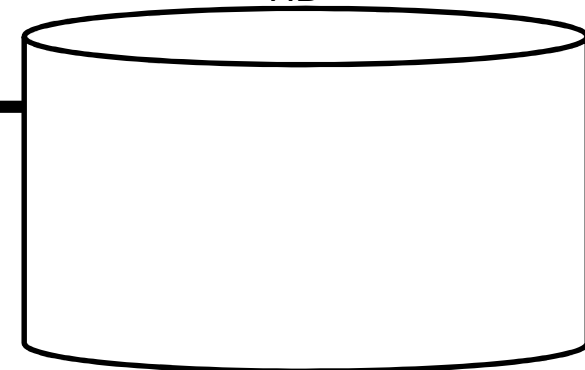
```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



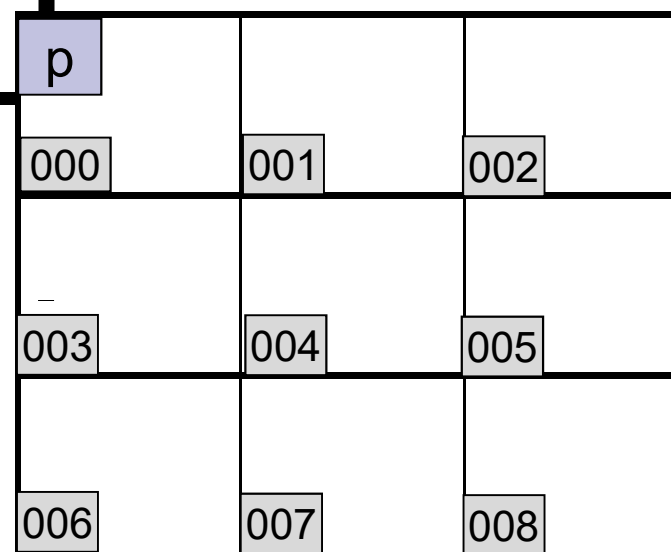
Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

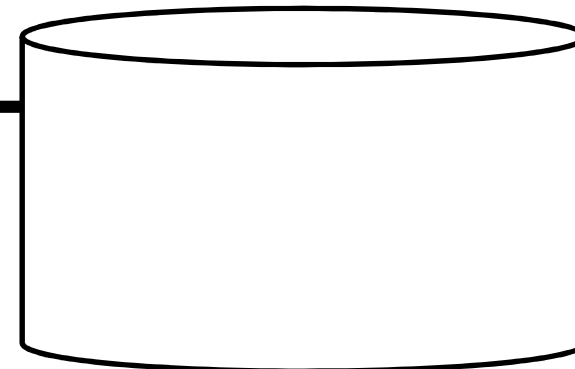
Tenta criar arquivo.bin no HD. O "b" em "wb" indica que o arquivo é binário.

Monitor / Teclado

Memória



HD



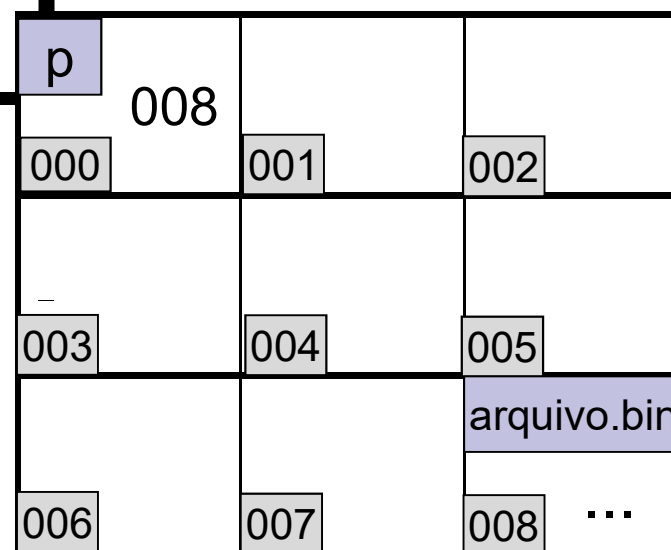
Arquivo Binário

*Assumimos que
foi bem sucedido.*

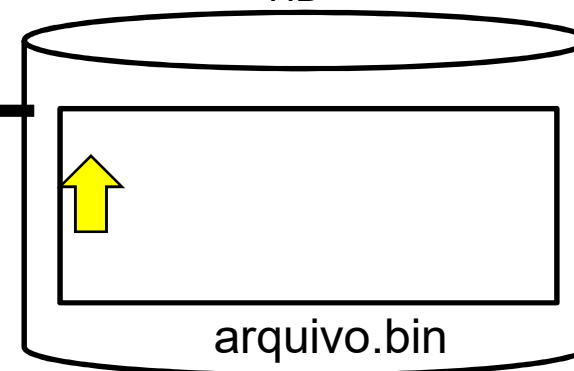
Monitor / Teclado

```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Memória



HD



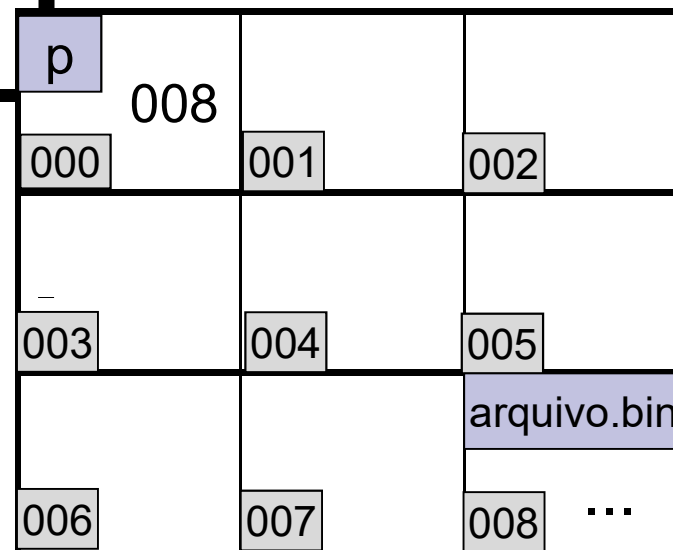
Arquivo Binário

$p == \text{NULL}$
 $= 0$

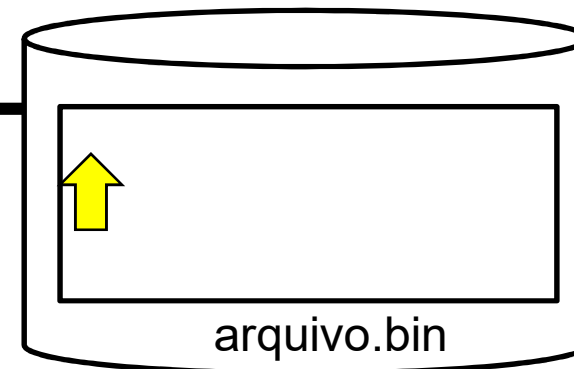
```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



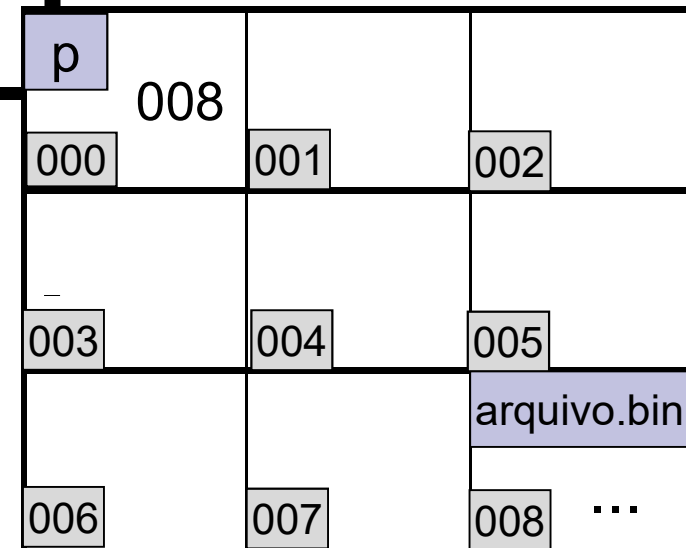
Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

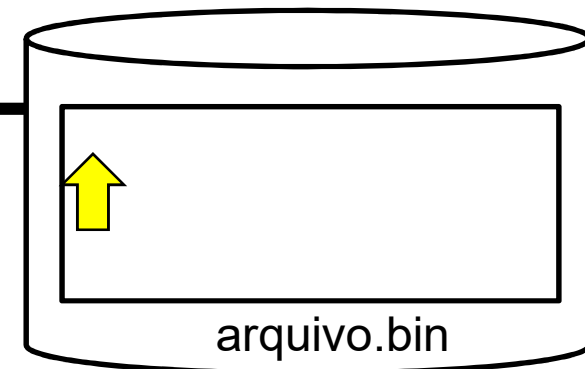
Monitor / Teclado

Arquivo criado com
sucesso!

Memória



HD



Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

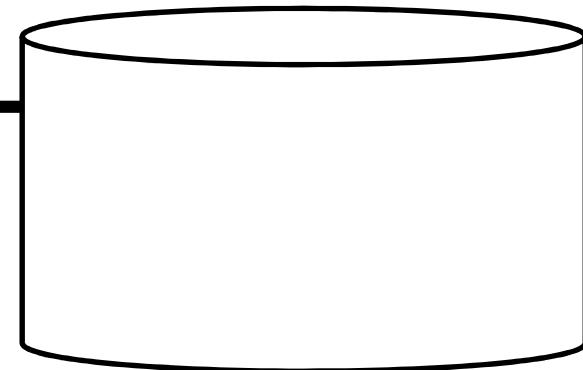
Monitor / Teclado

Arquivo criado com
sucesso!

Memória

p	008		
000		001	002
003		004	005
006		007	008

HD



Arquivo Binário

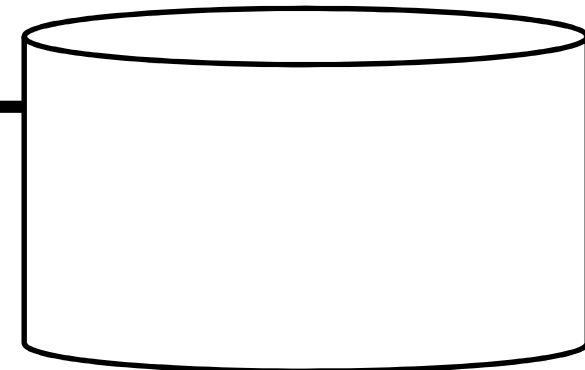
```
#include <stdio.h>
int main() {
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        printf("Arquivo criado com sucesso!\n");
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD





Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

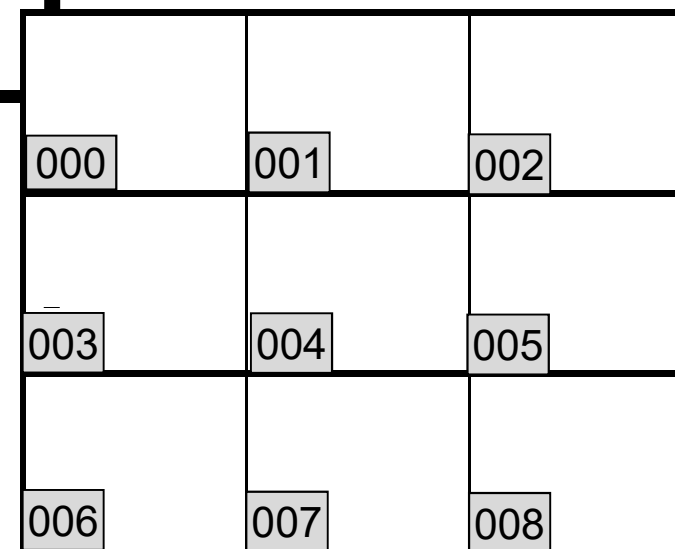
*Para ver o conteúdo de um arquivo binário:
<https://www.onlinehexeditor.com/>*

Arquivo Binário

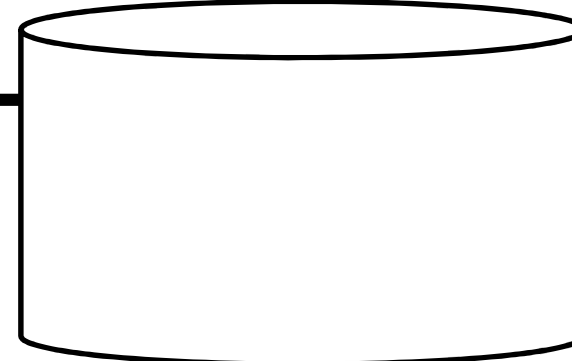
Monitor / Teclado



Memória



HD



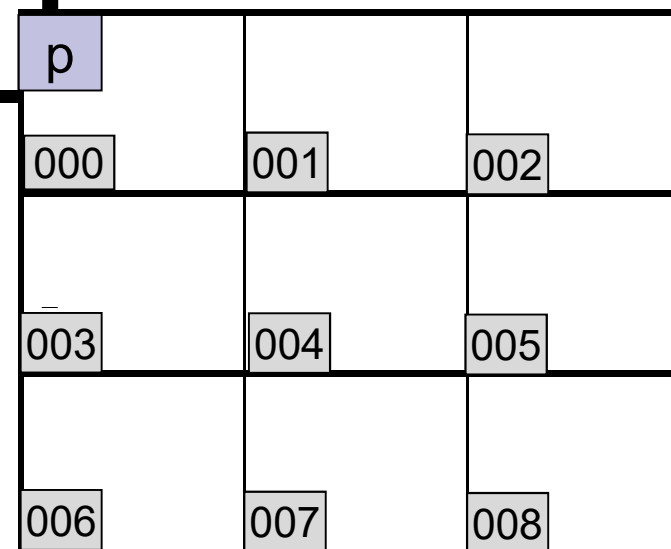
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Arquivo Binário

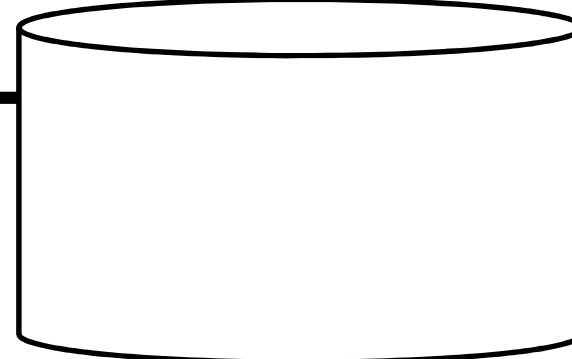
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

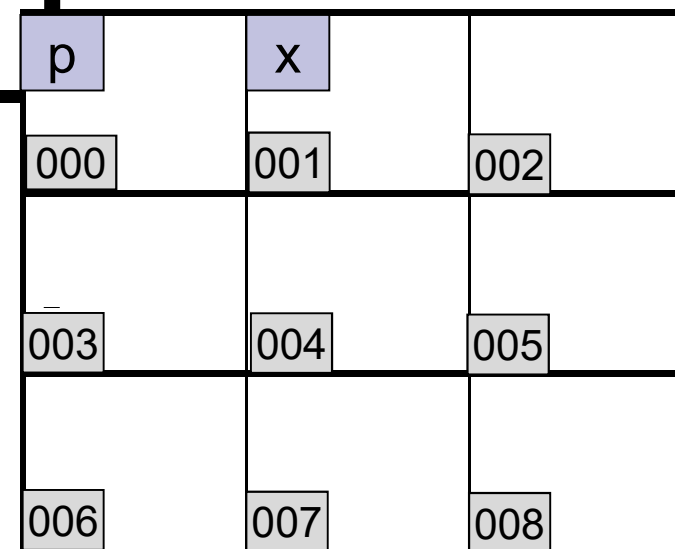


Arquivo Binário

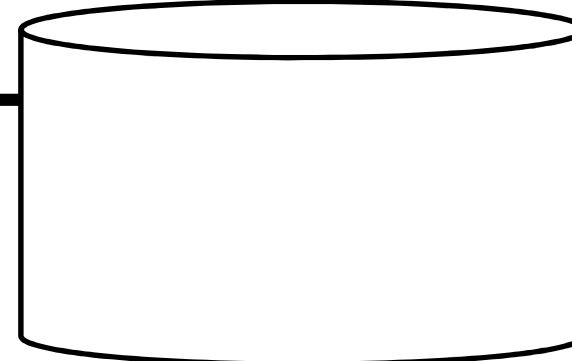
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

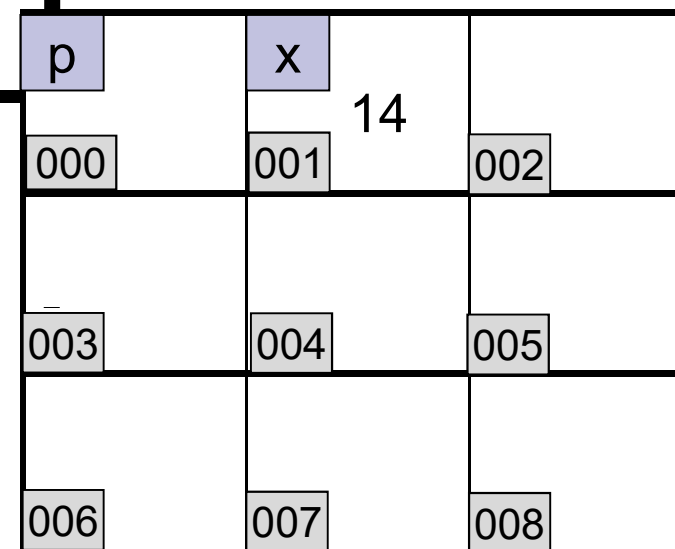


Arquivo Binário

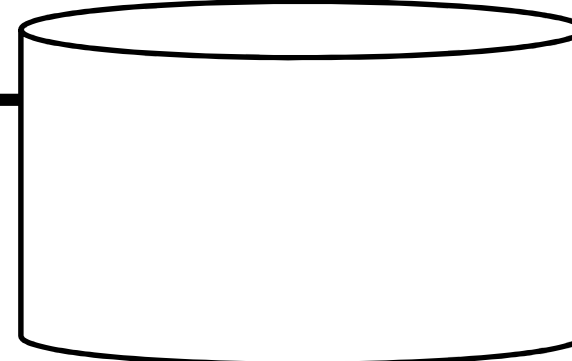
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

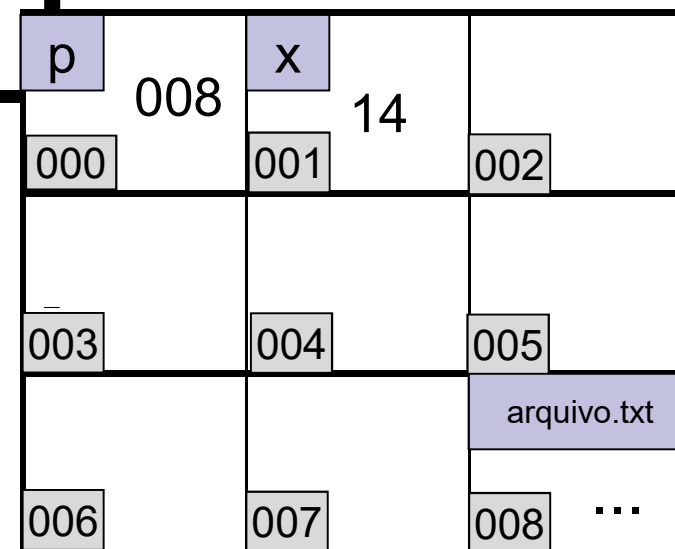


Arquivo Binário

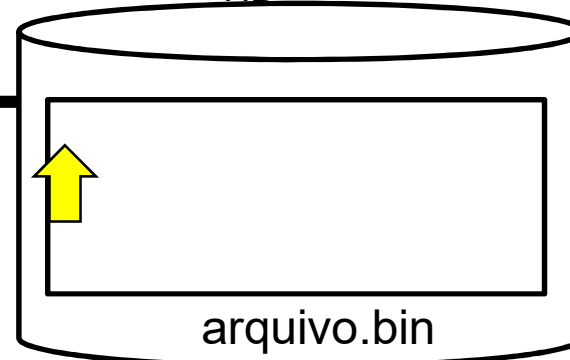
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

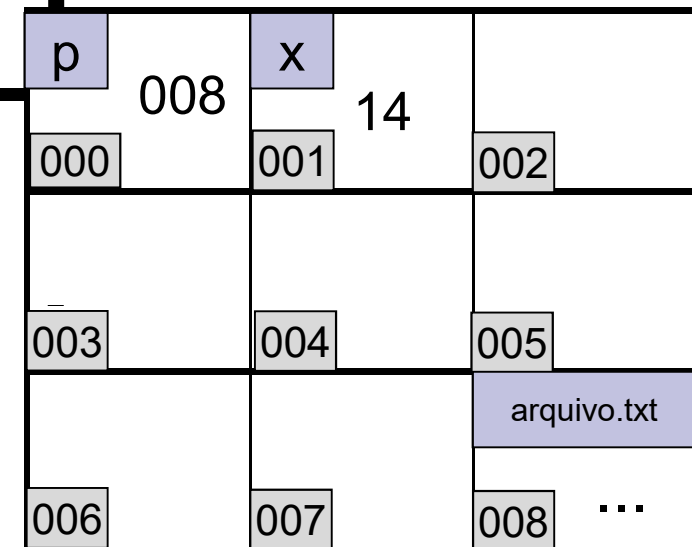


Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Arquivo Binário

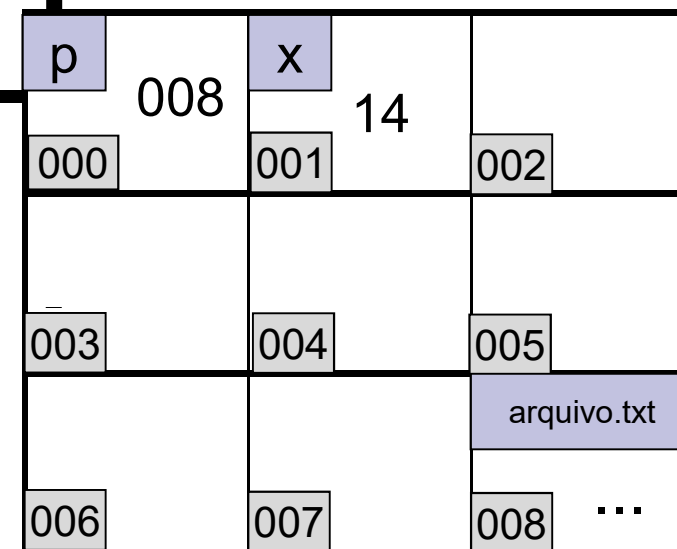
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        → fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

fwrite() recebe:

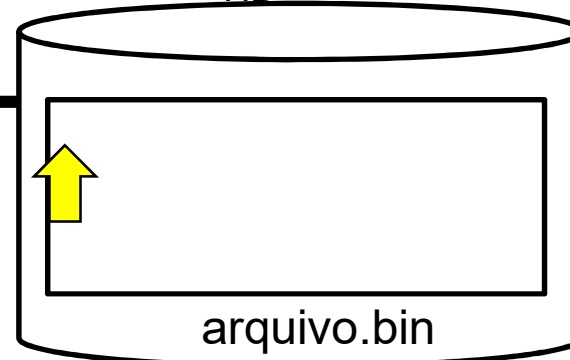
- 1) *&x, que é o endereço da variável que contém o dado a ser gravado*
- 2) *sizeof(x), que é o tamanho deste dado*
- 3) *1, que é a quantidade de dados que serão gravados (no caso, apenas 1 inteiro será gravado)*
- 4) *p, o apontador para a struct FILE*

Monitor / Teclado

Memória



HD

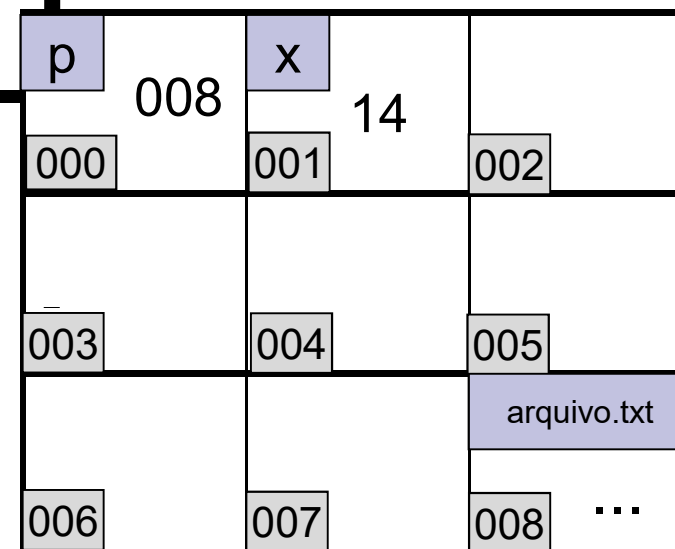


Arquivo Binário

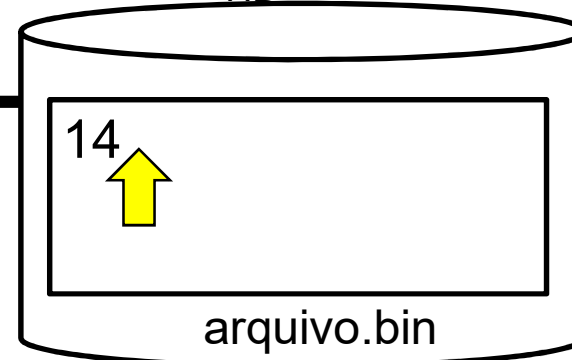
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        → fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

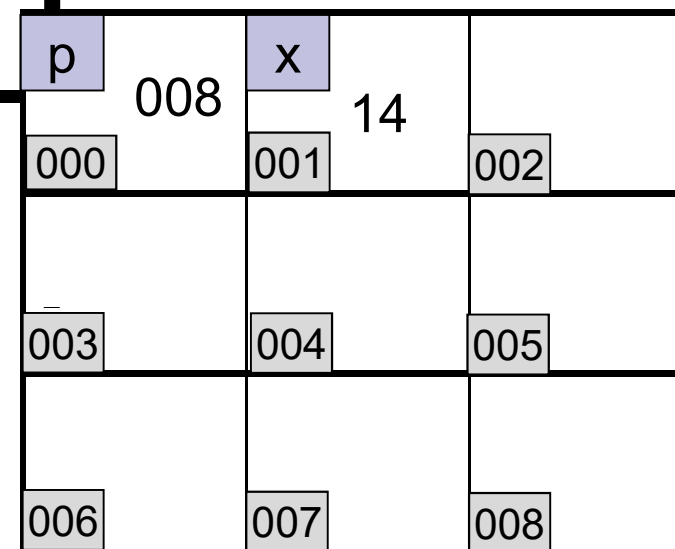


Arquivo Binário

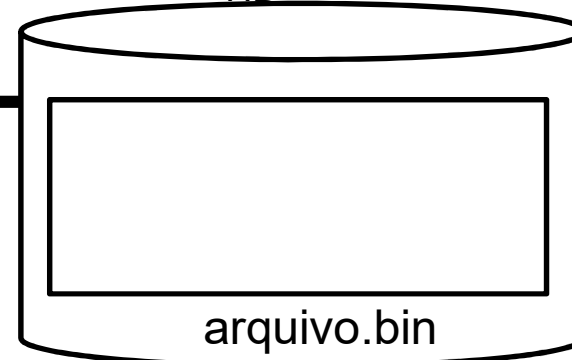
```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



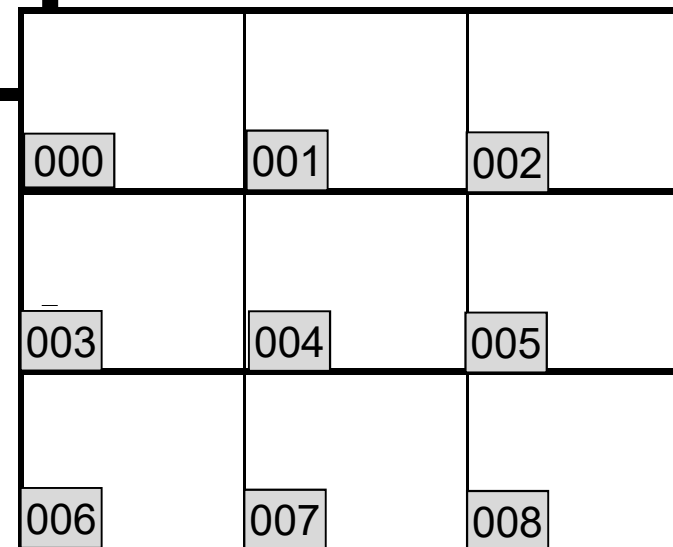
Arquivo Binário

```
#include <stdio.h>
int main() {
    FILE* p;
    int x;
    x = 14;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo!\n");
    }
    else {
        fwrite(&x, sizeof(x), 1, p);
    }
    fclose(p);
    return 0;
}
```

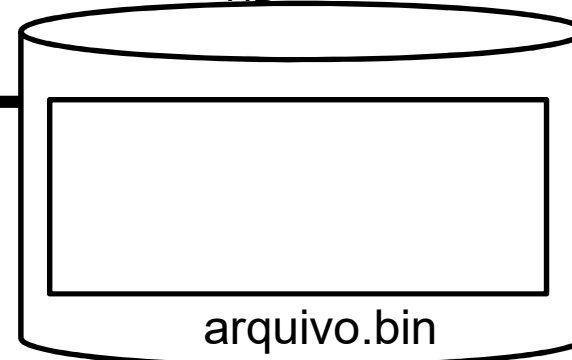
Para ver o conteúdo de um arquivo binário:
<https://www.onlinehexeditor.com/>

Monitor / Teclado

Memória



HD





Arquivo Binário

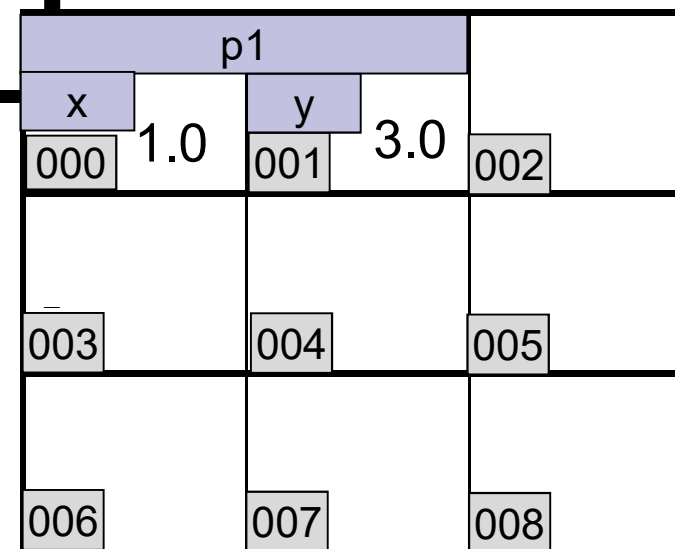
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Arquivo Binário

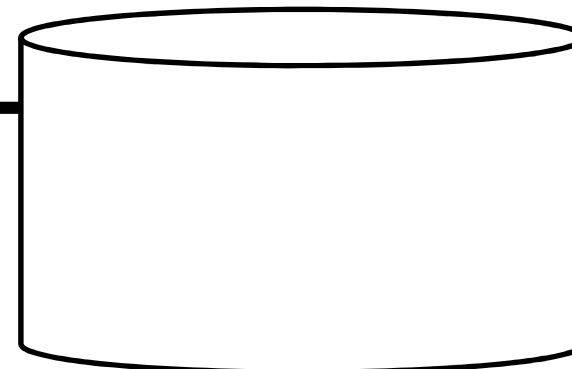
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

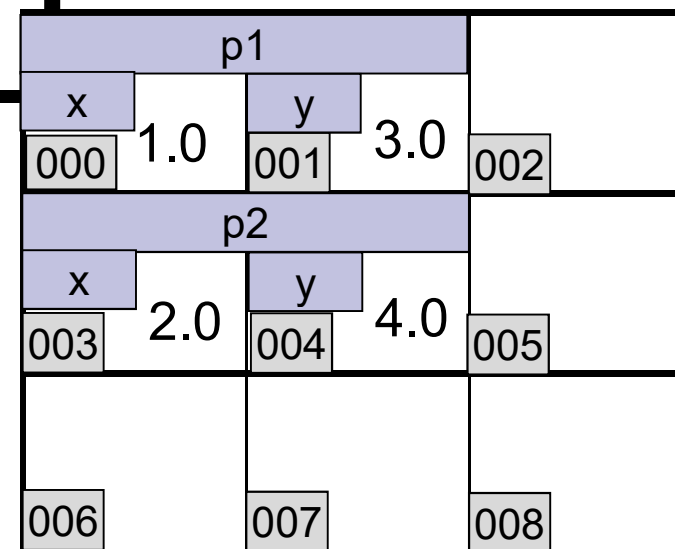


Arquivo Binário

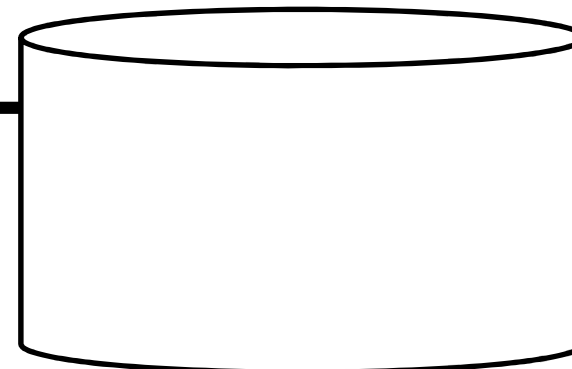
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    → Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

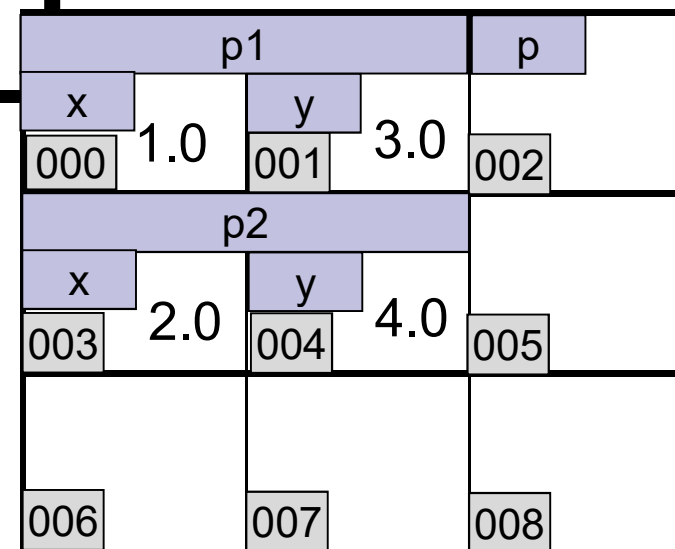


Arquivo Binário

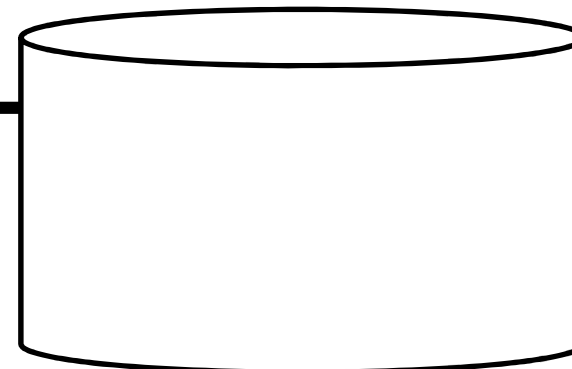
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    → FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

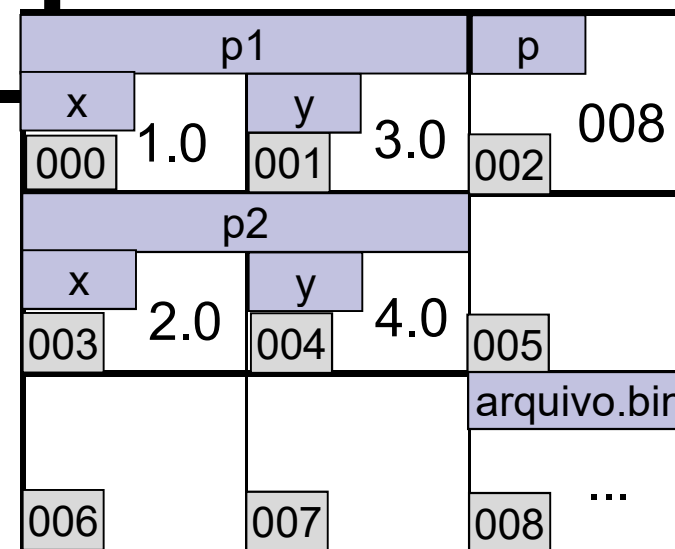


Arquivo Binário

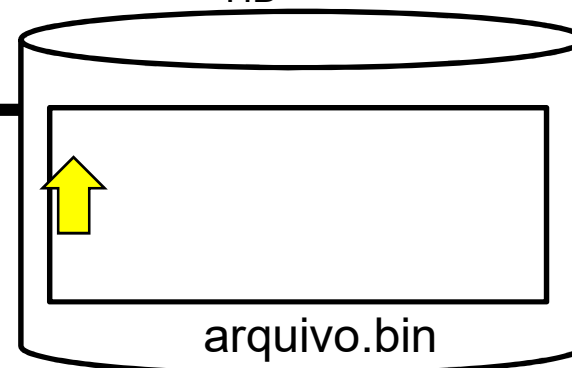
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    → p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



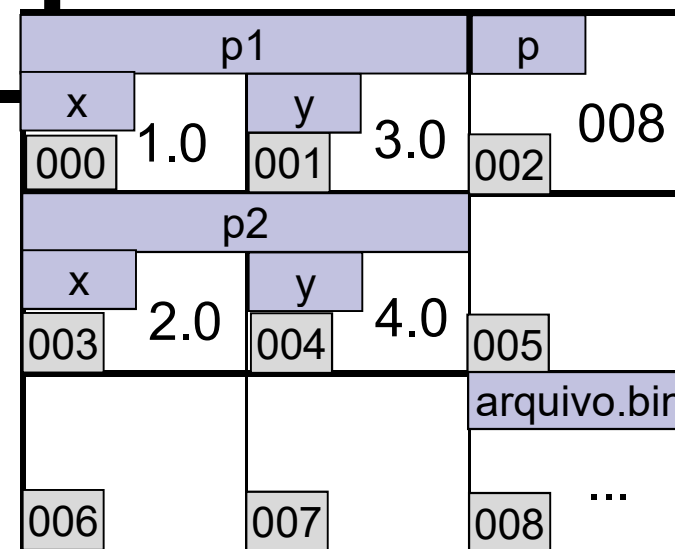
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0, 3.0};
    Ponto p2 = {2.0, 4.0};
    FILE* p;
    p = fopen("arquivo.bin", "wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1, sizeof(p1), 1, p);
        fwrite(&p2, sizeof(p2), 1, p);
    }
    fclose(p);
    return 0;
}
```

$p == NULL$
 $= 0$

Monitor / Teclado

Memória



HD

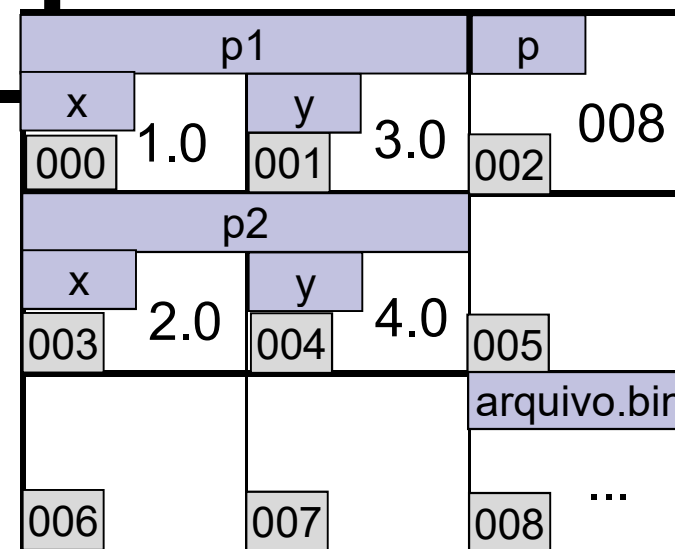


Arquivo Binário

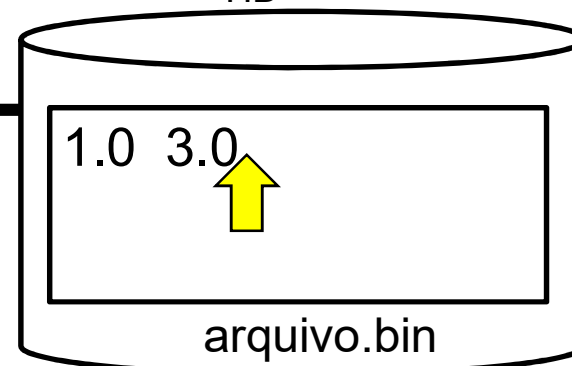
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        → fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

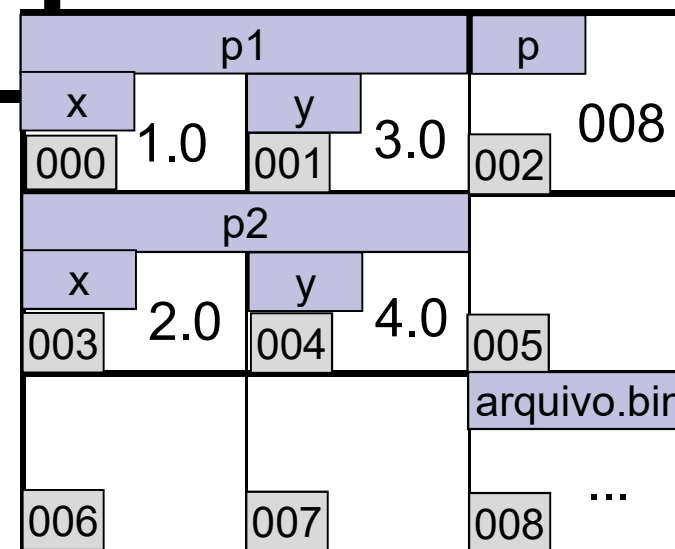


Arquivo Binário

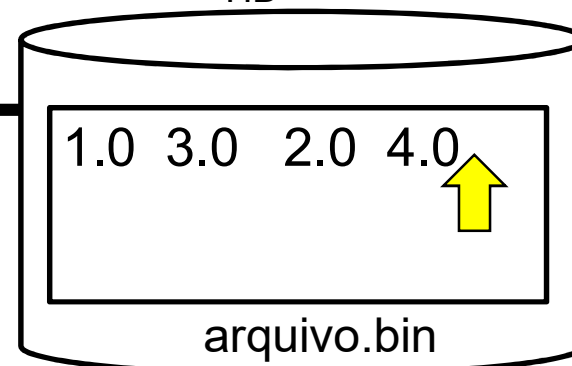
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        → fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

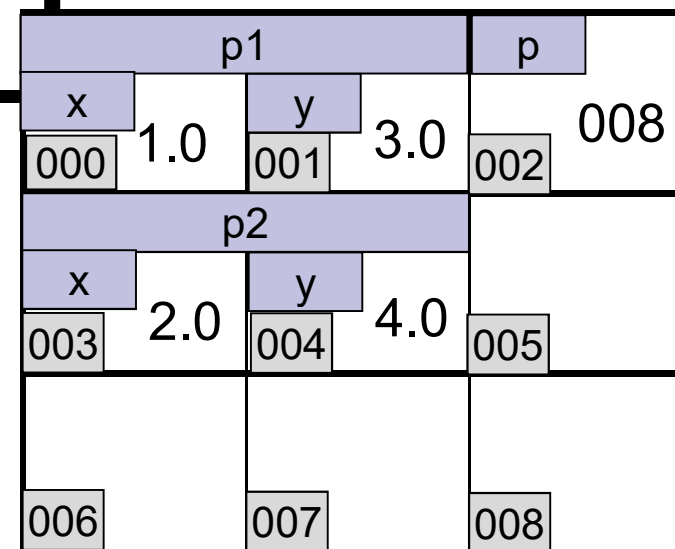


Arquivo Binário

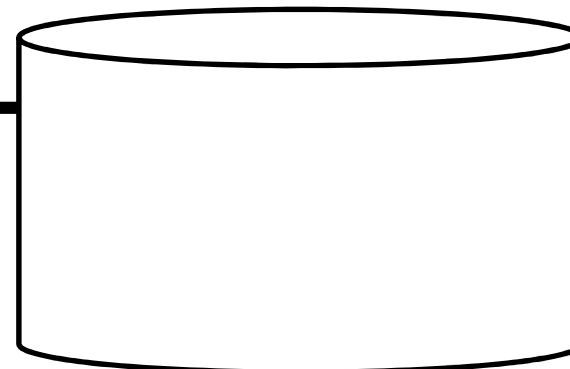
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

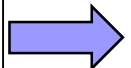


HD



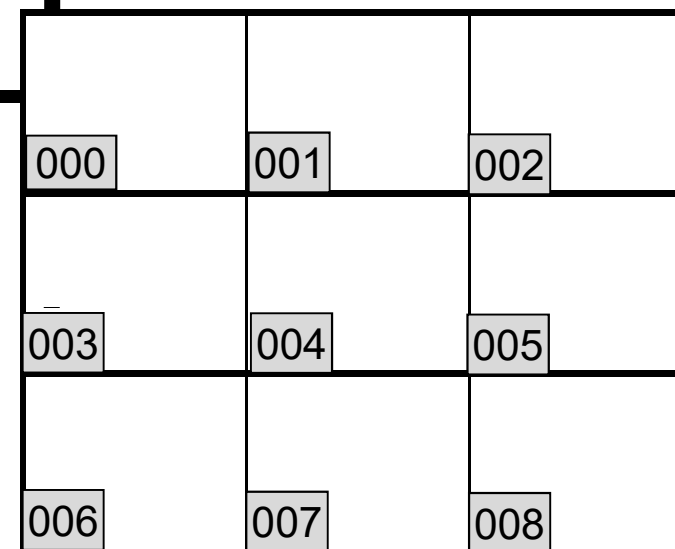
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto p1 = {1.0,3.0};
    Ponto p2 = {2.0,4.0};
    FILE* p;
    p = fopen("arquivo.bin","wb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        fwrite(&p1,sizeof(p1),1,p);
        fwrite(&p2,sizeof(p2),1,p);
    }
    fclose(p);
    return 0;
}
```

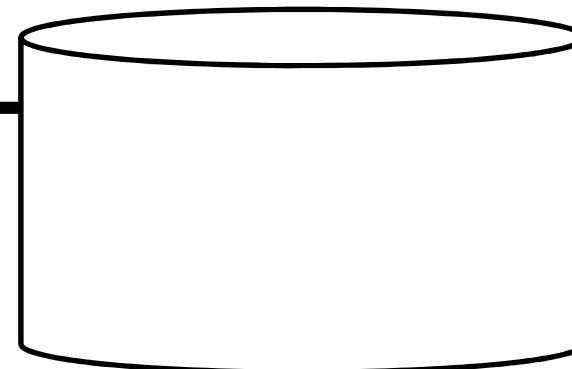


Monitor / Teclado

Memória



HD





Arquivo Binário

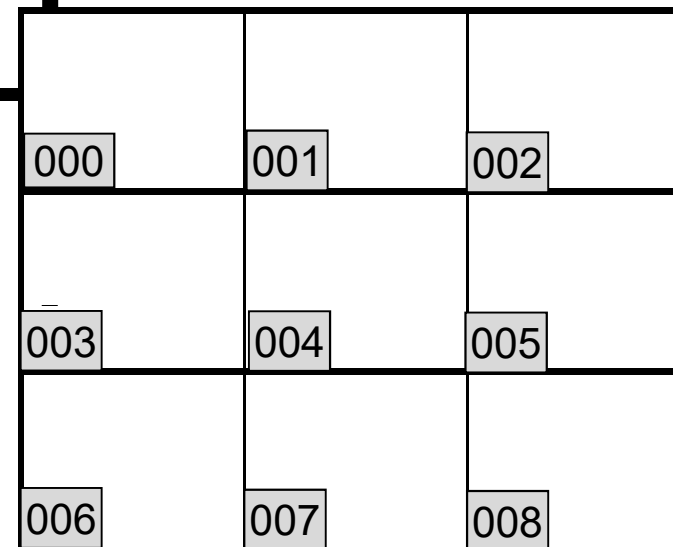
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```


Arquivo Binário

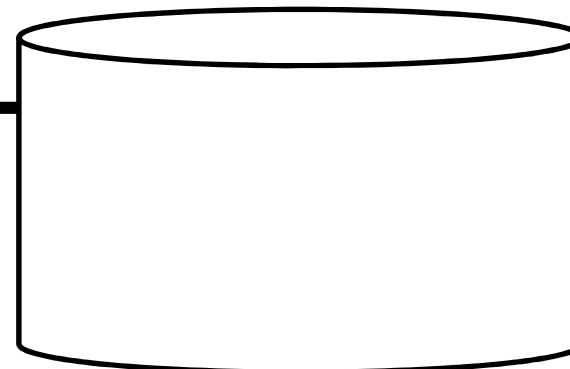
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

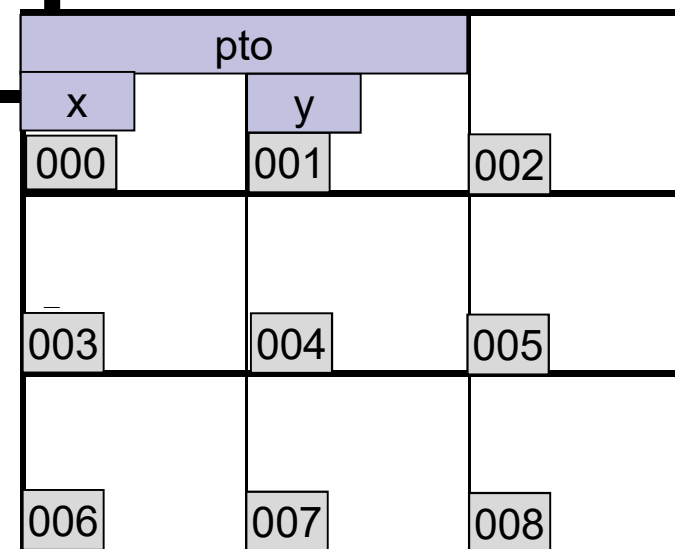


Arquivo Binário

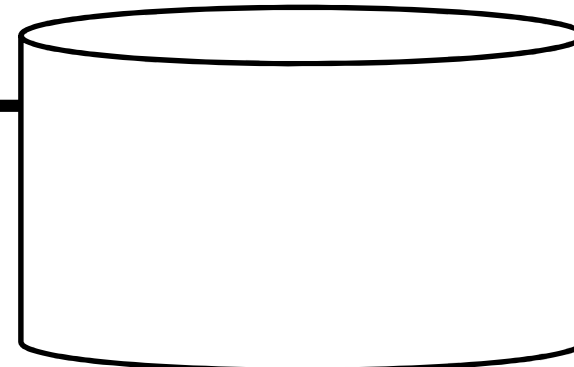
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

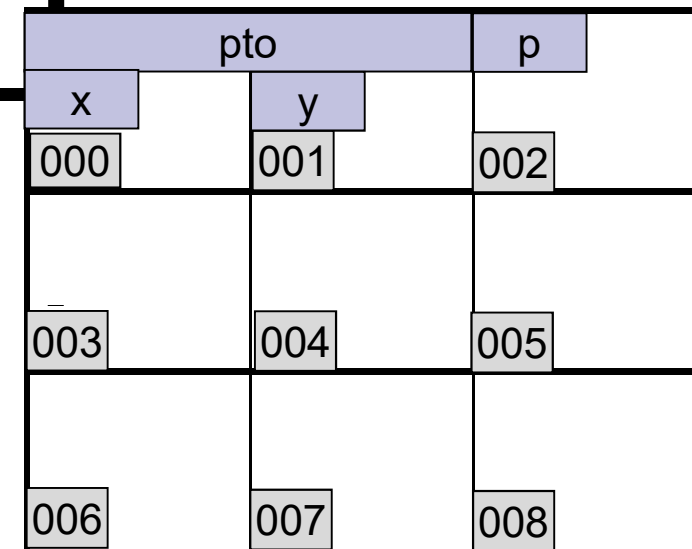


Arquivo Binário

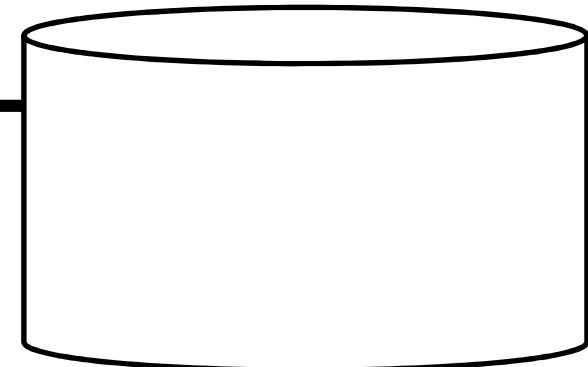
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    → FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

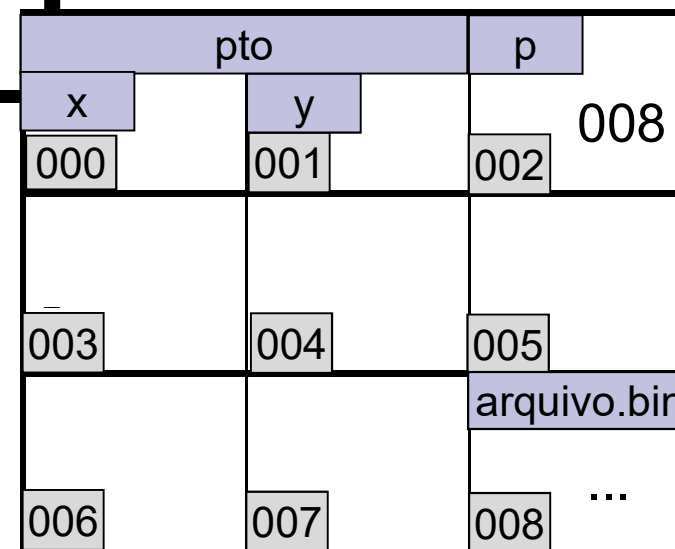


Arquivo Binário

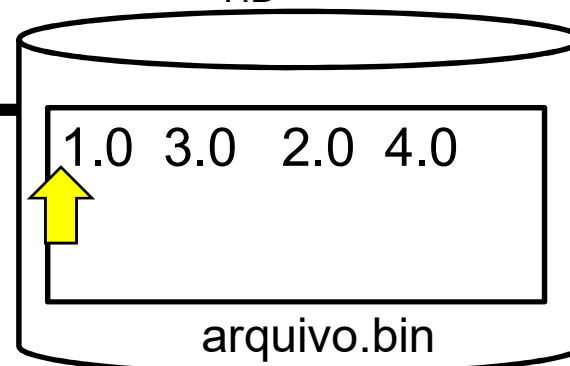
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    → p = fopen("arquivo.bin", "rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto, sizeof(pto), 1, p) == 1) {
            printf("(%.2f, %.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



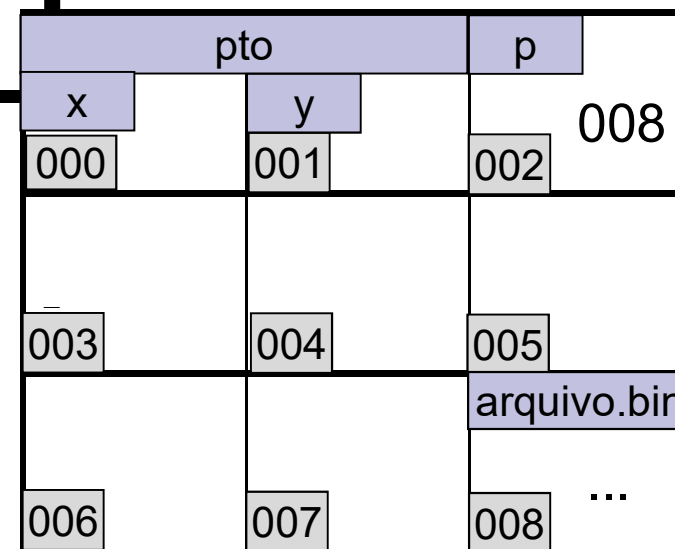
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

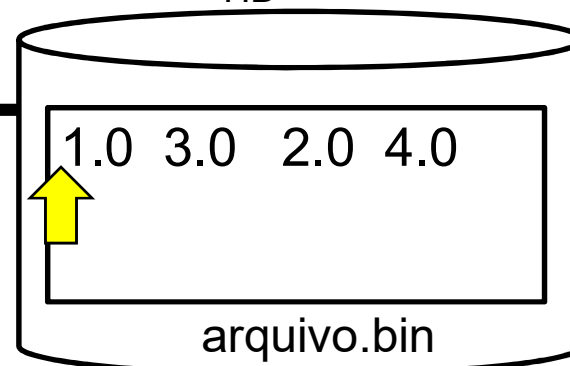
$p == NULL$
 $= 0$

Monitor / Teclado

Memória



HD



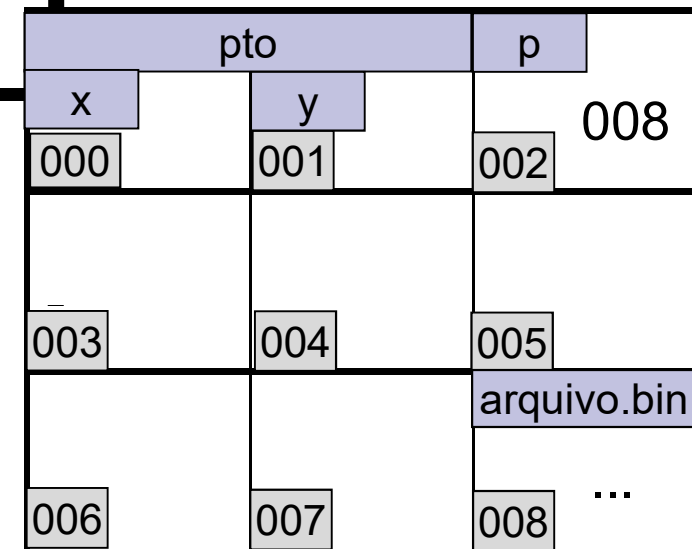
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        → while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

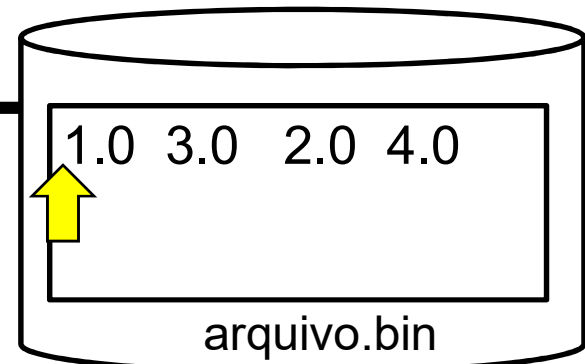
fread() tenta ler 1 valor. Se conseguir, retorna 1. Se não conseguir, retorna 0.

Monitor / Teclado

Memória



HD



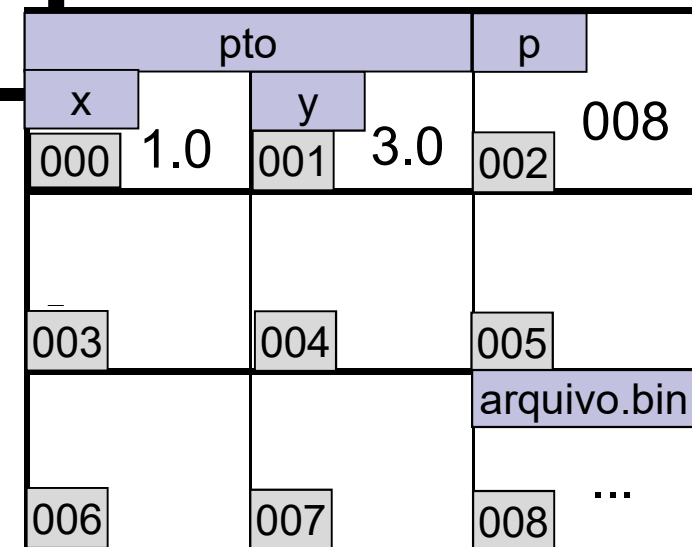
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        → while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

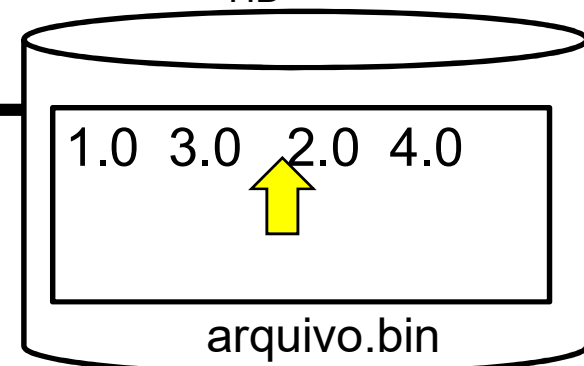
*A leitura vai ser bem sucedida.
fread() salva 1.0 em pto.x e
3.0 em pto.y e move a seta
amarela à frente.*

Monitor / Teclado

Memória



HD



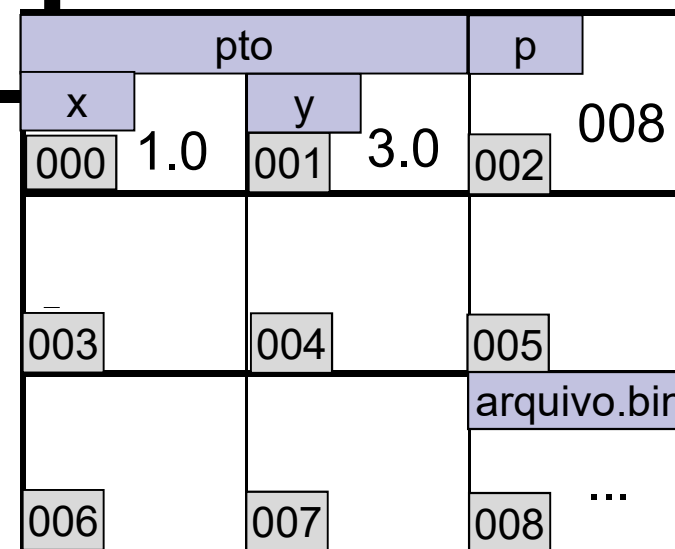
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

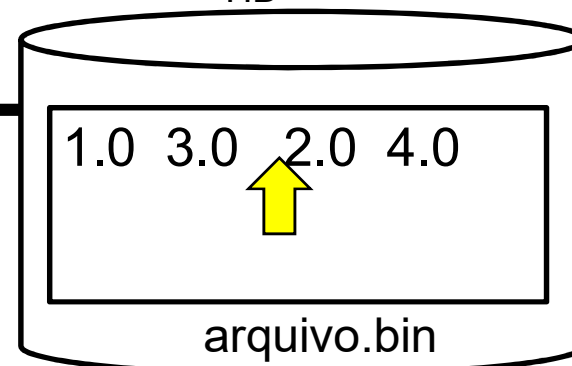
Monitor / Teclado

(1.0,3.0)

Memória



HD



Arquivo Binário

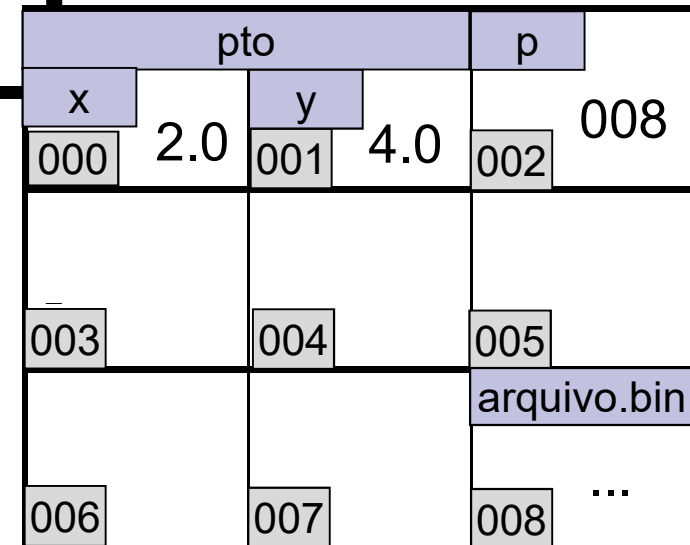
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        → while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
        fclose(p);
        return 0;
    }
}
```

*A leitura vai ser bem sucedida.
fread() salva 2.0 em pto.x e
4.0 em pto.y e move a seta
amarela à frente.*

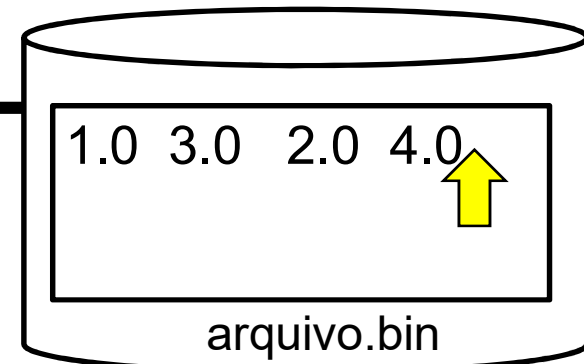
Monitor / Teclado

(1.0,3.0)

Memória



HD



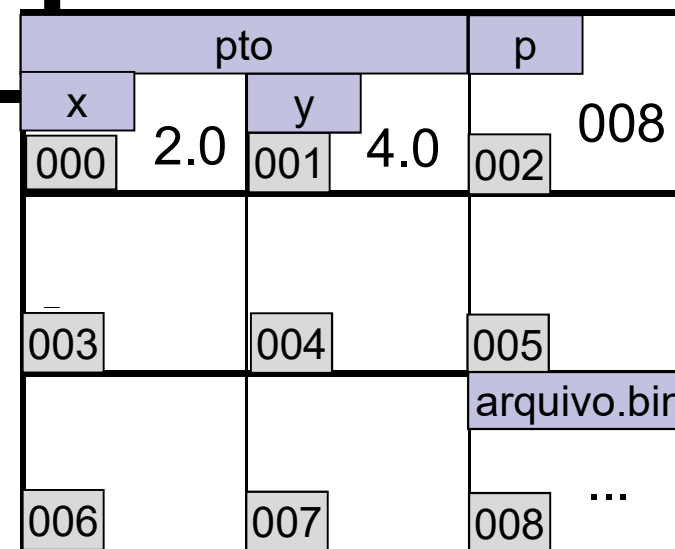
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

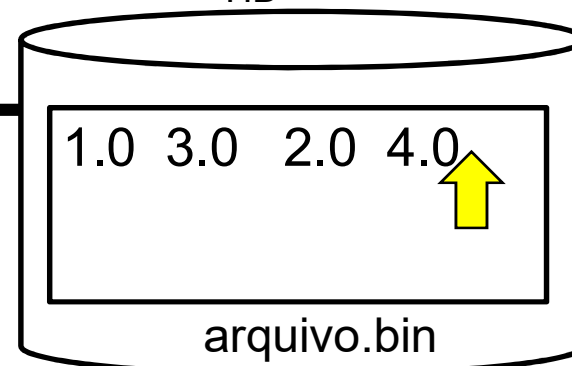
Monitor / Teclado

(1.0,3.0)
(2.0,4.0)

Memória



HD



Arquivo Binário

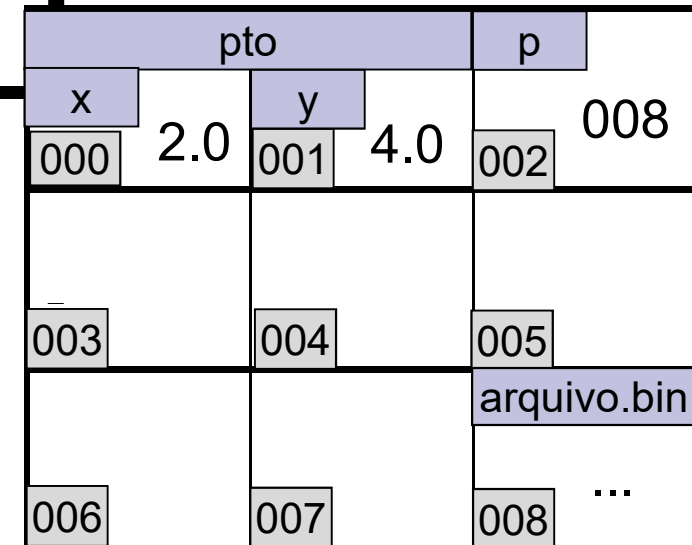
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) != 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

A leitura vai dar erro, pois o fim do arquivo foi atingido e fread() vai retornar 0, que é diferente de 1.

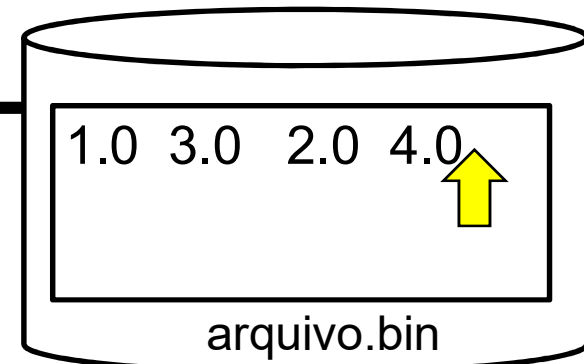
Monitor / Teclado

(1.0,3.0)
(2.0,4.0)

Memória



HD



Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

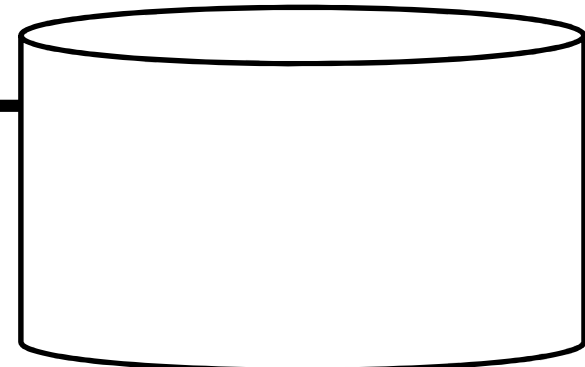
Monitor / Teclado

(1.0,3.0)
(2.0,4.0)

Memória

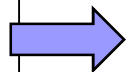
pto		p
x	y	
000 2.0	001 4.0	002 008
003	004	005
006	007	008

HD



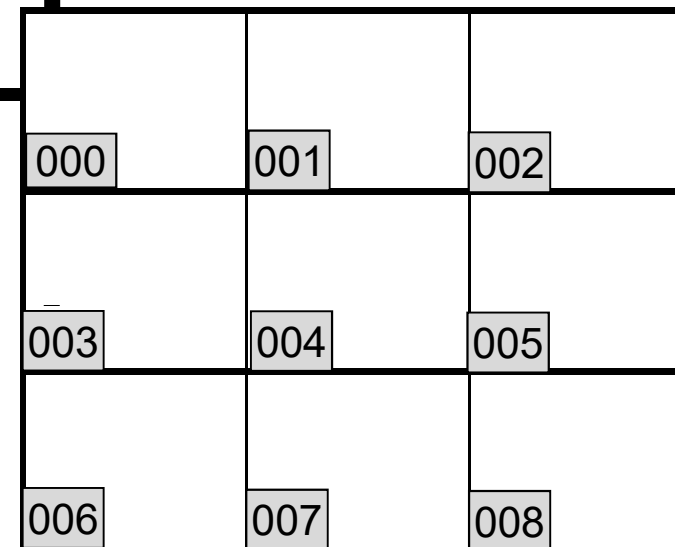
Arquivo Binário

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p;
    p = fopen("arquivo.bin","rb");
    if (p == NULL) {
        printf("Erro ao criar arquivo\n");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n",pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

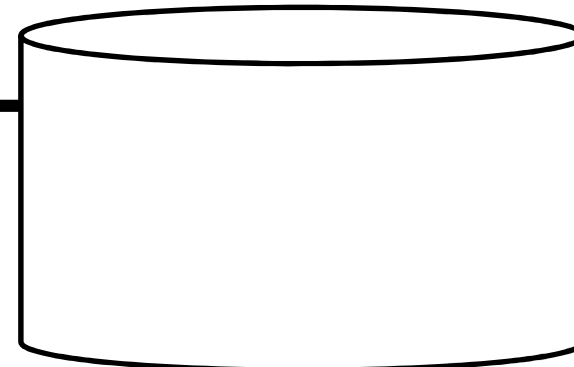


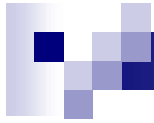
Monitor / Teclado

Memória



HD





Escrita de Vetor (versão 1.0)

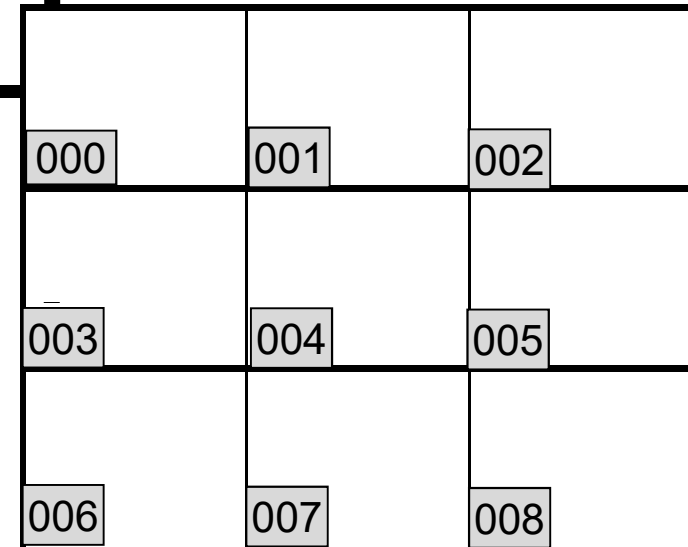
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] = {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Escrita de Vetor

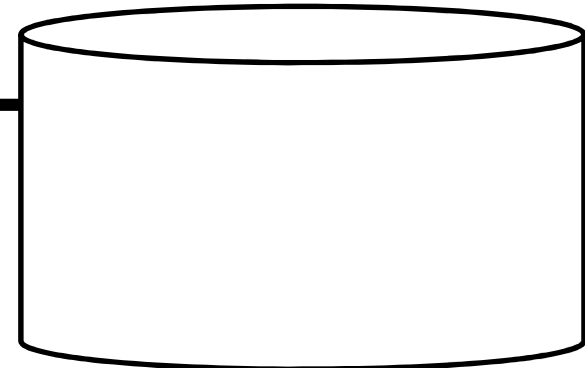
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i],sizeof(vetor[i]),1,p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

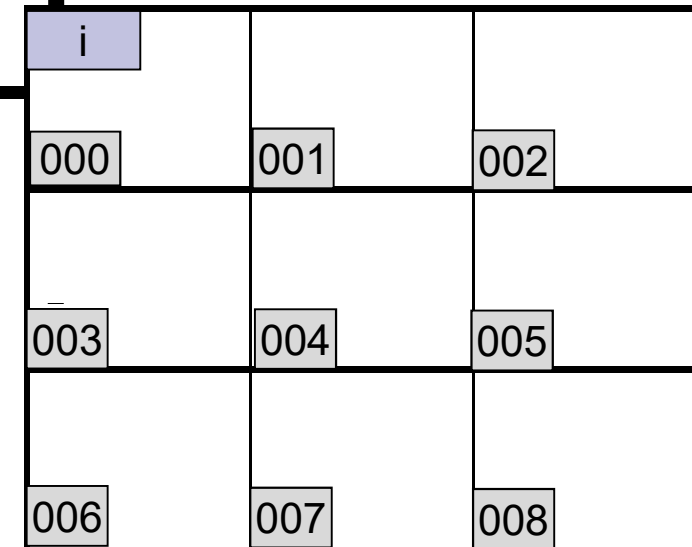


Escrita de Vetor

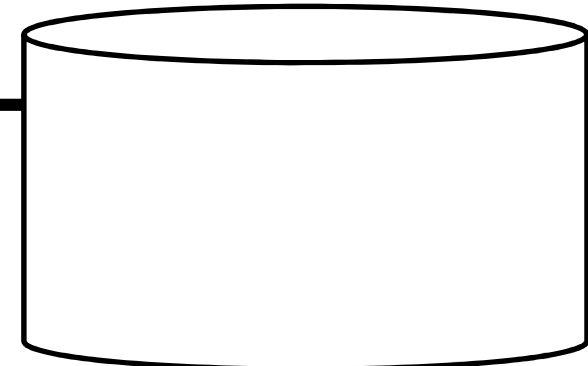
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i],sizeof(vetor[i]),1,p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Escrita de Vetor

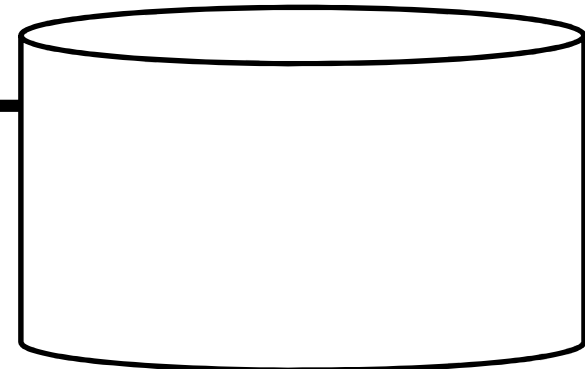
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i],sizeof(vetor[i]),1,p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
		x	y
000		001 1.0	002 2.0
vetor[1]			
		x	y
003	3.0	004 4.0	005
vetor[2]			
		x	y
006	5.0	007 6.0	008

HD



Escrita de Vetor

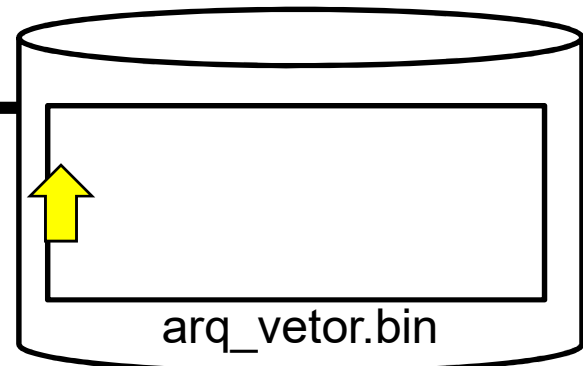
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
		x	y
000		001 1.0	002 2.0
vetor[1]			p
		x	y
003	3.0	004 4.0	005 008
vetor[2]			arq_vetor.bin
		x	y
006	5.0	007 6.0	008 ...

HD



Escrita de Vetor

Monitor / Teclado

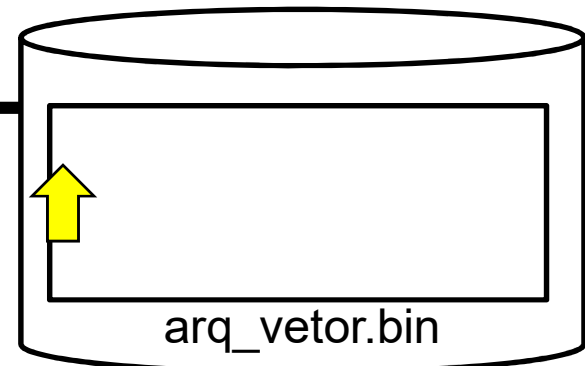
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

$p == \text{NULL}$
 $= 0$

Memória

i		vetor[0]	
		x	y
000		001 1.0	002 2.0
vetor[1]		p	
		x	y
003 3.0		004 4.0	005 008
vetor[2]		arq_vetor.bin	
		x	y
006 5.0		007 6.0	008 ...

HD



Escrita de Vetor

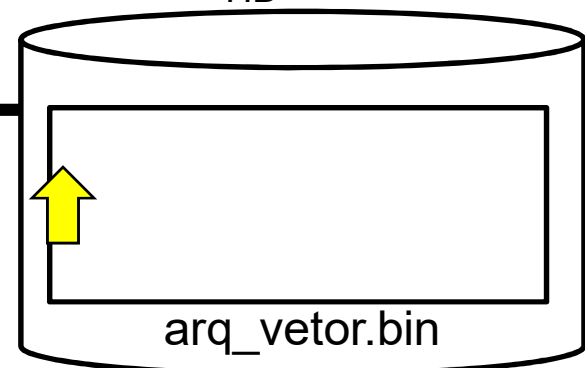
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]			
0		x	1.0	y	2.0
000		001		002	
vetor[1]				p	
x	3.0	y	4.0	008	
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		
006		007		008	...

HD



Escrita de Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

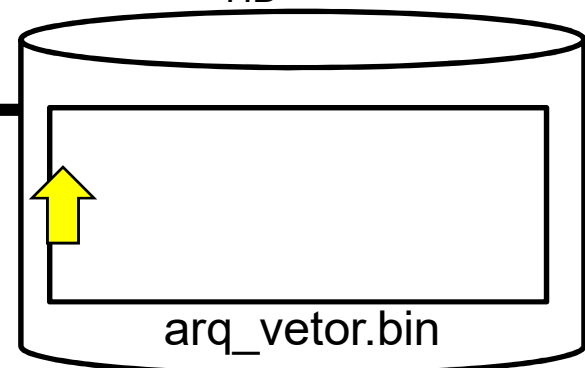
$0 < 3$
 $= 1$

Monitor / Teclado

Memória

i		vetor[0]			
0		x	1.0	y	2.0
000		001		002	
vetor[1]				p	
x	3.0	y	4.0	008	
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		
006		007		008	...

HD



Escrita de Vetor

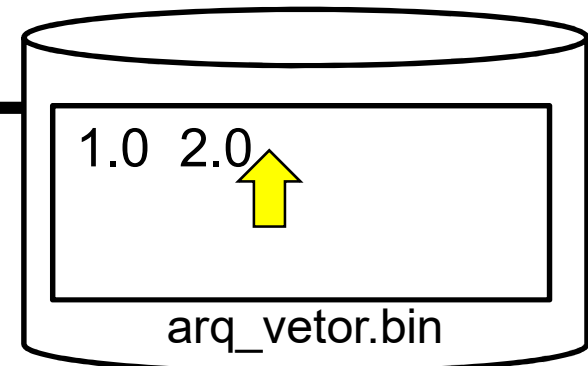
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            → fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
x	0	y	
000		001	002
		1.0	2.0
vetor[1]		p	
x		y	
003		004	005
		3.0	4.0
			008
vetor[2]		arq_vetor.bin	
x		y	
006		007	008
		5.0	6.0
			...

HD



Escrita de Vetor

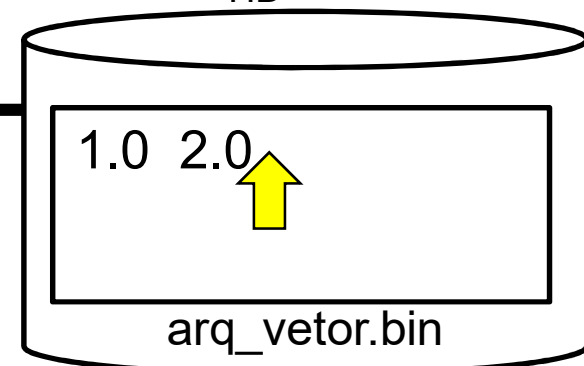
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
	1	x 1.0	y 2.0
000		001	002
vetor[1]		p	
x 3.0	y 4.0	008	
003	004	005	
vetor[2]		arq_vetor.bin	
x 5.0	y 6.0	...	
006	007	008	

HD



Escrita de Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

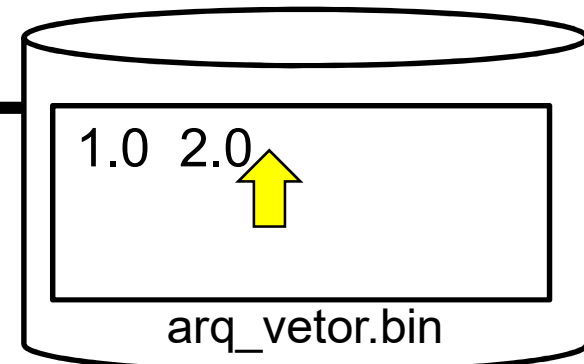
1 < 3
= 1

Monitor / Teclado

Memória

i		vetor[0]	
x	y	x	y
000	1	001	002
		1.0	2.0
vetor[1]		p	
x	y		
003	3.0	004	4.0
		005	008
vetor[2]		arq_vetor.bin	
x	y		
006	5.0	007	6.0
		008	...

HD



Escrita de Vetor

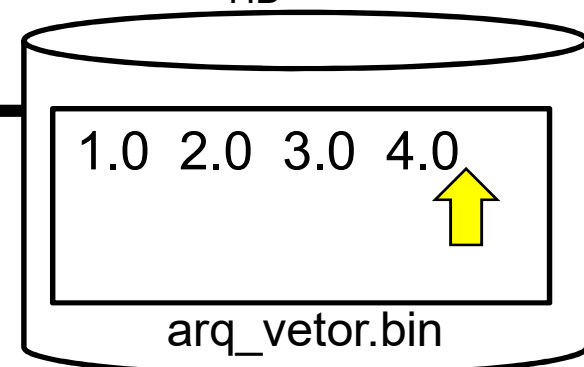
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            → fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
	1	x 1.0	y 2.0
000		001	002
vetor[1]		p	
x	3.0	y	4.0
003		004	005
vetor[2]		arq_vetor.bin	
x	5.0	y	6.0
006		007	008

HD



Escrita de Vetor

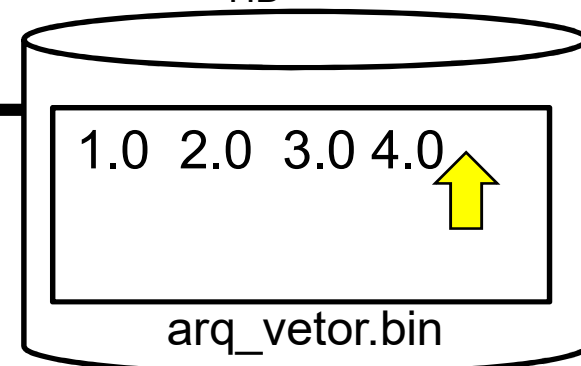
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
	2	x 1.0	y 2.0
000		001	002
vetor[1]		p	
x 3.0	y 4.0	008	
003	004	005	
vetor[2]		arq_vetor.bin	
x 5.0	y 6.0	...	
006	007	008	

HD



Escrita de Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

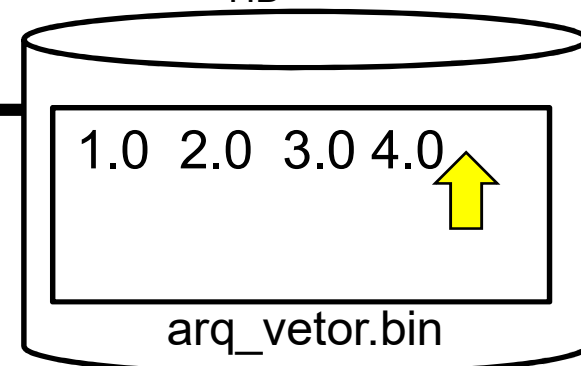
2 < 3
= 1

Monitor / Teclado

Memória

i		vetor[0]	
	2	x 1.0	y 2.0
000		001	002
		vetor[1]	
		x 3.0	y 4.0
003		004	005
		vetor[2]	
		x 5.0	y 6.0
006		007	008

HD



Escrita de Vetor

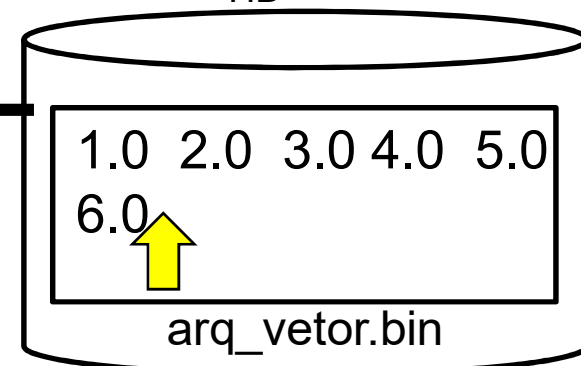
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            → fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
	2	x 1.0	y 2.0
000		001	002
vetor[1]		p	
x 3.0	y 4.0	008	
003	004	005	
vetor[2]		arq_vetor.bin	
x 5.0	y 6.0	...	
006	007	008	

HD



Escrita de Vetor

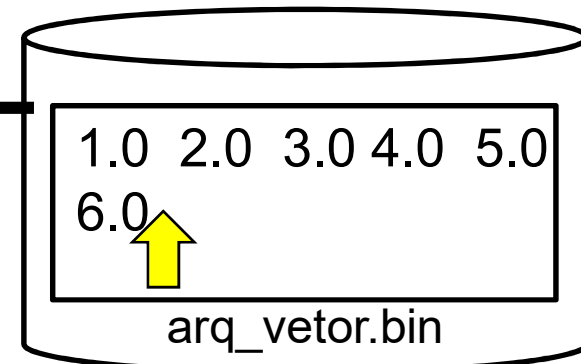
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
	3	x 1.0	y 2.0
000		001	002
vetor[1]		p	
x 3.0	y 4.0	008	
003	004	005	
vetor[2]		arq_vetor.bin	
x 5.0	y 6.0	...	
006	007	008	

HD



Escrita de Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i], sizeof(vetor[i]), 1, p);
        }
    }
    fclose(p);
    return 0;
}
```

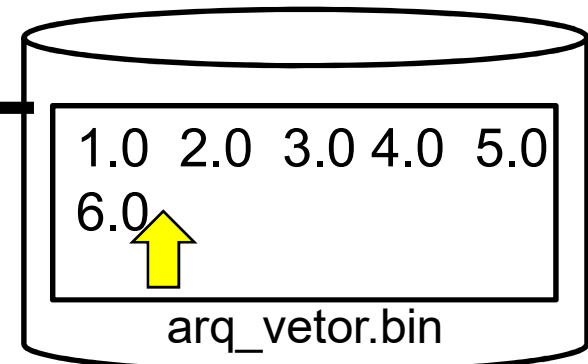
3 < 3
= 0

Monitor / Teclado

Memória

i		vetor[0]	
	2	x 1.0	y 2.0
000		001	002
		vetor[1]	
		x 3.0	y 4.0
003		004	005
		vetor[2]	
		x 5.0	y 6.0
006		007	008

HD



Escrita de Vetor

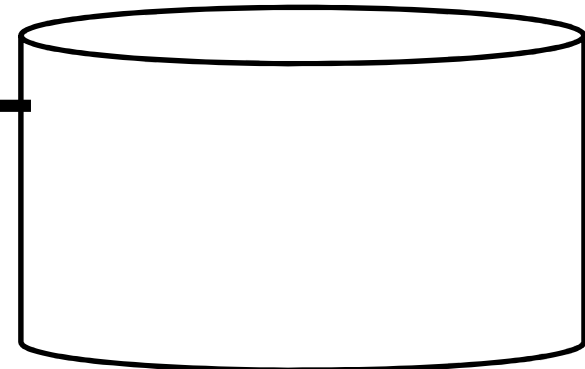
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i],sizeof(vetor[i]),1,p);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

i		vetor[0]	
	2	x 1.0	y 2.0
000		001	002
vetor[1]		p	
x	3.0	y	4.0
003		004	005
vetor[2]			
x	5.0	y	6.0
006		007	008

HD





Monitor / Teclado

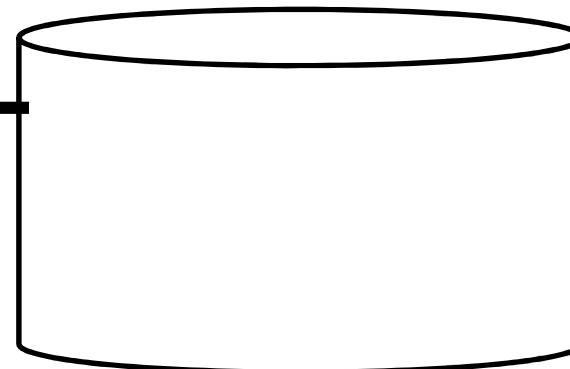
Escrita de Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    int i;
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            fwrite(&vetor[i],sizeof(vetor[i]),1,p);
        }
    }
    fclose(p);
    return 0;
}
```

Memória

000	001	002
003	004	005
006	007	008

HD





Escrita de vetor (versão 2.0)

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] = {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Grava os 3 pontos do vetor em um único fwrite().

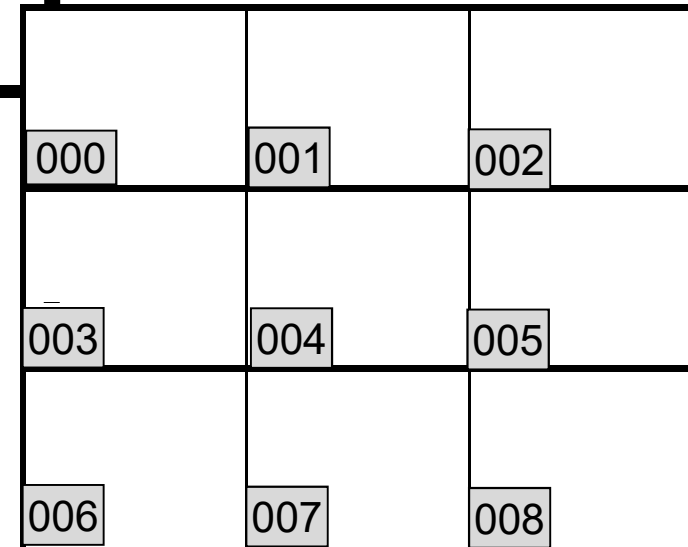
*Também podia ser:
fwrite(vetor, sizeof(vetor[0]), 3, p);*

Escrita de Vetor

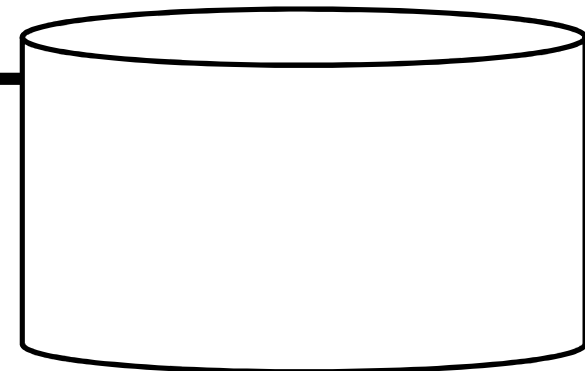
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Escrita de Vetor

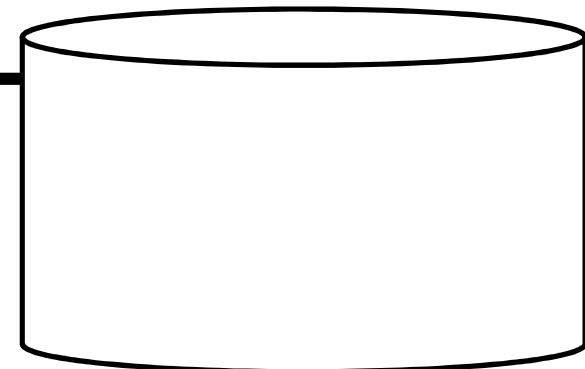
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

vetor[0]			
x	1.0	y	2.0
000		001	002
vetor[1]			
x	3.0	y	4.0
003		004	005
vetor[2]			
x	5.0	y	6.0
006		007	008

HD

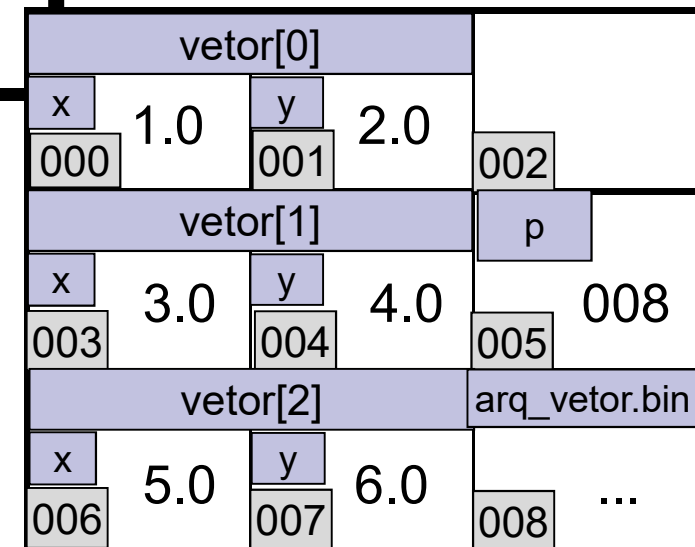


Escrita de Vetor

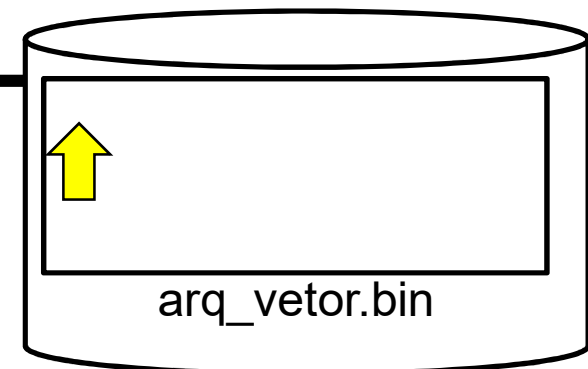
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

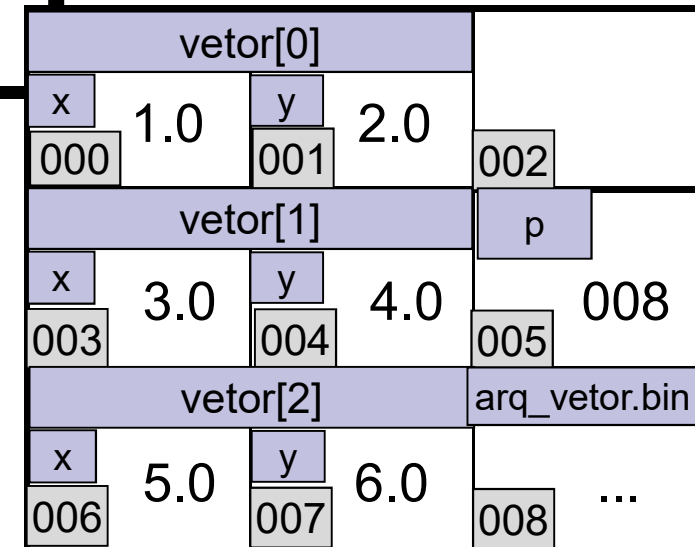


Escrita de Vetor

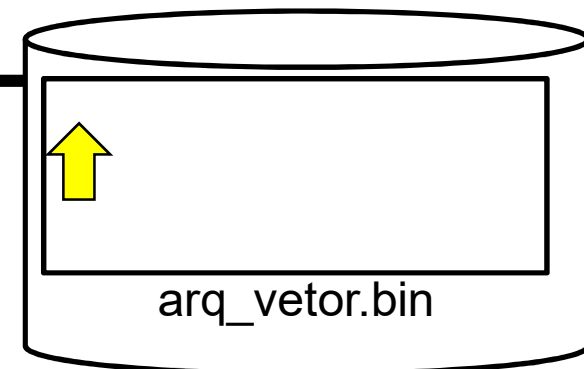
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



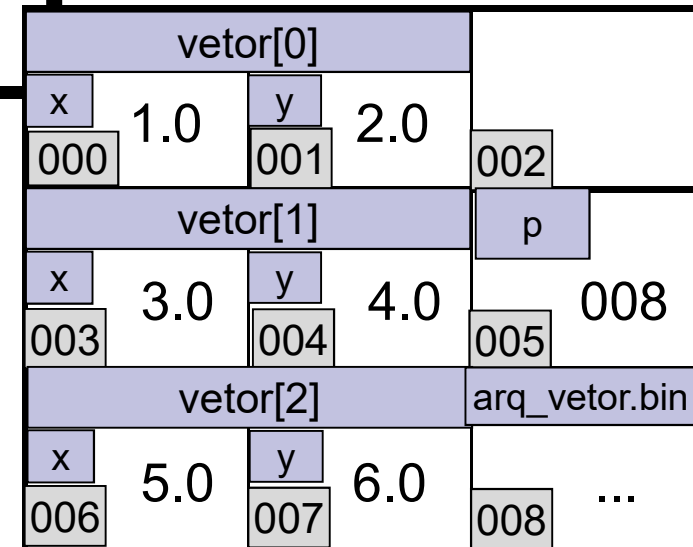
$p == NULL$
 $= 0$

Escrita de Vetor

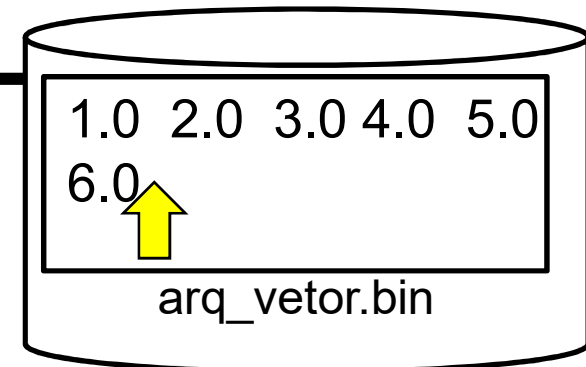
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

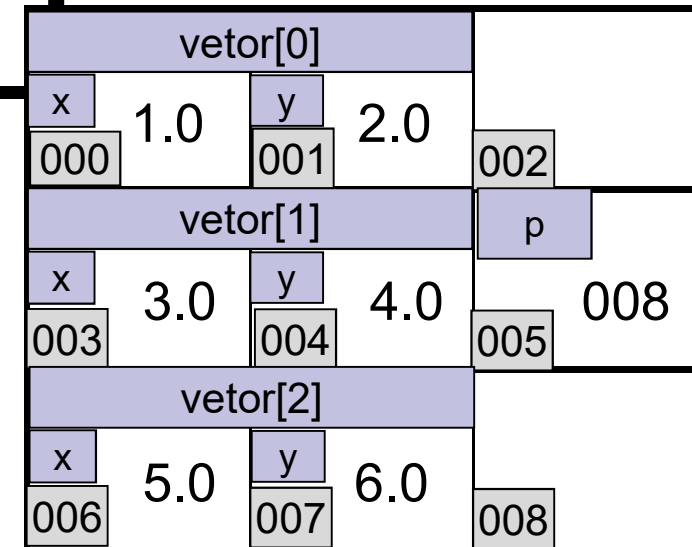


Escrita de Vetor

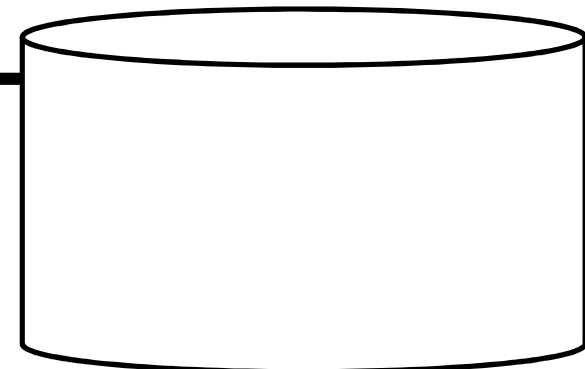
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

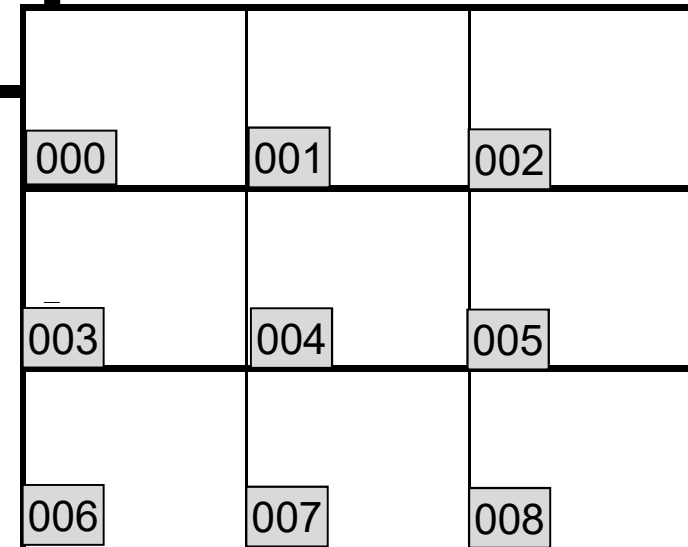


Escrita de Vetor

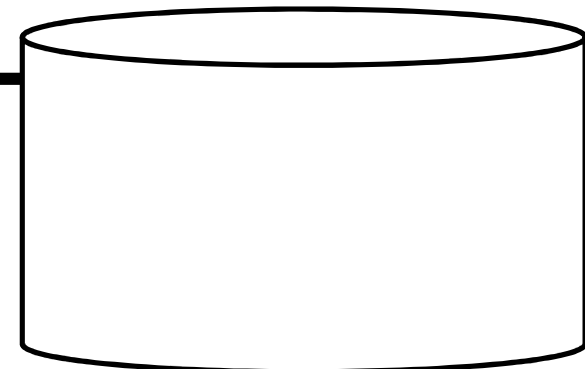
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3] =
        {{1.0,2.0},{3.0,4},{5.0,6.0}};
    FILE* p = fopen("arq_vetor.bin", "wb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        fwrite(vetor, sizeof(Ponto), 3, p);
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD





Leitura em vetor (versão 1.0)

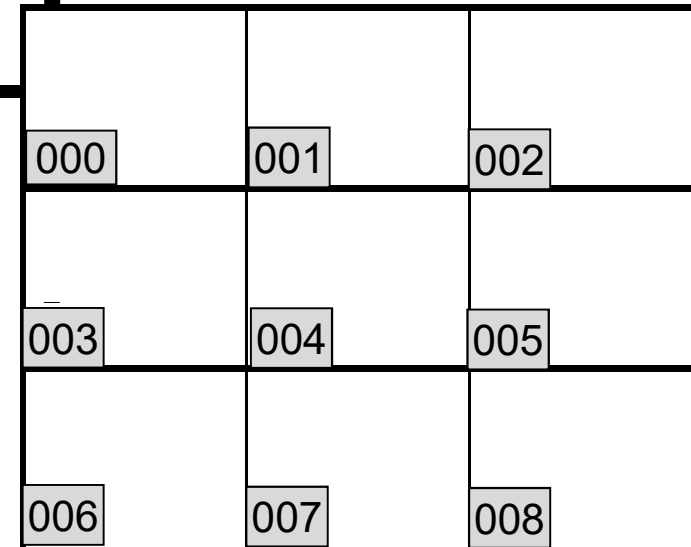
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p) == 1) {
                printf("(%.2f,%.2f)\n",vetor[i].x, vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Leitura em Vetor

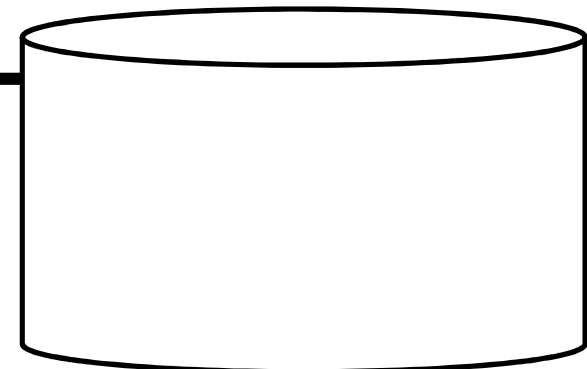
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

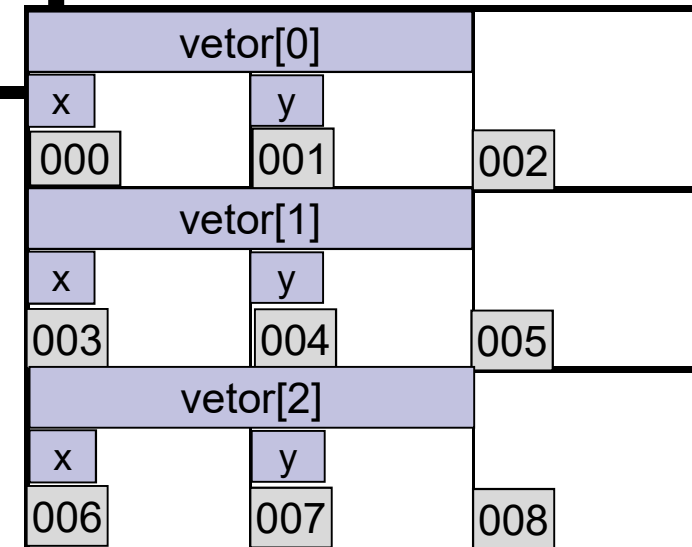


Leitura em Vetor

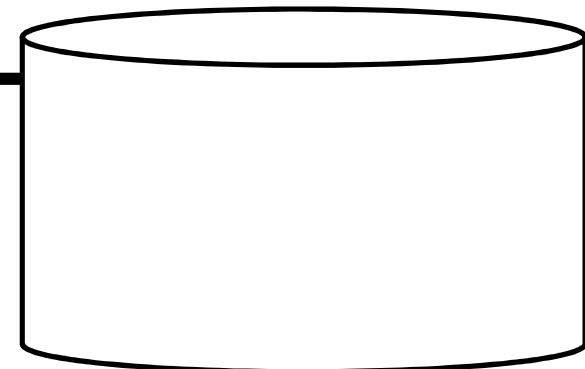
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f,%.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

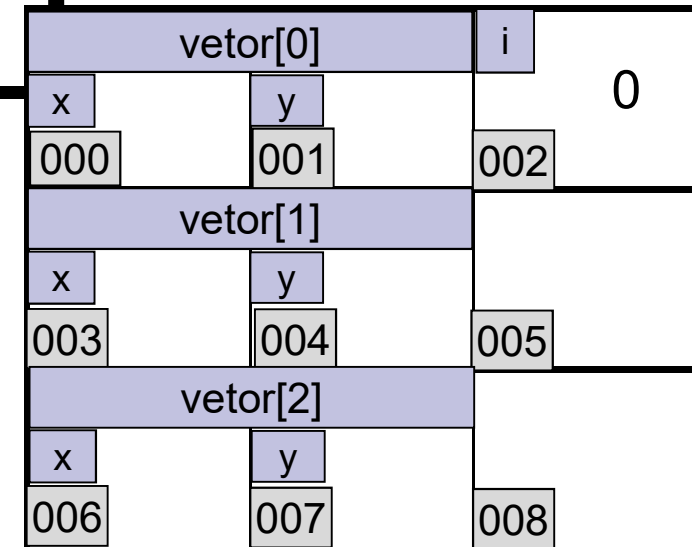


Leitura em Vetor

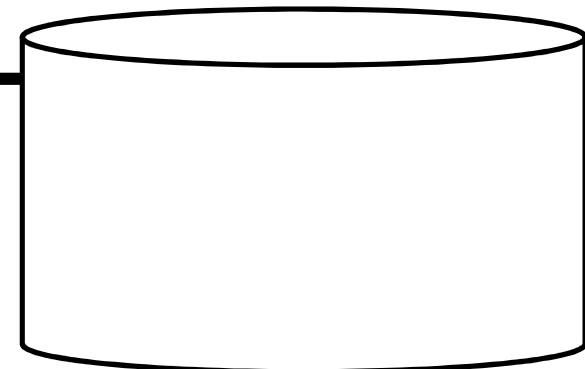
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Monitor / Teclado

Memória



HD

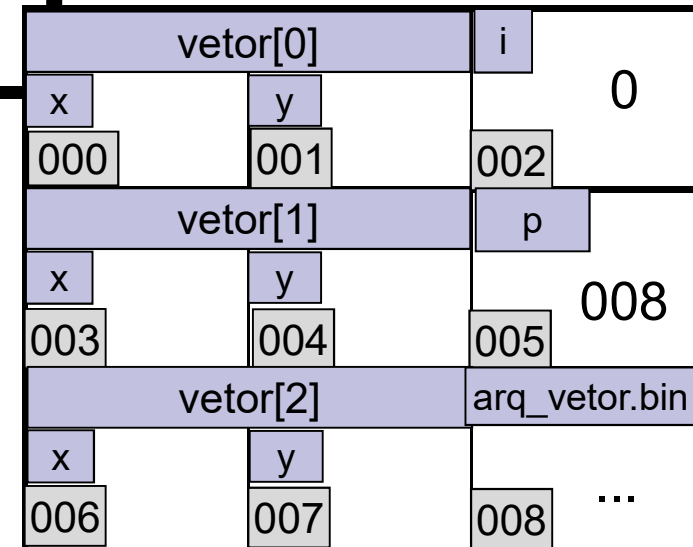


Leitura em Vetor

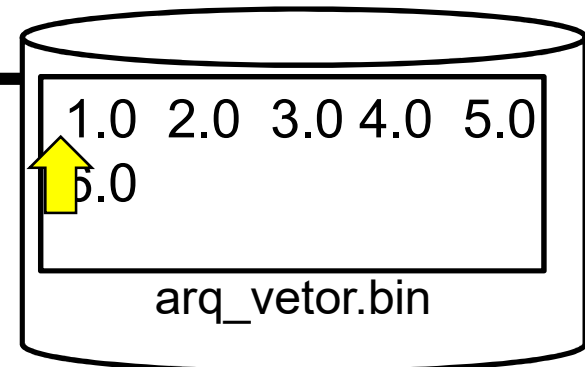
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f,%.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
            == 1) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

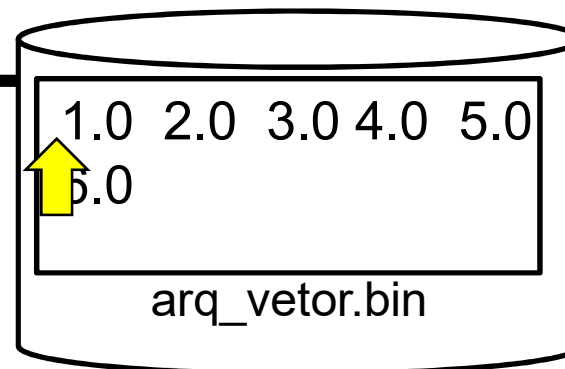
$p == \text{NULL}$
 $= 0$

Monitor / Teclado

Memória

vetor[0]			i
x	y		0
000	001	002	
vetor[1]			p
x	y		008
003	004	005	
vetor[2]			arq_vetor.bin
x	y		...
006	007	008	

HD

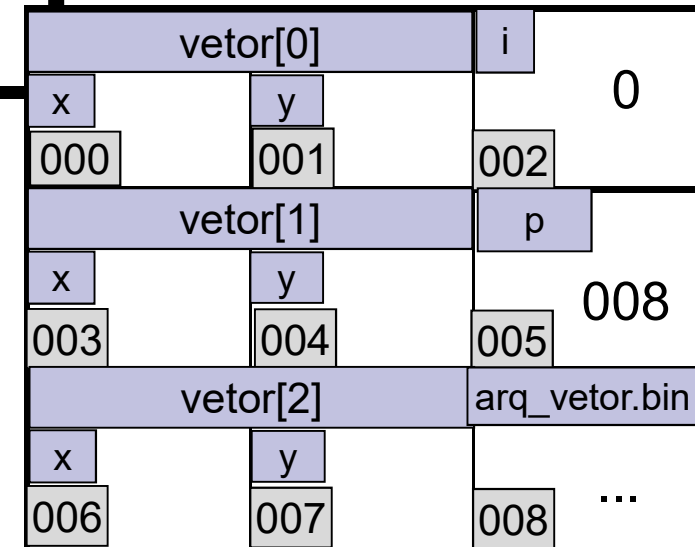


Leitura em Vetor

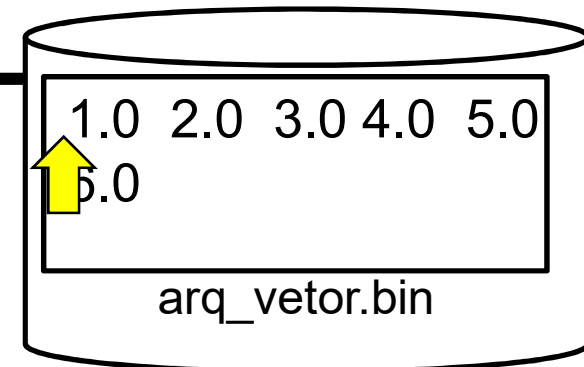
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Monitor / Teclado

Memória



HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Handwritten annotations:

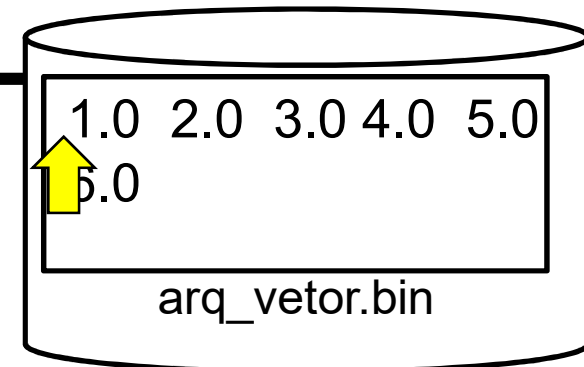
- A blue arrow points to the `for` loop.
- A red circle highlights the condition `i < 3`.
- Red text $0 < 3 = 1$ with an arrow pointing to the condition.

Monitor / Teclado

Memória

vetor[0]			i
x	y		0
000	001	002	
vetor[1]			p
x	y		008
003	004	005	
vetor[2]			arq_vetor.bin
x	y		...
006	007	008	

HD

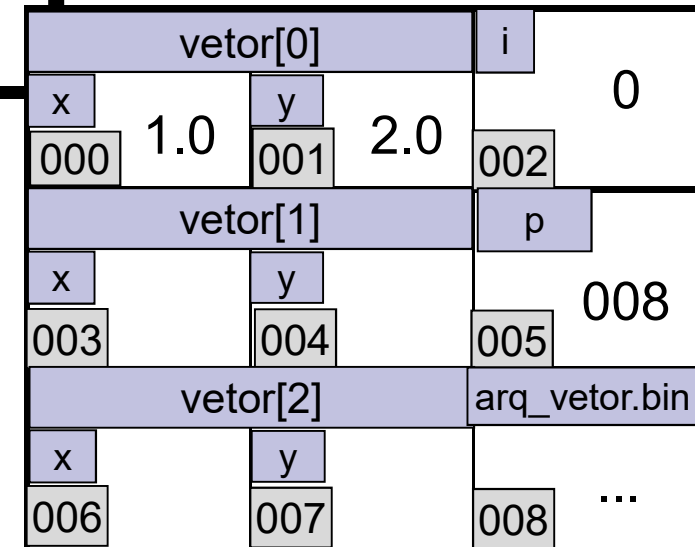


Leitura em Vetor

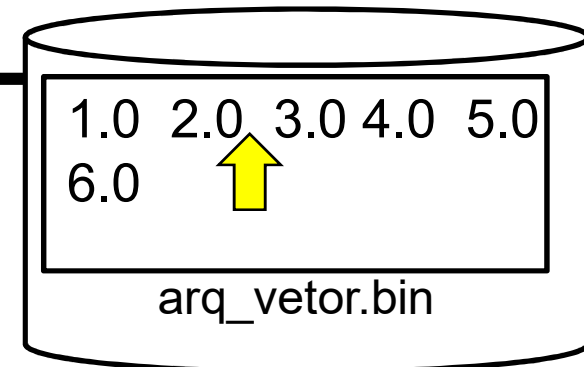
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f,%.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
== 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

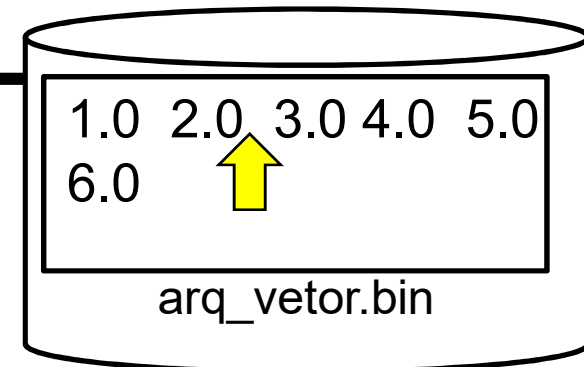
Monitor / Teclado

(1.0, 2.0)

Memória

vetor[0]		i
x	y	0
000 1.0	001 2.0	002
vetor[1]		p
x	y	008
003	004	005
vetor[2]		arq_vetor.bin
x	y	...
006	007	008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

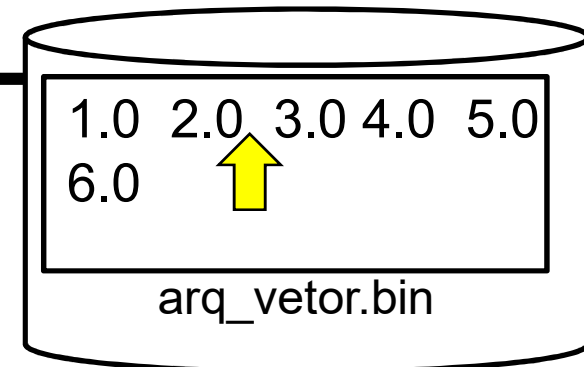
Monitor / Teclado

(1.0, 2.0)

Memória

vetor[0]		i
x	y	1
000 1.0	001 2.0	002
vetor[1]		p
x	y	008
003	004	005
vetor[2]		arq_vetor.bin
x	y	...
006	007	008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Handwritten notes:

- A red arrow points from the condition `i < 3` to the text `1 < 3 = 1`.
- A blue arrow points to the `for` loop.

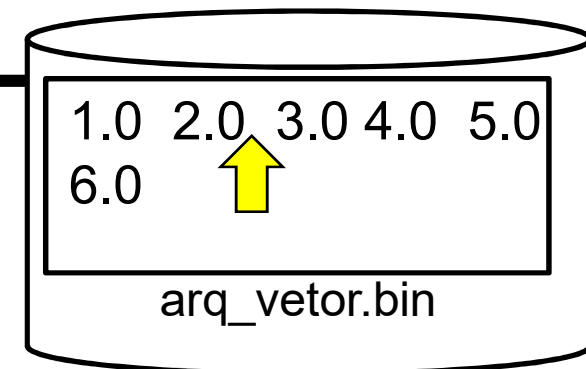
Monitor / Teclado

(1.0, 2.0)

Memória

vetor[0]		i	
x	y		1
000	001	002	
1.0	2.0		
vetor[1]		p	
x	y		008
003	004	005	
vetor[2]		arq_vetor.bin	
x	y		...
006	007	008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            → if (fread(&vetor[i], sizeof(Ponto), 1, p)
               == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

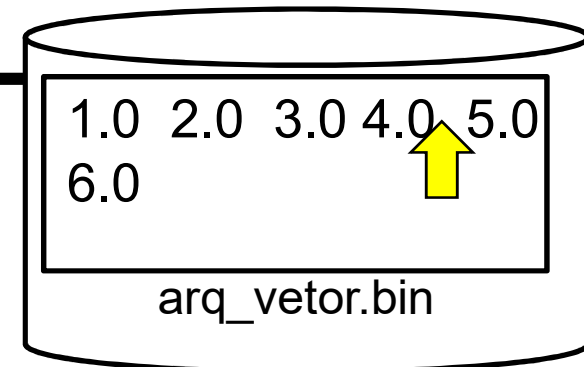
Monitor / Teclado

(1.0, 2.0)

Memória

vetor[0]		i
x	y	1
000 1.0	001 2.0	002
vetor[1]		p
x	y	008
003 3.0	004 4.0	005
vetor[2]		arq_vetor.bin
x	y	...
006	007	008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
== 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

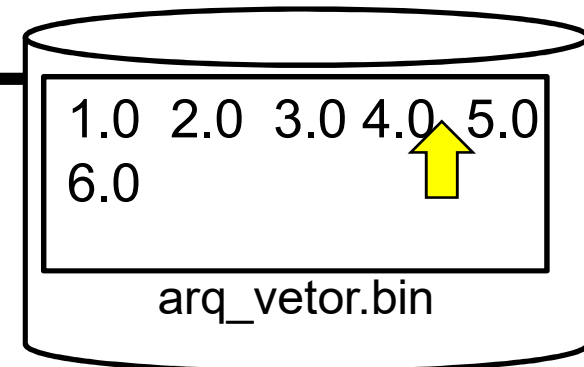
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)

Memória

vetor[0]		i
x	y	1
000 1.0	001 2.0	002
vetor[1]		p
x	y	008
003 3.0	004 4.0	005
vetor[2]		arq_vetor.bin
x	y	...
006	007	008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

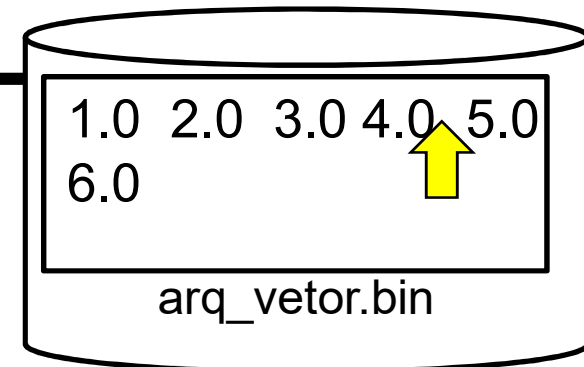
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)

Memória

vetor[0]		i
x	y	2
000 1.0	001 2.0	002
vetor[1]		p
x	y	008
003 3.0	004 4.0	005
vetor[2]		arq_vetor.bin
x	y	...
006	007	008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

2 < 3
= 1

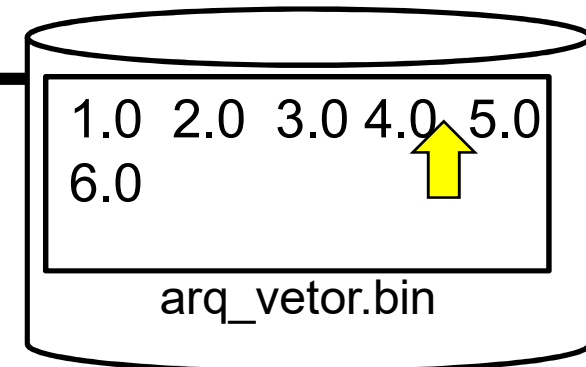
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)

Memória

vetor[0]		i
x	y	2
000 1.0	001 2.0	002
vetor[1]		p
x	y	008
003 3.0	004 4.0	005
vetor[2]		arq_vetor.bin
x	y	...
006	007	008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

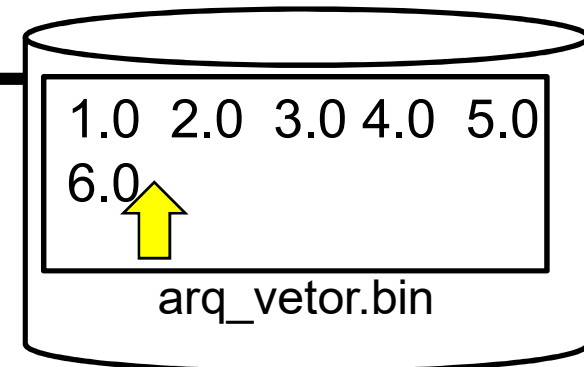
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		2
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

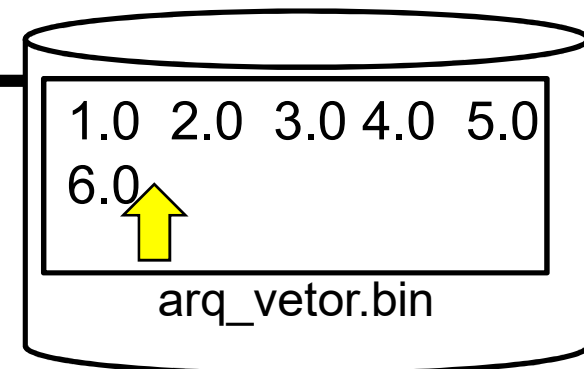
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)
(5.0, 6.0)

Memória

vetor[0]				i
x	1.0	y	2.0	2
000		001		002
vetor[1]				p
x	3.0	y	4.0	008
003		004		005
vetor[2]				arq_vetor.bin
x	5.0	y	6.0	...
006		007		008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

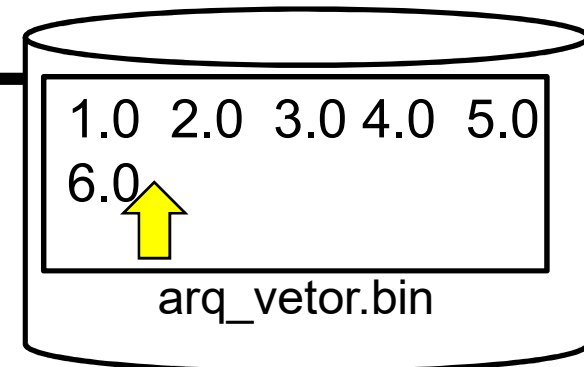
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)
(5.0, 6.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		3
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

$3 < 3$
 $= 1$

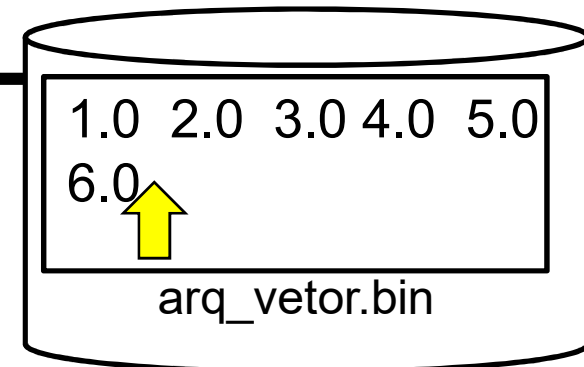
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)
(5.0, 6.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		3
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



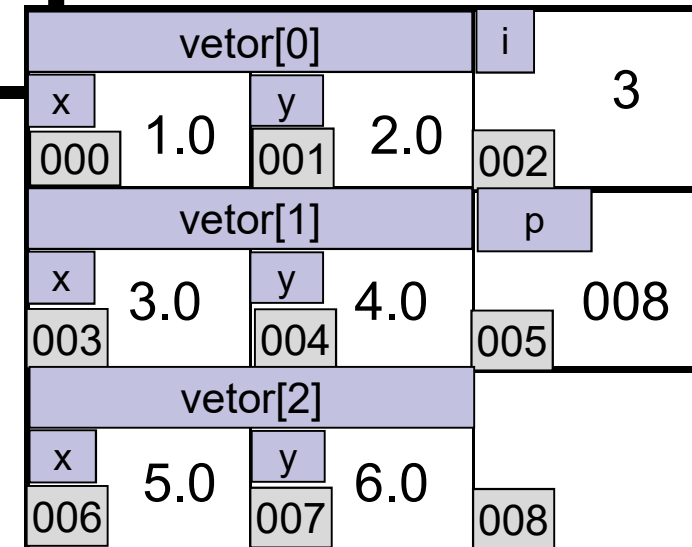
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
                == 1) {
                printf("(%.2f, %.2f)\n", vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

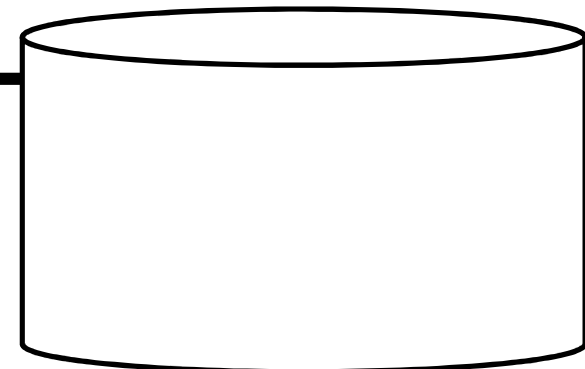
Monitor / Teclado

(1.0, 2.0)
(3.0, 4.0)
(5.0, 6.0)

Memória



HD

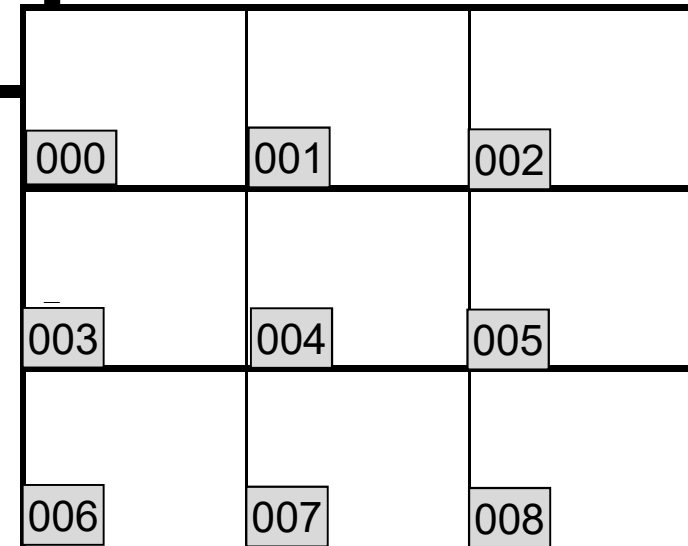


Leitura em Vetor

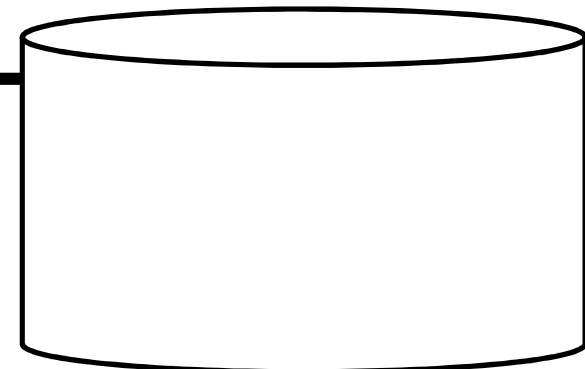
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        for (i = 0; i < 3; i++) {
            if (fread(&vetor[i], sizeof(Ponto), 1, p)
== 1) {
                printf("(%.2f,%.2f)\n", vetor[i].x,
vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

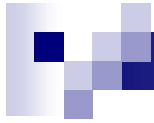
Monitor / Teclado

Memória



HD





Leitura em Vetor (versão 2.0)

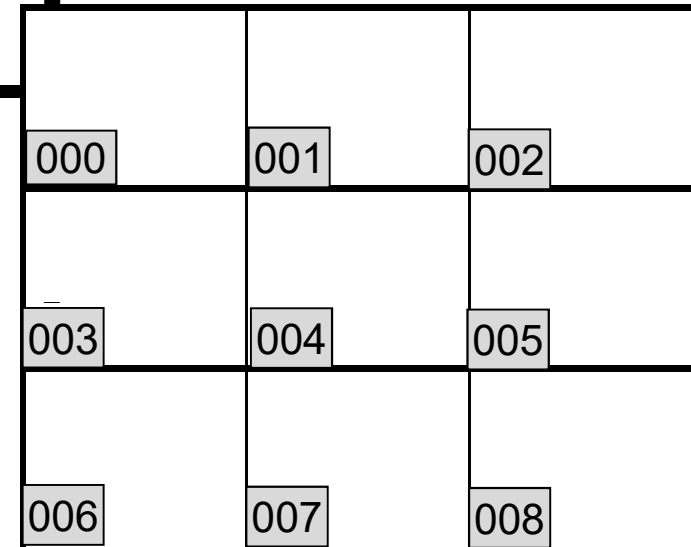
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) == 3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x, vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Leitura em Vetor

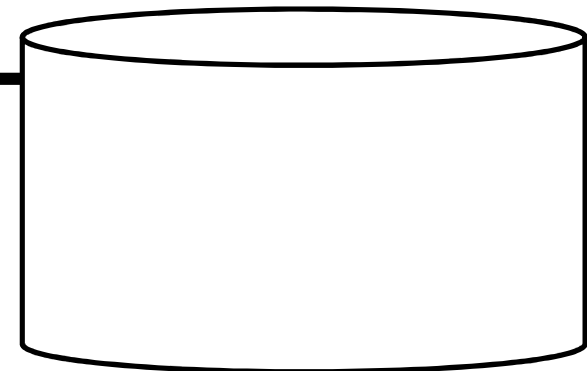
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

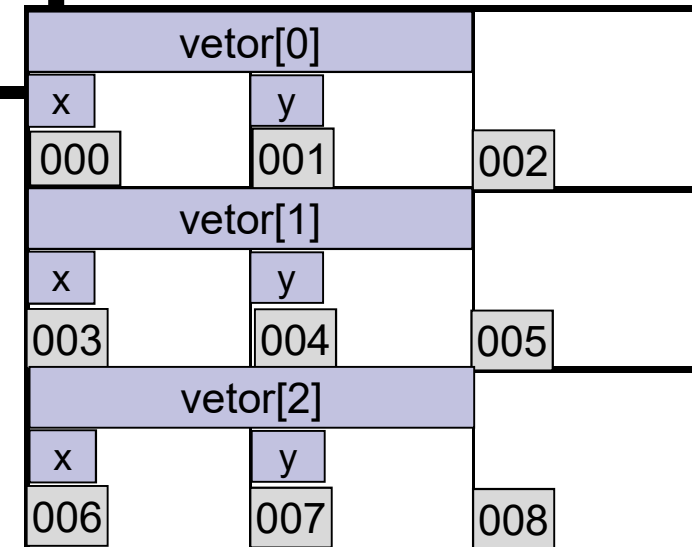


Leitura em Vetor

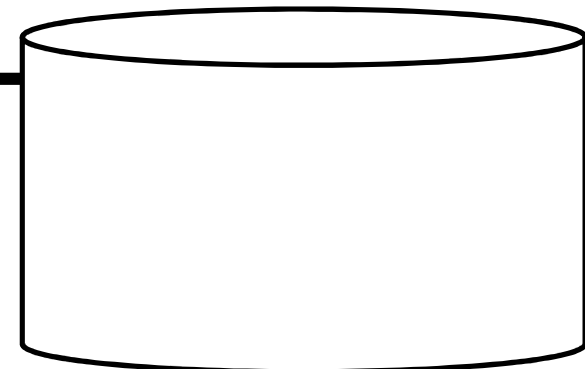
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Monitor / Teclado

Memória



HD

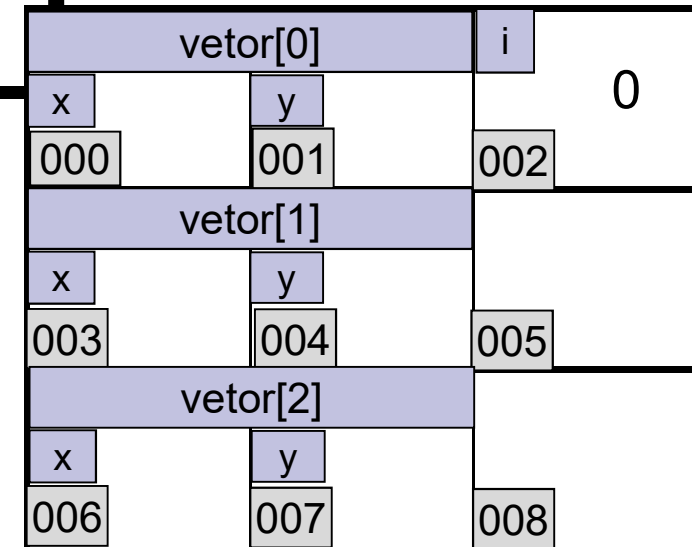


Leitura em Vetor

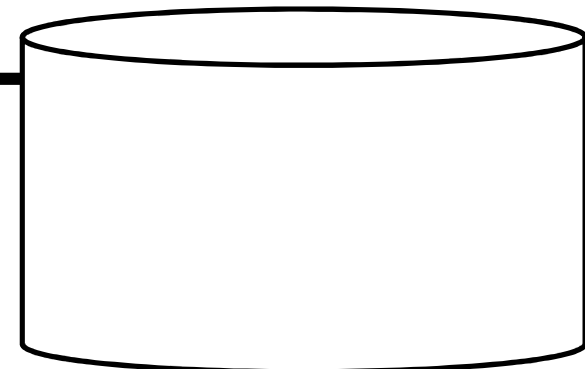
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Monitor / Teclado

Memória



HD

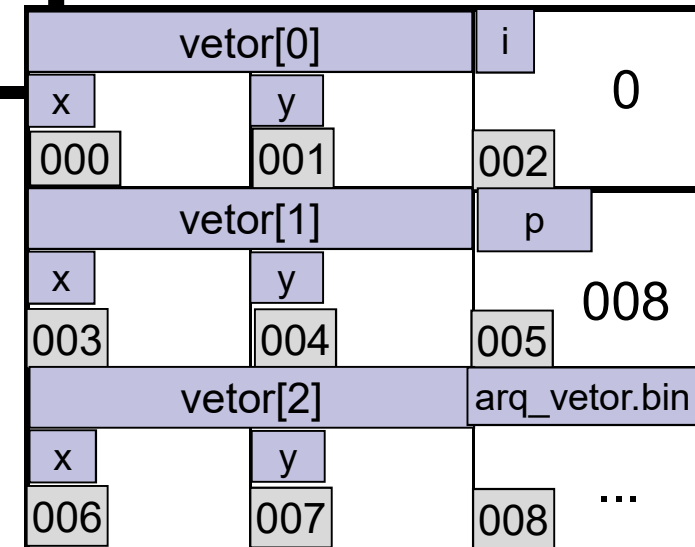


Leitura em Vetor

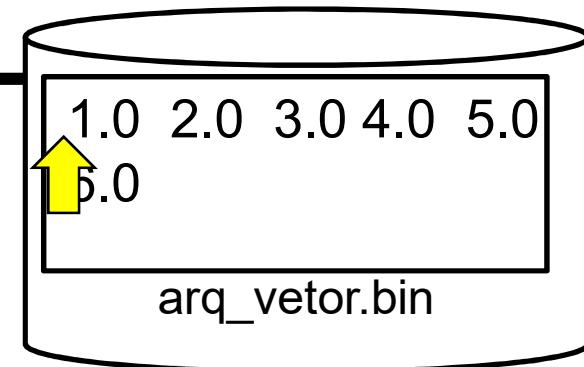
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    → FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Monitor / Teclado

Memória



HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

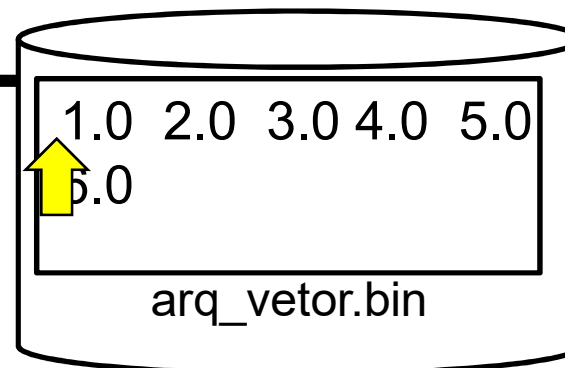
$p == NULL$
 $= 0$

Monitor / Teclado

Memória

vetor[0]			i
x	y		0
000	001	002	
vetor[1]			p
x	y		008
003	004	005	
vetor[2]			arq_vetor.bin
x	y		...
006	007	008	

HD



Leitura em Vetor

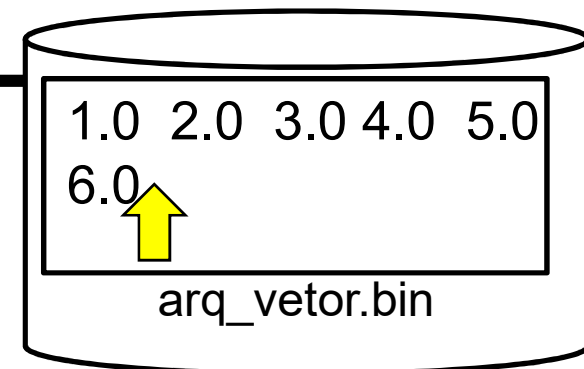
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

vetor[0]				i	
x	1.0	y	2.0		0
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

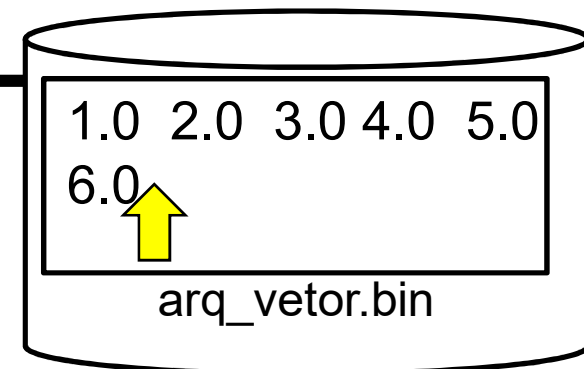
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

Monitor / Teclado

Memória

vetor[0]				i	
x	1.0	y	2.0		0
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

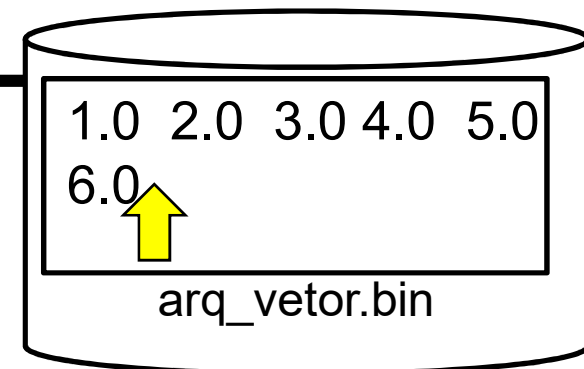
$0 < 3$
 $= 1$

Monitor / Teclado

Memória

vetor[0]				i	
x	1.0	y	2.0		0
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

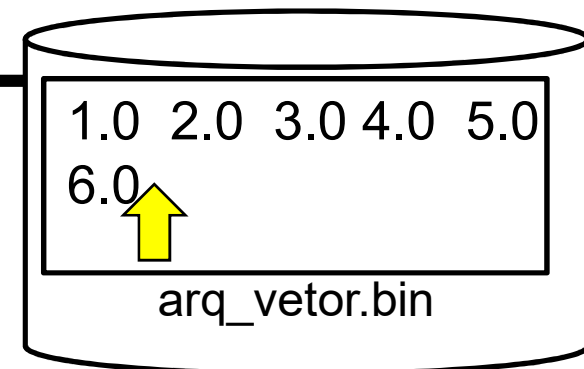
Monitor / Teclado

(1.0,2.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		0
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

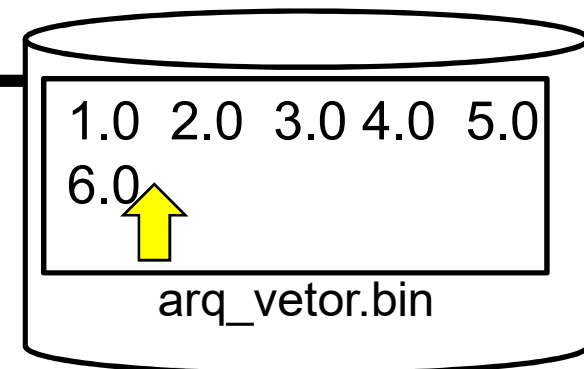
Monitor / Teclado

(1.0,2.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		1
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

$i < 3$
 $= 1$

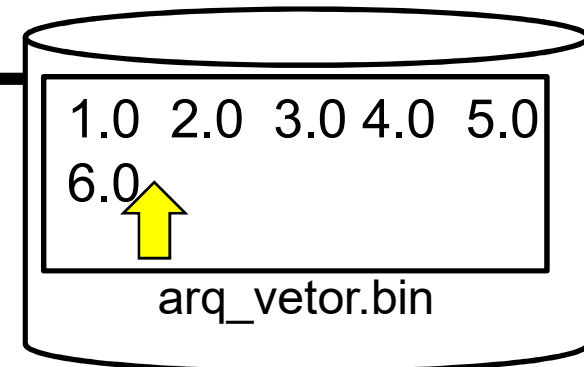
Monitor / Teclado

(1.0,2.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		1
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

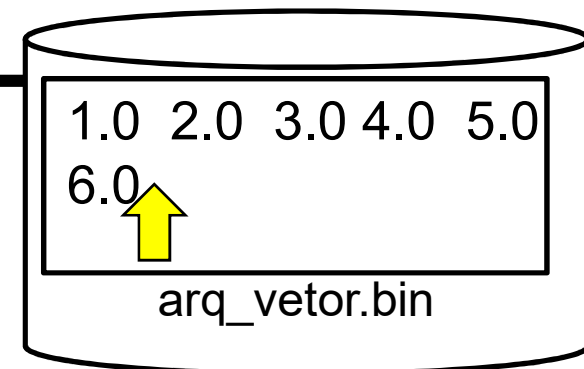
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		1
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

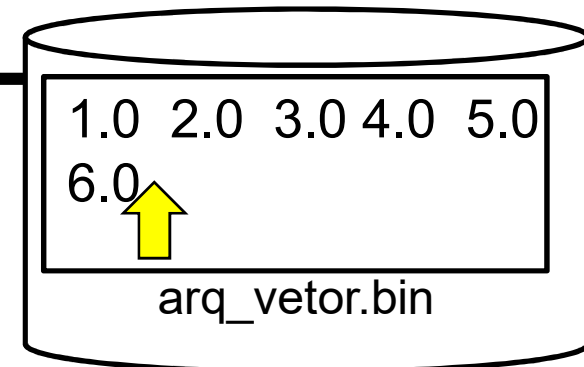
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)

Memória

vetor[0]				i
x	1.0	y	2.0	2
000		001		002
vetor[1]				p
x	3.0	y	4.0	008
003		004		005
vetor[2]				arq_vetor.bin
x	5.0	y	6.0	...
006		007		008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

2 < 3
= 1

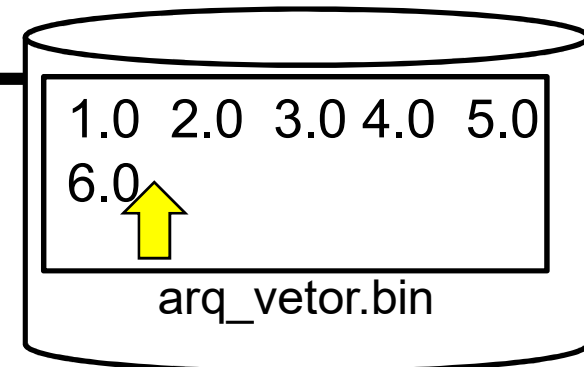
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		2
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

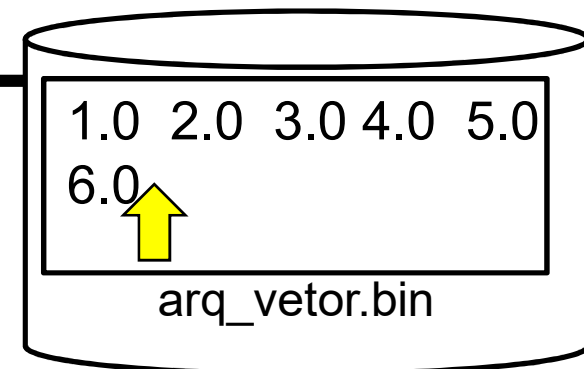
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)
(5,0, 6.0)

Memória

vetor[0]				i
x	1.0	y	2.0	2
000		001		002
vetor[1]				p
x	3.0	y	4.0	008
003		004		005
vetor[2]				arq_vetor.bin
x	5.0	y	6.0	...
006		007		008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

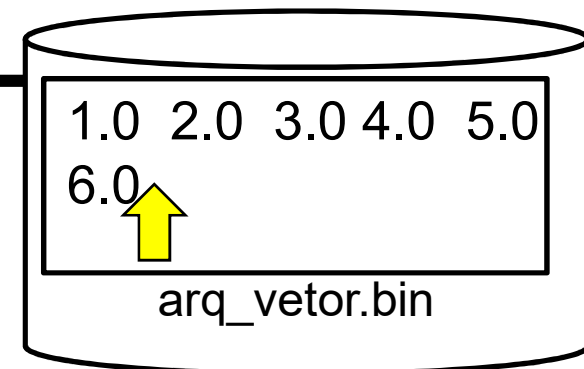
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)
(5.0, 6.0)

Memória

vetor[0]				i
x	1.0	y	2.0	3
000		001		002
vetor[1]				p
x	3.0	y	4.0	008
003		004		005
vetor[2]				arq_vetor.bin
x	5.0	y	6.0	...
006		007		008

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
        3) {
            → for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                vetor[i].y);
            }
        }
        fclose(p);
        return 0;
    }
}
```

3 < 3
= 0

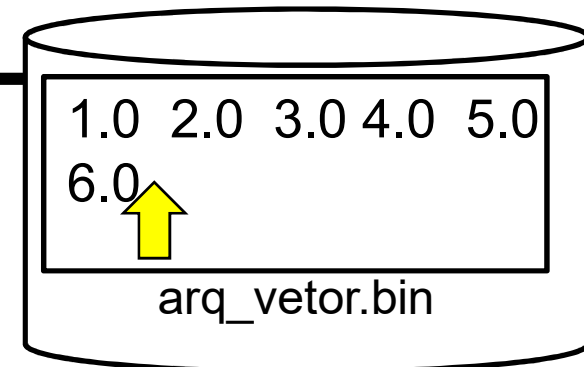
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)
(5,0, 6.0)

Memória

vetor[0]				i	
x	1.0	y	2.0		3
000		001		002	
vetor[1]				p	
x	3.0	y	4.0		008
003		004		005	
vetor[2]				arq_vetor.bin	
x	5.0	y	6.0		...
006		007		008	

HD



Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

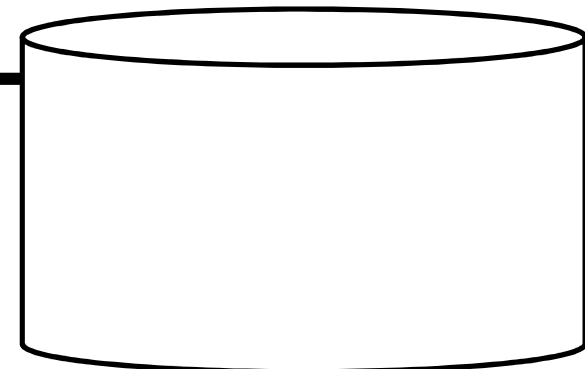
Monitor / Teclado

(1.0,2.0)
(3.0, 4.0)
(5,0, 6.0)

Memória

vetor[0]				i
x	1.0	y	2.0	3
000		001		002
vetor[1]				p
x	3.0	y	4.0	008
003		004		005
vetor[2]				
x	5.0	y	6.0	
006		007		008

HD

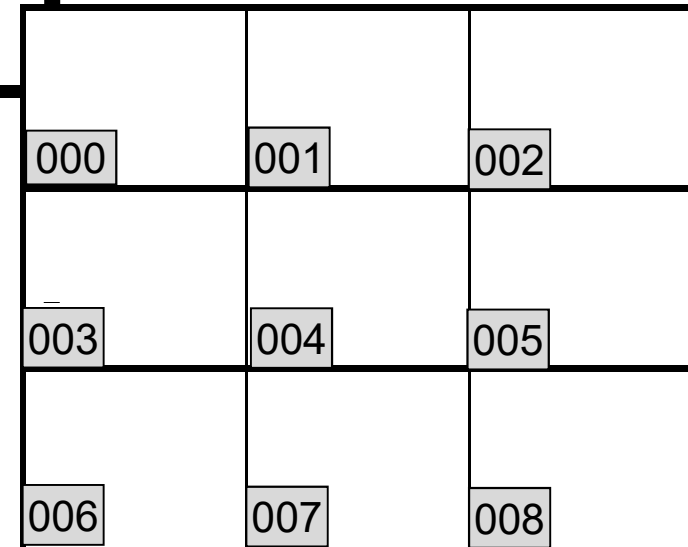


Leitura em Vetor

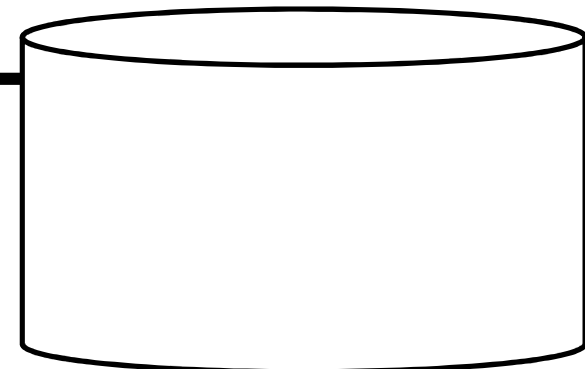
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto vetor[3];
    int i = 0;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        if (fread(vetor, sizeof(Ponto), 3, p) ==
            3) {
            for (i = 0; i < 3; i++) {
                printf("(%.2f,%.2f)\n",vetor[i].x,
                    vetor[i].y);
            }
        }
    }
    fclose(p);
    return 0;
}
```

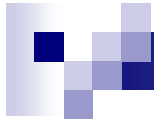
Monitor / Teclado

Memória



HD





Leitura em Vetor (versão 3.0)

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto, sizeof(pto), 1, p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Leitura em Vetor

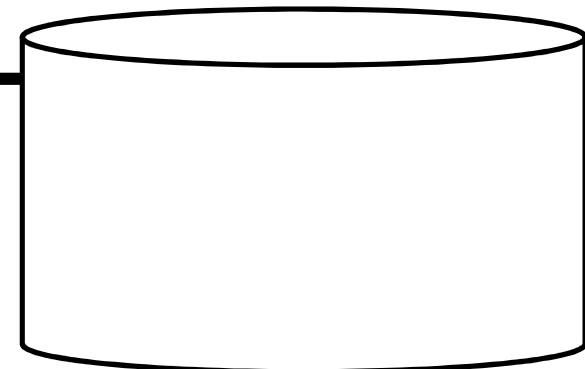
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD

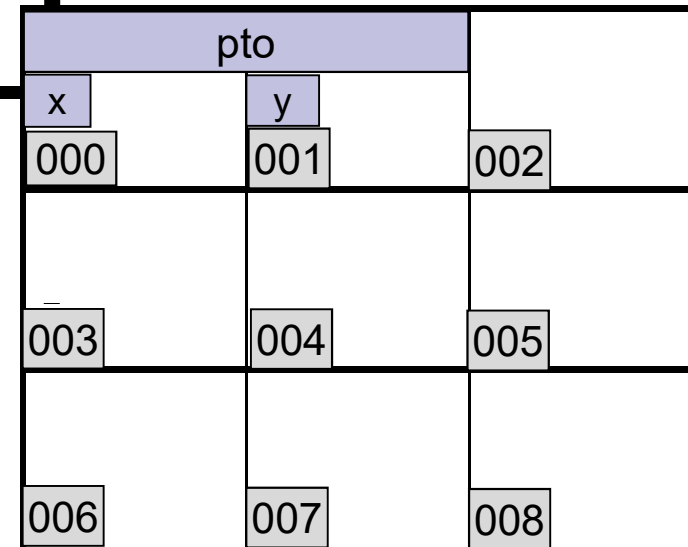


Leitura em Vetor

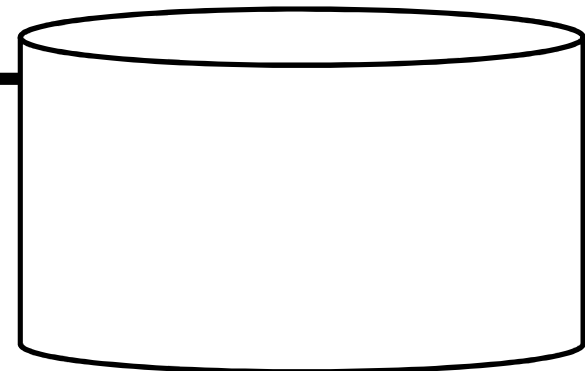
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

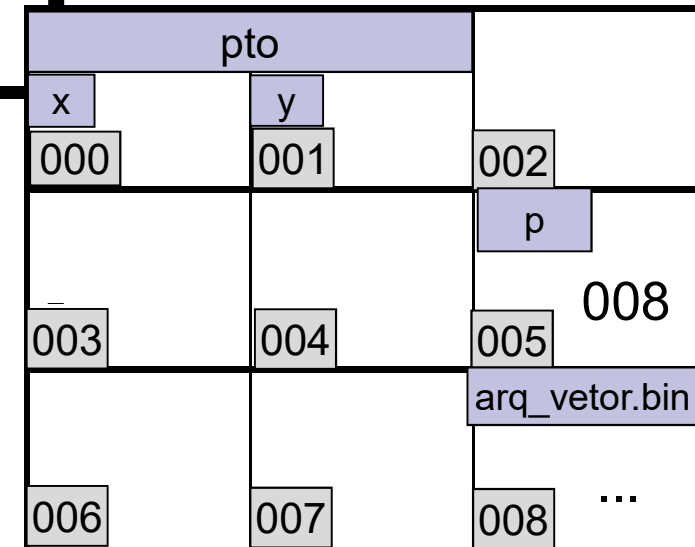


Leitura em Vetor

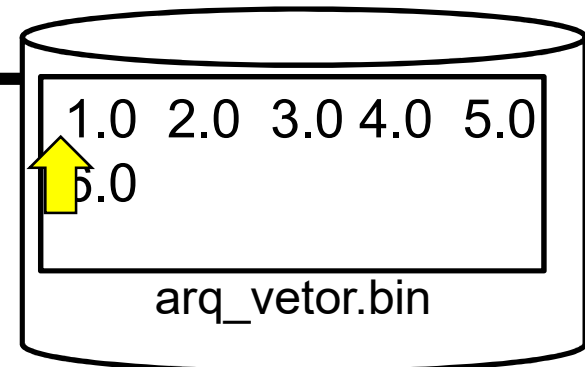
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

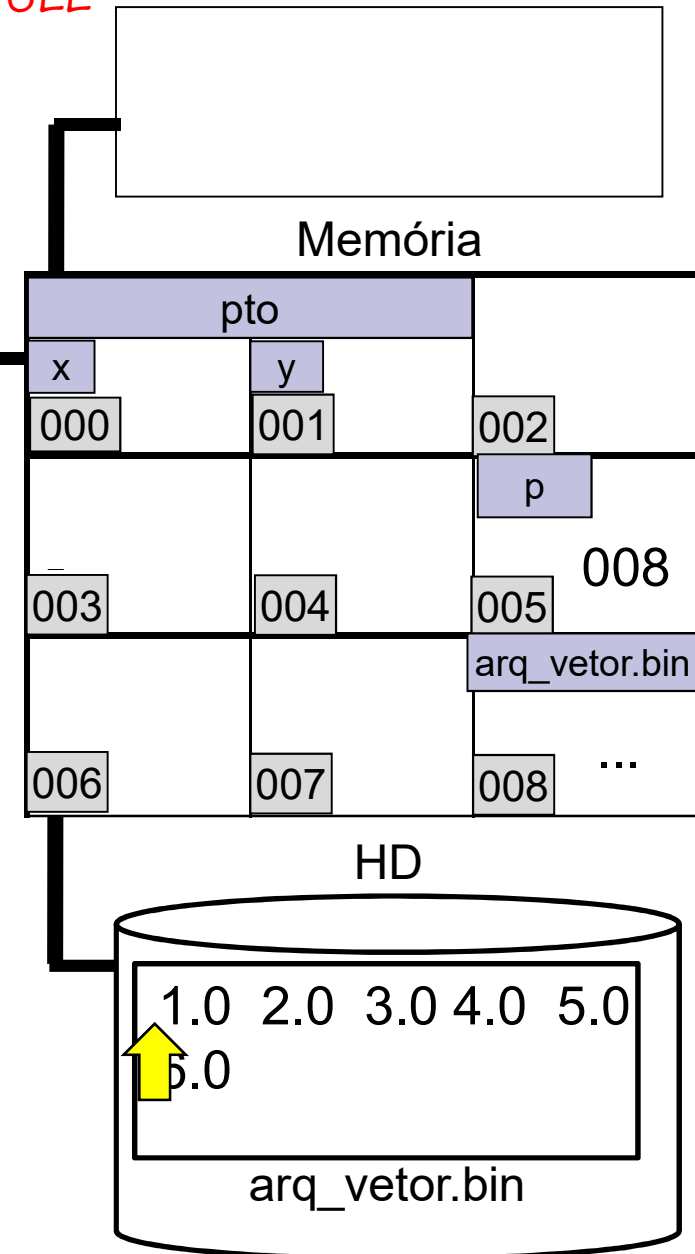




Monitor / Teclado

Memória

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```



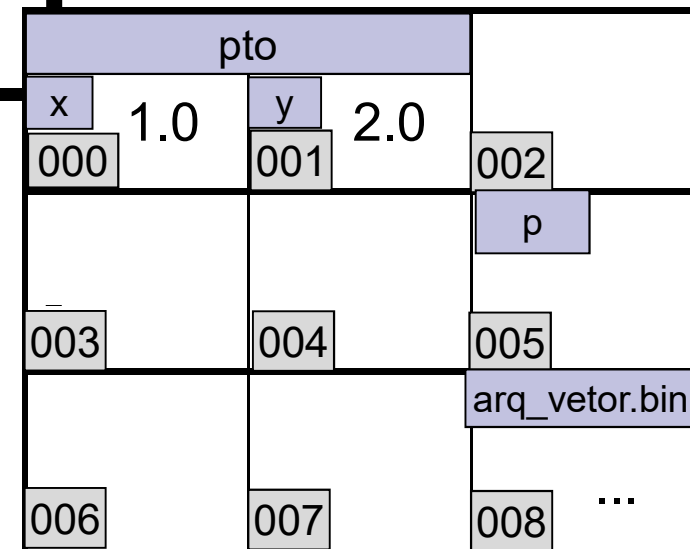
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → while (fread(&pto, sizeof(ponto), 1, p)
           == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
                pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

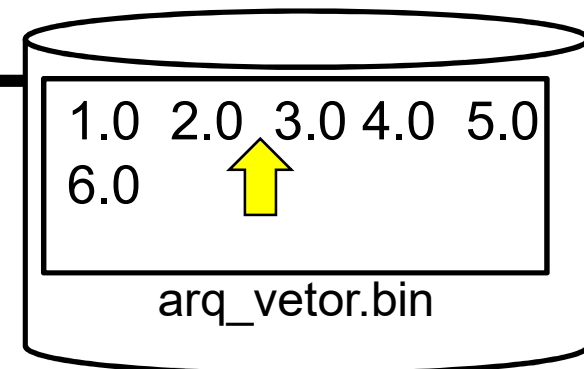
*A leitura vai ser bem sucedida.
fread() salva 1.0 em pto.x e
2.0 em pto.y e move a seta
amarela à frente.*

Monitor / Teclado

Memória



HD



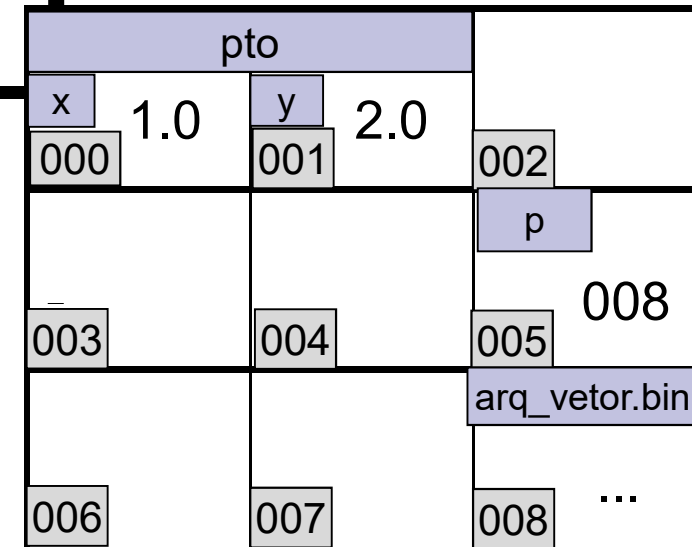
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

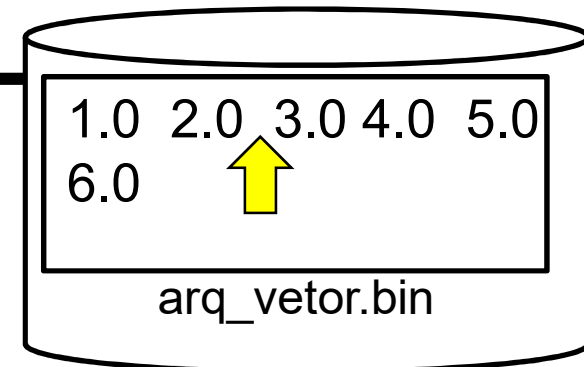
Monitor / Teclado

(1.0,2.0)

Memória



HD



Leitura em Vetor

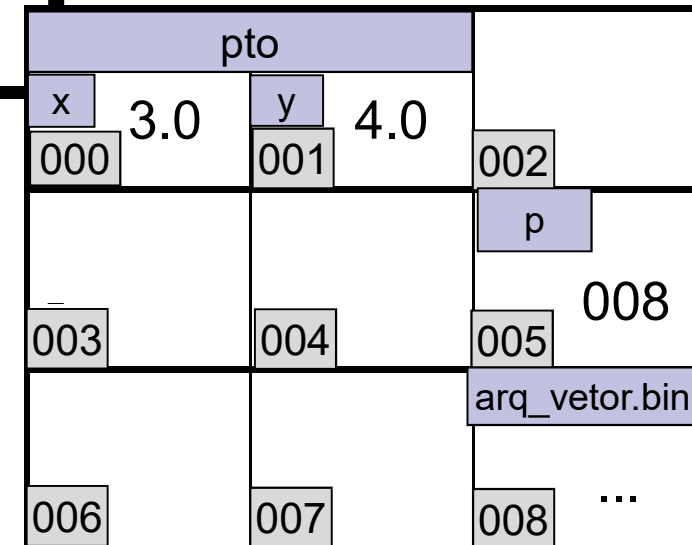
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → while (fread(&pto, sizeof(ponto), 1, p)
            == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
                pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 3.0 em pto.x e
4.0 em pto.y e move a seta
amarela à frente.*

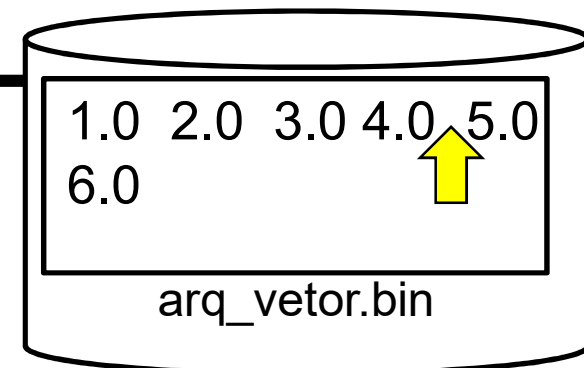
Monitor / Teclado

(1.0,2.0)

Memória



HD



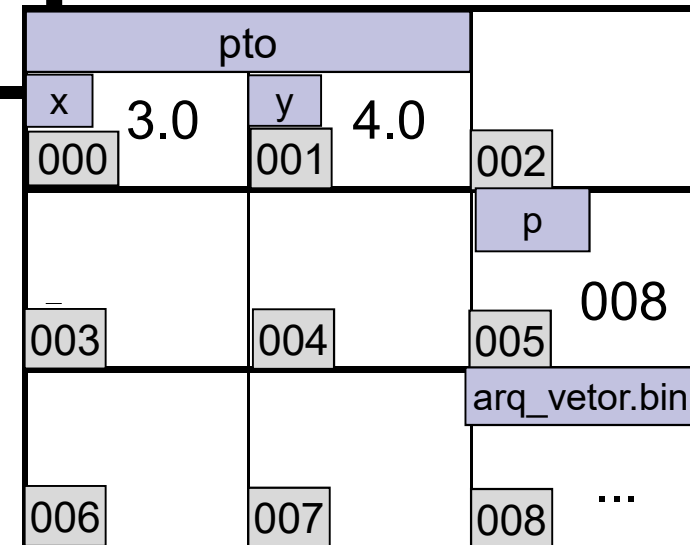
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

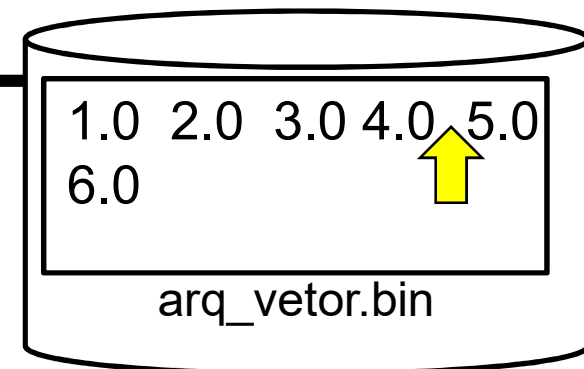
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)

Memória



HD



Leitura em Vetor

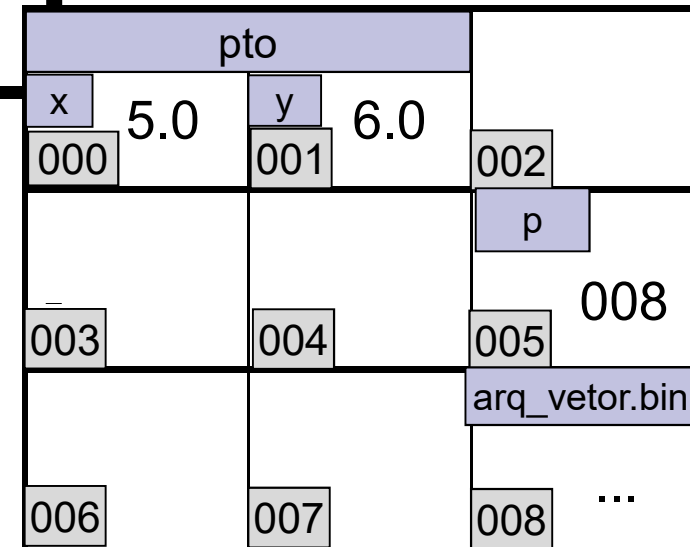
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto, sizeof(ponto), 1, p)
            == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
                pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 5.0 em pto.x e
6.0 em pto.y e move a seta
amarela à frente.*

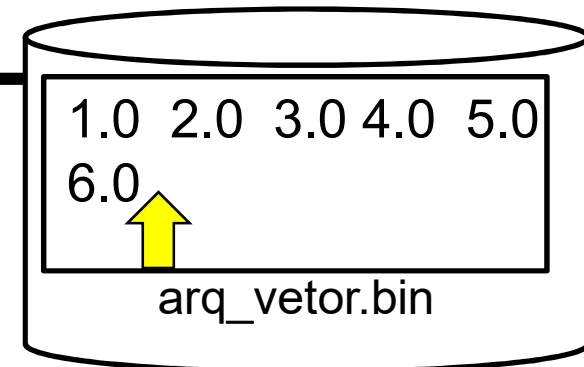
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)

Memória



HD



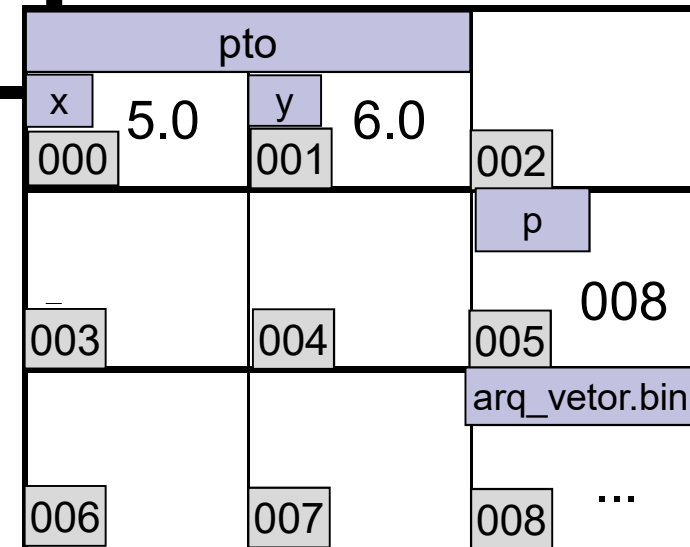
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

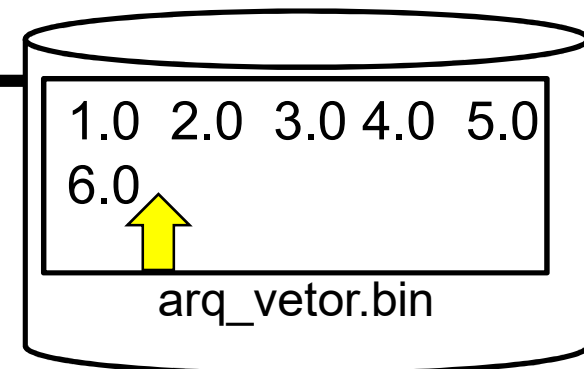
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5,0,6.0)

Memória



HD



Leitura em Vetor

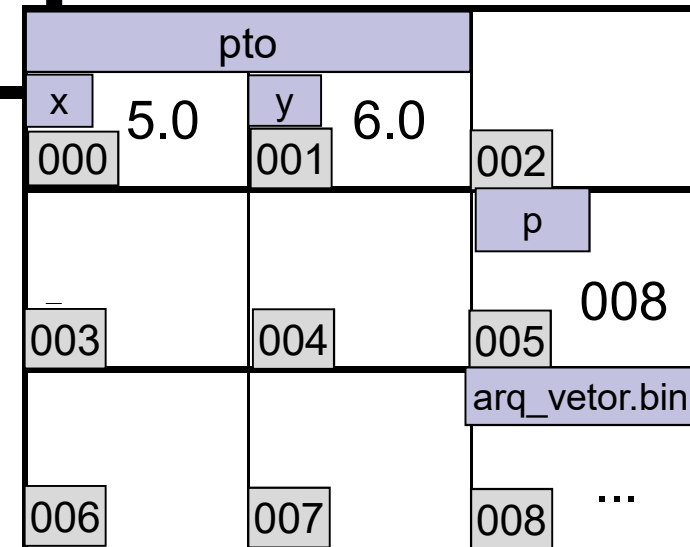
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → while (fread(&pto, sizeof(ponto), 1, p)
           == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
                pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

A leitura vai ser mal sucedida. fread() atingiu o final de arquivo e retorna 0. Como 0 é diferente de 1, a comparação dá falso e o laço termina.

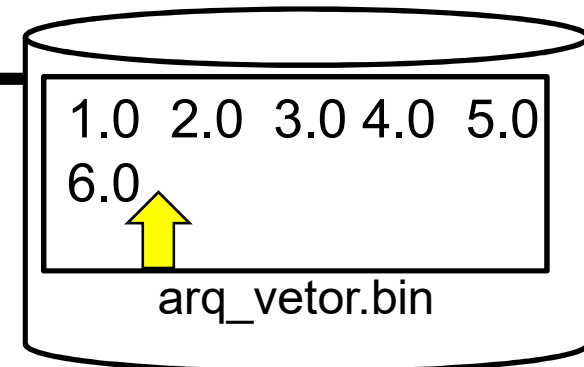
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



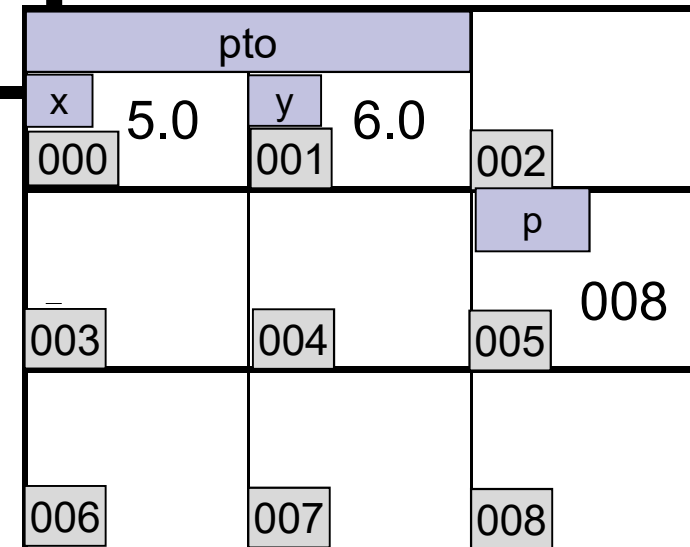
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

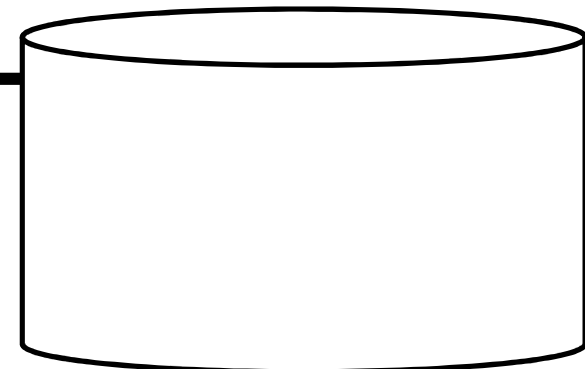
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



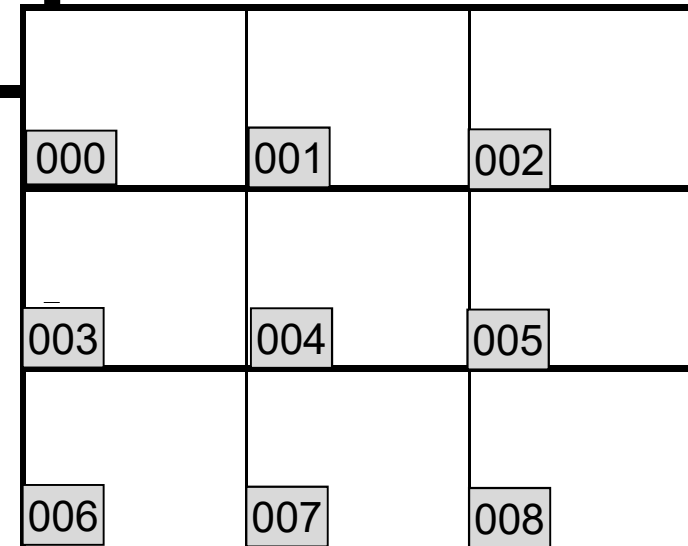
Leitura em Vetor

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(ppto),1,p)
        == 1) {
            printf("(%.2f,%.2f)\n", pto.x,
            pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

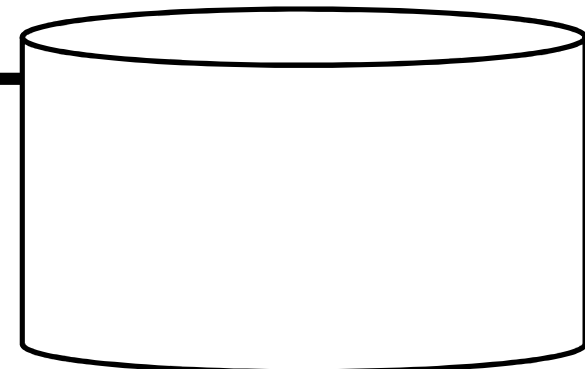
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD





rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

rewind()

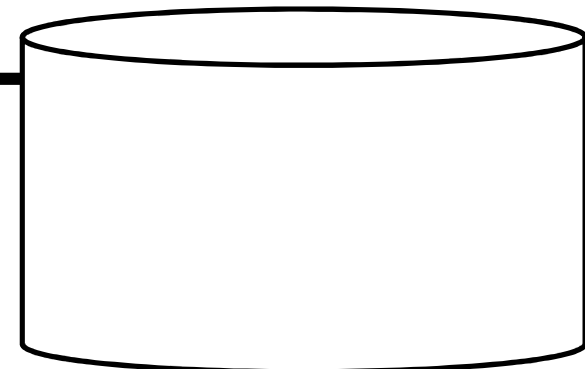
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD

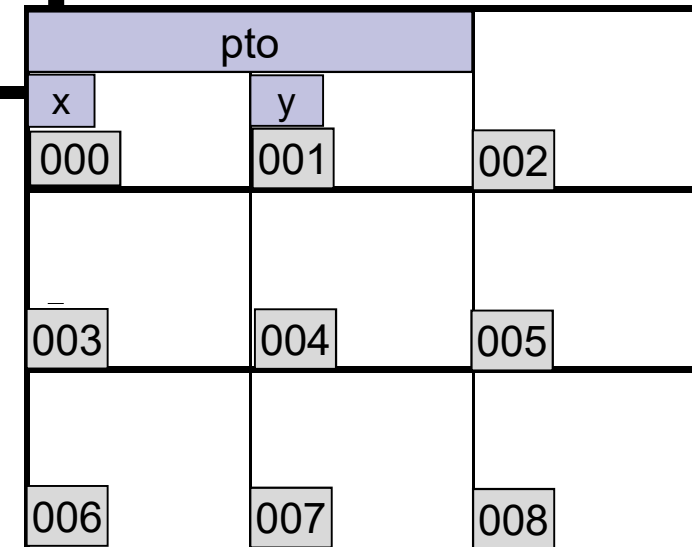


rewind()

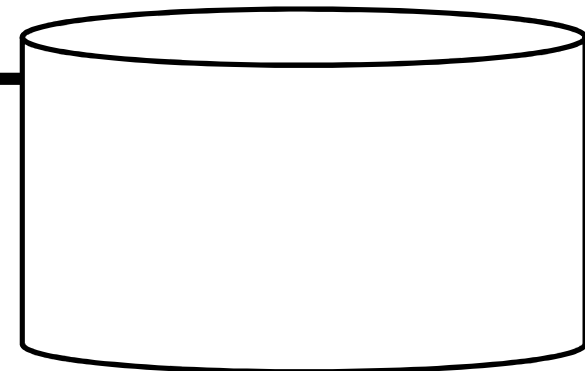
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

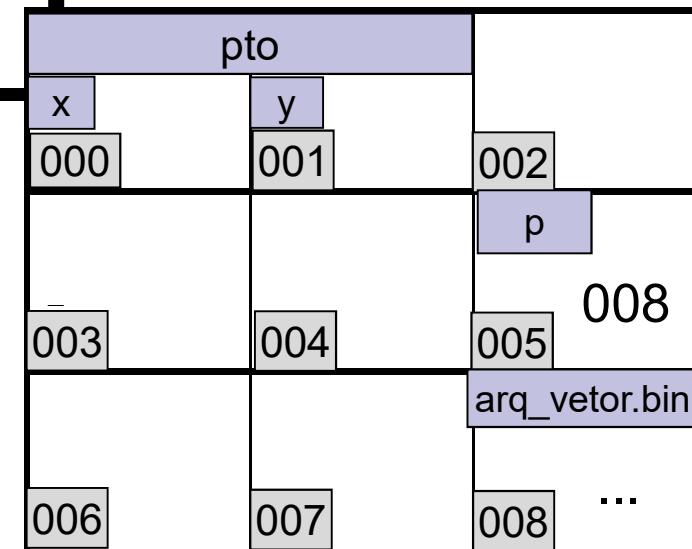


rewind()

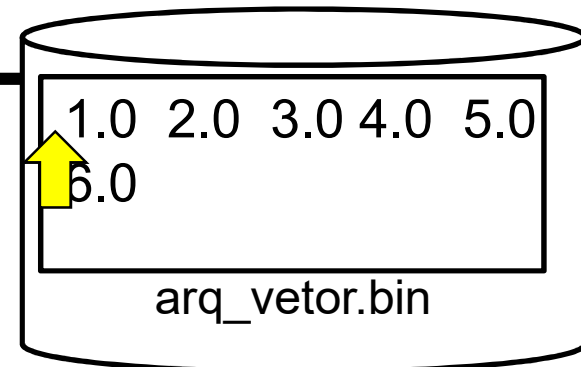
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



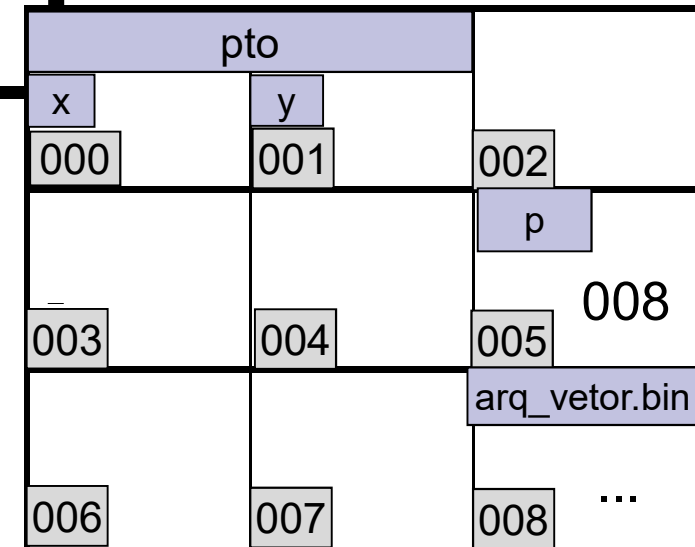
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

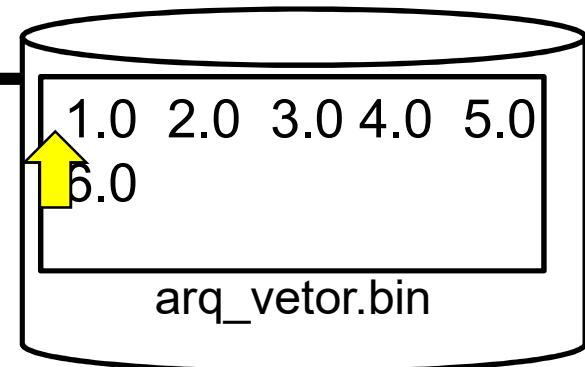
*p == NULL
= 0*

Monitor / Teclado

Memória



HD



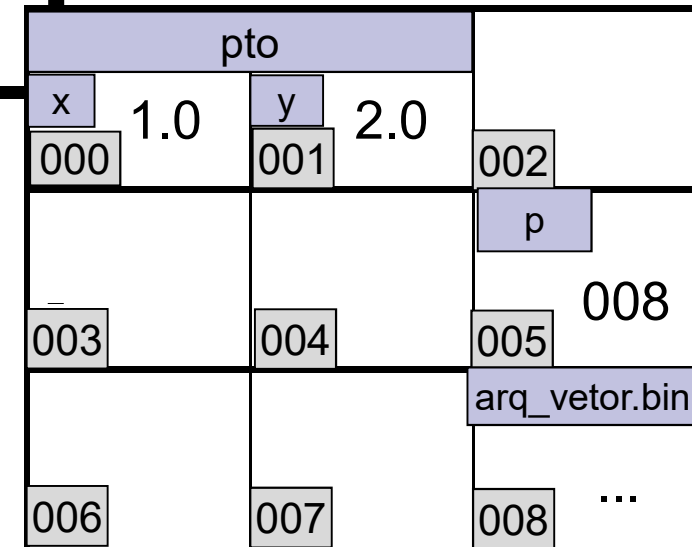
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

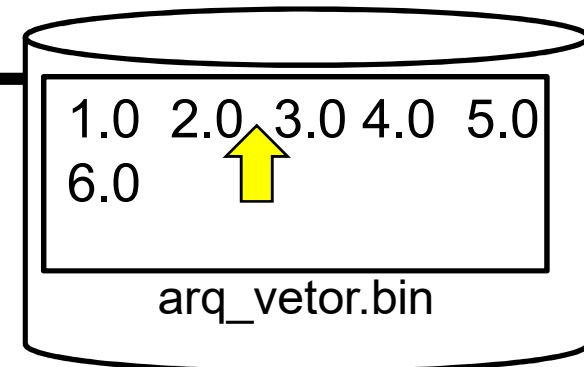
*A leitura vai ser bem sucedida.
fread() salva 1.0 em pto.x e
2.0 em pto.y e move a seta
amarela à frente.*

Monitor / Teclado

Memória



HD



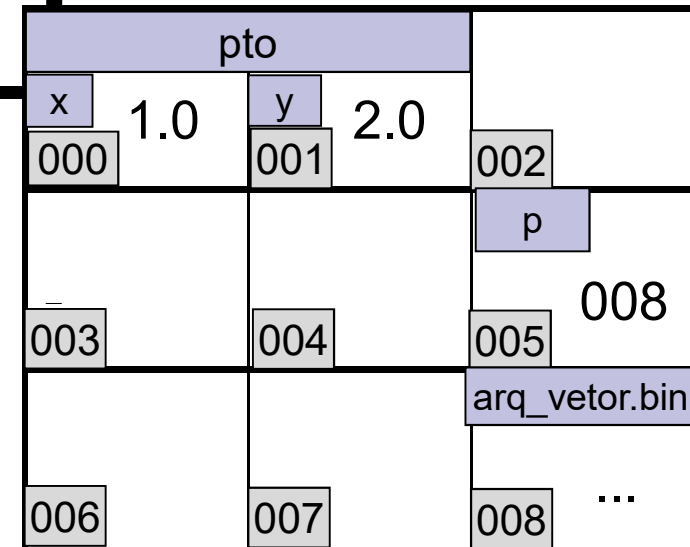
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

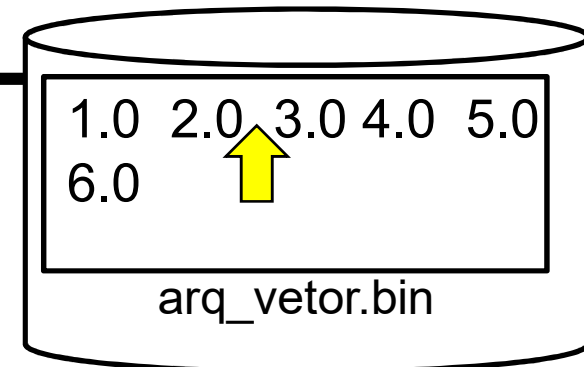
Monitor / Teclado

(1.0,2.0)

Memória



HD



rewind()

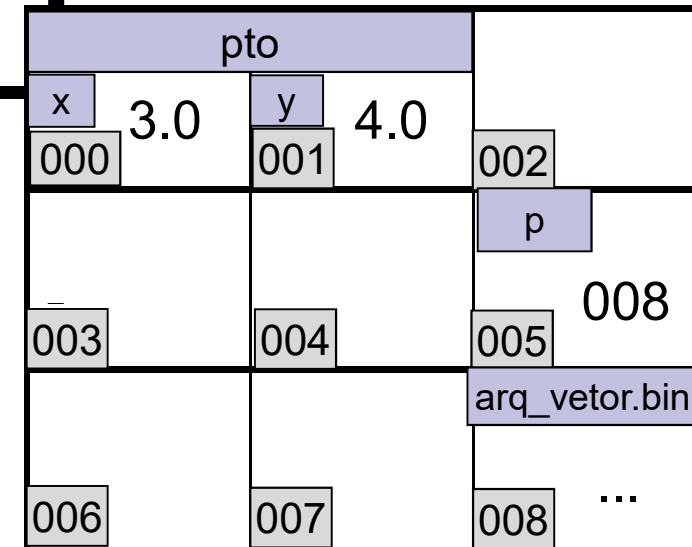
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 3.0 em pto.x e
4.0 em pto.y e move a seta
amarela à frente.*

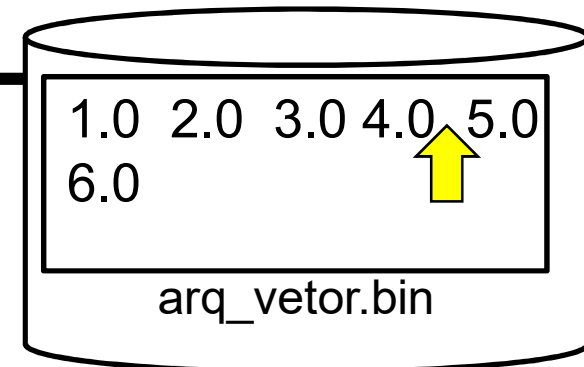
Monitor / Teclado

(1.0,2.0)

Memória



HD



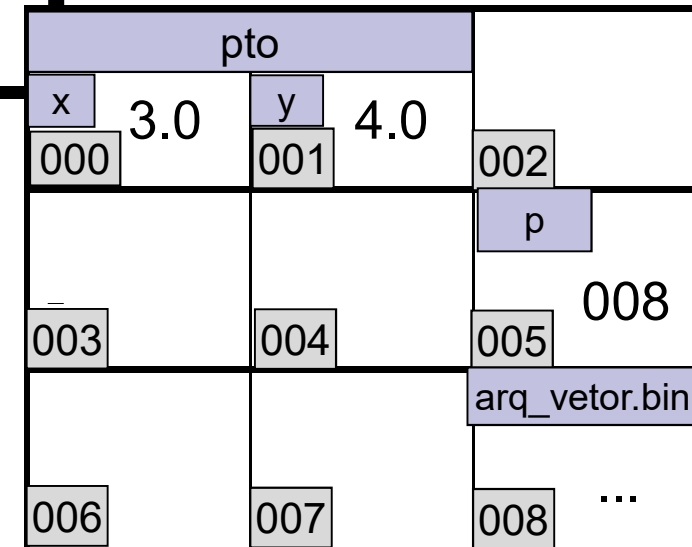
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

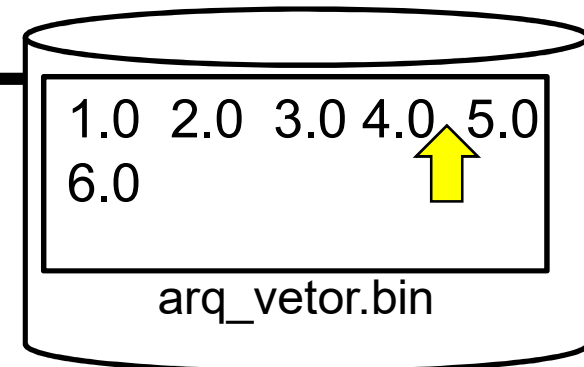
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)

Memória



HD



rewind()

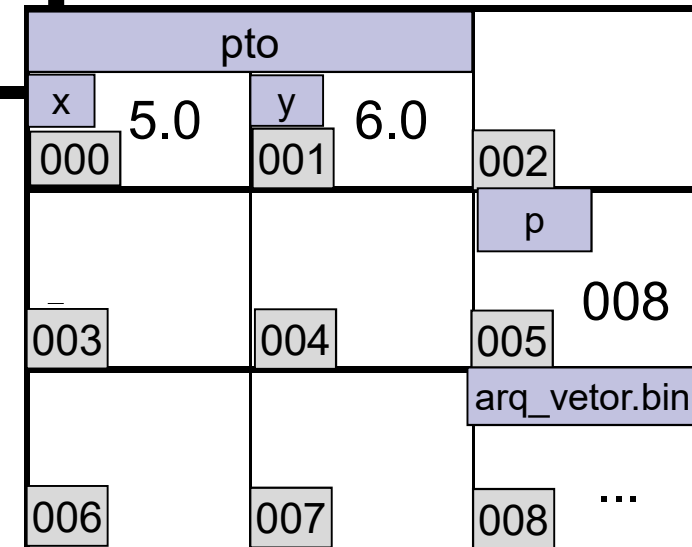
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto, sizeof(pto), 1, p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto, sizeof(pto), 1, p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 5.0 em pto.x e
6.0 em pto.y e move a seta
amarela à frente.*

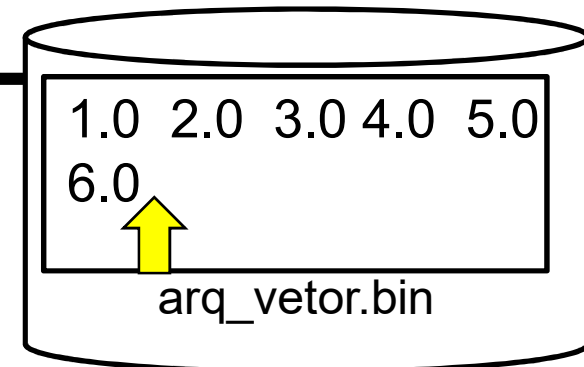
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)

Memória



HD



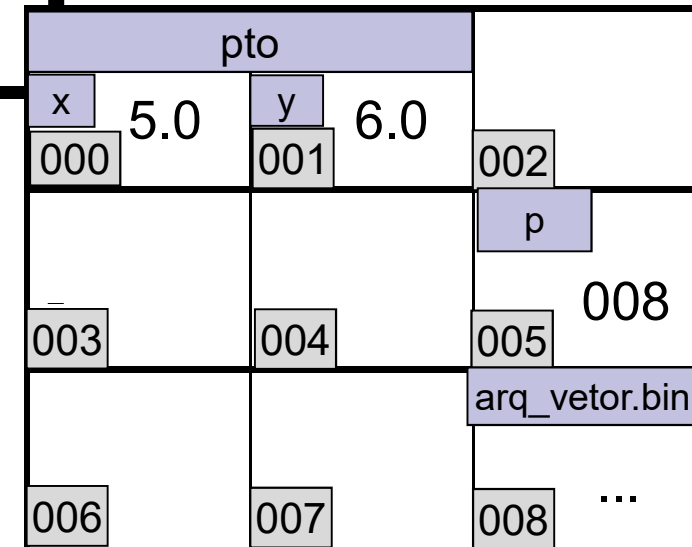
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

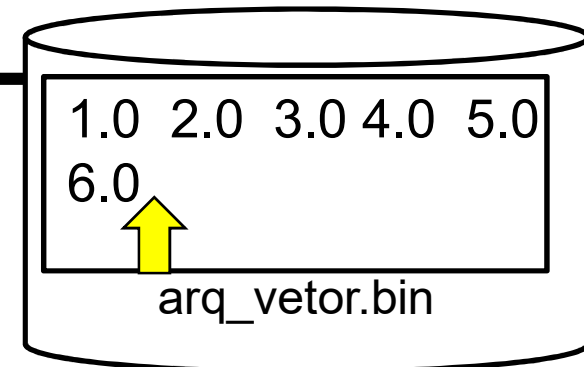
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



rewind()

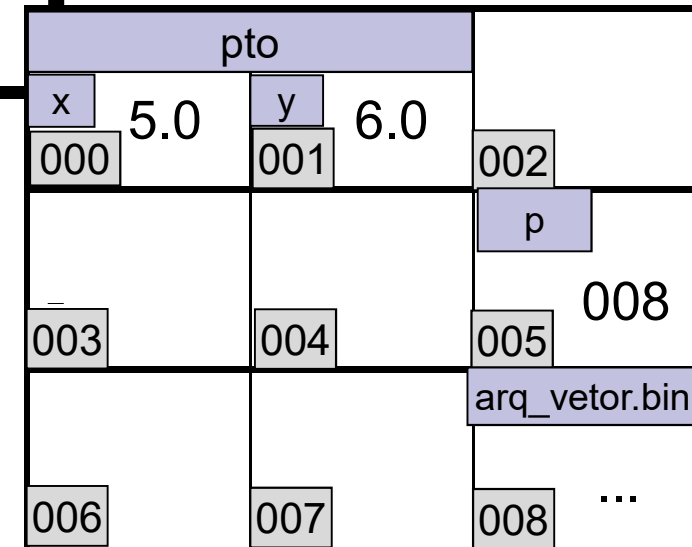
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser mal sucedida.
fread() retorna 0, pois atingiu
o final do arquivo.*

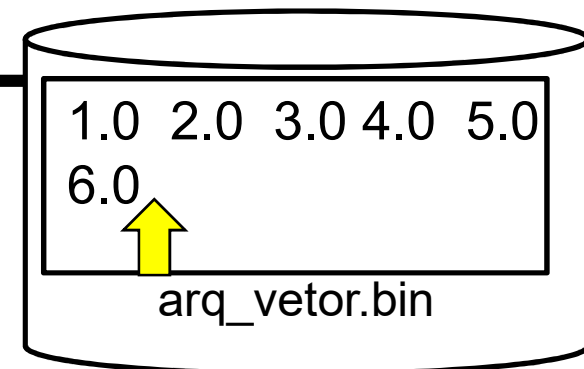
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



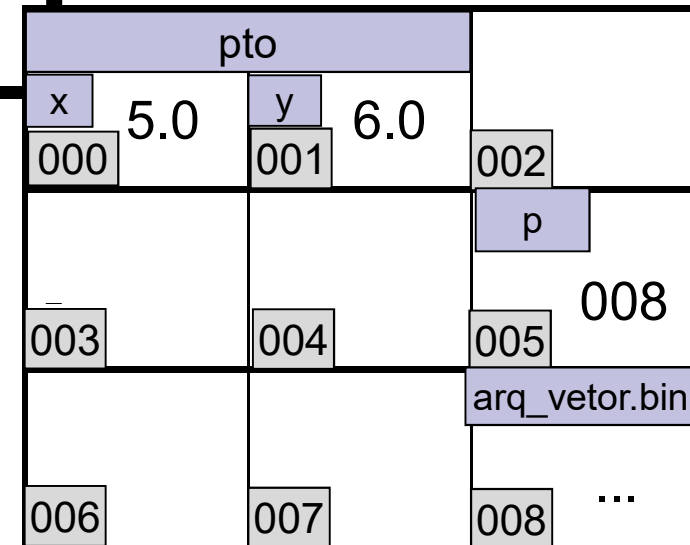
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        ↗ rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

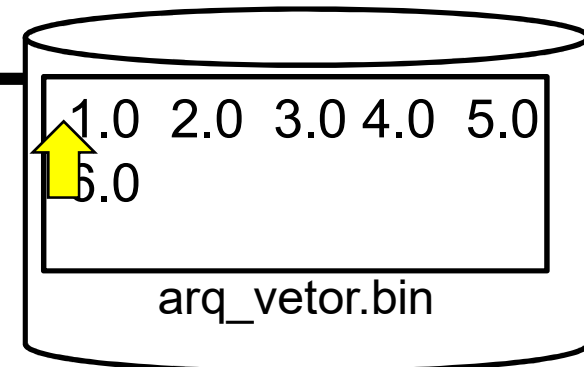
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



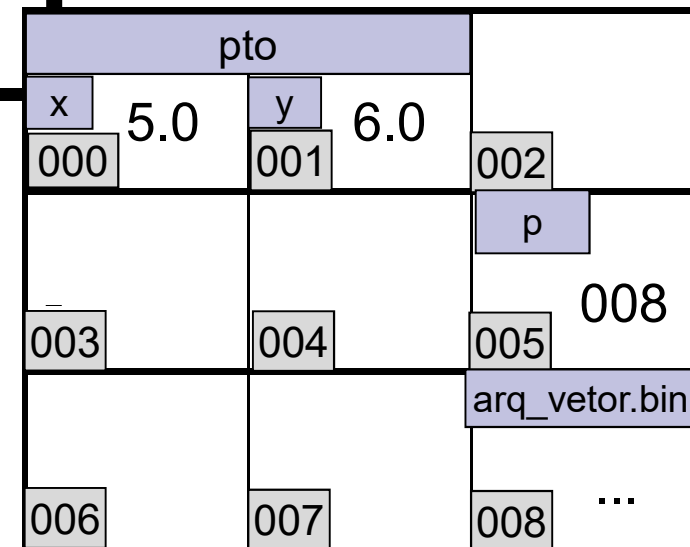
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

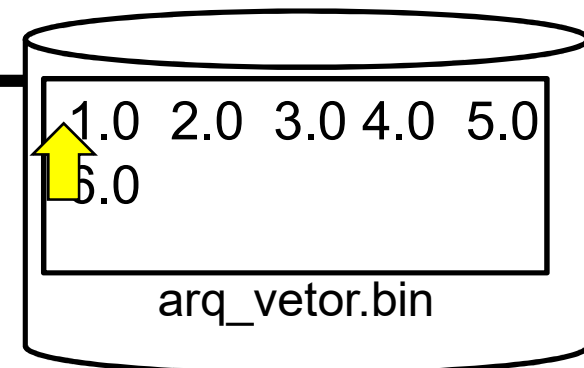
Monitor / Teclado

(3.0,4.0)
(5.0,6.0)
rewind()! De novo!

Memória



HD



rewind()

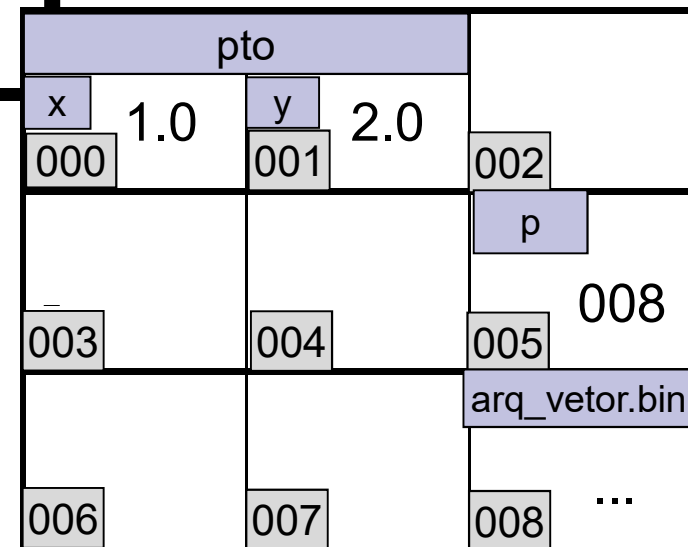
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 1.0 em pto.x e
2.0 em pto.y e move a seta
amarela à frente.*

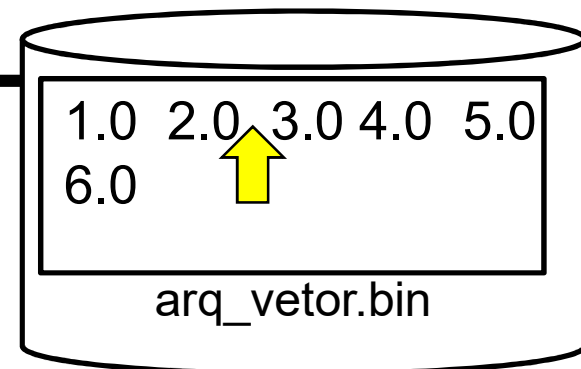
Monitor / Teclado

(3.0,4.0)
(5.0,6.0)
rewind()! De novo!

Memória



HD



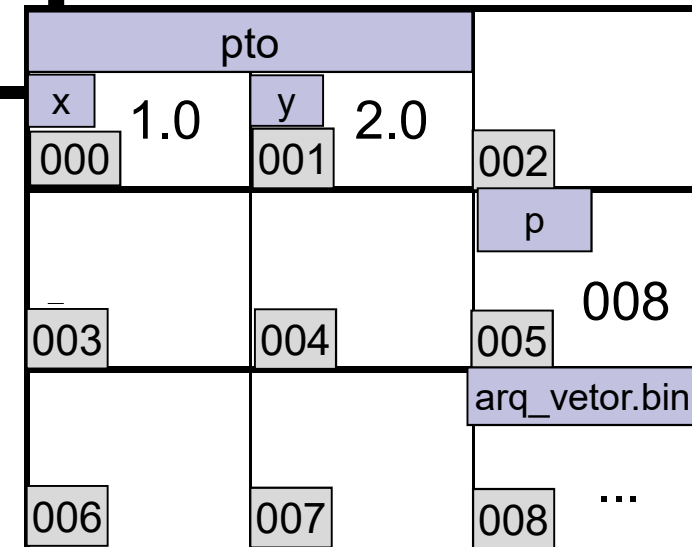
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

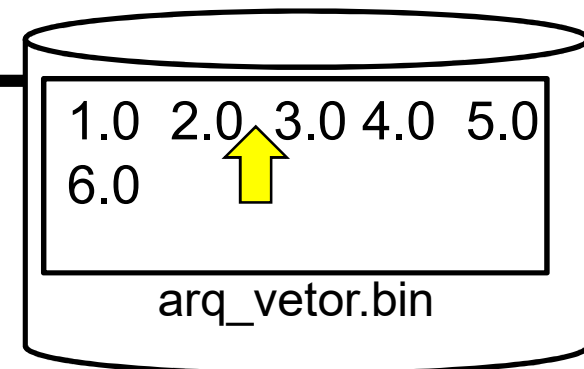
Monitor / Teclado

(5.0,6.0)
rewind()! De novo!
(1.0,2.0)

Memória



HD



rewind()

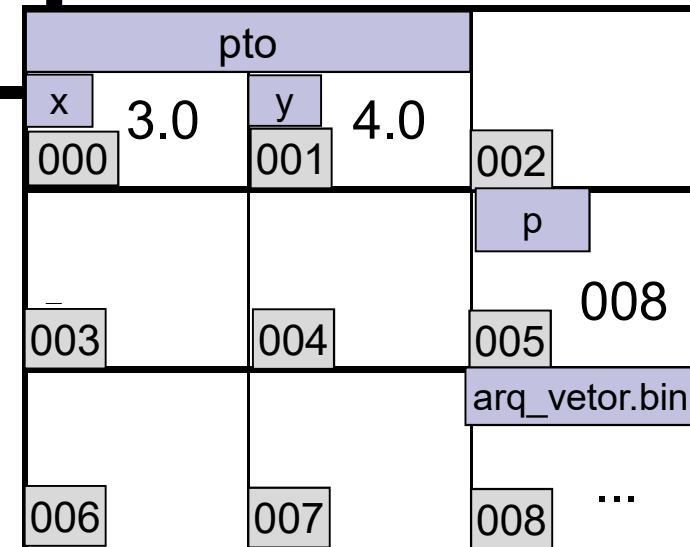
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 3.0 em pto.x e
4.0 em pto.y e move a seta
amarela à frente.*

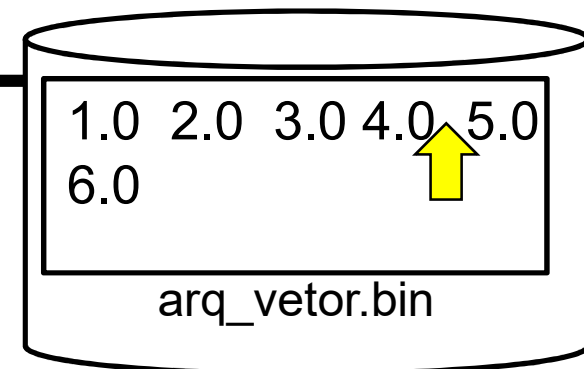
Monitor / Teclado

(5.0,6.0)
rewind()! De novo!
(1.0,2.0)

Memória



HD



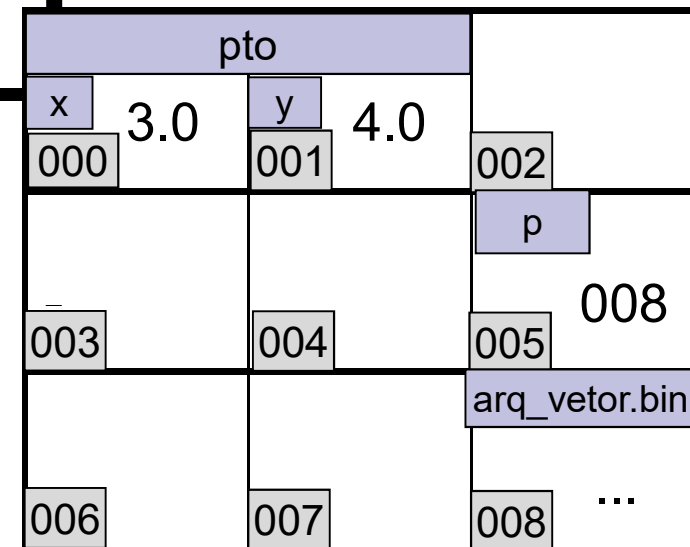
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

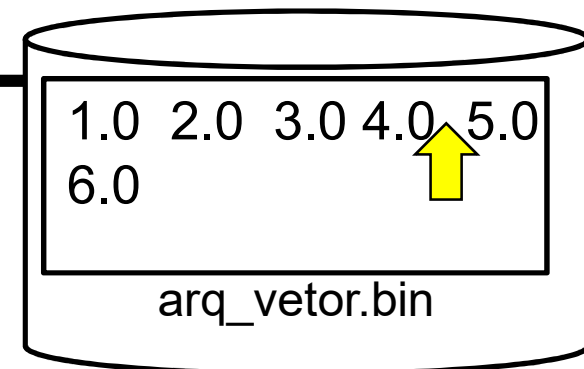
Monitor / Teclado

rewind()! De novo!
(1.0,2.0)
(3.0,4.0)

Memória



HD



rewind()

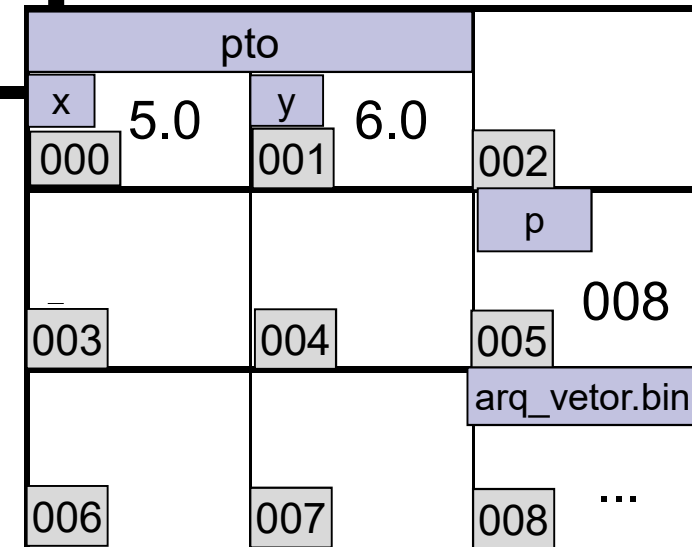
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser bem sucedida.
fread() salva 5.0 em pto.x e
6.0 em pto.y e move a seta
amarela à frente.*

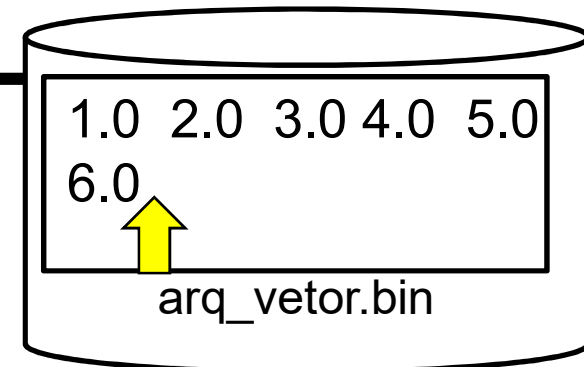
Monitor / Teclado

rewind()! De novo!
(1.0,2.0)
(3.0,4.0)

Memória



HD



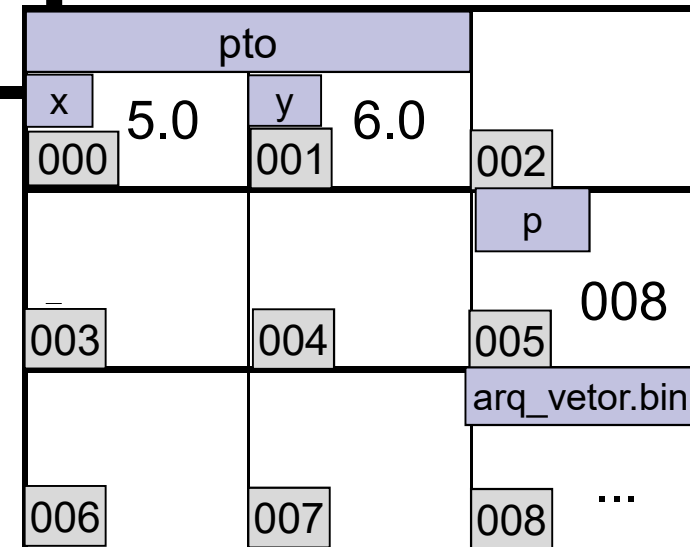
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

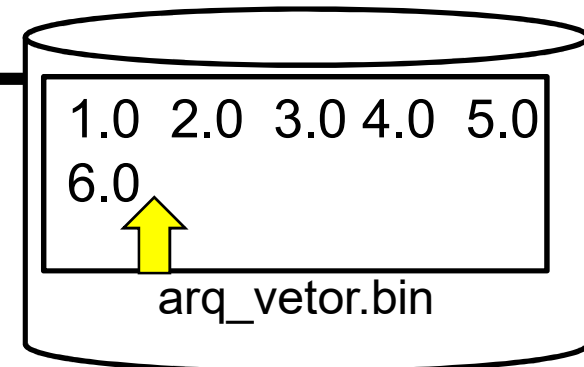
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



rewind()

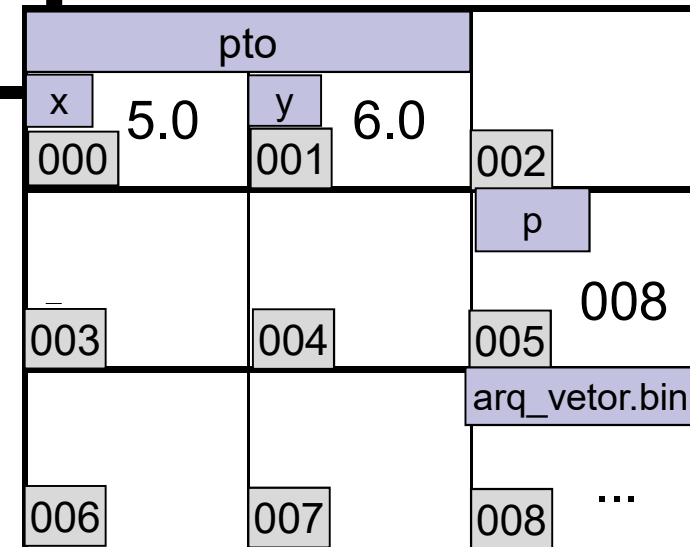
```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

*A leitura vai ser mal sucedida.
fread() retorna 0, pois atingiu
o final do arquivo.*

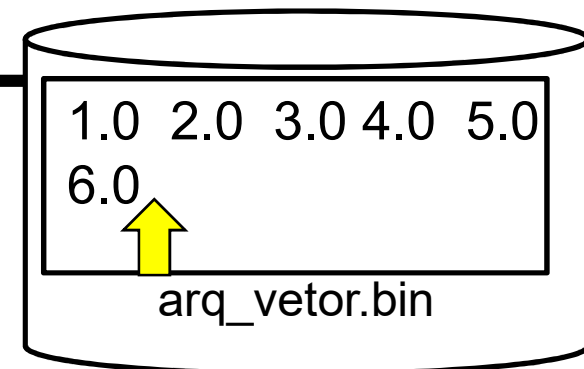
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



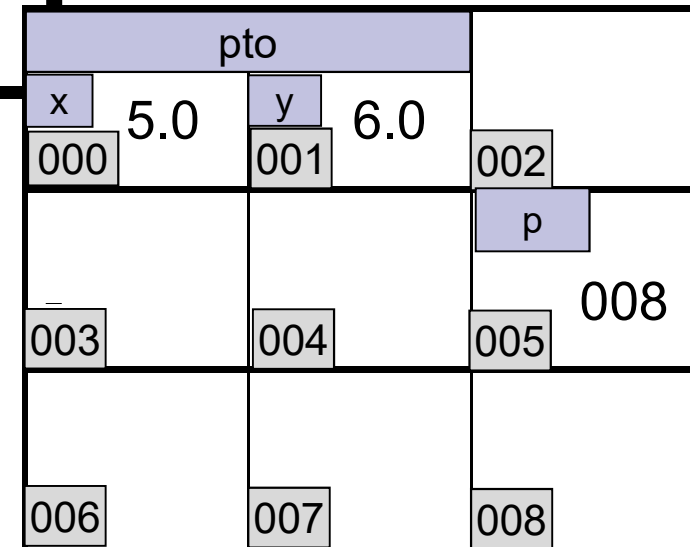
rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

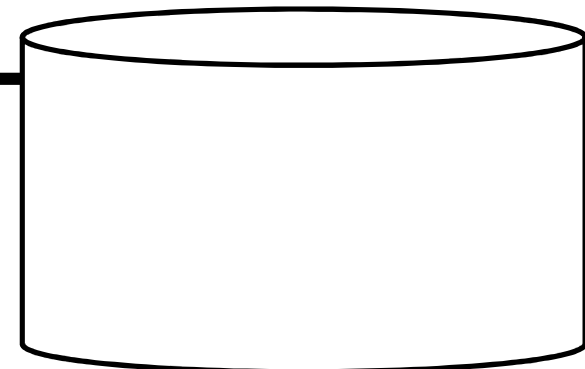
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória



HD



rewind()

```
#include <stdio.h>
typedef struct ponto {float x,y;} Ponto;
int main() {
    Ponto pto;
    FILE* p = fopen("arq_vetor.bin", "rb");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
        rewind(p);
        printf("rewind()! De novo!\n");
        while (fread(&pto,sizeof(pto),1,p) == 1) {
            printf("(%.2f,%.2f)\n", pto.x, pto.y);
        }
    }
    fclose(p);
    return 0;
}
```

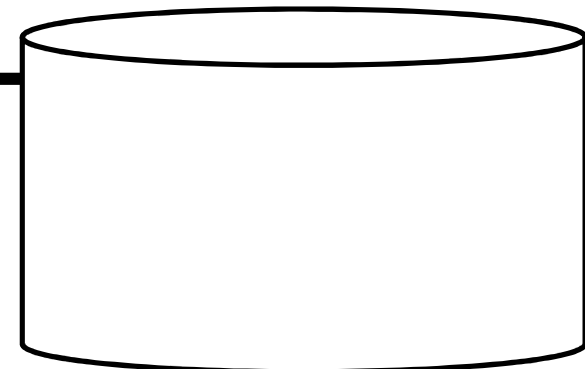
Monitor / Teclado

(1.0,2.0)
(3.0,4.0)
(5.0,6.0)

Memória

000	001	002
003	004	005
006	007	008

HD





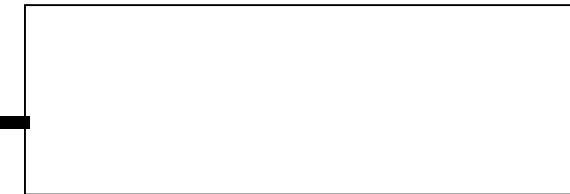
rewind()

```
#include <stdio.h>

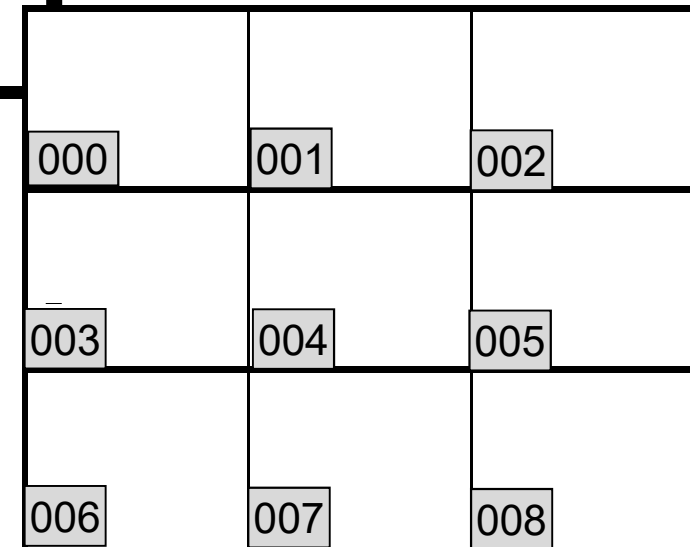
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```


rewind()

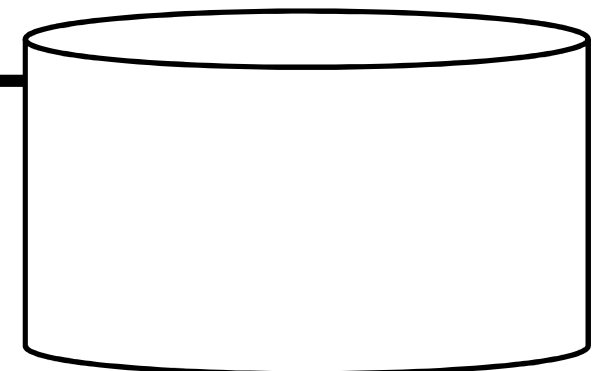
Monitor / Teclado



Memória



HD



```
#include <stdio.h>
```

```
int main() {
```

```
    int x;
```

```
    FILE* p = fopen("arquivo.txt", "w+");
```

```
    if (p == NULL) {
```

```
        printf("Erro!");
```

```
    }
```

```
    else {
```

```
        x = 3;
```

```
        fprintf(p, "%d\n", x);
```

```
        x = 8;
```

```
        fprintf(p, "%d\n", x);
```

```
        rewind(p);
```

```
        while (fscanf(p, "%d", &x) == 1) {
```

```
            printf("%d\n", x);
```

```
        }
```

```
    }
```

```
    fclose(p);
```

```
    return 0;
```

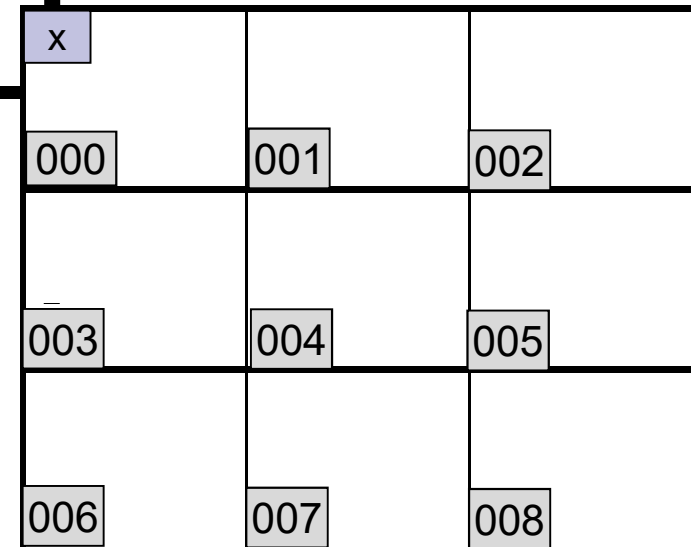
```
}
```

rewind()

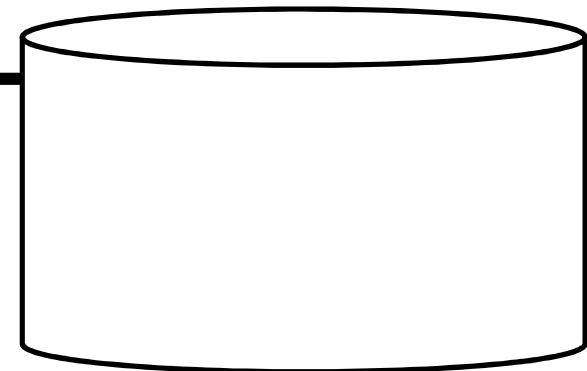
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

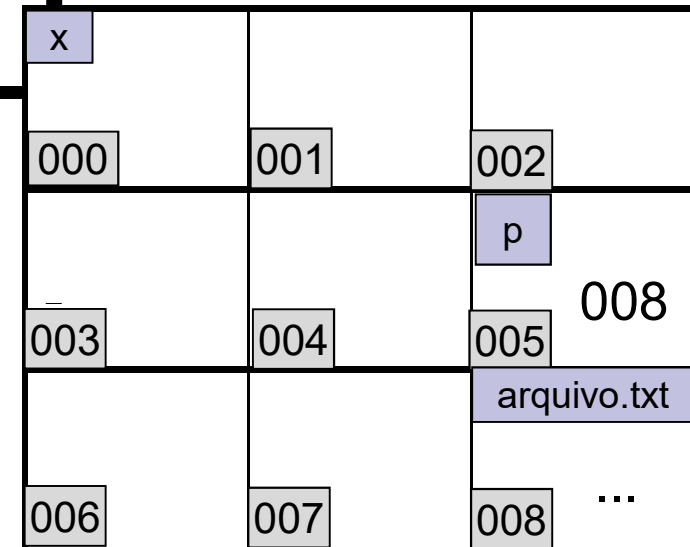


rewind()

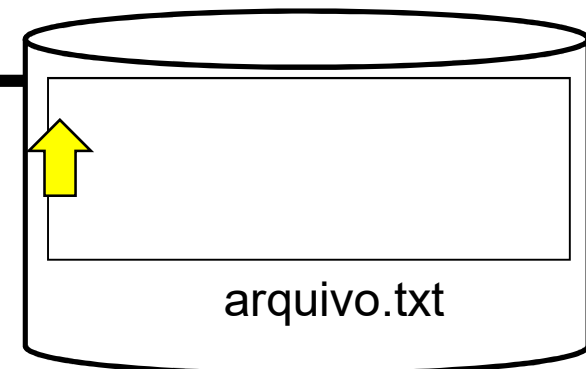
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



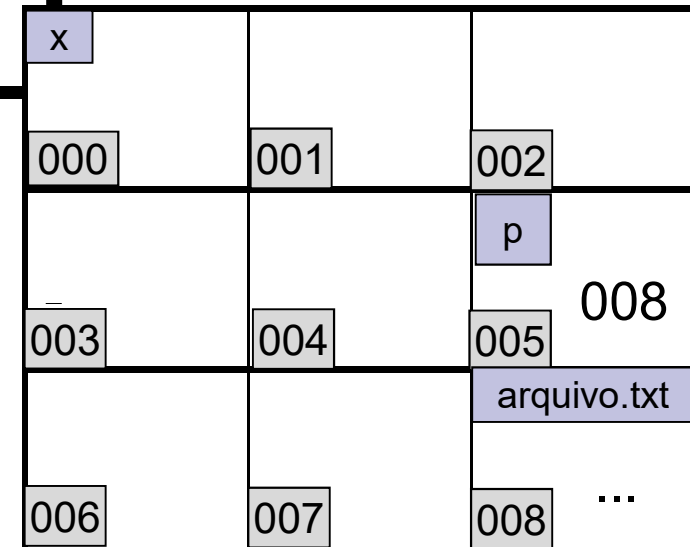
rewind()

```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

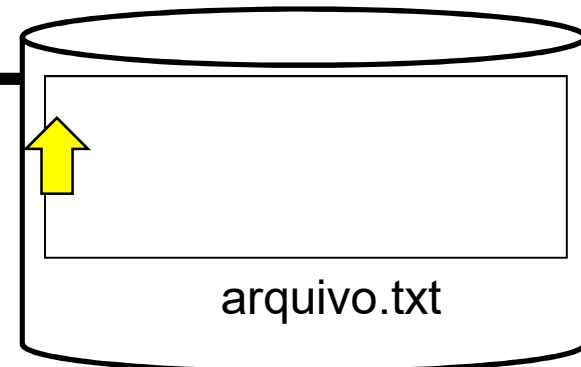
*p == NULL
= 0*

Monitor / Teclado

Memória



HD

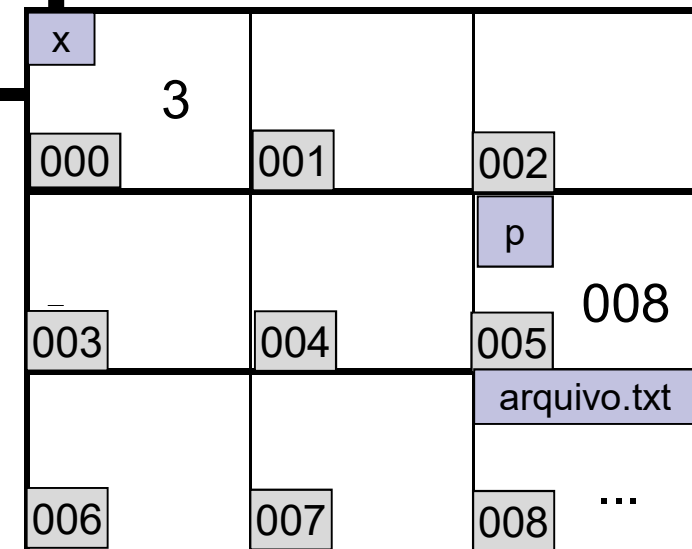


rewind()

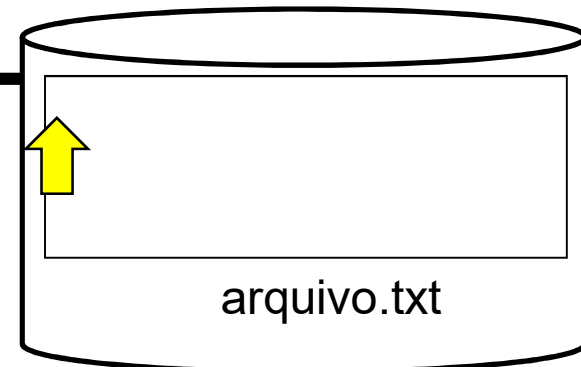
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        → x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

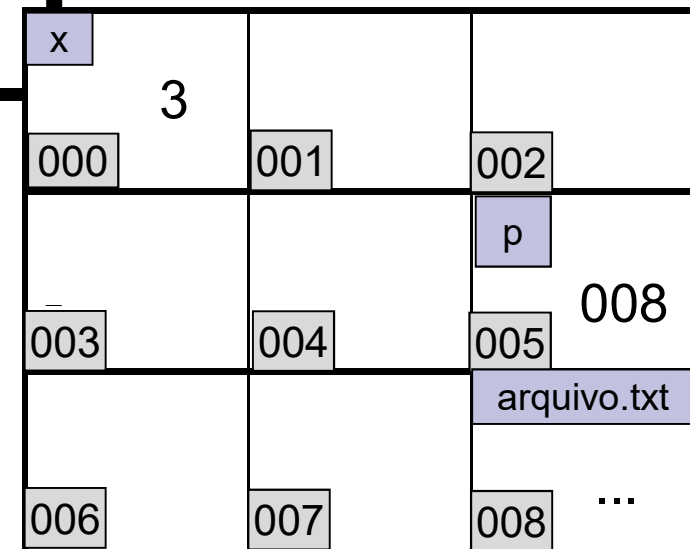


rewind()

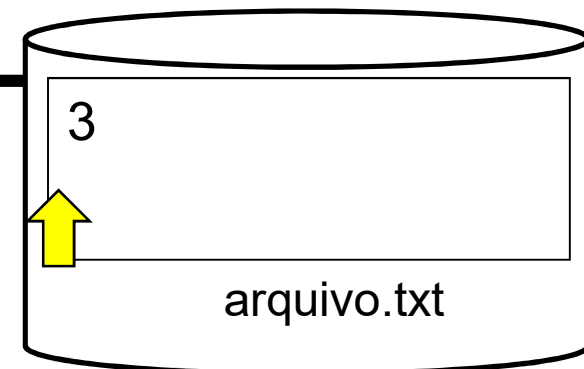
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        ➡ fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

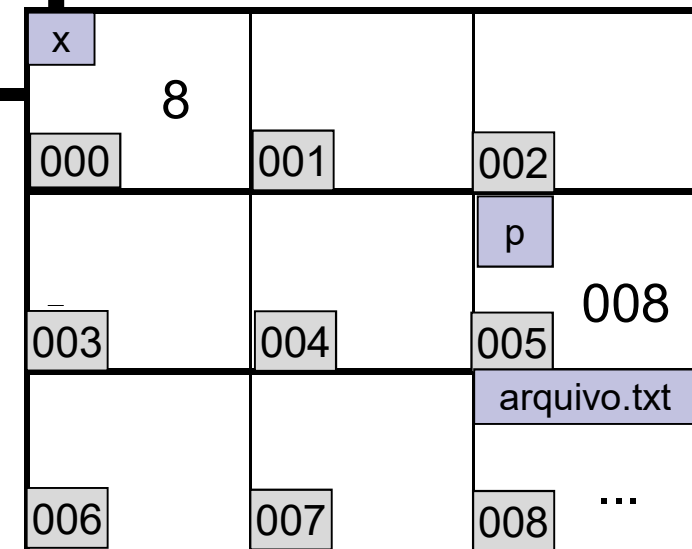


rewind()

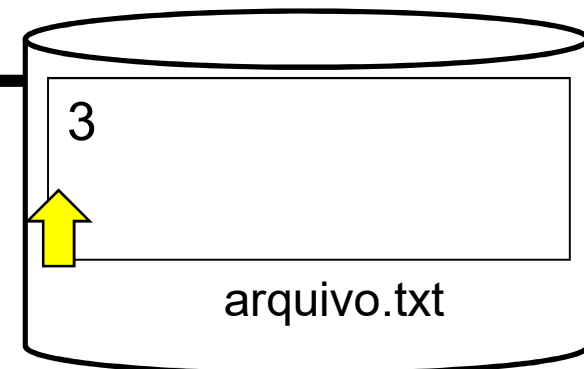
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

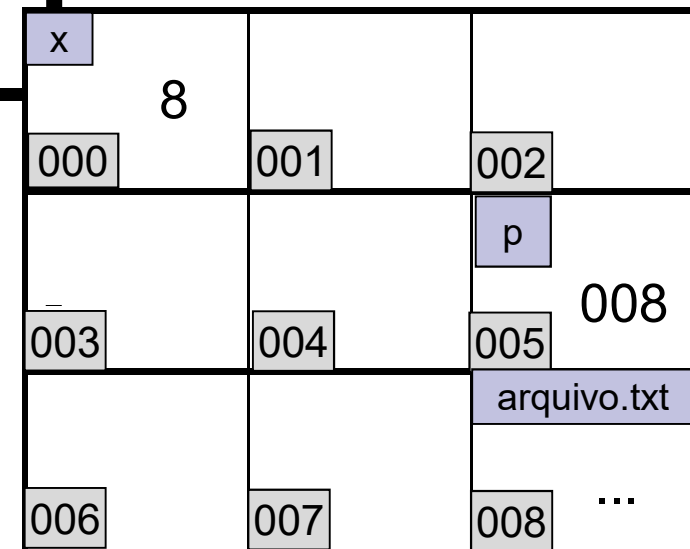


rewind()

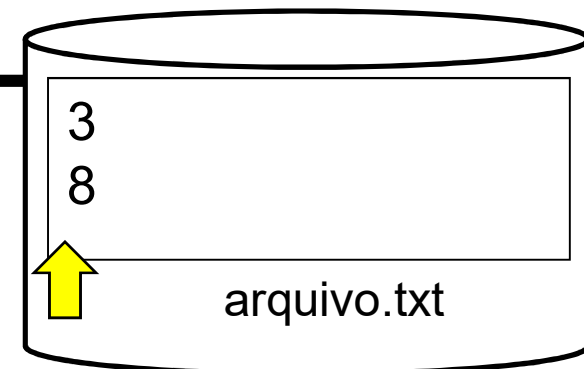
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        ➡ fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

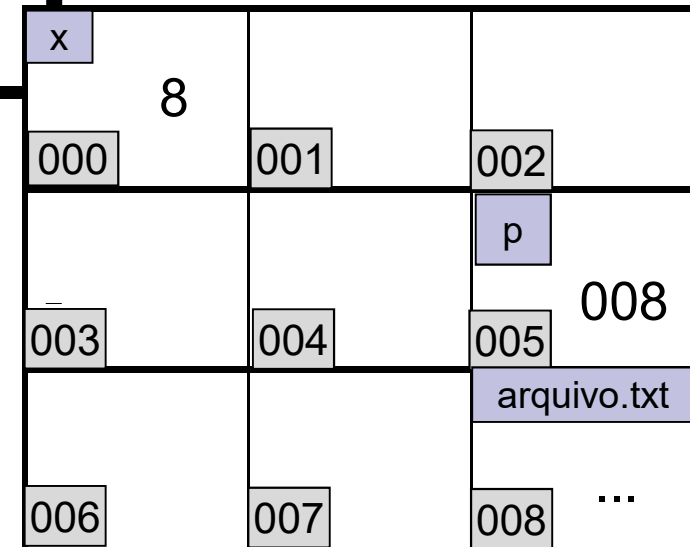


rewind()

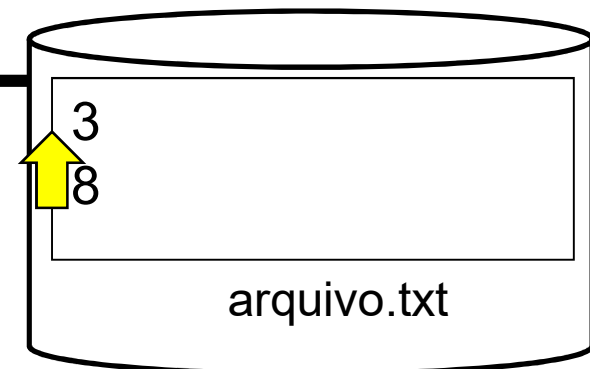
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        → rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



rewind()

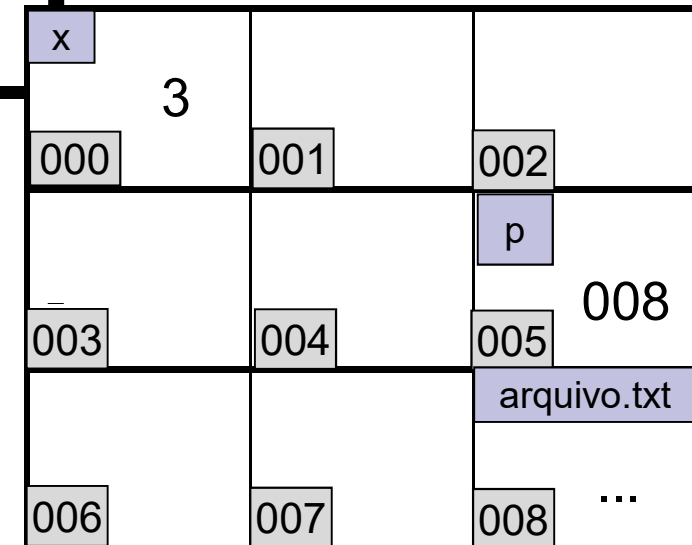
*fscanf(p, "%d", &x) == 1
= 1 == 1
= 1*

```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

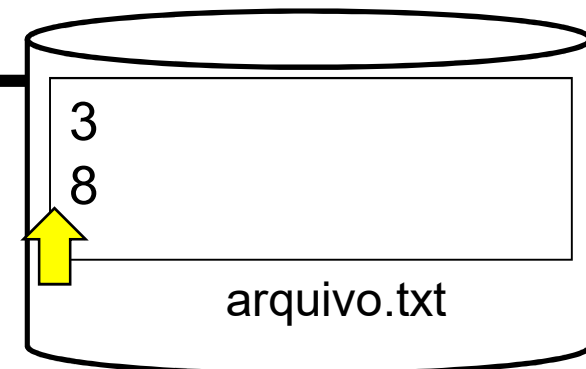
*Vamos assumir que fscanf()
retornou 1. Neste caso, ele grava
na variável x o valor 3 retorna 1
para o main()*

Monitor / Teclado

Memória



HD



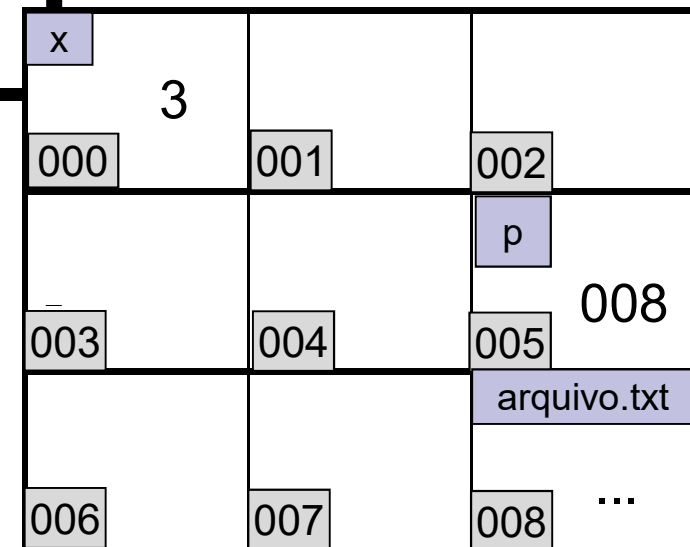
rewind()

```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

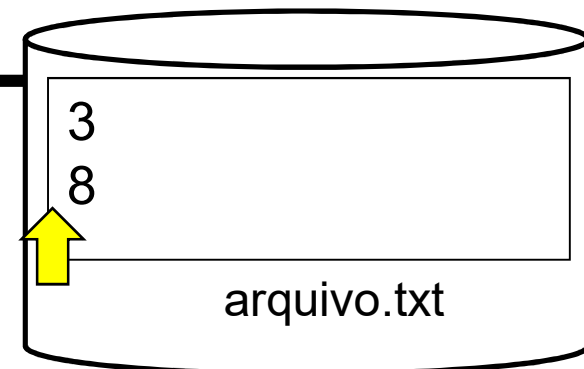
Monitor / Teclado

3

Memória



HD



rewind()

*fscanf(p, "%d", &x) == 1
= 1 == 1
= 1*

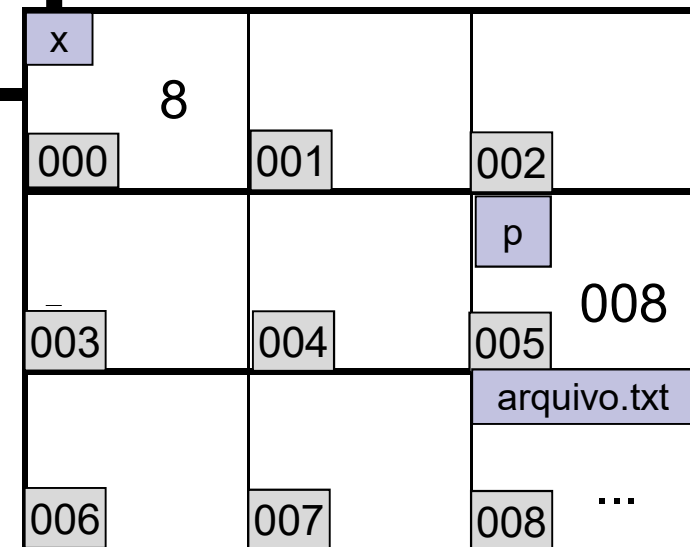
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

*Vamos assumir que fscanf()
retornou 1. Neste caso, ele grava
na variável x o valor 8 retorna 1
para o main()*

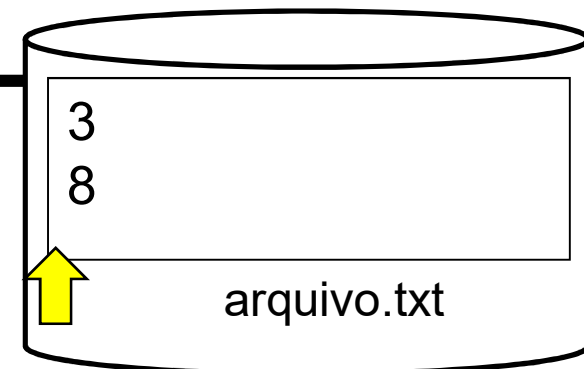
Monitor / Teclado



Memória



HD



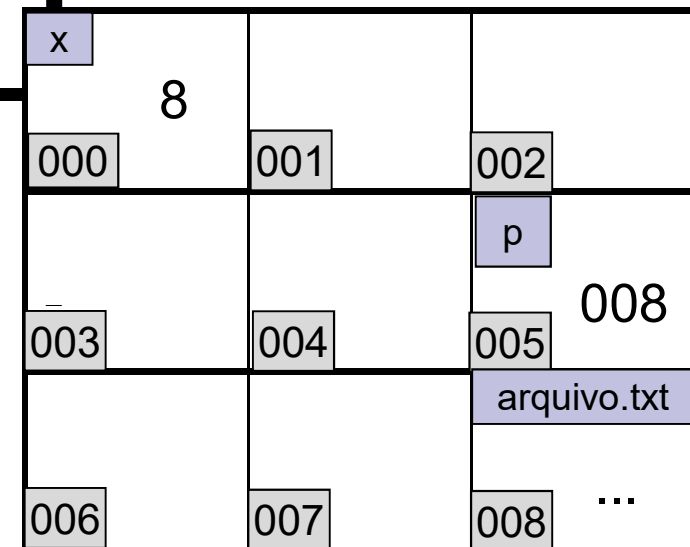
rewind()

```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

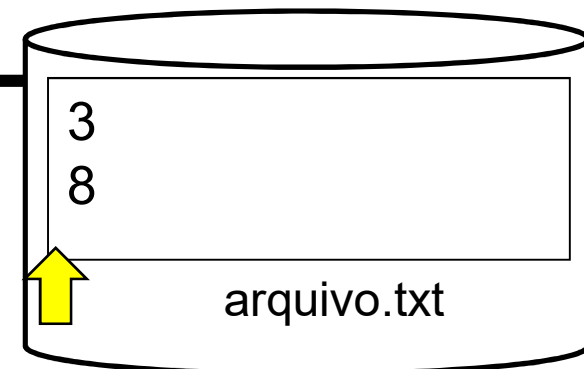
Monitor / Teclado



Memória



HD



rewind()

*fscanf(p, "%d", &x) == 1
= 0 == 1
= 0*

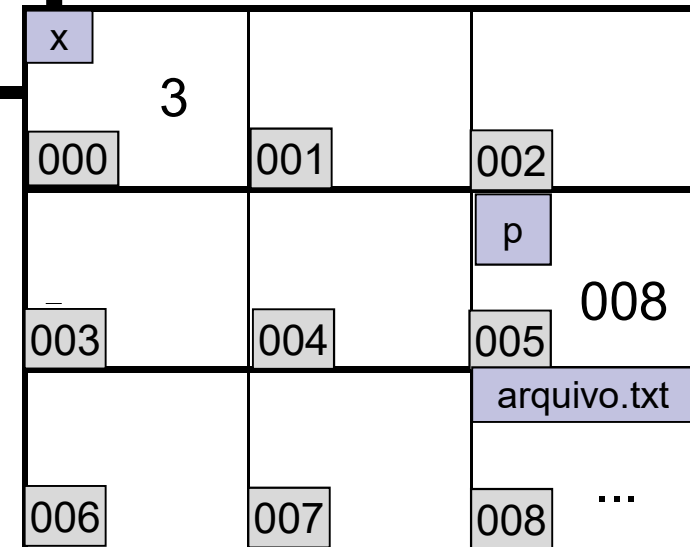
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

Neste caso, fscanf() não consegue ler 1 inteiro e retorna 0.

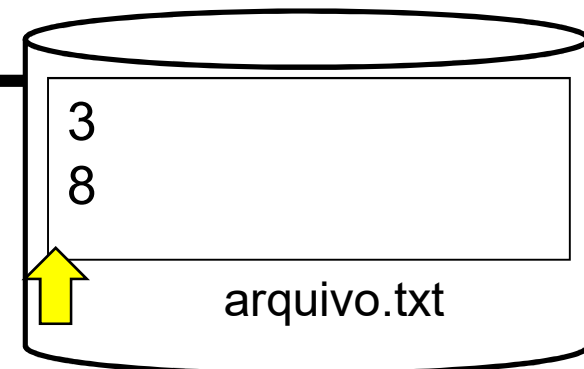
Monitor / Teclado



Memória



HD



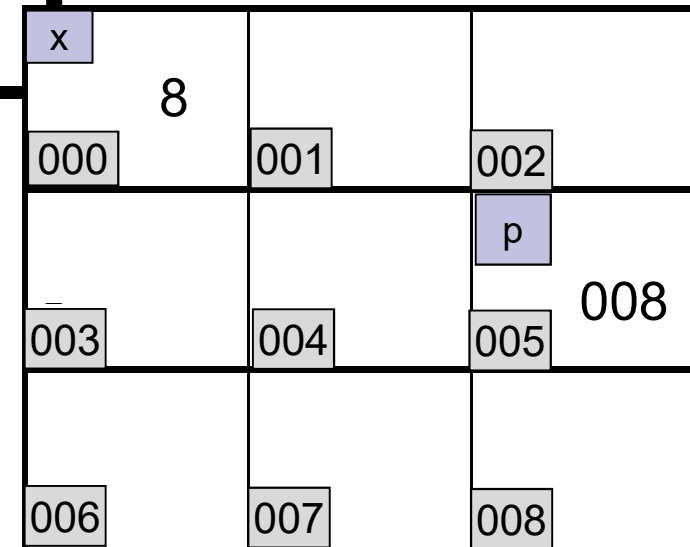
rewind()

```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

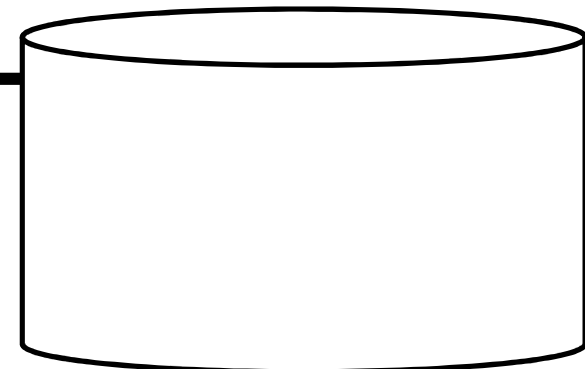
Monitor / Teclado

3
8

Memória



HD



rewind()

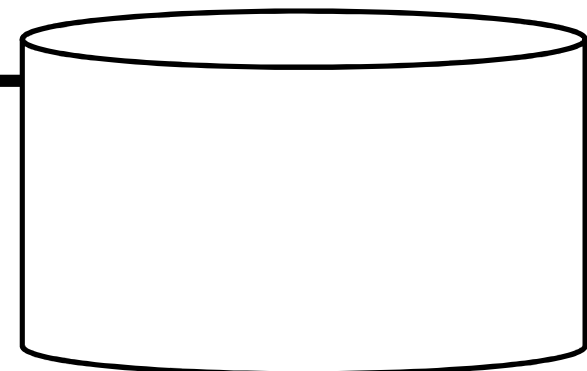
```
#include <stdio.h>
int main() {
    int x;
    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

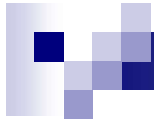
Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD





Exercício

- Abra arquivo.txt no Bloco de Notas e grave:

1

2

3

4

Exercício

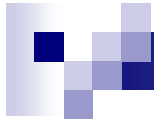
■ Agora, execute o programa abaixo

```
#include <stdio.h>
int main() {
    int x;

    FILE* p = fopen("arquivo.txt", "w+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

*w: cria um novo arquivo.
Se arquivo.txt já existir,
apaga e cria em branco.
Posiciona a seta amarela
no início do arquivo.*

*w+: faz o que o w faz e
permite escrita (fprintf)
e leitura (fscanf)*



Exercício

- De novo, abra arquivo.txt no Bloco de Notas e grave:

1

2

3

4

Exercício

■ Agora, execute o programa abaixo

```
#include <stdio.h>
int main() {
    int x;

    FILE* p = fopen("arquivo.txt", "r+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

r: abre arquivo.txt mas não apaga nada. O arquivo.txt precisa existir. Se não existir, dá erro. Posiciona a seta amarela no início do arquivo.

r+: faz o que o r faz e permite escrita (fprintf) e leitura (fscanf).



Exercício

- Mais uma vez, abra arquivo.txt no Bloco de Notas e grave:

1

2

3

4

Exercício

■ Agora, execute o programa abaixo

```
#include <stdio.h>
int main() {
    int x;

    FILE* p = fopen("arquivo.txt", "a+");
    if (p == NULL) {
        printf("Erro!");
    }
    else {
        x = 3;
        fprintf(p, "%d\n", x);
        x = 8;
        fprintf(p, "%d\n", x);
        rewind(p);
        while (fscanf(p, "%d", &x) == 1) {
            printf("%d\n", x);
        }
    }
    fclose(p);
    return 0;
}
```

a: abre arquivo.txt. Se não existir, cria um em branco. E posiciona a seta amarela no final do arquivo.

a+: faz o que o a faz e permite escrita (fprintf) e leitura (fscanf). A escrita começa no final do arquivo.



Operações

- Abertura

- ☐ O arquivo é preparado para manipulação

- Leitura

- ☐ Dados do arquivo são levados para a memória

- Escrita

- ☐ Dados da memória são levados para o arquivo

- Fechamento

- ☐ Término do uso arquivo e salvamento de dados pendentes



Abertura

```
FILE* fopen(char *nome, char* modo);
```

- **nome** : nome do arquivo

- **modo** : modo de operação. Símbolos:

- ☐ **r** : leitura

- ☐ **w** : escrita

- ☐ **a** : escrita a partir do final do arquivo

- ☐ **b** : modo binário

- ☐ **+** : pode tanto leitura e escrita

- **FILE*** : retorna um apontador do arquivo



Abertura

```
FILE* fopen(char* nome, char* modo) ;
```

■ Uso dos modos de operação

- “r” – Abertura de arquivo **texto** para leitura. O arquivo precisa existir. Se não existir, retorna NULL (dá erro). Ao abrir com sucesso, posiciona a seta amarela no início do arquivo.



Abertura

```
FILE* fopen(char* nome, char* modo) ;
```

■ Uso dos modos de operação

- “w” – Abertura de arquivo **texto** para escrita. Se o arquivo não existir, será criado em branco. Se existir, apaga o conteúdo e deixa o arquivo em branco. Ao abrir com sucesso, posiciona a seta amarela no início do arquivo.



Abertura

```
FILE* fopen(char* nome, char* modo) ;
```

■ Uso dos modos de operação

- “a” – Abertura de arquivo **texto** para escrita. Se não existir, cria um novo em branco. Se existir, abre o arquivo e posiciona a seta amarela no final do arquivo (ou seja, um fprintf() adiciona mais texto aos textos já existentes).



Abertura

```
FILE* fopen(char* nome, char* modo) ;
```

■ Uso dos modos de operação

- ☐ “r+” – Faz o que o “r” faz, além de permitir leitura e escrita.
- ☐ “w+” – Faz o que o “w” faz, além de permitir leitura e escrita.
- ☐ “a+” – Faz o que o “a” faz, além de permitir leitura e escrita.



Abertura

```
FILE* fopen(char* nome, char* modo) ;
```

■ Uso dos modos de operação

- ☐ “rb” – Faz o que o “r” faz, só que para binário.
- ☐ “wb” – Faz o que o “w” faz, só que para binário
- ☐ “ab” – Faz o que o “a” faz, só que para binário



Abertura

```
FILE* fopen(char* nome, char* modo) ;
```

■ Uso dos modos de operação

- “rb+” – Faz o que o “r+” faz, só que para binário.
- “wb+” – Faz o que o “w+” faz, só que para binário.
- “ab+” – Faz o que o “a+” faz, só que para binário.



Computação Eletrônica

Arquivos